
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

MACHINE LEARNING

From Hypotheses to Decisions

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



Every example drawn from IT project management

Effort and schedules, tasks and sprints, tickets and releases, risk and technical debt.

What this volume covers. The whole of the HEC course *Machine Learning* — every topic in the HEC 2023 course outline, and every learning outcome that HEC 2025 attaches to the course — in one resequenced volume, taught from first principles to working code. It is the full-semester companion to the short Summer 2026 handout, which covered a subset of these topics for an eight-week session.

Six Parts • Twenty-Two Chapters • Six Appendices

How to Use This Handout

This is not a reference manual to be searched; it is a **path** to be walked. The chapters are ordered so that every idea rests on ideas you have already met. Reading out of order is possible, but each chapter tells you honestly what it assumes.

The shape of the book

The course moves through six parts, and the movement is deliberate:

- **Part I — Foundations.** What learning from data means, the mathematics it runs on, the workflow and toolkit every later chapter uses, and the oldest precise account of learning: concept learning as a search through a space of hypotheses.
- **Part II — Linear Models and Generalization.** Linear and logistic regression, then the two ideas the rest of the course depends on: why a model that fits its training data can still fail, and how to measure a model honestly.
- **Part III — Supervised Learning Algorithms.** Decision trees, Bayesian learning, nearest neighbours, support vector machines, neural networks, and ensembles that combine them.
- **Part IV — Learning Without Full Labels.** Clustering, dimensionality reduction, self-organizing maps, and semi-supervised learning with the EM algorithm.
- **Part V — Sequences, Decisions and Reinforcement.** Hidden Markov models, Markov decision processes, and agents that learn by trial and error.
- **Part VI — Responsible Practice.** The ethics of deploying a learned model, and the end-to-end project in which you build one.

💡 Why Evaluation Comes Before the Algorithms

Part II ends with two chapters about *judging* models rather than building them: Chapter 7 and Chapter 8. They come before decision trees, SVMs and neural networks on purpose. Every algorithm in Part III is introduced with a claim that it works; without the tools of Part II you have no way to check that claim, and no way to choose between two algorithms that both seem to work. Students who learn the algorithms first and evaluation last tend to report training accuracy for the rest of their careers.

The furniture in every chapter

Each chapter is built from the same parts, always in the same order, so you always know where you are:

Element	What it is for
🎯 Learning Outcomes	What you will be able to <i>do</i> after the chapter. Written as actions, not topics.
🔗 Before You Start	The specific earlier chapters this one leans on.
🔑 Key Terms	Vocabulary introduced here. All terms also appear in the glossary (Appendix D).
📖 Definition	A precise statement of a term. Learn these exactly.
💡 Concept	The intuition behind a definition — why it is built this way.

Element	What it is for
 Worked Example	A complete calculation with real numbers. Work through it with pen and paper; examination numericals follow the same pattern.
 Common Pitfalls	Mistakes that are made every semester. Read these twice.
 Try It Yourself	A short Python lab you can run immediately. Every lab is also in the <code>code/</code> folder as a runnable file.
 Checkpoint	Three quick self-tests. Answers in Appendix C.
 Chapter Summary	The chapter compressed to what must be retained.
 Review Questions	Multiple-choice, short-answer and applied questions for assessment practice.

How to study with it

1. **Read the outcomes first.** They tell you what to look for.
2. **Do not skip the worked examples.** Reading a calculation is not the same as being able to perform one. Cover the answer and try it.
3. **Answer the checkpoint before moving on.** If you cannot, the next chapter will be harder than it needs to be.
4. **Run the labs.** Fifteen minutes of running code teaches more than an hour of reading about it. Then change one number and predict what will happen before you run it again.
5. **Connect each algorithm to a project decision.** For every chapter, say which project-management question it could answer — how long, how risky, which queue, when to refactor. If you can, you understand it.

A Note on Notation and Rigour

This handout uses mathematics where mathematics clarifies — the gradient descent update, Bayes' theorem, the margin of an SVM, the Bellman equation — but it proves nothing that is not needed to use the result correctly. Every formula is accompanied by an explanation in words and, wherever possible, a numeric example you can reproduce by hand. If a formula looks intimidating, read the words around it first and return to the symbols afterwards.

Course Specification

This page is the course file. It states what HEC requires of a course called *Machine Learning*, and shows where in this handout each requirement is met.

Course particulars

Item	Value
Course title	Machine Learning
Programme	BS Information Technology (domain elective); the same course is a BSAI domain core and a BSDS domain elective under HEC 2023, and an elective in the Data Science, AI and Software Engineering specializations of the HEC 2025 BS Computer Science
Credit hours	3 (2–3): two credit hours of lectures and one credit hour of laboratory, which is three contact hours of lab per week
Prerequisite	Artificial Intelligence. Probability and statistics, and programming in Python, are assumed (Chapter 2 and Chapter 3 revise what is needed)
Duration	Sixteen teaching weeks, with a midterm after Week 8 and a final examination after Week 16 (Appendix F)
Course introduction	The HEC aims, verbatim: “a) Present the basic machine learning concepts; b) Present a range of machine learning algorithms along with their strengths and weaknesses; c) Apply machine learning algorithms to solve problems of moderate complexity.”

Course learning outcomes

HEC 2023 gives the course five learning outcomes. HEC 2025 publishes a different list for each specialization that carries the course; together they add evaluation, bias–variance reasoning, modern toolkits and ethics. The seven outcomes below are the union, and each is traced to the chapters that deliver it.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	Explain core machine learning concepts, paradigms and theories, and select an appropriate model for a task	C2 Understand	1–4, 7
CLO-2	Apply supervised learning techniques to classification and regression problems of moderate complexity	C3 Apply	5, 6, 9–14

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-3	Apply unsupervised and semi-supervised techniques — clustering, dimensionality reduction, self-organizing maps, EM — to real data	C3 Apply	15–17
CLO-4	Apply reinforcement learning algorithms to environments with sequential, uncertain dynamics	C3 Apply	18–20
CLO-5	Analyze the performance of machine learning models using evaluation metrics, validation and bias–variance trade-offs	C4 Analyze	7, 8, and every lab
CLO-6	Develop and test an end-to-end machine learning solution of reasonable size using modern toolkits	C6 Create	3, 22, and every lab
CLO-7	Discuss the ethical considerations of applying machine learning to societal problems	A3 Valuing	21, and the fairness check of 22

Coverage of the HEC 2023 course outline

Every topic in the published outline, in the order HEC lists it, with the chapter that teaches it. Nothing is omitted.

HEC 2023 outline topic	Chapter	Where exactly
Introduction to machine learning	1, 2, 3	Definition, paradigms, the design of a learning system; the mathematics and the workflow it needs
Concept learning: general-to-specific ordering of hypotheses	4	The more-general-than relation; FIND-S
Version spaces algorithm	4	Version spaces; LIST-THEN-ELIMINATE
Candidate elimination algorithm	4	Full trace on the release-decision data; inductive bias
Supervised learning: decision trees	9	ID3, entropy and information gain, gain ratio, Gini
Naive Bayes	10	Bayes' theorem, MAP hypotheses, naive Bayes with smoothing
Artificial neural networks	13	Perceptron, delta rule, multilayer networks, backpropagation
Support vector machines	12	Maximum margin, soft margin, kernels
Overfitting, noisy data, and pruning	7, 9	Bias–variance and regularisation (7); pruning trees against noise (9)
Measuring classifier accuracy	8	Confusion matrix, precision, recall, ROC, confidence intervals for error, comparing two classifiers

HEC 2023 outline topic	Chapter	Where exactly
Linear and logistic regression	5, 6	Least squares and gradient descent; the sigmoid and cross-entropy
Unsupervised learning: hierarchical agglomerative clustering	15	Linkage criteria, dendrograms
k-means partitional clustering	15	Lloyd's algorithm, choosing k
Self-organizing maps (SOM)	16	Competitive learning, neighbourhoods, the U-matrix
k-nearest-neighbour algorithm	11	Distance, choice of k , weighting, the curse of dimensionality
Semi-supervised learning with EM using labeled and unlabeled data	17	Gaussian mixtures, the EM algorithm, EM over labelled plus unlabelled data
Reinforcement learning: hidden Markov models	18	Forward algorithm, Viterbi, Baum–Welch as EM
Monte Carlo inference	20	Estimation by sampling; Monte Carlo value estimation and control
Exploration vs. exploitation trade-off	20	Multi-armed bandits, ϵ -greedy, UCB
Markov decision processes	19	Bellman equations, value and policy iteration
Ensemble learning: committees of multiple hypotheses; bagging, boosting	14	Voting, bagging and random forests, AdaBoost and gradient boosting

💡 Three Places Where This Handout Departs from the Outline's Grouping

The HEC outline is a list of topics, not a teaching order, and three of its groupings would mislead a student if followed literally.

- **k -nearest-neighbour** is listed under *unsupervised* learning. It is a supervised method — it needs labels to vote with — so it is taught in Part III with the other classifiers (Chapter 11).
- **Hidden Markov models** are listed under *reinforcement* learning. An HMM makes no decisions and receives no reward; it is a model of sequences. It is taught at the start of Part V because its Markov machinery is exactly what Markov decision processes build on (Chapter 18).
- **Overfitting and measuring accuracy** are listed after the supervised algorithms. They are taught before them (Part II), for the reason given on the previous page.

Every topic is still taught; only the order changes.

Coverage of the HEC 2025 learning outcomes

HEC 2025 publishes learning outcomes but no outline. The outcomes it attaches to *Machine Learning* in each specialization are all met:

Specialization	Distinctive outcomes it adds	Met in
Software Engineering	Analyze performance using evaluation metrics (C4); implement with modern toolkits (C3); discuss ethical considerations (A3)	Ch 8; Ch 3 and labs; Ch 21

Specialization	Distinctive outcomes it adds	Met in
Data Science	Apply ML to classification problems and to real-world problems (C3)	Chs 6, 9–14; project (Ch 22)
Artificial Intelligence	Differentiate ML paradigms and select appropriate models; implement, train and evaluate SVMs, decision trees and k -means; analyse performance with metrics	Chs 1, 7; Chs 9, 12, 15; Ch 8
Robotics	Supervised, unsupervised and reinforcement learning; regression, SVM, PCA and clustering; bias–variance trade-offs	Chs 1, 5, 12, 15, 16, 19–20; Ch 7
Network Infrastructure and Cloud Computing	Design and develop end-to-end ML solutions for real-world datasets	Ch 22 and the labs

⚠ One HEC 2025 List Belongs to a Different Course

The Internet of Things specialization lists, under the name *Machine Learning*, outcomes about CNNs, RNNs and transformers and about deploying models on devices. Those are the outcomes of a *Deep Learning* course. This handout introduces neural networks thoroughly (Chapter 13) and signposts the deep architectures, but does not teach them; a department offering this course in the IoT specialization should pair it with Deep Learning rather than stretch it.

What this course deliberately leaves to other courses

Machine Learning overlaps heavily with Data Mining and with Introduction to Data Science. Following the split recommended in the department’s HEC curriculum map, this course owns *prediction* — generalization, model selection, evaluation, ensembles and regularisation — and leaves *discovery* to Data Mining: association rules and frequent patterns, large-scale preprocessing, outlier mining and graph mining are not taught here. Deep architectures (CNNs, RNNs, transformers) belong to Deep Learning, and deployment and monitoring to MLOps.

Assessment

Instrument	Weight	What it assesses
Quizzes (three)	10%	CLO-1, CLO-2: definitions and one short numerical each
Assignments (two)	10%	CLO-2, CLO-5: a by-hand numerical and a coded experiment each
Project	15%	CLO-5, CLO-6, CLO-7: the end-to-end project of Chapter 22
Midterm examination	25%	Chapters 1–8
Final examination	40%	The whole course, weighted towards Chapters 9–21

These weights are a suggestion. Replace them with your department’s scheme; the mapping from instruments to outcomes is what a course file actually needs to show.

Recommended texts

Set by HEC.

1. Tom M. Mitchell, *Machine Learning*, McGraw-Hill, 1997. The source of the concept-learning, decision-tree, Bayesian-learning and instance-based chapters; its worked examples are reproduced here.
2. Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012. The reference for EM, HMMs and the probabilistic view.

Recommended alongside.

3. Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 3rd ed., O'Reilly, 2022. For the labs.
4. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani and Jonathan Taylor, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023. Free online; the clearest treatment of the bias–variance trade-off.
5. Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. Free online; the source for Chapters 19 and 20.

The Course at a Glance

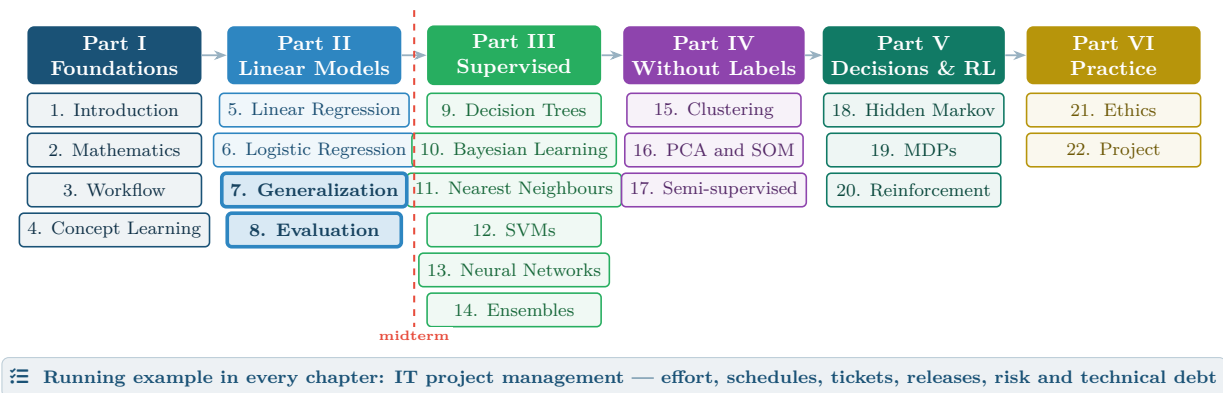


Figure F.1: Course map. The six parts run left to right in the order they are taught; the chapters in each column are the order within that part. The two outlined chapters of Part II are the hinge of the course, and the dashed line marks the midterm.

The Running Example: IT Project Management

Every example in this handout comes from one setting: a software organisation delivering IT projects. Keeping to one setting keeps the course simple — the vocabulary of tasks, sprints, tickets and releases is learned once, and each new algorithm is met as a new answer to a familiar kind of question.

Recurring example	Chapters
Story points and the hours stories really take	1, 5
Tasks and projects that finish late	1, 3, 6, 11–14, 16, 22
Release approvals and sprint goals	4, 9, 10
Code changes, defects and releases	2, 8, 10
Tickets: their text, type, queue and resolution time	6, 8, 10, 15, 17
A project's hidden weekly health	18, 20
Technical debt: ship features or refactor?	19, 20
Screening candidates for developer roles	21

💡 What Makes Project Data Hard

Project data is *small* — a team completes a few hundred tasks a year — so overfitting (Chapter 7) is a constant danger. It is *noisy and human-reported*: estimates, statuses and root causes are typed in by busy people. Its interesting events are *rare*: most tasks finish, most changes are clean, so accuracy misleads (Chapter 8). And a prediction *changes the behaviour* of the team that hears it, so a model of last year's projects describes a world the model itself will alter. Every chapter meets at least one of these.

Two of the set text's classic tables — Mitchell's *EnjoySport* (Chapter 4) and *PlayTennis* (Chapter 9, Chapter 10) — appear here with their attributes renamed as release and sprint decisions, so every number can still be checked against the book.

Notation Used in This Handout

Symbol	Meaning
n	Number of training examples (rows)
d	Number of features (columns, attributes)
$\mathbf{x}_i$	The i -th example, a vector of d feature values; bold lower case is always a vector
x_{ij}	The value of feature j in example i
y_i	The true target (label) of example i
$\hat{y}_i$	The model's <i>prediction</i> of y_i ; the hat always means “estimated”
X	The feature matrix, n rows by d columns; capitals are matrices
$\mathbf{w}, b$	Weights and bias (intercept) of a linear model
θ	All parameters of a model, collected together
h, H	A hypothesis, and the hypothesis space it is drawn from (Chapter 4)
$L(\theta)$	Loss — how wrong the model with parameters θ is on the data
∇L	Gradient: the vector of partial derivatives of L
α or η	Learning rate (step size)
λ	Regularisation strength
$P(A B)$	Probability of A given B
$\mathbb{E}[X]$	Expected (average) value of X
s, a, r	State, action and reward, in Part V
γ	Discount factor for future rewards, in Part V
π	A policy (Part V); also the initial-state distribution of an HMM (Chapter 18)

⚠ Two Symbols Students Confuse Every Year

y versus $\hat{y}$: y is the *truth*, $\hat{y}$ is the *guess*. Every loss function in this book is some way of summarising the gaps between them.

Training error versus *true* error: the first is measured on the data the model learned from, the second is what happens on data it has never seen. The whole of Part II is about the gap between them.

Prerequisite Self-Check

You are ready for Part I if you can answer these. If several give you trouble, work through Chapter 2 slowly and keep Appendix B beside you.

1. Compute the dot product of $(1, 2, 3)$ and $(4, 0, -1)$.
2. What is the derivative of $f(w) = (w - 3)^2$, and where is f smallest?
3. A test is positive for 90% of sick people and 10% of healthy ones; 1% of people are sick. Roughly what fraction of positive tests are sick people?
4. What is $\log_2 8$? What is $\log_2 1$?
5. In a table of network connections, what does one *row* usually represent? What does one *column* represent?
6. Write a Python loop that prints the squares of 1 to 5.

*Answers: (1) $4+0-3=1$ (2) $2(w-3)$, smallest at $w=3$ (3) about 8% — $0.009/(0.009+0.099)$
(4) 3; 0 (5) one row = one connection (an example); one column = one attribute (a feature)
(6) e.g. `for i in range(1,6): print(i**2)`*

Contents

How to Use This Handout	ii
Course Specification	iv
The Course at a Glance	ix
List of Figures	xxii
I Foundations	1
1 What Machine Learning Is	3
1.1 Why Learn a Program Instead of Writing One?	3
1.2 A Precise Definition	4
1.3 The Four Paradigms	5
1.4 Designing a Learning System	6
1.5 Learning as Search Through a Hypothesis Space	7
1.6 A Short History	7
1.7 Where Machine Learning Sits	8
2 Mathematics for Machine Learning	11
2.1 Four Kinds of Mathematics, Four Jobs	11
2.2 Vectors and the Dot Product	12
2.3 Matrices: A Whole Dataset at Once	12
2.4 Derivatives and Gradients	13
2.5 Gradient Descent	13
2.6 Probability	14
2.7 Expectation, Variance and the Gaussian	15
2.8 Maximum Likelihood	15
2.9 Information and Entropy	16
3 The Machine Learning Workflow and Toolkit	19
3.1 One Experiment, Start to Finish	19
3.2 Examples, Features and Labels	20
3.3 Splitting the Data	20
3.4 Preparing Features	21
3.5 The scikit-learn Interface	21
3.6 Pipelines and Data Leakage	22
3.7 Baselines	23

4	Concept Learning and the Version Space	26
4.1	Concept Learning as a Task	26
4.2	The Inductive Learning Hypothesis	27
4.3	The General-to-Specific Ordering	28
4.4	Find-S: The Most Specific Consistent Hypothesis	28
4.5	Version Spaces	29
4.6	The Candidate-Elimination Algorithm	29
4.7	Using a Partially Learned Concept	30
4.8	Inductive Bias	31
II	Linear Models and Generalization	35
5	Linear Regression	37
5.1	The Model	37
5.2	The Loss: Mean Squared Error	37
5.3	Least Squares by Formula	38
5.4	Fitting by Gradient Descent	39
5.5	Multiple and Polynomial Features	40
5.6	Measuring a Regression Model	40
6	Logistic Regression	44
6.1	Why Not Just Use Linear Regression?	44
6.2	The Sigmoid and the Log-Odds	45
6.3	The Decision Boundary	45
6.4	The Loss: Cross-Entropy	46
6.5	Fitting by Gradient Descent	47
6.6	More Than Two Classes	47
7	Overfitting, Bias–Variance and Regularisation	51
7.1	The Goal Is Generalisation	51
7.2	Underfitting and Overfitting	51
7.3	The Bias–Variance Decomposition	52
7.4	Noisy Data	54
7.5	Diagnosing With Learning Curves	54
7.6	Regularisation	54
7.7	Model Selection With Cross-Validation	55
8	Measuring Classifier Performance	59
8.1	Measuring Classifier Accuracy Is Not One Number	59
8.2	The Confusion Matrix	59
8.3	Precision, Recall and the Threshold	60
8.4	Threshold-Free Summaries: ROC and PR Curves	61
8.5	More Than Two Classes	62
8.6	How Sure Is the Number? Confidence Intervals for Error	62

8.7	Is One Classifier Really Better? McNemar's Test	63
8.8	Handling Class Imbalance	63
III	Supervised Learning Algorithms	66
9	Decision Tree Learning	68
9.1	Trees as Hypotheses	68
9.2	Choosing the Question: Entropy and Information Gain	69
9.3	Beyond the Basic Algorithm	70
9.4	Overfitting, Noisy Data and Pruning	71
9.5	The Inductive Bias of ID3	72
10	Bayesian Learning and Naive Bayes	75
10.1	Learning as Updating Belief	75
10.2	Bayes' Theorem for Hypotheses	75
10.3	The Bayes Optimal Classifier	77
10.4	Naive Bayes	77
10.5	The Zero-Frequency Problem and Laplace Smoothing	78
10.6	Continuous Features: Gaussian Naive Bayes	79
10.7	Text Classification	79
11	Instance-Based Learning: k-Nearest Neighbours	83
11.1	Learning by Remembering	83
11.2	The k -Nearest Neighbour Algorithm	84
11.3	Distance and Scale	85
11.4	Choosing k	85
11.5	Beyond Classification	86
11.6	The Curse of Dimensionality	86
11.7	The Cost of Being Lazy	86
12	Support Vector Machines	89
12.1	Which Line?	89
12.2	The Maximum-Margin Hyperplane	89
12.3	Soft Margins and the Hinge Loss	90
12.4	The Kernel Trick	91
12.5	Practicalities	93
13	Artificial Neural Networks	95
13.1	From Neurons to Units	95
13.2	The Perceptron	96
13.3	Linear Separability and XOR	97
13.4	Gradient Descent and the Delta Rule	97
13.5	Multilayer Networks and Backpropagation	97
13.6	Activation Functions and Outputs	99
13.7	Training Well: Overfitting and Its Cures	99

14 Ensemble Learning	103
14.1 Committees of Hypotheses	103
14.2 Bagging	104
14.3 Random Forests	105
14.4 Boosting	105
14.5 Stacking	106
 IV Learning Without Full Labels	 110
15 Clustering	112
15.1 Finding Groups Without Labels	112
15.2 k -Means	112
15.3 Hierarchical Agglomerative Clustering	114
15.4 How Many Clusters?	115
15.5 Beyond the HEC Outline	115
 16 Dimensionality Reduction and Self-Organizing Maps	 118
16.1 Why Fewer Dimensions	118
16.2 Principal Component Analysis	119
16.3 How Many Components?	120
16.4 Self-Organizing Maps	121
 17 Semi-Supervised Learning and the EM Algorithm	 125
17.1 Few Labels, Many Examples	125
17.2 Self-Training	126
17.3 Mixture Models and Hidden Variables	126
17.4 The EM Algorithm	127
17.5 EM With Labelled and Unlabelled Data	128
 V Sequences, Decisions and Reinforcement	 132
18 Hidden Markov Models	134
18.1 When Order Matters	134
18.2 Markov Chains	134
18.3 Hidden Markov Models	135
18.4 Evaluation: The Forward Algorithm	136
18.5 Decoding: The Viterbi Algorithm	136
18.6 Learning: Baum–Welch	137
18.7 Monte Carlo Estimation	138
 19 Markov Decision Processes	 141
19.1 Decisions Whose Consequences Unfold	141
19.2 The Markov Decision Process	142

19.3 Value Functions and the Bellman Equations	143
19.4 Value Iteration	143
19.5 Policy Iteration	144
20 Reinforcement Learning	147
20.1 Learning From Consequences	147
20.2 Monte Carlo Methods	148
20.3 Exploration and Exploitation	148
20.4 Temporal-Difference Learning	149
20.5 Q-Learning	150
20.6 From Tables to Deep Reinforcement Learning	151
VI Responsible Practice	154
21 Responsible and Ethical Machine Learning	156
21.1 Why a Machine Learning Course Has an Ethics Chapter	156
21.2 Where Bias Enters	157
21.3 Measuring Fairness	157
21.4 Privacy	158
21.5 Transparency and Explainability	159
21.6 Accountability and Security	159
22 The Machine Learning Project	162
22.1 What the Project Is For	162
22.2 Choosing and Scoping a Problem	162
22.3 The Plan	163
22.4 Running Experiments Honestly	164
22.5 The Report and the Presentation	165
22.6 Assessment	165
Appendices	169
A Python and scikit-learn Reference	170
A.1 Setting Up	170
A.2 NumPy in Ten Lines	170
A.3 pandas in Ten Lines	170
A.4 The scikit-learn Pattern	171
A.5 Models, Chapter by Chapter	171
A.6 Evaluation Calls	172
A.7 Errors Students Meet Every Semester	172

B	Formula Sheet	173
B.1	Mathematics (Chapter 2)	173
B.2	Workflow and Linear Models (Chapters 3, 5, 6)	173
B.3	Generalisation and Evaluation (Chapters 7, 8)	174
B.4	Supervised Algorithms (Chapters 9–14)	174
B.5	Unsupervised, Sequences and Decisions (Chapters 15–21)	174
C	Answers to Checkpoints	176
D	Glossary	181
E	Resources	185
E.1	Textbooks	185
E.2	Online Courses	185
E.3	Datasets	185
E.4	Tools	186
E.5	The Summer 2026 Handout	186
F	Sixteen-Week Teaching Schedule	187
F.1	The Schedule	187
F.2	Dependency Map	188
F.3	Laboratory Sessions	188

List of Figures

F.1	Course map. The six parts run left to right in the order they are taught; the chapters in each column are the order within that part. The two outlined chapters of Part II are the hinge of the course, and the dashed line marks the midterm. . . .	ix
P1.1	Reading path through Part I. Chapter 3 (highlighted) sets up the workflow that every later lab repeats; Chapter 4 formalises the idea of learning introduced in Chapter 1.	2
1.1	Traditional programming against machine learning. The learned model is then used exactly like a program: new data in, answers out.	4
1.2	The four paradigms, distinguished by what the experience tells the learner about the right answer: all of it, none of it, a little of it, or only whether what it did turned out well.	5
1.3	Learning as search. The data narrows the hypothesis space to those hypotheses consistent with it; something other than the data must choose among the survivors. . . .	7
1.4	Selected milestones. Most of what this course teaches was in place by 2001; what changed after 2012 was mainly scale — data, computation, and the depth of the networks.	7
2.1	A matrix–vector product scores every example in one operation. Each row of X is one example; each entry of $\hat{y}$ is that row’s dot product with the weights.	13
2.2	Gradient descent on $L(w) = (w - 3)^2$ from $w = 0$. With a small learning rate each step closes a fixed fraction of the gap; with one that is too large, each step jumps past the minimum by more than the last, and the weights diverge.	14
2.3	The likelihood of seven delivered sprints in ten, as a function of p . It is largest at $p = 0.7$ — the observed proportion, which is what maximum likelihood gives for any such count.	16
3.1	The workflow every lab follows. The loop between training and validation may run dozens of times; the test set, split off in step 3, is used exactly once, in step 8. . . .	19
3.2	The three-way split. Only the test score is an honest estimate of performance on new data, and it stays honest only if it is looked at once.	20
3.3	Train–test contamination. Fitting any preparation step before the split lets the test rows influence training; a pipeline fitted on the training data alone cannot make this mistake.	22
4.1	The more-general-than ordering. A more general hypothesis covers a superset of the instances; hypotheses whose covered sets merely overlap, like h_1 and h_3 , are not comparable.	28
4.2	The version space is everything between the two boundaries. Each example moves one boundary towards the other; if they meet, the concept has been learned exactly. . . .	30
4.3	The final version space for the release decisions: two maximally general hypotheses, one maximally specific one, and the three between them. A line joins each pair related by $\geq_g$	30
P2.1	Reading path through Part II. Regression gives the first models; Chapters 7 and 8 (highlighted) give the tools to judge every model that follows.	36

5.1	The least-squares line through five user stories. Each red segment is a residual; the line is the one that makes the sum of their squares smallest.	38
5.2	Residual plots. Left: residuals scattered evenly around zero, as they should be. Right: a U-shape, meaning the true relationship is curved; adding an x^2 feature would remove it.	41
6.1	The same on-time (green) and late (red) projects modelled two ways. The straight line leaves $[0, 1]$ and is pulled by easy cases; the sigmoid stays between 0 and 1 and flattens out where the answer is already clear.	44
6.2	The decision boundary of the model $z = 4 - 5c - 3s$ is the line $5c + 3s = 4$. The dashed lines, where $z = \pm \ln 3$, are where the probability of finishing late is 0.75 and 0.25: probability changes gradually across the boundary.	46
6.3	Cross-entropy punishes confident mistakes without limit. Being right at 0.9 costs 0.11; being wrong at 0.99 costs 4.6.	46
7.1	Three polynomials fitted to the same ten points (blue); the dashed curve is the true function. Training error falls from left to right and reaches zero at degree 9, which is nonetheless the worst of the three on new data.	52
7.2	Training and test error for the polynomials of Figure 7.1, degree 0 to 9, with the test error measured on 2,000 fresh samples. Training error only ever falls; test error falls, bottoms out, and rises. The shape of that second curve is the whole of this chapter.	52
7.3	Bias and variance as target practice. The goal is the top-left target; in practice lowering one tends to raise the other.	53
7.4	Learning curves. With high bias the curves meet early at an unacceptable level; with high variance there is a wide gap that more data would narrow.	54
7.5	Why lasso sets weights to zero. The regularised solution is where the smallest error ellipse (blue) first touches the region allowed by the penalty (green). A circle is touched at a generic point; a diamond is usually touched at a corner, where one weight is exactly zero.	55
7.6	Five-fold cross-validation on the training data. The test set is not part of the rotation.	56
8.1	A defect predictor's confusion matrix on 5,000 code changes, and the five metrics computed from it. Precision reads along the "flagged" row; recall reads down the "truly defective" column.	60
8.2	Left: ROC curves sweep every threshold; the shaded area is model A's AUC. Right: the same two models on precision and recall. With rare positives the PR plot shows how many alerts are false, which the ROC plot hides.	61
8.3	How the uncertainty in a measured 12% error rate shrinks with test-set size. The width falls as $1/\sqrt{n}$: four times the data halves it.	62
P3.1	Reading path through Part III. The five learners can be taught in any order after Part II; Chapter 14 (highlighted) combines them, and is where trees in particular become state of the art.	67
9.1	The tree ID3 learns from the fourteen past sprints. Under each leaf are the training sprints that reach it; every leaf is pure. It is equivalent to the rule: the goal is met if (requirements Changing and load Normal) or requirements Frozen or (requirements Vague and Few dependencies).	68
9.2	Two impurity measures for a two-class node. Both are zero for a pure node and largest for an even split; they almost always pick the same attribute.	70

9.3	Accuracy against tree size on data with 15% label noise. Training accuracy climbs to 100% as the tree memorises the noise; test accuracy peaks early and then declines.	71
10.1	Prior times likelihood, normalised, gives the posterior. A flag that strongly favours “defect” is not enough to overturn a prior that strongly favours “clean”.	76
10.2	The naive Bayes assumption drawn as a network: the class causes each attribute, and the attributes do not influence each other once the class is known.	78
10.3	Left: the two class-conditional densities; at 750 lines the fail density is nine times higher. Right: after weighting by the 5% prior, the posterior crosses 0.5 only at 772 lines. The boundary sits beyond the midpoint of the means because the rare class needs more evidence.	79
11.1	Six past projects and one new project. Which class the new project receives depends on how many neighbours are asked. (Axes scaled so that 10 points = one unit.) . .	84
11.2	Cross-validated error of k -NN on the (standardised) project data of this chapter’s lab. Small k has high variance, large k high bias; the familiar U-shape.	85
11.3	The curse of dimensionality. For 500 uniformly random points, the ratio of the nearest to the farthest distance from a query rises towards 1 as dimensions are added: “nearest” stops meaning anything.	86
12.1	The maximum-margin hyperplane. The solid line separates the classes; the dashed lines bound the widest empty band; the points on the dashed lines are the support vectors.	90
12.2	Three losses as a function of $yf(\mathbf{x})$, positive when the example is correctly classified. The hinge is exactly zero for points beyond the margin, which is why only support vectors affect the SVM; the logistic loss is never quite zero.	91
12.3	Lifting data into more dimensions. The inner class (blue) and the outer class (red) cannot be separated on the line, but after adding x^2 as a second feature a horizontal line separates them; in the original space that line is two thresholds.	92
13.1	A perceptron: a weighted sum of the inputs, then a threshold. The bias is a weight on a constant input of 1. With the weights shown on the right it computes logical AND.	96
13.2	AND and OR are linearly separable; XOR is not. A second layer of units, each drawing its own line, is needed to represent it.	97
13.3	The network of the worked example, with the unit outputs of the forward pass written in each unit. Errors computed at the output are passed backwards along the same connections to assign blame to the hidden units.	98
13.4	Three activation functions. The sigmoid and tanh flatten out for large $ z $, where their gradients vanish; ReLU does not saturate for positive inputs, which is why it trains deep networks faster.	99
13.5	Training and validation loss of a two-hidden-layer network on data with 20% label noise. Training loss goes to zero; validation loss is lowest after five epochs and rises thereafter. Early stopping keeps the epoch-5 weights.	99
14.1	How majority voting amplifies weak but independent voters. Even 55%-accurate members reach 84% with a hundred voters — if, and only if, their mistakes are independent.	104

14.2	Bagging trains independent models on resampled data and averages them. Boosting trains models one after another, re-weighting the data towards the examples still being misclassified, and combines them with a weighted vote.	105
14.3	Test error against ensemble size on a 3,000-example problem with 5% label noise. Every ensemble roughly halves the error of a single tree. Averaging methods level off quickly; boosting keeps improving for longer.	106
P4.1	Reading path through Part IV. k -means (Chapter 15) is the hard-assignment special case of the EM algorithm in Chapter 17, which reappears in Chapter 18 to train hidden Markov models.	111
15.1	Three snapshots of k -means on the worked example. Assignment and update alternate until no point changes cluster.	113
15.2	Dendrograms for the points 1, 2, 4, 9, 11. The merges happen in almost the same order, but at different heights: complete linkage keeps groups compact and makes the gap between them obvious.	114
15.3	Choosing k for 600 points generated from four clusters. The WCSS curve bends sharply at $k = 4$ and the silhouette peaks there: both agree with the truth.	115
16.1	The worked example. PCA rotates the axes to line up with the directions of greatest variance; dropping PC2 projects every point onto the green line.	120
16.2	Cumulative explained variance for forty weekly project metrics. The curve rises steeply for four components and then flattens: four underlying factors drive almost everything the dashboard shows, and the rest is noise.	120
16.3	A self-organizing map. Competition picks one winner; cooperation spreads the update over its neighbourhood on the grid, which is what makes the map smooth.	121
17.1	The cluster assumption. With two labelled points (blue circle, red square) a classifier puts the boundary between them, through the middle of a cluster. The unlabelled points (grey) reveal two clusters and a gap, and the boundary belongs in the gap.	126
17.2	EM on the worked example. Each point is coloured by its responsibilities; $x = 3$ starts out shared between the components. Two iterations move the second Gaussian onto the group it explains.	128
17.3	The EM loop. Whenever a model would be easy to fit if some variable were observed, EM fills that variable in with probabilities and fits as if it were observed.	128
P5.1	Reading path through Part V. Each chapter adds one thing to the last: an HMM observes, an MDP chooses with a known model, and reinforcement learning chooses without one.	133
18.1	The hidden Markov model used throughout this chapter. The project office never sees the state, only a weekly status report whose probabilities depend on it.	135
18.2	The Viterbi trellis for the worked example. Every hidden-state sequence is a path through the trellis; Viterbi keeps only the best path into each node.	137
19.1	The agent–environment loop. Every problem in Part V’s last two chapters has this shape.	141
19.2	The technical-debt MDP of the worked examples. Shipping features earns value but lets debt build up; refactoring costs a sprint’s value and restores the codebase.	142

20.1 Ten-armed bandits (600 random problems, averaged). Greedy settles early on a mediocre arm; ϵ -greedy keeps improving; UCB explores hardest at first — its worst start — and earns the most thereafter. 149

P6.1 Reading path through Part VI. The project draws directly on the workflow, the model selection and the evaluation of Parts I and II, and must answer the questions of Chapter 21. 155

21.1 Five places bias enters a machine learning system. Removing a protected attribute addresses none of them. 157

21.2 The worked audit. Every criterion shows a gap, but they mean different things: the selection gap partly reflects different base rates; the TPR gap is unequal treatment of equally qualified applicants. 158

22.1 The project across the last six weeks of the semester. The test set is opened only once, after experiments are frozen in Week 14. 164

F.1 Chapter dependencies. The outlined chapters (3, 7 and 8) are the ones every later chapter relies on; the arrows mark the strongest direct dependencies. 188

PART I

Foundations

Before any algorithm, three questions: what does it mean for a program to learn, what mathematics does learning run on, and what does a single experiment look like from raw data to an honest number? Part I answers them, and ends with the oldest precise answer to the first: learning as a search through a space of hypotheses.

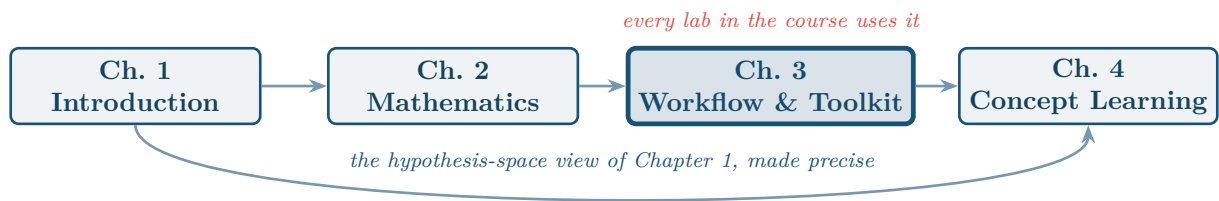


Figure P1.1: Reading path through Part I. Chapter 3 (highlighted) sets up the workflow that every later lab repeats; Chapter 4 formalises the idea of learning introduced in Chapter 1.

What Machine Learning Is

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** machine learning precisely, and name the task, performance measure and experience of a given learning problem.
- **Distinguish** learning a program from writing one, and machine learning from the wider field of artificial intelligence.
- **Classify** a problem as supervised (regression or classification), unsupervised, semi-supervised or reinforcement learning.
- **Describe** the four design decisions in building a learning system.
- **Explain** why a learner must assume something the data does not say in order to generalise at all.

🔗 Before You Start

The course *Artificial Intelligence*, in which you met agents, search and reasoning under uncertainty. Nothing in this handout is assumed yet.

📖 Key Terms in This Chapter

machine learning • task, performance measure, experience • training data • feature • label • model • supervised learning • classification • regression • unsupervised learning • semi-supervised learning • reinforcement learning • target function • hypothesis • hypothesis space • generalisation • inductive bias

1.1 Why Learn a Program Instead of Writing One?

Suppose a project office asks you for a program that warns, at the start of a sprint, which tasks will finish late. You could sit down and write rules: *if a task is 8 story points or more, and it waits on another team, and its requirements changed this week, flag it*. The first version catches some late tasks. Then the teams change, the tools change, a new client arrives — and the rules stop working. You add more. After a year you have two hundred rules that contradict one another, nobody understands, and that still miss the task that slipped this morning.

Now suppose instead that you collect five thousand completed tasks, each marked on time or late, and hand them to a program whose job is to *find* the rules. When the situation changes you do not rewrite anything: you collect new examples and let the program find new rules. That is the whole idea of machine learning, and Figure 1.1 shows it in one picture.

💡 When Learning Beats Writing

Machine learning earns its cost in four situations: when the rules are too numerous to write (routing thousands of tickets a day to the right team); when nobody can articulate them (judging from a ticket's wording how urgent it is); when they change faster than a person can rewrite them (estimating effort while the team and its tools change); and when they

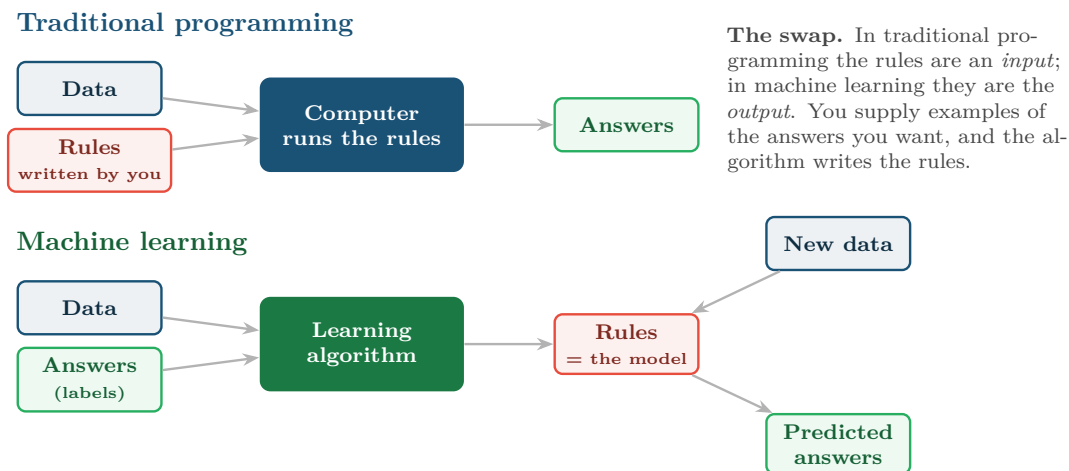


Figure 1.1: Traditional programming against machine learning. The learned model is then used exactly like a program: new data in, answers out.

must be different for every case (each team's own pace). If you can write the rule down in an afternoon and it will not change, write it — a rule is cheaper, faster and easier to explain than any model.

1.2 A Precise Definition

The definition used throughout this course is Tom Mitchell's, from the set text. It is worth memorising because it forces you to say exactly what you mean before you start.

Machine Learning (Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

The three letters are not decoration. A project that cannot name its P cannot tell whether it is making progress; a project that cannot name its E does not know what data to collect. Most failed machine learning projects failed here, before any algorithm was chosen.

Worked Example: Naming T , P and E

State the task, performance measure and experience for three problems.

1. Late-task warning.

- T : classify each task in the new sprint as likely late or on time.
- P : the fraction of late tasks flagged, with no more than one false alarm in every three flags.
- E : completed tasks from past sprints, each with its record and whether it finished late.

2. Sprint effort estimation.

- T : predict the hours a user story will take.
- P : average absolute error, in hours, on stories not yet estimated.
- E : past stories with their descriptions, story points and actual hours.

3. A scheduler learning to assign tickets.

- T : choose which engineer receives each incoming support ticket.
- P : average time until tickets are resolved.

- E : the scheduler’s own past assignments — no labelled examples of the “right” engineer, only how each assignment turned out.

Notice the choice of P in problem 1. “Accuracy” would have been the obvious answer and the wrong one: if 5% of tasks run late, a model that says “on time” for every task is 95% accurate and useless. Choosing P is a design decision with consequences, and Chapter 8 is devoted to it.

1.3 The Four Paradigms

What varies most between learning problems is the *experience* — in particular, what the data tells you about the right answer. That gives four paradigms, sketched in Figure 1.2.

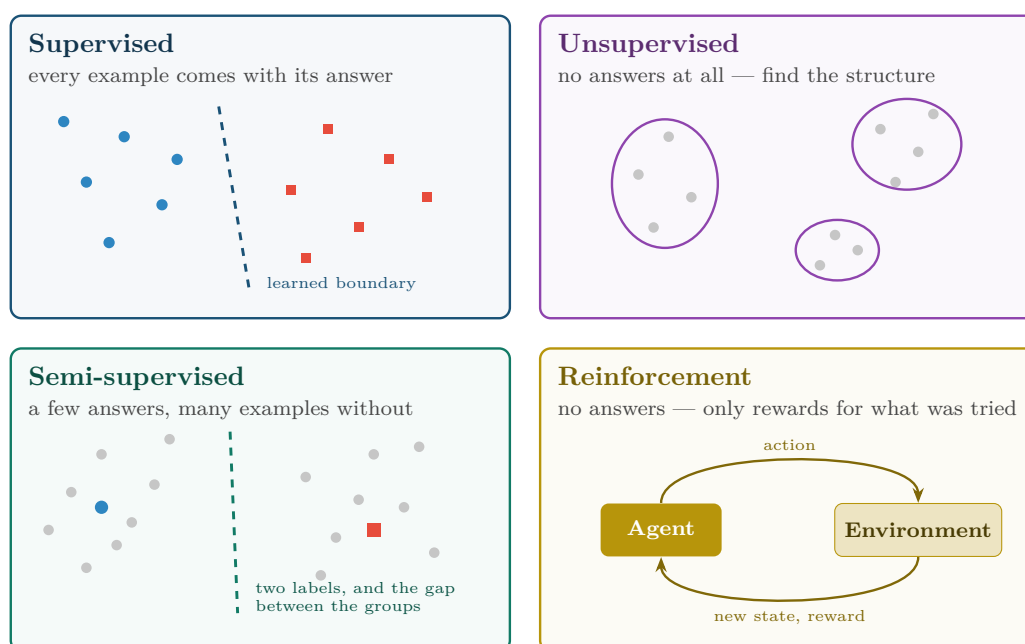


Figure 1.2: The four paradigms, distinguished by what the experience tells the learner about the right answer: all of it, none of it, a little of it, or only whether what it did turned out well.

The Four Paradigms

Supervised learning: every training example is a pair $(\mathbf{x}, y)$ of features and the correct answer (**label**). If y is a category the task is **classification**; if y is a number it is **regression**.

Unsupervised learning: examples $\mathbf{x}$ with no labels. The task is to find structure — groups, directions of variation, a compact representation.

Semi-supervised learning: a few labelled examples and many unlabelled ones, used together.

Reinforcement learning: an agent acts in an environment and receives a reward. Nobody says what the right action was; the agent must discover which actions lead, eventually, to high reward.

Paradigm	Experience	Learns	Example	Chapters
Supervised, regression	Features and a numeric target	A function to a number	Hours a user story will take	5, 7, 11, 13, 14
Supervised, classification	Features and a category	A function to a class	Will this task finish late?	6, 8–14
Unsupervised	Features only	Groups, directions, maps	Kinds of project in a portfolio	15, 16
Semi-supervised	Few labels, many unlabelled	A classifier using both	Triage of ten thousand tickets when two hundred are labelled	17
Reinforcement	States, actions, rewards	A policy: what to do in each state	When to pay down technical debt	19, 20

⚠ Classification or Regression? Look at the Target, Not the Features

The paradigm is decided by what y is. “Predict a project’s final cost” is regression; “predict whether the project will overrun its budget” is classification — same projects, same features, different y . Numbers used as category codes (team 7, office 3) are a common trap: ask whether averaging two values of y would mean anything.

1.4 Designing a Learning System

Mitchell’s set text frames the construction of any learner as four decisions, made in order. They are shown here for a problem this course returns to repeatedly: predicting, two weeks into a project, whether it will finish late.

📅 Worked Example: The Four Design Decisions

1. Choose the training experience. Records of past projects with their data at Week 2 and whether they finished late. Is it representative? Only if past projects resemble this one — a new kind of client or a new technology stack breaks that.

2. Choose the target function. What exactly should be learned?

- Option A: $V : \text{project} \rightarrow \{\text{on time, late}\}$.
- Option B: $V : \text{project} \rightarrow [0, 1]$, the probability of finishing late.

Option B is more useful: it lets the project office review the riskiest projects first.

3. Choose a representation for the approximation $\hat{V}$ of the target. For example, a weighted sum of features:

$$\hat{V}(\mathbf{x}) = \sigma(w_0 + w_1 \text{ requirement changes} + w_2 \text{ team experience} + w_3 \text{ tasks done by Week 2})$$

where σ squashes any number into $[0, 1]$ (Chapter 6). Choosing this form rules out, for good or ill, every pattern it cannot express.

4. Choose a learning algorithm that sets the weights $w_0, \dots, w_3$ from the training experience — here, gradient descent on a loss (Chapter 2).

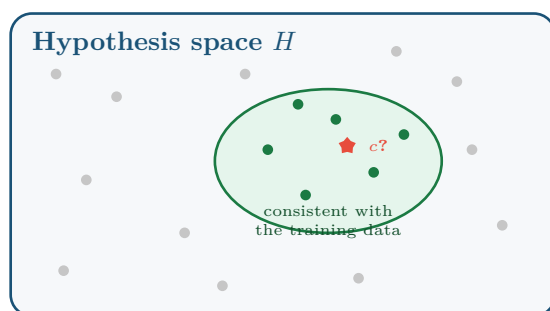
The first three decisions matter more than the fourth. A poor choice of experience or target cannot be rescued by a better algorithm.

1.5 Learning as Search Through a Hypothesis Space

Decision 3 deserves a closer look. Once you fix a representation, you have fixed the set of all functions the learner could possibly produce.

Hypothesis and Hypothesis Space

A **hypothesis** h is one candidate function from inputs to outputs — one particular setting of the weights, one particular decision tree. The **hypothesis space** H is the set of all hypotheses the representation allows. The unknown true function the data came from is the **target function** c (or f). **Learning** is a search through H for a hypothesis that agrees with c — not just on the training data, but on data not yet seen.



Grey dots: hypotheses the representation allows but the data rules out.

Green region: every hypothesis that fits the training data. Usually there are many.

Star: the true target, if it is in H at all.

Which green hypothesis to return? The data cannot say — they all fit it equally well. The learner's *preference* among them is its **inductive bias**.

Figure 1.3: Learning as search. The data narrows the hypothesis space to those hypotheses consistent with it; something other than the data must choose among the survivors.

Why a Learner Must Assume Something

A new team's first three stories took 2 hours (1 point), 4 hours (2 points) and 6 hours (3 points). How long will a 4-point story take? "8 hours" is the natural answer, but the rule "2 hours per point up to 3 points, and 0 above" fits the three stories just as perfectly. Nothing in the data prefers one to the other. You prefer 8 because you assume the simpler pattern continues. That assumption — the **inductive bias** — is not a defect to be removed: a learner with no bias at all cannot generalise beyond the examples it has seen. Chapter 4 proves this, and every later chapter names the bias of its algorithm.

1.6 A Short History

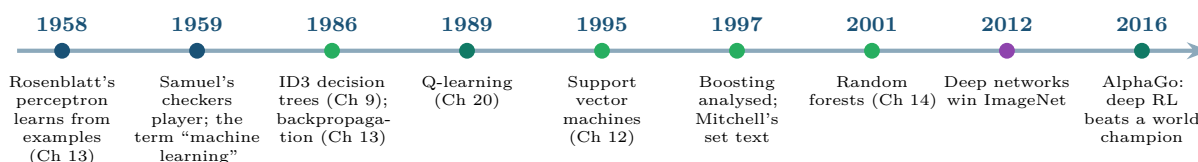


Figure 1.4: Selected milestones. Most of what this course teaches was in place by 2001; what changed after 2012 was mainly scale — data, computation, and the depth of the networks.

The ideas in this course are older than most students expect. Frank Rosenblatt's perceptron learned from examples in 1958; Arthur Samuel's checkers program, which improved by playing against itself, gave the field its name in 1959. Decision trees, backpropagation, reinforcement learning, support vector machines and ensembles were all in place by 2001. What changed after 2012 was mostly *scale* — data, computing power, and the depth of the neural networks that both

made practical. That is why a course built on a 1997 set text is not out of date: the deep learning course that follows this one is built on exactly these foundations.

1.7 Where Machine Learning Sits

In the *Artificial Intelligence* course you built agents that search, reason and plan using knowledge someone supplied. Machine learning is the part of AI in which the knowledge is *acquired from data* instead. Deep learning is, in turn, the part of machine learning that uses neural networks with many layers. A scheduling tool that plans a project with task durations typed in by the manager is AI but not ML; the same tool with durations learned from past projects is both.

! Common Pitfalls

- **Choosing the algorithm first.** “Let’s use a neural network” before T , P and E are written down. Decide what success means, and what data exists, before anything else.
- **Judging a model on the data it learned from.** Any sufficiently flexible model can memorise its training data. Performance means performance on examples it has not seen (Chapter 7).
- **Treating a category code as a number.** Team 7 is not “more” than team 3; averaging them means nothing.
- **Believing unsupervised means objective.** Every clustering reflects choices — of features, distance and number of groups — made by a person (Chapter 15).
- **Calling any automation “machine learning”.** A threshold written by a person is a rule, not a model, however useful it is.

🔧 Try It Yourself: A Rule Against a Learned Model

Write a rule by hand, then let an algorithm learn one, and compare them on tasks neither has seen. The lab makes its own data: 1,200 completed tasks from past sprints, each with its size in story points, the number of other tasks it waited on, its assignee’s years of experience and the number of requirement changes, and whether it finished late.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

rng = np.random.default_rng(1)
n = 1200
points = rng.choice([1, 2, 3, 5, 8, 13], n) # story points
depends = rng.integers(0, 4, n) # tasks it waited on
years = rng.integers(0, 11, n) # assignee's experience
changes = rng.integers(0, 5, n) # requirement changes
z = -3.3 + 0.18 * points + 0.9 * depends + 0.7 * changes - 0.3 * years
y = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int) # 1 = late
X = np.column_stack([points, depends, years, changes])
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

# --- 1. a rule written by a person: big tasks run late -----
rule = (X_te[:, 0] >= 8).astype(int) # 8 or 13 points -> late
print("hand-written rule :", round(accuracy_score(y_te, rule), 3))

# --- 2. a rule found by a learning algorithm -----
tree = DecisionTreeClassifier(max_depth=3, random_state=0).fit(X_tr, y_tr)
acc = accuracy_score(y_te, tree.predict(X_te))
print("learned tree      :", round(acc, 3))
```

```
# --- 3. the baseline every result must beat -----
majority = int(y_tr.mean() > 0.5)          # here: 0, "on time"
acc = accuracy_score(y_te, [majority] * len(y_te))
print("always 'majority' :", round(acc, 3))
```

The hand-written rule does *worse* than simply predicting “on time” for every task; the tree beats both. Change the threshold 0.5 in the rule, and the depth 3 of the tree, and watch the numbers. Which one is easier to improve by hand?

🔍 Checkpoint — Test Yourself

1. A project tool marks a task “at risk” whenever its logged hours pass the estimate by half. Is this machine learning? Use the definition.
2. Classify each as regression, classification, clustering or reinforcement learning: (a) predicting next sprint’s velocity in story points; (b) grouping past projects with no labels; (c) deciding whether a change request will be approved; (d) a scheduler learning to order the backlog from the delivery times that follow.
3. Why can a learner with no inductive bias not classify an example it has never seen?

📋 Chapter Summary

- Machine learning reverses traditional programming: you supply examples of the answers and the algorithm produces the rules.
- A program learns if its performance P at task T improves with experience E . Name all three before choosing an algorithm.
- The paradigm is set by the experience: labelled (supervised), unlabelled (unsupervised), partly labelled (semi-supervised) or rewarded (reinforcement). Regression and classification differ only in y .
- Designing a learner means choosing the experience, the target function, its representation and a learning algorithm — in that order of importance.
- Learning is search through a hypothesis space. The data narrows it; the inductive bias chooses among what is left, and without one no generalisation is possible.

🔧 Review Questions

1. **(MCQ)** Predicting the number of help-desk tickets tomorrow is: (a) classification (b) regression (c) clustering (d) reinforcement learning.
2. **(MCQ)** In Mitchell’s definition, E stands for: (a) error (b) experience (c) expectation (d) evaluation.
3. **(MCQ)** Ten thousand support tickets, of which 200 are labelled with their category, are data for: (a) supervised (b) unsupervised (c) semi-supervised (d) reinforcement learning.
4. **(Short)** Give one project-management problem for which you would write rules rather than learn a model, and explain why.
5. **(Short)** Explain, with an example, the difference between the target function and a hypothesis.
6. **(Short)** Why is “accuracy” often the wrong performance measure? Give a numeric example.
7. **(Applied)** A software company wants to suggest which developer should review each pull request. Specify T , P and E ; state which paradigm applies; and describe the four design decisions you would make.

8. (**Applied**) For the late-task warning in the first worked example, describe what could make the training experience unrepresentative of the sprints the model will face next year, and what you would do about it.

Mathematics for Machine Learning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Compute** dot products, norms and distances, and write a linear model as a dot product.
- **Explain** what the product $X\mathbf{w}$ computes for a whole dataset at once.
- **Differentiate** a simple loss function and **perform** gradient descent steps by hand, including choosing a learning rate.
- **Apply** Bayes' theorem to a base-rate problem.
- **Estimate** a parameter by maximum likelihood, and **compute** the entropy of a distribution.

🔗 Before You Start

School algebra, and the probability you used in *Artificial Intelligence*. Chapter 1 for the idea of a model with weights $w_0, \dots, w_d$. Appendix B collects every formula in this chapter on one page.

📖 Key Terms in This Chapter

vector • dot product • norm • Euclidean distance • cosine similarity • matrix • transpose • derivative • partial derivative • gradient • chain rule • gradient descent • learning rate • conditional probability • Bayes' theorem • expectation • variance • Gaussian • likelihood • maximum likelihood • entropy

2.1 Four Kinds of Mathematics, Four Jobs

Machine learning uses four pieces of mathematics, and each does one job. It is worth knowing which is which, because when a formula appears later you will know what it is *for* before you know what it says.

Mathematics	Its job	Where it appears
Linear algebra	<i>Represent</i> examples and models	Every model: an example is a vector, a dataset is a matrix, a linear model is a dot product (Chs 5, 6, 12, 13, 16)
Calculus	<i>Optimise</i> : find the weights that minimise a loss	Gradient descent in Chs 5, 6 and 13; backpropagation is the chain rule
Probability	<i>Reason under uncertainty</i>	Naive Bayes (Ch 10), EM (Ch 17), HMMs (Ch 18), every stochastic environment in Part V
Information theory	<i>Measure surprise</i>	Decision-tree splits (Ch 9) and the cross-entropy loss (Ch 6)

2.2 Vectors and the Dot Product

Vector, Dot Product and Norm

A **vector** $\mathbf{x} = (x_1, \dots, x_d)$ is an ordered list of d numbers — in this course, usually the features of one example.

The **dot product** of two vectors of the same length multiplies them element by element and adds up:

$$\mathbf{w} \cdot \mathbf{x} = \mathbf{w}^\top \mathbf{x} = \sum_{j=1}^d w_j x_j$$

The **norm** (length) of a vector is $\|\mathbf{x}\| = \sqrt{\mathbf{x} \cdot \mathbf{x}} = \sqrt{\sum_j x_j^2}$, and the **Euclidean distance** between two vectors is the norm of their difference, $\|\mathbf{x} - \mathbf{z}\|$.

The dot product is the most important operation in this course. A linear model — linear regression, logistic regression, a perceptron, an SVM — computes one dot product and adds a bias: $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$. Each weight says how much its feature pushes the prediction up or down.

Worked Example: One Task as a Vector

A task has requirement clarity 0.9 (the fraction of its acceptance criteria written down), test coverage 0.6 and 4 open dependencies: $\mathbf{x} = (0.9, 0.6, 4)$. A model has weights $\mathbf{w} = (2, 3, 0.25)$ and bias $b = -3$.

Score: $\mathbf{w} \cdot \mathbf{x} + b = (2)(0.9) + (3)(0.6) + (0.25)(4) - 3 = 1.8 + 1.8 + 1.0 - 3 = \mathbf{1.6}$.

Distance to a second task $\mathbf{z} = (0.5, 0.4, 1)$:

$$\|\mathbf{x} - \mathbf{z}\| = \sqrt{0.4^2 + 0.2^2 + 3^2} = \sqrt{0.16 + 0.04 + 9} = \sqrt{9.2} \approx \mathbf{3.03}$$

Look at where the distance came from. Of the 9.2 under the root, 9 is dependencies. Clarity and coverage — arguably the more important differences — contribute almost nothing, simply because they are measured on a scale of 0 to 1 and dependencies are not. Any method built on distance (Chapter 11, Chapter 15) needs the features put on a common scale first (Chapter 3).

💡 Cosine Similarity: Direction Without Length

Sometimes length is a nuisance. Two tickets reporting the same fault, one ten times longer, have word-count vectors pointing the same way but of very different lengths. **Cosine similarity** compares directions only: $\cos \theta = \frac{\mathbf{x} \cdot \mathbf{z}}{\|\mathbf{x}\| \|\mathbf{z}\|}$, which is 1 for vectors pointing the same way, 0 for perpendicular ones and -1 for opposite ones.

2.3 Matrices: A Whole Dataset at Once

Data Matrix and Matrix–Vector Product

Stack n examples as the rows of a **matrix** X with n rows and d columns. Then $X\mathbf{w}$ is a vector of n numbers: row i of the result is the dot product of example i with $\mathbf{w}$. The **transpose** $X^\top$ swaps rows and columns.

You will meet one more matrix operation, in Chapter 5: the **inverse** A^{-1} of a square matrix, which undoes multiplication by A the way dividing by 3 undoes multiplying by 3. You will never compute one by hand; NumPy does.

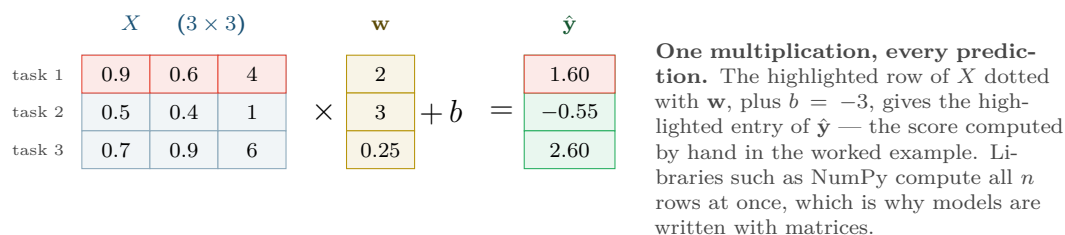


Figure 2.1: A matrix–vector product scores every example in one operation. Each row of X is one example; each entry of $\hat{\mathbf{y}}$ is that row’s dot product with the weights.

2.4 Derivatives and Gradients

Derivative, Partial Derivative, Gradient

The **derivative** $\frac{dL}{dw}$ of a function $L(w)$ is its slope: how much L changes per unit increase in w . For a function of several weights, the **partial derivative** $\frac{\partial L}{\partial w_j}$ is the slope in the direction of w_j alone, holding the others fixed. The **gradient** collects them into a vector:

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \dots, \frac{\partial L}{\partial w_d} \right)$$

The gradient points in the direction in which L increases fastest.

You need only four rules, and Appendix B lists them: the derivative of w^k is $k w^{k-1}$; of a constant is 0; of a sum is the sum of the derivatives; and the **chain rule**: if $L = f(g(w))$ then $\frac{dL}{dw} = f'(g(w)) \cdot g'(w)$. For example, $L(w) = (w - 3)^2$ has outer function u^2 and inner function $u = w - 3$, so $\frac{dL}{dw} = 2(w - 3) \cdot 1 = 2(w - 3)$. The chain rule is the whole of backpropagation (Chapter 13).

2.5 Gradient Descent

Training a model means finding weights that make a **loss** $L(\mathbf{w})$ small. For most models there is no formula for the best weights, so we walk downhill instead.

Gradient Descent

Start from any weights. Repeatedly take a small step against the gradient:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L(\mathbf{w})$$

The **learning rate** $\alpha > 0$ sets the step size. Stop when the loss stops decreasing.

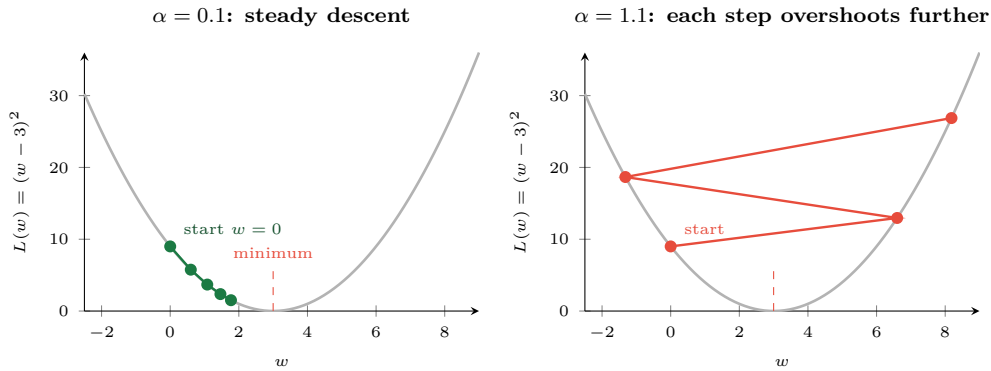


Figure 2.2: Gradient descent on $L(w) = (w-3)^2$ from $w = 0$. With a small learning rate each step closes a fixed fraction of the gap; with one that is too large, each step jumps past the minimum by more than the last, and the weights diverge.

Worked Example: Four Steps of Gradient Descent

Minimise $L(w) = (w-3)^2$ starting from $w = 0$ with $\alpha = 0.1$. The gradient is $L'(w) = 2(w-3)$.

Step	w	$L'(w) = 2(w-3)$	$w - 0.1 L'(w)$
0	0	-6	$0 + 0.6 = 0.6$
1	0.6	-4.8	$0.6 + 0.48 = 1.08$
2	1.08	-3.84	$1.08 + 0.384 = 1.464$
3	1.464	-3.072	$1.464 + 0.3072 = 1.7712$

The pattern. The gap to the minimum goes $3 \rightarrow 2.4 \rightarrow 1.92 \rightarrow 1.536 \rightarrow 1.2288$: every step multiplies it by $1 - 2\alpha = 0.8$. So

- $\alpha = 0.1$: the gap shrinks by 20% a step — steady.
- $\alpha = 0.5$: the factor is 0, so one step lands exactly on 3.
- $\alpha = 1.1$: the factor is -1.2 — the weight flips to the far side and lands further away each time: $0 \rightarrow 6.6 \rightarrow -1.32 \rightarrow 8.18$. It diverges (Figure 2.2, right).

Real losses are not this simple, and the right α is found by trying several (Chapter 7), but the three behaviours — slow, just right, divergent — are exactly the ones you will see.

2.6 Probability

Conditional Probability and Bayes' Theorem

The **conditional probability** of A given B is $P(A | B) = \frac{P(A \text{ and } B)}{P(B)}$. Reversing the condition gives **Bayes' theorem**:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}, \quad P(B) = P(B | A) P(A) + P(B | \text{not } A) P(\text{not } A)$$

$P(A)$ is the **prior**, $P(B | A)$ the **likelihood** and $P(A | B)$ the **posterior**. Two events are **independent** if $P(A \text{ and } B) = P(A) P(B)$.

Worked Example: A Blocked Release You Should Not Panic About

An automated release check catches 99% of builds that would break production, and wrongly blocks 1% of good builds. One build in a thousand is broken. The check blocks a build. What is the probability that it really is broken?

Let A = broken build, B = blocked. $P(A) = 0.001$, $P(B | A) = 0.99$, $P(B | \text{not } A) = 0.01$.

$$P(B) = (0.99)(0.001) + (0.01)(0.999) = 0.00099 + 0.00999 = 0.01098$$

$$P(A | B) = \frac{0.00099}{0.01098} \approx \mathbf{0.090}$$

Only 9% of blocked builds are broken, from a check that is “99% accurate”. There are so many more good builds that their 1% of false blocks outnumbers the real failures ten to one. This is the **base-rate** effect, and it is why Chapter 8 insists on precision rather than accuracy. Naive Bayes (Chapter 10) is this calculation, done for every class.

2.7 Expectation, Variance and the Gaussian

The **expectation** (mean) of a random variable is its probability-weighted average, $\mathbb{E}[X] = \sum_x x P(x)$; from a sample it is estimated by the sample mean $\bar{x}$. The **variance** is the expected squared distance from the mean, $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$, and its square root is the standard deviation σ . For the values 2, 4, 4, 4, 5, 5, 7, 9 the mean is 5, the squared deviations sum to 32, the variance is $32/8 = 4$ and $\sigma = 2$.

The **Gaussian** (normal) distribution with mean μ and variance σ^2 has density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

It appears in Gaussian naive Bayes (Chapter 10) and Gaussian mixtures (Chapter 17), and whenever a model assumes that its errors are small, symmetric and unsurprising.

2.8 Maximum Likelihood

Most learning algorithms can be read as answering one question: *which parameters make the observed data most probable?*

Likelihood and Maximum Likelihood Estimation

For data D and parameters θ , the **likelihood** $\mathcal{L}(\theta) = P(D | \theta)$ is the probability of the data under those parameters, read as a function of θ . The **maximum likelihood estimate** $\hat{\theta}_{ML}$ is the θ that maximises it. In practice one maximises the **log-likelihood** $\log \mathcal{L}(\theta)$ instead: the logarithm turns products into sums and has the same maximum.

Worked Example: How Often Does the Team Deliver?

A team delivered everything it committed to in 7 of its last 10 sprints. Let p be the probability that a sprint delivers, and treat sprints as independent. The likelihood of that record is $\mathcal{L}(p) = p^7(1 - p)^3$, and the log-likelihood is $\ell(p) = 7 \ln p + 3 \ln(1 - p)$. Set its derivative to zero:

$$\frac{d\ell}{dp} = \frac{7}{p} - \frac{3}{1 - p} = 0 \Rightarrow 7(1 - p) = 3p \Rightarrow \hat{p} = \mathbf{0.7}$$

The answer is what intuition says — the observed fraction — but the method is general: the same argument gives the least-squares line of Chapter 5 and the cross-entropy loss of

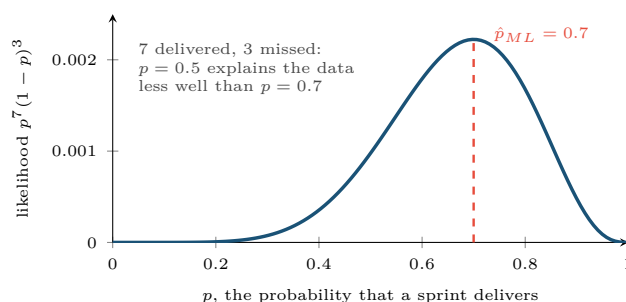


Figure 2.3: The likelihood of seven delivered sprints in ten, as a function of p . It is largest at $p = 0.7$ — the observed proportion, which is what maximum likelihood gives for any such count.

Chapter 6, and the EM algorithm of Chapter 17 is a way of doing it when some data is missing.

⚠ Maximum Likelihood Trusts Small Samples Too Much

One sprint, one success: $\hat{p}_{ML} = 1$, a team that can never miss. A word that never appeared in the training tickets gets probability zero, and a single zero makes a whole product zero. Chapter 10 fixes this with **Laplace smoothing**: add one imaginary observation of every outcome.

2.9 Information and Entropy

📖 Entropy

The **entropy** of a distribution $(p_1, \dots, p_k)$ measures its uncertainty, in bits:

$$H = -\sum_{i=1}^k p_i \log_2 p_i \quad (\text{with } 0 \log_2 0 = 0)$$

It is 0 when one outcome is certain, and largest — $\log_2 k$ bits — when all k outcomes are equally likely.

🧮 Worked Example: Entropy of a Set of Examples

Fourteen past sprints, of which 9 met their sprint goal and 5 did not:

$$H = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.410 + 0.531 = \mathbf{0.940} \text{ bits}$$

A 7–7 split would give exactly 1 bit; a 14–0 split, 0 bits. A decision tree (Chapter 9) chooses the question that lowers this number the most — that is, the question whose answer leaves the least uncertainty behind.

⚠ Common Pitfalls

- **Forgetting the minus sign in gradient descent.** $\mathbf{w} + \alpha \nabla L$ climbs the loss. The update subtracts.
- **Reading a diverging loss as “needs more steps”.** If the loss grows, the learning rate is too large; more steps make it worse.
- **Confusing $P(A | B)$ with $P(B | A)$.** “The check catches 99% of broken builds” is $P(\text{blocked} | \text{broken})$; the release manager wants $P(\text{broken} | \text{blocked})$, which was 9%.

- **Computing distances on unscaled features.** The feature with the largest units decides the answer.
- **Multiplying many small probabilities.** The product underflows to zero on a computer; add log-probabilities instead.

Try It Yourself: The Chapter in NumPy

```
import numpy as np

# --- vectors and the dot product -----
x = np.array([0.9, 0.6, 4.0])
w = np.array([2.0, 3.0, 0.25])
print("score w.x + b :", round(w @ x - 3, 2))          # 1.6
z = np.array([0.5, 0.4, 1.0])
print("distance      :", round(np.linalg.norm(x - z), 2)) # 3.03

# --- a whole dataset at once -----
X = np.array([[0.9, 0.6, 4], [0.5, 0.4, 1], [0.7, 0.9, 6]])
print("X w + b      :", X @ w - 3)                    # [1.6 -0.55 2.6]

# --- gradient descent on L(w) = (w - 3)^2 -----
for alpha in (0.1, 0.5, 1.1):
    wt = 0.0
    path = [wt]
    for step in range(4):
        grad = 2 * (wt - 3)
        wt = wt - alpha * grad
        path.append(round(wt, 3))
    print(f"alpha={alpha}: {path}")

# --- Bayes by simulation: how many blocked builds are broken? -----
rng = np.random.default_rng(0)
n = 1_000_000
broken = rng.random(n) < 0.001
blocked = np.where(broken, rng.random(n) < 0.99, rng.random(n) < 0.01)
print("P(broken | blocked), simulated:",
      round(broken[blocked].mean(), 3))                # about 0.09

# --- entropy -----
def entropy(counts):
    p = np.array(counts) / np.sum(counts)
    p = p[p > 0]
    return -(p * np.log2(p)).sum()
print("H(9 yes, 5 no) :", round(entropy([9, 5]), 3))    # 0.94
print("H(7 yes, 7 no) :", round(entropy([7, 7]), 3))    # 1.0
```

Change 0.001 in the simulation to 0.05 — a codebase where broken builds are common — and predict the new answer before you run it.

Checkpoint — Test Yourself

1. Compute the Euclidean distance between (1, 2) and (4, 6).
2. Take one gradient descent step on $L(w) = (w + 1)^2$ from $w = 2$ with $\alpha = 0.25$. Is the new w closer to the minimum?
3. Compute the entropy of the distribution (0.5, 0.25, 0.25).

Chapter Summary

- An example is a vector, a dataset is a matrix, and a linear model is a dot product plus a bias; $X\mathbf{w}$ scores every example at once.
- Distances are dominated by the features with the largest units, so scale features before comparing examples.
- The gradient points uphill; gradient descent steps against it. Too small a learning rate is slow, too large diverges.
- Bayes' theorem reverses a conditional probability. When the prior is small, even an accurate test produces mostly false alarms.
- Maximum likelihood chooses the parameters that make the data most probable; work with the log-likelihood.
- Entropy measures uncertainty in bits: zero when the outcome is certain, largest when every outcome is equally likely.

Review Questions

1. **(MCQ)** The dot product of $(1, -1, 2)$ and $(3, 1, 1)$ is: (a) 2 (b) 4 (c) 6 (d) 0.
2. **(MCQ)** If the loss increases after every gradient descent step, the most likely cause is: (a) too few steps (b) a learning rate that is too large (c) too much data (d) a learning rate that is too small.
3. **(MCQ)** A new ticket is equally likely to belong to any of six categories. The entropy of its category is: (a) 1 bit (b) $\log_2 6 \approx 2.58$ bits (c) 6 bits (d) 0 bits.
4. **(Short)** Explain in words what the gradient of a loss function tells you, and why gradient descent subtracts it.
5. **(Short)** Why do we maximise the log-likelihood rather than the likelihood?
6. **(Short)** Give an example, not from this chapter, where $P(A | B)$ and $P(B | A)$ are very different.
7. **(Applied)** A code-review bot flags 2% of clean pull requests and 95% of those that contain a critical defect; 0.2% of pull requests contain one. Compute the probability that a flagged pull request contains a critical defect, and state what this means for the reviewers.
8. **(Applied)** Perform three gradient descent steps on $L(w) = 2(w - 5)^2$ from $w = 1$ with $\alpha = 0.1$. Find the factor by which the gap to the minimum shrinks each step, and the largest learning rate for which descent still converges.

The Machine Learning Workflow and Toolkit

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** the stages of a machine learning experiment, from framing the problem to reporting the result.
- **Split** data into training, validation and test sets, and **explain** what each one is for.
- **Prepare** features: impute missing values, encode categories and scale numbers.
- **Use** the scikit-learn estimator interface and **combine** preparation and model in a pipeline.
- **Recognise** and **prevent** data leakage, and **establish** a baseline before building any model.

🔗 Before You Start

Chapter 1 (examples, features, labels, the paradigms) and Chapter 2 (vectors, distance, and why feature scale matters). Basic Python; Appendix A is a reference for every library call used in the labs.

📖 Key Terms in This Chapter

workflow • feature types • training set • validation set • test set • stratified split • imputation • one-hot encoding • standardisation • min-max scaling • estimator • transformer • `fit` / `predict` / `transform` • pipeline • data leakage • baseline

3.1 One Experiment, Start to Finish

Every chapter from here on teaches a new algorithm, and every lab runs it inside the same sequence of steps. Learn the sequence once, here, and the algorithms become interchangeable parts. Figure 3.1 shows it.

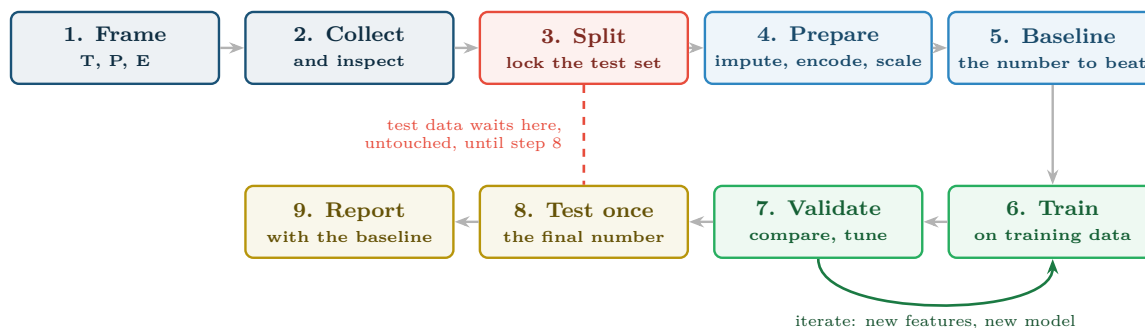


Figure 3.1: The workflow every lab follows. The loop between training and validation may run dozens of times; the test set, split off in step 3, is used exactly once, in step 8.

💡 Why the Order Matters

Steps 3 and 8 are the only two that touch the test set, and they are placed as far apart as possible on purpose. Everything between them — every choice of feature, algorithm and setting — is made while looking only at training and validation data. If any of those choices is made while looking at the test set, the final number measures how well you fitted the test set, not how well the model will do on the next task, ticket or project.

3.2 Examples, Features and Labels

A dataset for supervised learning is a table: one row per **example**, one column per **feature**, and one column for the **label**. Before choosing any algorithm, decide what type each feature is, because the type decides how it must be prepared.

Feature type	Examples	Preparation
Numeric, continuous	Budget spent, hours logged, test coverage	Impute gaps; scale for distance- and gradient-based models
Numeric, count	Open bugs, change requests	As above; a log transform often helps with long tails
Categorical, nominal	Team, project type, technology stack	One-hot encode; never treat the codes as numbers
Categorical, ordinal	Priority (low to critical), severity level	Map to ordered integers, or one-hot if the gaps are not equal
Text	Ticket description, commit message	Turn into counts or weights of words (Chapter 10)
Identifier	Task ID, ticket number, employee number	Usually drop. An ID that predicts the label is a leak, not a pattern

3.3 Splitting the Data

📖 Training, Validation and Test Sets

The **training set** is what the algorithm learns from. The **validation set** is used to compare models and choose settings. The **test set** is held back and used once, at the end, to estimate performance on new data. A typical split is 60/20/20; with little data, the training and validation roles are combined by cross-validation (Chapter 7).

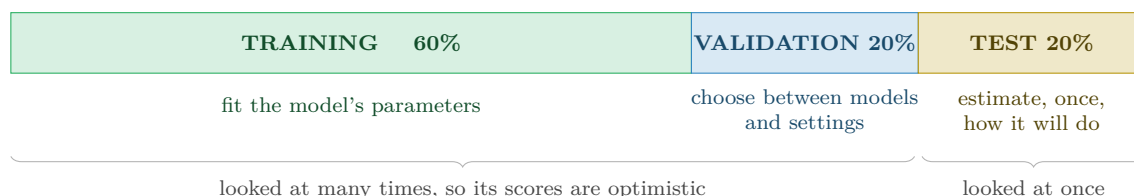


Figure 3.2: The three-way split. Only the test score is an honest estimate of performance on new data, and it stays honest only if it is looked at once.

⚠️ Split Before You Do Anything Else

Split first, and stratify: `train_test_split(..., stratify=y)` keeps the proportion of each class the same in every part, which matters when one class is rare. Two further rules apply

to data with structure. If there are several rows per project or per developer, keep all of an entity's rows on the same side of the split. If the data is ordered in time, train on the past and test on the future — a random split lets the model peek at tomorrow.

3.4 Preparing Features

Missing values. Most algorithms cannot accept a blank. Either drop the rows (wasteful, and biased if the gaps are not random) or **impute**: fill each gap with the median of that feature in the training data. When a value is missing *for a reason* — a timesheet never filled in during a crunch, a project that never kept a risk register — add a yes/no column recording that it was missing; the absence is itself informative.

Categories. **One-hot encoding** replaces a categorical feature with one 0/1 column per category. `pctype ∈ {Web, Mobile, Data}` becomes three columns, exactly one of which is 1 in each row. Coding the project types as 1, 2 and 3 instead would tell the model that a Data project is “three times” a Web project, which is nonsense that a linear model will happily use.

Scaling. Chapter 2 showed a distance dominated by the feature with the largest units. **Standardisation** rescales each feature to mean 0 and standard deviation 1:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Min-max scaling maps each feature onto $[0, 1]$ instead: $x' = (x - \min)/(\max - \min)$. Tree-based models (Chapter 9, Chapter 14) do not need scaling; nearest neighbours, SVMs, neural networks, k -means, PCA and anything trained by gradient descent do.

Worked Example: Standardising with Training Statistics

Four past releases had test coverage 0.6, 0.8, 0.9, 0.7 and 2, 4, 6, 8 open bugs.

Coverage: $\mu = 0.75$; $\sigma = \sqrt{\frac{0.15^2 + 0.05^2 + 0.15^2 + 0.05^2}{4}} = \sqrt{0.0125} = 0.112$.

Open bugs: $\mu = 5$; $\sigma = \sqrt{\frac{9+1+1+9}{4}} = \sqrt{5} = 2.236$.

A new release arrives with coverage 0.5 and 10 open bugs:

$$z_{\text{cov}} = \frac{0.5 - 0.75}{0.112} = -2.24, \quad z_{\text{bugs}} = \frac{10 - 5}{2.236} = +2.24$$

Both are now equally unusual — 2.24 standard deviations from the training mean, in opposite directions — where before the bug difference (5) was twenty times the coverage difference (0.25). Note that μ and σ came from the *training* releases only. The new release is transformed with the training statistics, never with statistics that include the new release.

3.5 The scikit-learn Interface

Every algorithm in this course is available in `scikit-learn`, and every one of them is used in the same way. That uniformity is the reason the labs can swap a decision tree for an SVM by changing one line.

Estimator and Transformer

An **estimator** learns from data with `fit(X, y)` and then makes predictions with `predict(X)` (and, for classifiers, `predict_proba(X)` for probabilities). A **transformer** learns with `fit(X)` and changes data with `transform(X)`; a scaler learns μ and σ in `fit` and applies them in `transform`. Anything learned during `fit` is stored in attributes ending in an underscore, such as `coef_` or `mean_`.

Class	Chapter	What it is
LinearRegression, Ridge, Lasso	5, 7	Least-squares regression, with and without a penalty
LogisticRegression	6	Linear classifier with probabilities
DecisionTreeClassifier	9	ID3/CART-style tree
GaussianNB, MultinomialNB	10	Naive Bayes
KNeighborsClassifier	11	k -nearest neighbours
SVC	12	Support vector machine
MLPClassifier	13	Multilayer neural network
RandomForestClassifier, AdaBoostClassifier	14	Ensembles
KMeans, AgglomerativeClustering	15	Clustering
PCA	16	Principal component analysis
GaussianMixture, SelfTrainingClassifier	17	EM mixtures; semi-supervised wrapper

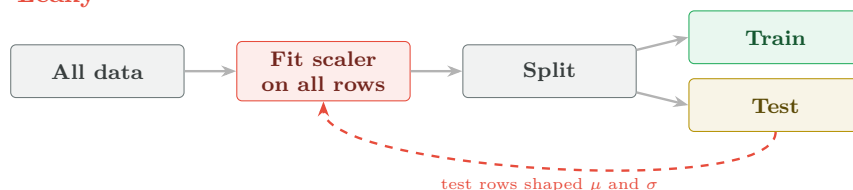
3.6 Pipelines and Data Leakage

Data Leakage

Data leakage is any route by which information that will not be available at prediction time reaches the model during training or evaluation. Leakage makes evaluation scores optimistic, sometimes spectacularly so, and the model then fails in use.

Leakage comes in two forms. **Target leakage** is a feature that is recorded after, or because of, the outcome: `days_overdue` in a model of which tasks will finish late. **Train–test contamination** is subtler: preparing the data using statistics that include the test rows. Fit a scaler on the full dataset and then split, and the training data has been shifted by the test data’s mean — the model has seen a summary of the exam.

Leaky



A Pipeline makes the correct order automatic: calling `fit` on the pipeline fits every step on the training data only, and `predict` applies the stored statistics to new data.

Correct

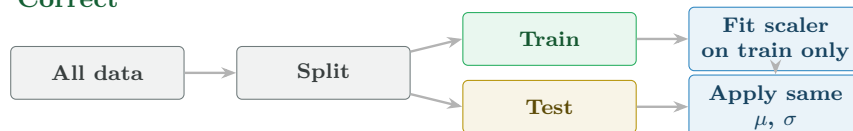


Figure 3.3: Train–test contamination. Fitting any preparation step before the split lets the test rows influence training; a pipeline fitted on the training data alone cannot make this mistake.

Pipeline

A scikit-learn **pipeline** chains transformers and a final estimator into one object with a single `fit` and `predict`. Because every step is fitted only when the pipeline is fitted, and only on the data passed to it, a pipeline used inside a split or a cross-validation loop cannot leak. A `ColumnTransformer` applies different preparation to different columns — scaling to the numbers, one-hot encoding to the categories.

3.7 Baselines

Baseline

A **baseline** is the score of the simplest possible predictor: always predict the most common class (classification) or always predict the training mean (regression). Every model must be compared with it, and a model that does not beat it clearly has learned nothing useful.

Baselines are humbling. On the late-task data of Chapter 1’s lab, “always on time” already scores 66% — more than the hand-written rule. On commit data where 99% of commits are clean, “always clean” scores 99% — which is why a defect predictor’s accuracy means nothing until it is set beside the baseline (Chapter 8).

Common Pitfalls

- **Preparing before splitting.** Imputing, scaling or selecting features on the full dataset leaks the test set into training. Put every step in a pipeline.
- **Using the test set to choose.** Every look at the test set that changes a decision turns it into a validation set. Use it once.
- **Label-encoding a nominal feature.** Project-type codes 1, 2, 3 impose an order that does not exist. One-hot encode.
- **Keeping an identifier.** If `task_id` improves accuracy, IDs were assigned in a way that encodes the outcome — a leak.
- **Reporting a score without the baseline.** “92% accurate” is meaningless until you know that “always no” scores 90%.

Try It Yourself: A Leak-Proof Pipeline

Six hundred synthetic past projects with a missing-value problem and one categorical feature; the label is whether the project finished late. The whole workflow, in the order of Figure 3.1.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier

# --- 600 past projects, with gaps -----
rng = np.random.default_rng(0)
n = 600
df = pd.DataFrame({
    "clarity": rng.uniform(0.3, 1.0, n),          # requirement clarity
    "coverage": rng.uniform(0.0, 1.0, n),         # test coverage
    "changes": rng.poisson(5, n).astype(float),   # change requests
    "ptype": rng.choice(["Web", "Mobile", "Data"], n),
```

```

})
risk = 2.6 - 4 * df.clarity - 2 * df.coverage + 0.1 * df.changes
df["late"] = (risk + rng.normal(0, 0.5, n) > 0).astype(int)
df.loc[rng.random(n) < 0.1, "coverage"] = np.nan          # 10% missing
print("late rate:", df.late.mean().round(3))

X, y = df.drop(columns="late", df["late"])

# --- 1. split FIRST; the test set is locked away -----
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)

# --- 2. every preparation step lives inside the pipeline -----
num = ["clarity", "coverage", "changes"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
                          StandardScaler()), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["ptype"]),
])
model = Pipeline([("prep", prep), ("clf", LogisticRegression())])

# --- 3. a baseline first, then the model -----
base = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
print("baseline accuracy:", round(base.score(X_te, y_te), 3))
model.fit(X_tr, y_tr)
print("model accuracy   :", round(model.score(X_te, y_te), 3))

# --- 4. look inside: the scaler learned from training rows only -----
scaler = model.named_steps["prep"].named_transformers_["num"][-1]
print("scaler means:", scaler.mean_.round(2))

```

The baseline scores about 0.68 and the model about 0.84. Now swap the model: import `DecisionTreeClassifier` from `sklearn.tree` and put it where `LogisticRegression()` was. Nothing else changes — that is the point of the interface.

🔍 Checkpoint — Test Yourself

1. Why must the scaler's mean and standard deviation be computed from the training set only?
2. A dataset has a column `method` with values `Scrum`, `Kanban` and `Waterfall`. How should it be encoded, and why not as 1, 2, 3?
3. A model predicting which projects will be cancelled scores 97% accuracy. What single number do you ask for before you are impressed?

📋 Chapter Summary

- Every experiment follows the same workflow: frame, collect, split, prepare, baseline, train, validate, test once, report.
- The training set fits the model, the validation set chooses between models, and the test set — used once — estimates performance on new data.
- Impute gaps (and record that they were gaps), one-hot encode nominal categories, and scale numbers for any method based on distance or gradients.
- Every scikit-learn estimator has `fit` and `predict`; every transformer has `fit` and `transform`. Pipelines chain them.
- Leakage — through a feature recorded after the outcome or through preparation fitted on test rows — produces scores the model cannot reproduce in use. Pipelines prevent the second kind.

- Always report the baseline beside the model.

Review Questions

1. **(MCQ)** Which set is used to choose between a decision tree and an SVM? (a) training (b) validation (c) test (d) all three.
2. **(MCQ)** Which model does *not* need its features scaled? (a) k -nearest neighbours (b) SVM (c) decision tree (d) k -means.
3. **(MCQ)** A column recording the date a project was formally closed as failed, in a model predicting project failure, is an example of: (a) target leakage (b) stratification (c) imputation (d) a baseline.
4. **(Short)** Explain the difference between `fit`, `transform` and `predict`.
5. **(Short)** Why should a time-ordered dataset not be split at random?
6. **(Short)** When is it worth adding a “value was missing” column alongside an imputed feature?
7. **(Applied)** Training values of a feature are 10, 20, 30, 40. Compute the standardised value of a new observation of 45, showing μ and σ . Then compute its min–max scaled value, and say what is unusual about the result.
8. **(Applied)** A colleague scales the whole dataset, selects the ten features most correlated with the label, then runs 5-fold cross-validation and reports 94%. Identify every leak in this procedure and rewrite it so that it cannot leak.

Concept Learning and the Version Space

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Foundations

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Represent** a concept as a conjunction of attribute constraints, and **order** hypotheses from specific to general.
- **Trace** the FIND-S algorithm on a set of training examples, and **state** its limitations.
- **Define** the version space, and **trace** the CANDIDATE-ELIMINATION algorithm through the S and G boundaries by hand.
- **Classify** a new instance with a partially learned concept, and **choose** the most informative query.
- **Explain** why an unbiased learner cannot generalise, and **state** the inductive bias of CANDIDATE-ELIMINATION.

🔗 Before You Start

Chapter 1 (hypotheses, hypothesis spaces and inductive bias, introduced informally in Figure 1.3). This chapter makes those ideas exact.

📖 Key Terms in This Chapter

concept learning • instance space • target concept • attribute constraint • consistent hypothesis • more-general-than-or-equal-to • FIND-S • version space • LIST-THEN-ELIMINATE • general boundary G • specific boundary S • CANDIDATE-ELIMINATION • query • unbiased learner • inductive bias

4.1 Concept Learning as a Task

Much of learning is learning a *category*: which releases are ready to ship, which tasks will slip, which tickets are urgent. Each such category is a **concept**, and it can be described as a yes/no function over the things being classified.

📖 Concept Learning

Concept learning is inferring a boolean-valued function from training examples of its input and output. Formally:

- the **instance space** X is the set of all things that could be classified;
- the **target concept** is an unknown function $c : X \rightarrow \{0, 1\}$;
- the training examples D are pairs $\langle x, c(x) \rangle$;
- the learner searches a hypothesis space H for an h with $h(x) = c(x)$ for every $x \in X$.

The running example of this chapter is a *change advisory board*, which decides whether each software release may go to production. Each release is described by six attributes, and four of the board's decisions have been observed. (The table is Mitchell's *EnjoySport* example with its attributes renamed, so every number in this chapter can be checked against the set text.)

Release	Tests	Review	Docs	Rollback	Traffic	Scope	Approve
1	Pass	Done	Complete	Ready	Quiet	Same	Yes
2	Pass	Done	Partial	Ready	Quiet	Same	Yes
3	Fail	Pending	Partial	Ready	Quiet	Changed	No
4	Pass	Done	Partial	Ready	Peak	Changed	Yes

Tests is Pass, Flaky or Fail; Review is Done or Pending; Docs is Complete or Partial; Rollback (a tested plan to undo the release) is Ready or None; Traffic, the load expected at release time, is Quiet or Peak; and Scope records whether the scope has Changed since the last review.

Hypotheses as Conjunctions of Constraints

Each hypothesis is a vector of six **constraints**, one per attribute. Each constraint is one of:

- a specific value, such as *Done* — only that value is acceptable;
- “?” — any value is acceptable;
- “ $\emptyset$ ” — no value is acceptable.

A hypothesis classifies an instance as positive if the instance satisfies every constraint. So $\langle \text{Pass}, ?, ?, \text{Ready}, ?, ? \rangle$ means “tests pass and a rollback is ready, whatever else”; $\langle ?, ?, ?, ?, ?, ? \rangle$ approves every release; and a hypothesis containing $\emptyset$ approves none.

Worked Example: How Big Are These Spaces?

Tests has 3 values and the other five attributes have 2 each.

- **Instances:** $3 \times 2^5 = 96$ possible releases.
- **Hypotheses, as written:** each constraint can also be “?” or “ $\emptyset$ ”, so $5 \times 4^5 = 5120$.
- **Hypotheses, by meaning:** every hypothesis containing a $\emptyset$ classifies every release as negative, so they all mean the same thing. Counting that as one: $1 + 4 \times 3^5 = 973$.
- **All possible concepts** over 96 releases: every subset of the releases is a concept, so $2^{96} \approx 7.9 \times 10^{28}$.

The conjunctive language can express 973 of the 7.9×10^{28} possible concepts. That restriction looks like a weakness. By the end of the chapter it will turn out to be the only reason learning is possible at all.

4.2 The Inductive Learning Hypothesis

A learner sees only the training examples. The best it can guarantee is a hypothesis that fits them. Everything rests on an assumption, which Mitchell names explicitly:

The Inductive Learning Hypothesis

Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other unobserved examples.

This is an assumption, not a theorem. It fails whenever the future does not resemble the training data — which is why Chapter 7 and Chapter 8 measure performance on held-out data rather than trusting it.

4.3 The General-to-Specific Ordering

Hypotheses are not an unordered heap. Some are strictly more permissive than others, and that structure is what makes efficient search possible.

More-General-Than-or-Equal-To

Hypothesis h_j is **more general than or equal to** h_k , written $h_j \geq_g h_k$, if every instance that h_k classifies as positive, h_j also classifies as positive. h_j is strictly more general ($h_j >_g h_k$) if, in addition, $h_k \not\geq_g h_j$. The relation is a **partial order**: some pairs of hypotheses are not comparable at all.

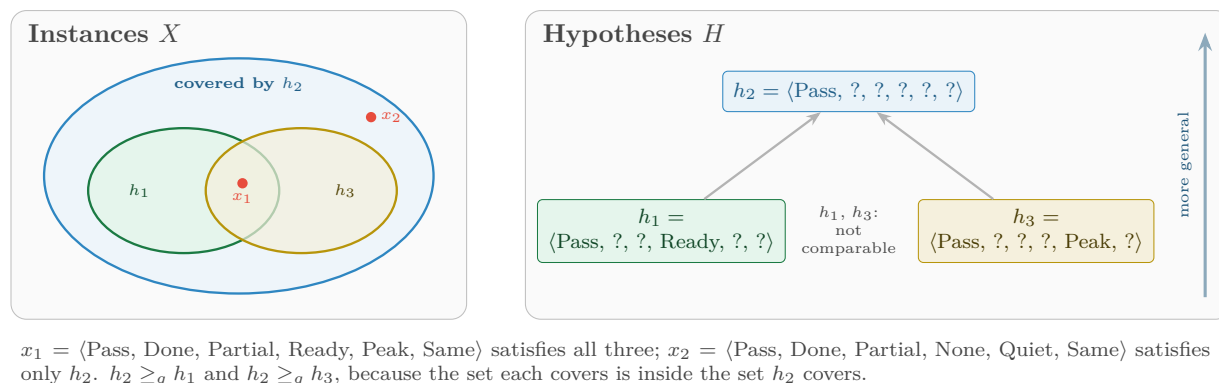


Figure 4.1: The more-general-than ordering. A more general hypothesis covers a superset of the instances; hypotheses whose covered sets merely overlap, like h_1 and h_3 , are not comparable.

4.4 Find-S: The Most Specific Consistent Hypothesis

The simplest algorithm that uses the ordering starts at the most specific hypothesis and generalises only as far as each positive example forces it to.

The Find-S Algorithm

1. Initialise h to the most specific hypothesis $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$.
2. For each *positive* training example x : for each constraint in h that x does not satisfy, replace it by the next more general constraint that x does satisfy ($\emptyset \rightarrow$ the value of x ; a different value $\rightarrow$ “?”).
3. Ignore negative examples. Output h .

Worked Example: Find-S on the Release Decisions

Start: $h_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$.

Release 1 (+): nothing is satisfied, so copy the example: $h_1 = \langle \text{Pass, Done, Complete, Ready, Quiet, Same} \rangle$.

Release 2 (+): Docs is Partial, not Complete, so generalise that constraint: $h_2 = \langle \text{Pass, Done, ?, Ready, Quiet, Same} \rangle$.

Release 3 (−): ignored. (It is correctly classified as negative anyway — its tests failed.)

Release 4 (+): Traffic and Scope differ, so generalise both: $h_4 = \langle \text{Pass, Done, ?, Ready, ?, ?} \rangle$.

Output: “tests pass, review done and a rollback ready”. Why is it safe to ignore negatives? If the target concept is in H , it is at least as general as h (which only ever generalised as far

as positives forced). A negative example is excluded by the target, so it is excluded by the more specific h as well.

⚠ What Find-S Cannot Tell You

FIND-S returns *a* consistent hypothesis, but not whether it is the only one; it cannot say whether it has converged; it cannot detect noisy or inconsistent data, because it never looks at negatives; and it always prefers the most specific answer, for no reason other than that it started there. The version space fixes the first three.

4.5 Version Spaces

📖 Consistent Hypothesis and Version Space

A hypothesis h is **consistent** with training examples D if $h(x) = c(x)$ for every example $\langle x, c(x) \rangle$ in D . The **version space** $VS_{H,D}$ is the set of all hypotheses in H consistent with D — every hypothesis the data has not yet ruled out.

The obvious way to find it is LIST-THEN-ELIMINATE: write down every hypothesis in H , and for each training example cross out the ones it contradicts. What remains is the version space. For this example's 973 hypotheses this is feasible; for any realistic hypothesis space it is not. The trick of the next section is that the version space never needs to be listed — two boundaries are enough.

4.6 The Candidate-Elimination Algorithm

📖 General and Specific Boundaries

The **general boundary** G is the set of maximally general hypotheses in the version space; the **specific boundary** S is the set of maximally specific ones. Every hypothesis in the version space lies between them: it is more general than or equal to some member of S and more specific than or equal to some member of G .

📖 The Candidate-Elimination Algorithm

Initialise G to $\{\langle ?, ?, ?, ?, ?, ? \rangle\}$ and S to $\{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$. For each training example d :

- **If d is positive:** remove from G every hypothesis that does not cover d . Replace each $s \in S$ that does not cover d by its minimal generalisations that do, keeping only those that are more specific than some member of G .
- **If d is negative:** remove from S every hypothesis that covers d . Replace each $g \in G$ that covers d by its minimal specialisations that do not, keeping only those that are more general than some member of S , and then drop any that is less general than another member of G .

📖 Worked Example: Candidate-Elimination on the Release Decisions

Start. $S_0 = \{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$, $G_0 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$.

Release 1 (+) $\langle \text{Pass, Done, Complete, Ready, Quiet, Same} \rangle$. G_0 covers it: keep. S_0 does not: generalise minimally.

$S_1 = \{\langle \text{Pass, Done, Complete, Ready, Quiet, Same} \rangle\}$, $G_1 = G_0$.

Release 2 (+) $\langle \text{Pass, Done, Partial, Ready, Quiet, Same} \rangle$. Docs differs, so it becomes “?”.

$S_2 = \{\langle \text{Pass, Done, ?, Ready, Quiet, Same} \rangle\}$, $G_2 = G_0$.

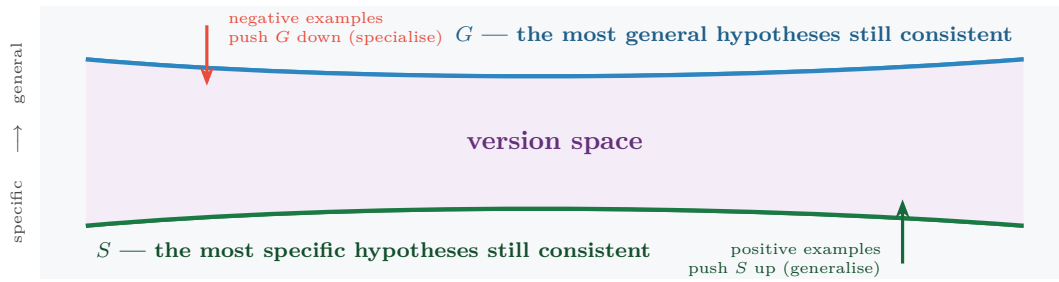


Figure 4.2: The version space is everything between the two boundaries. Each example moves one boundary towards the other; if they meet, the concept has been learned exactly.

Release 3 (–) ⟨Fail, Pending, Partial, Ready, Quiet, Changed⟩. S_2 already excludes it. G_2 covers it, so specialise ⟨?, ?, ?, ?, ?, ?⟩ one attribute at a time to a value the negative example does not have:

- Tests: Pass or Flaky. Only *Pass* is still more general than S_2 .
- Review: *Done*. Kept.
- Docs: Complete — but S_2 has “?” there, so this is not more general than S_2 . Dropped.
- Rollback: None — contradicts S_2 ’s Ready. Dropped.
- Traffic: Peak — contradicts S_2 ’s Quiet. Dropped.
- Scope: *Same*. Kept.

$S_3 = S_2$, $G_3 = \{\langle \text{Pass}, ?, ?, ?, ?, ? \rangle, \langle ?, \text{Done}, ?, ?, ?, ? \rangle, \langle ?, ?, ?, ?, ?, \text{Same} \rangle\}$.

Release 4 (+) ⟨Pass, Done, Partial, Ready, Peak, Changed⟩. ⟨?, ?, ?, ?, ?, Same⟩ does not cover it: removed from G . S generalises Traffic and Scope.

$S_4 = \{\langle \text{Pass}, \text{Done}, ?, \text{Ready}, ?, ? \rangle\}$, $G_4 = \{\langle \text{Pass}, ?, ?, ?, ?, ? \rangle, \langle ?, \text{Done}, ?, ?, ?, ? \rangle\}$.

Check. S_4 is exactly what FIND-S returned. The difference is that CANDIDATE-ELIMINATION also knows the other end: the version space holds six hypotheses, drawn in Figure 4.3.

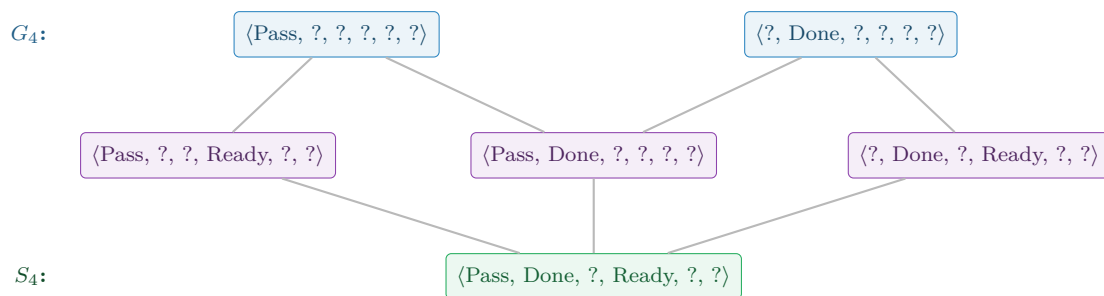


Figure 4.3: The final version space for the release decisions: two maximally general hypotheses, one maximally specific one, and the three between them. A line joins each pair related by $\geq_g$.

4.7 Using a Partially Learned Concept

Four examples have not pinned the concept down: six hypotheses survive. They can still classify new releases, by voting.

Worked Example: Classifying by Vote

- $\langle \text{Pass, Done, Complete, Ready, Peak, Changed} \rangle$: satisfies S_4 , and so every hypothesis above it. **6 yes, 0 no** — approved with the same confidence as if the concept were fully learned.
- $\langle \text{Fail, Pending, Complete, None, Quiet, Same} \rangle$: satisfies neither member of G_4 , so no hypothesis. **0 yes, 6 no**.
- $\langle \text{Pass, Done, Complete, None, Quiet, Same} \rangle$: **3 yes, 3 no**. The data genuinely cannot decide.
- $\langle \text{Pass, Pending, Complete, Ready, Quiet, Same} \rangle$: **2 yes, 4 no** — a majority, but not a certainty.

Two tests are enough for the unanimous cases: an instance satisfying every member of S is positive for the whole version space; one satisfying no member of G is negative for all of it. And if the learner may *ask* the board for a decision, the best question is the third release: whatever the answer, it eliminates half the version space. Repeating that halves it each time, so the concept is found in about $\log_2 |VS|$ queries — here $\log_2 6 \approx 2.6$, so three.

4.8 Inductive Bias

Why stop at conjunctions? Why not use a hypothesis space that can express *every* one of the 2^{96} possible concepts, so that the target is certainly in it?

The Futility of an Unbiased Learner

Give CANDIDATE-ELIMINATION a hypothesis space containing every subset of X . After any training set, S is the disjunction of the positive examples seen — “release 1 or release 2 or release 4” — and G is “anything except release 3”. The version space contains every concept between them. For any release *not* in the training data, exactly half of those concepts call it positive and half negative: the vote is always a tie. The unbiased learner can classify the examples it has seen and nothing else. It has memorised; it has not learned.

Inductive Bias

The **inductive bias** of a learner is the set of assumptions that, added to the training data, make its classification of new instances follow *deductively*. The inductive bias of CANDIDATE-ELIMINATION is: *the target concept is contained in the hypothesis space H* — here, it is a conjunction of attribute constraints.

Learners can be ranked by the strength of their bias. A **rote learner**, which stores examples and classifies only exact matches, has no bias and never generalises. CANDIDATE-ELIMINATION assumes the target is in H , and classifies a new instance only when the whole version space agrees. FIND-S assumes that as well, and in addition that every instance is negative unless the evidence forces it to be positive — a stronger bias, so it classifies everything. Every algorithm in the rest of this course has a bias, and its chapter names it.

The Weakness Every Later Chapter Fixes

CANDIDATE-ELIMINATION assumes the data is *noise-free*. Record one approved release as rejected by mistake and the true concept is eliminated; with enough errors S and G cross, and the version space is empty. Real data always contains errors. The algorithms of Part III therefore replace “consistent with every example” by “makes few errors” and accept a hypothesis that is probably approximately right. Concept learning is where the vocabulary comes from; it is not how practical systems are built.

! Common Pitfalls

- **Specialising G to values that contradict S .** Every new member of G must still be more general than a member of S ; check each candidate against S before keeping it.
- **Forgetting to prune G on a positive example.** Members of G that do not cover a positive example are removed, as at Release 4.
- **Believing Find-S's answer is unique.** It is one of the six hypotheses in the version space — the most specific.
- **Reading “unbiased” as “better”.** A learner with no bias cannot generalise at all.
- **Running Candidate-Elimination on noisy data.** One mislabelled example can empty the version space.

🔧 Try It Yourself: Candidate Elimination in Forty Lines

The whole of this chapter, run: FIND-S, the boundary trace of the worked example, and a brute-force check of the version space by listing every hypothesis.

```
from itertools import product

# Release decisions. "?" = any value, "0" = no value (empty constraint)
VALUES = [("Pass", "Flaky", "Fail"), ("Done", "Pending"),
          ("Complete", "Partial"), ("Ready", "None"), ("Quiet", "Peak"),
          ("Same", "Changed")]
D = [((("Pass", "Done", "Complete", "Ready", "Quiet", "Same"), True),
      (("Pass", "Done", "Partial", "Ready", "Quiet", "Same"), True),
      (("Fail", "Pending", "Partial", "Ready", "Quiet", "Changed"), False),
      (("Pass", "Done", "Partial", "Ready", "Peak", "Changed"), True))]
show = lambda hs: " ".join("<" + " ".join(h) + ">" for h in sorted(hs))

def covers(h, x):
    # does h classify x as positive?
    return all(c == "?" or c == v for c, v in zip(h, x))

def more_general(h1, h2):
    # h1 >=g h2
    return "0" in h2 or all(a == "?" or a == b for a, b in zip(h1, h2))

def generalize(s, x):
    # minimal generalisation of s covering x
    return tuple(v if c == "0" else (c if c == v else "?")
                for c, v in zip(s, x))

def specializations(g, x):
    # minimal specialisations of g excluding x
    return [g[:i] + (v,) + g[i + 1:] for i, c in enumerate(g) if c == "?"
            for v in VALUES[i] if v != x[i]]

# --- Find-S -----
h = ("0",) * 6
for x, positive in D:
    if positive:
        h = generalize(h, x)
print("Find-S:", show([h]))

# --- Candidate-Elimination -----
S, G = {("0",) * 6}, {("?",) * 6}
for k, (x, positive) in enumerate(D, 1):
    if positive:
        G = {g for g in G if covers(g, x)}
```

```

    S = {generalize(s, x) for s in S}
    S = {s for s in S if any(more_general(g, s) for g in G)}
else:
    S = {s for s in S if not covers(s, x)}
    new = {g for g in G if not covers(g, x)}
    for g in G - new:
        new |= {h for h in specializations(g, x)
                if any(more_general(h, s) for s in S)}
    G = {g for g in new
         if not any(o != g and more_general(o, g) for o in new)}
print(f"release {k}  S: {show(S)}\n          G: {show(G)}")

# --- brute force: list every hypothesis, keep the consistent ones -----
H = list(product(*[v + ("?",) for v in VALUES]))
VS = [h for h in H if all(covers(h, x) == p for x, p in D)]
print(len(VS), "hypotheses in the version space:")
print(show(VS))
new_release = ("Pass", "Done", "Complete", "None", "Quiet", "Same")
yes = sum(covers(h, new_release) for h in VS)
print(f"vote on {new_release}: {yes} yes, {len(VS) - yes} no")

```

Now add a fifth example, a copy of Release 1 recorded as *No* — a data-entry mistake — and run the trace again. What happens to S , G and the version space?

🔍 Checkpoint — Test Yourself

1. Is $\langle \text{Pass}, ?, ?, \text{Ready}, ?, ? \rangle \geq_g \langle \text{Pass}, \text{Done}, ?, \text{Ready}, ?, ? \rangle$? Is the reverse true?
2. Run FIND-S on the release decisions with Release 4 removed. What does it return?
3. Why does an unbiased learner's vote on an unseen instance always end in a tie?

📋 Chapter Summary

- Concept learning infers a yes/no function from examples by searching a hypothesis space — here, conjunctions of attribute constraints.
- Hypotheses are partially ordered from specific to general; the ordering lets a learner search without listing every hypothesis.
- FIND-S returns the most specific hypothesis consistent with the positive examples, but cannot say whether it is the only one.
- The version space is every hypothesis consistent with the data; CANDIDATE-ELIMINATION represents it by its boundaries S and G , which positive and negative examples move towards each other.
- A partially learned concept classifies by vote; the most informative query splits the version space in half.
- Without an inductive bias there is no generalisation. The bias of CANDIDATE-ELIMINATION is that the target is in H — and it assumes noise-free data, which later chapters drop.

🔧 Review Questions

1. (MCQ) FIND-S ignores: (a) positive examples (b) negative examples (c) the first example (d) nothing.
2. (MCQ) A negative example can change: (a) S only (b) G only, though it may also remove members of S (c) neither (d) the hypothesis space.
3. (MCQ) With 3 attributes of 3 values each, the number of semantically distinct conjunctive hypotheses is: (a) 27 (b) 64 (c) 65 (d) 2^{27} .

4. **(Short)** Define the version space, and explain why CANDIDATE-ELIMINATION does not need to list it.
5. **(Short)** State the inductive learning hypothesis. Why is it an assumption rather than a theorem?
6. **(Short)** Explain how a single mislabelled training example affects CANDIDATE-ELIMINATION.
7. **(Applied)** Trace CANDIDATE-ELIMINATION on the release decisions presented in the order Release 3, Release 1, Release 2, Release 4. Is the final version space the same? Is any intermediate boundary different?
8. **(Applied)** For the final version space of this chapter, find a release that splits the vote 2 to 4 other than the one in the worked example, and one that splits it 3 to 3. Explain why the second is the better query.

PART II

Linear Models and Generalization

Two linear models, then the two ideas the rest of the course stands on. A model is judged by how it does on data it has never seen, not on the data it learned from; and “how it does” must be measured with a number that matches what you actually care about. Chapters 7 and 8 are the hinge of the course.

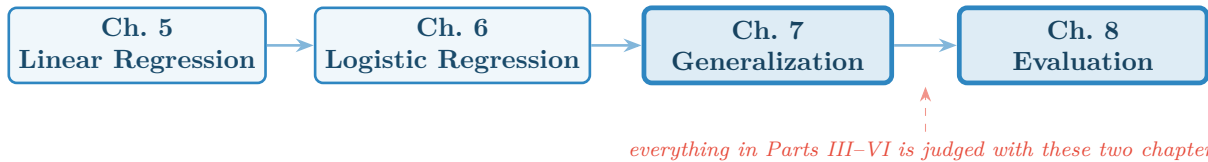


Figure P2.1: Reading path through Part II. Regression gives the first models; Chapters 7 and 8 (highlighted) give the tools to judge every model that follows.

Linear Regression

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Linear Models and Generalization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Write** the linear regression model and its mean squared error loss, and **interpret** each weight.
- **Fit** a simple linear regression by hand with the least-squares formulas, and **state** the normal equation for many features.
- **Derive** the gradient descent update for linear regression and **perform** a step by hand.
- **Extend** the model with multiple and polynomial features.
- **Evaluate** a regression model with MAE, RMSE and R^2 against a baseline, and **read** a residual plot.

🔗 Before You Start

Chapter 2 (dot products, derivatives, gradient descent, maximum likelihood) and Chapter 3 (splits, scaling, baselines).

📖 Key Terms in This Chapter

regression • linear model • weight • bias (intercept) • residual • mean squared error • least squares • normal equation • batch, stochastic and mini-batch gradient descent • multiple regression • polynomial features • MAE • RMSE • R^2 • residual plot

5.1 The Model

Linear regression predicts a number as a weighted sum of the features plus a constant:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b = \mathbf{w} \cdot \mathbf{x} + b$$

With one feature this is the straight line $\hat{y} = wx + b$ of school algebra. The **weight** w_j is the change in the prediction for a one-unit increase in feature j with the other features held fixed; the **bias** (or intercept) b is the prediction when every feature is zero.

Linear regression is the first model in the course because every later model either extends it (logistic regression, the perceptron, the SVM, a neural network's output layer) or is compared against it. It is also, very often, the right answer: fast, stable, and explainable in one sentence per feature.

5.2 The Loss: Mean Squared Error

Residual and Mean Squared Error

The **residual** of example i is $e_i = y_i - \hat{y}_i$, the vertical gap between the observed value and the prediction. The **mean squared error** of the model on n examples is

$$\text{MSE}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Fitting a linear regression means choosing $\mathbf{w}$ and b to make the MSE as small as possible — the method of **least squares**.

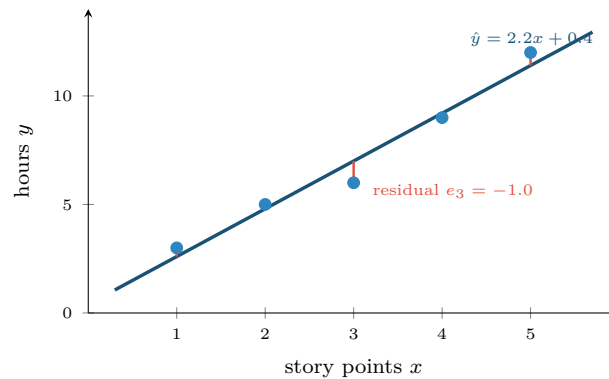


Figure 5.1: The least-squares line through five user stories. Each red segment is a residual; the line is the one that makes the sum of their squares smallest.

Why Square the Residuals?

Three reasons. Squaring makes every error positive, so over- and under-predictions cannot cancel. It makes the loss smooth, so it has a derivative everywhere and a single minimum that a formula can find. And it is what maximum likelihood gives (Chapter 2): if the noise around the true line is Gaussian, the most likely line is exactly the least-squares line. The price is that one large error counts a great deal — an error of 10 costs a hundred times an error of 1 — so least squares is pulled hard by outliers.

5.3 Least Squares by Formula

For one feature, setting the derivatives of the MSE with respect to w and b to zero gives a closed-form answer:

$$w = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad b = \bar{y} - w\bar{x}$$

The line always passes through the point of means $(\bar{x}, \bar{y})$.

Worked Example: Fitting Hours to Story Points

Five completed user stories: story points $x = 1, 2, 3, 4, 5$ took $y = 3, 5, 6, 9, 12$ hours.

Means: $\bar{x} = 3$, $\bar{y} = 35/5 = 7$.

Deviations: $x - \bar{x} = -2, -1, 0, 1, 2$ and $y - \bar{y} = -4, -2, -1, 2, 5$.

Slope:

$$w = \frac{(-2)(-4) + (-1)(-2) + (0)(-1) + (1)(2) + (2)(5)}{4 + 1 + 0 + 1 + 4} = \frac{8 + 2 + 0 + 2 + 10}{10} = 2.2$$

Intercept: $b = 7 - 2.2 \times 3 = 0.4$.

Reading it: each extra story point adds 2.2 hours; the intercept, 0.4 hours, is the fixed overhead of any story. A new 6-point story is predicted to take $2.2 \times 6 + 0.4 = 13.6$ hours.
Residuals: predictions 2.6, 4.8, 7.0, 9.2, 11.4 give residuals 0.4, 0.2, -1.0 , -0.2 , 0.6, which sum to zero — they always do for a least-squares line with an intercept. The sum of squares is $0.16 + 0.04 + 1 + 0.04 + 0.36 = 1.6$, so $\text{MSE} = 0.32$.

The Normal Equation

With d features, put a column of ones in front of X to absorb the bias, so the model is $\hat{\mathbf{y}} = X\boldsymbol{\theta}$. The least-squares parameters solve the **normal equation**

$$X^\top X \boldsymbol{\theta} = X^\top \mathbf{y} \implies \boldsymbol{\theta} = (X^\top X)^{-1} X^\top \mathbf{y}$$

For the five stories,

$$X^\top X = \begin{pmatrix} 5 & 15 \\ 15 & 55 \end{pmatrix}, \quad X^\top \mathbf{y} = \begin{pmatrix} 35 \\ 127 \end{pmatrix},$$

whose solution is $b = 0.4$, $w = 2.2$ as before.

The normal equation is exact and needs no learning rate, but inverting a $d \times d$ matrix costs roughly d^3 operations, and it fails outright when two features are perfect copies of each other ($X^\top X$ has no inverse). For many features, or data that does not fit in memory, gradient descent is used instead.

5.4 Fitting by Gradient Descent

Differentiating the MSE with the chain rule gives the gradient:

$$\frac{\partial \text{MSE}}{\partial w_j} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}, \quad \frac{\partial \text{MSE}}{\partial b} = \frac{2}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

Read it in words: each weight moves in proportion to the average of *(error) × (its feature)*. A feature that is large exactly when the model under-predicts has its weight pushed up.

Worked Example: One Gradient Descent Step

Start from $w = 0$, $b = 0$ on the five stories, with $\alpha = 0.05$. Every prediction is 0, so the errors $\hat{y} - y$ are $-3, -5, -6, -9, -12$ and the MSE is $(9 + 25 + 36 + 81 + 144)/5 = 59$.

$$\frac{\partial}{\partial w} = \frac{2}{5} [(-3)(1) + (-5)(2) + (-6)(3) + (-9)(4) + (-12)(5)] = \frac{2}{5} (-127) = -50.8$$

$$\frac{\partial}{\partial b} = \frac{2}{5} (-3 - 5 - 6 - 9 - 12) = \frac{2}{5} (-35) = -14$$

Update: $w \leftarrow 0 - 0.05(-50.8) = 2.54$, $b \leftarrow 0 - 0.05(-14) = 0.70$.

The new predictions 3.24, 5.78, 8.32, 10.86, 13.40 give an MSE of 2.29 — down from 59 in one step. Repeating the update converges to $w = 2.2$, $b = 0.4$, the same answer as the formula.

Variant	Each step uses	Trade-off
Batch	All n examples	Smooth, exact steps; slow when n is large
Stochastic (SGD)	One example, chosen at random	Very cheap steps; noisy path that jitters around the minimum
Mini-batch	A batch of, say, 32–256 examples	The practical default; what neural networks use (Chapter 13)

⚠ Scale the Features Before Gradient Descent

If one feature is measured in thousands and another in fractions, the MSE surface is a long narrow valley: a learning rate small enough for the steep direction crawls along the shallow one. Standardising the features (Chapter 3) makes the valley round and gradient descent converge in far fewer steps. The normal equation does not care.

5.5 Multiple and Polynomial Features

With several features each weight is a *partial* effect: the change in $\hat{y}$ for one more unit of x_j *with the other features held fixed*. That is why the weight of a feature can change sign when another, correlated feature is added.

A linear model can also fit curves. Add powers of a feature as new features — x , x^2 , x^3 — and fit $\hat{y} = w_1x + w_2x^2 + w_3x^3 + b$. The model is still *linear in the weights*, so everything in this chapter applies unchanged; only the features are new. The danger is obvious as soon as it is said: with enough powers, the curve can pass through every training point. Chapter 7 begins exactly there.

5.6 Measuring a Regression Model

📊 MAE, RMSE and R^2

Mean absolute error: $\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$ — the typical size of an error, in the units of y .

Root mean squared error: $\text{RMSE} = \sqrt{\text{MSE}}$ — also in the units of y , but weighting large errors more.

Coefficient of determination: $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ — the fraction of the variance in y the model explains. 1 is perfect; 0 is no better than always predicting the mean; on test data it can be negative.

🧮 Worked Example: Scoring the Story-Point Model

Residuals 0.4, 0.2, −1.0, −0.2, 0.6.

- $\text{MAE} = (0.4 + 0.2 + 1.0 + 0.2 + 0.6)/5 = \mathbf{0.48}$ hours.
- $\text{RMSE} = \sqrt{1.6/5} = \sqrt{0.32} = \mathbf{0.57}$ hours.
- $\sum (y - \bar{y})^2 = 16 + 4 + 1 + 4 + 25 = 50$, so $R^2 = 1 - 1.6/50 = \mathbf{0.968}$.

Caution. These are *training* scores on five points fitted by two parameters. They say the line describes these five stories well. Whether it predicts the next story well can only be answered on stories it has not seen.

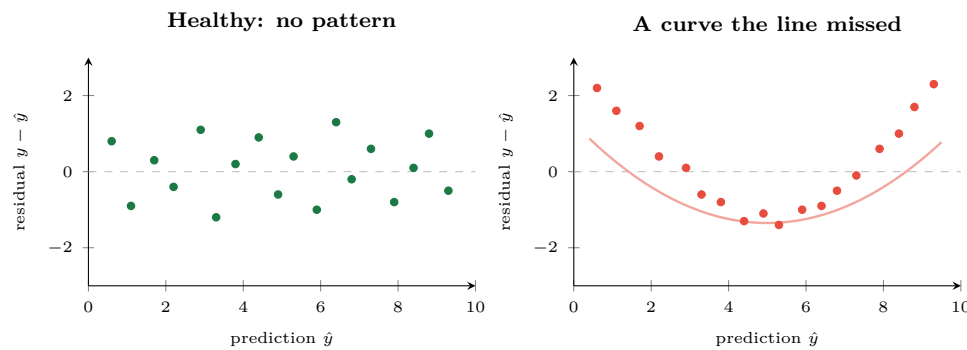


Figure 5.2: Residual plots. Left: residuals scattered evenly around zero, as they should be. Right: a U-shape, meaning the true relationship is curved; adding an x^2 feature would remove it.

⚠ Always Compare With the Mean

The baseline for regression is “predict the training mean for everyone”, which has $R^2 = 0$. On the 400 completed stories of this chapter’s lab it has an RMSE of 11.8 hours; linear regression brings that to 4.3. Whether that improvement is worth having is a question about the application, not the algorithm — but without the baseline nobody could even ask it.

! Common Pitfalls

- **Reading a weight as a cause.** $w = 2.2$ hours per point describes the data; it does not say that adding points to a story would add hours. Chapter 21 returns to this.
- **Comparing raw weights across features.** A weight’s size depends on its feature’s units. Standardise first, then compare.
- **Extrapolating.** The line was fitted on 1–5 points; a 40-point epic is outside anything it has seen.
- **Ignoring the residual plot.** A good R^2 with a curved residual pattern means a better model is available.
- **Letting one outlier drive the fit.** Squared loss gives a single wild point enormous influence. Look at the data first.

🔧 Try It Yourself: Least Squares Three Ways

The worked example by formula, then by gradient descent, then a larger dataset: 400 completed user stories, with the hours each took predicted from five features — story points, files touched, requirement changes, the developer’s years of experience, and whether the story touches the database.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# --- 1. the worked example, by formula -----
x = np.array([1, 2, 3, 4, 5.0])      # story points
y = np.array([3, 5, 6, 9, 12.0])    # hours
w = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean()) ** 2).sum()
b = y.mean() - w * x.mean()
print(f"least squares:    w = {w:.2f}, b = {b:.2f}")

# --- 2. the same line by gradient descent -----
```

```

wg, bg, alpha = 0.0, 0.0, 0.05
for step in range(2000):
    err = (wg * x + bg) - y          # y_hat - y
    wg -= alpha * 2 * (err * x).mean()
    bg -= alpha * 2 * err.mean()
print(f"gradient descent: w = {wg:.2f}, b = {bg:.2f}")

# --- 3. 400 completed stories, against the mean baseline -----
rng = np.random.default_rng(5)
n = 400
points = rng.choice([1, 2, 3, 5, 8, 13], n)
files = rng.poisson(2 + 0.5 * points)      # files touched
changes = rng.poisson(1.0, n)             # requirement changes
years = rng.integers(0, 11, n)            # developer experience
db = (rng.random(n) < 0.3).astype(int)    # touches the database
t = (2 + 2.2 * points + 0.8 * files + 1.5 * changes - 0.3 * years
     + 3 * db + rng.normal(0, 4, n)).clip(min=1) # actual hours
X = np.column_stack([points, files, changes, years, db])
names = ["points", "files", "changes", "years", "db"]
X_tr, X_te, t_tr, t_te = train_test_split(X, t, test_size=0.25,
                                          random_state=0)

scaler = StandardScaler().fit(X_tr)
model = LinearRegression().fit(scaler.transform(X_tr), t_tr)
pred = model.predict(scaler.transform(X_te))
base = np.full_like(t_te, t_tr.mean()) # always predict the mean
for name, p in [("baseline (mean)", base), ("linear regression", pred)]:
    print(f"{name:18s} MAE {mean_absolute_error(t_te, p):5.1f} "
          f"RMSE {mean_squared_error(t_te, p) ** 0.5:5.1f} "
          f"R2 {r2_score(t_te, p):5.2f}")
for i in np.argsort(-np.abs(model.coef_))[:3]:
    print(f" weight on {names[i]:7s}: {float(model.coef_[i]):6.1f}")

```

Change alpha to 0.2 in part 2 and run it again. Explain what you see using the worked example of Chapter 2.

🔍 Checkpoint — Test Yourself

1. Fit $\hat{y} = wx + b$ to the points (0, 1), (1, 3), (2, 5) by the least-squares formulas.
2. A model has $R^2 = -0.2$ on the test set. What does that mean?
3. Why is $\hat{y} = w_1x + w_2x^2 + b$ still called a *linear* model?

📋 Chapter Summary

- Linear regression predicts $\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$; each weight is the effect of one more unit of its feature, others held fixed.
- Least squares minimises the mean squared error; it is the maximum likelihood fit under Gaussian noise, and it is sensitive to outliers.
- One feature: $w = \sum(x - \bar{x})(y - \bar{y}) / \sum(x - \bar{x})^2$, $b = \bar{y} - w\bar{x}$. Many: the normal equation.
- Gradient descent moves each weight by the average of error times feature; scale features first. Mini-batch is the practical default.
- Polynomial features let a linear model fit curves — and overfit.
- Report MAE or RMSE in the units of y , and R^2 , beside the mean baseline; check the residual plot for patterns.

 Review Questions

1. **(MCQ)** The least-squares line always passes through: (a) the origin (b) $(\bar{x}, \bar{y})$ (c) the first data point (d) the largest residual.
2. **(MCQ)** Which metric is most affected by one very large error? (a) MAE (b) RMSE (c) the median absolute error (d) all equally.
3. **(MCQ)** $R^2 = 0$ means the model: (a) is perfect (b) does no better than predicting the mean (c) has no bias (d) is overfitted.
4. **(Short)** Explain in words the gradient of the MSE with respect to one weight.
5. **(Short)** Give two situations where you would use gradient descent rather than the normal equation.
6. **(Short)** Why can the weight of a feature change sign when another feature is added to the model?
7. **(Applied)** For the points $(1, 2)$, $(2, 2)$, $(3, 5)$, $(4, 6)$, compute the least-squares line, the residuals, the MAE, the RMSE and R^2 .
8. **(Applied)** Starting from $w = 0$, $b = 0$ with $\alpha = 0.1$, perform one gradient descent step on the data of the previous question and compute the MSE before and after.

Logistic Regression

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Linear Models and Generalization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why linear regression is unsuitable for classification.
- **Compute** a class probability with the sigmoid function, and **interpret** a weight as a change in log-odds and an odds ratio.
- **Find** and **draw** the decision boundary of a logistic regression model.
- **Compute** the cross-entropy loss and **perform** a gradient descent step by hand.
- **Extend** the model to several classes with one-vs-rest and softmax.

🔗 Before You Start

Chapter 5 (the linear model, gradient descent on a loss) and Chapter 2 (maximum likelihood, logarithms, the chain rule).

🔑 Key Terms in This Chapter

binary classification • sigmoid (logistic) function • odds • log-odds (logit) • odds ratio • decision boundary • threshold • cross-entropy (log loss) • linear separability • one-vs-rest • softmax

6.1 Why Not Just Use Linear Regression?

Code a binary label as $y = 1$ (late) or $y = 0$ (on time), fit a least-squares line, and call anything above 0.5 late. It half works — and it breaks in two ways, both visible on the left of Figure 6.1. The line predicts values below 0 and above 1, which cannot be probabilities. And a few examples far from the boundary — a small, well-specified project, which of course finished on time — drag the line towards them and move the 0.5 crossing, misclassifying projects near the boundary that were previously correct. The model is being penalised for being *too* right about easy cases.

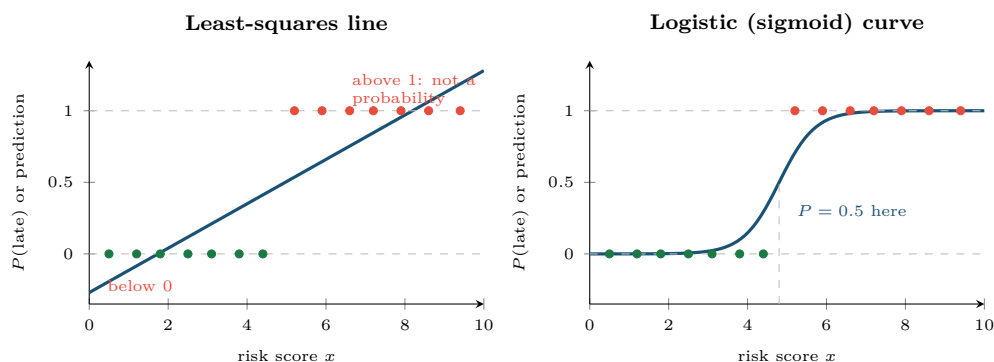


Figure 6.1: The same on-time (green) and late (red) projects modelled two ways. The straight line leaves $[0, 1]$ and is pulled by easy cases; the sigmoid stays between 0 and 1 and flattens out where the answer is already clear.

6.2 The Sigmoid and the Log-Odds

Logistic Regression

Compute a linear score $z = \mathbf{w} \cdot \mathbf{x} + b$ exactly as in linear regression, then squash it into $(0, 1)$ with the **sigmoid** (logistic) function:

$$P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$\sigma(0) = 0.5$; large positive z gives a probability near 1 and large negative z a probability near 0. Predict class 1 when the probability exceeds a **threshold**, by default 0.5.

Inverting the sigmoid shows what the linear score means. If $p = \sigma(z)$ then

$$z = \ln \frac{p}{1 - p}$$

The ratio $p/(1 - p)$ is the **odds** — a probability of 0.8 is odds of 4 to 1 — and its logarithm is the **log-odds** or **logit**. So logistic regression is a linear model *of the log-odds*. Each weight is the change in log-odds for a one-unit increase in its feature, and e^{w_j} is the **odds ratio**: the factor by which the odds are multiplied.

Worked Example: Probabilities and Odds for Two Projects

A fitted model of finishing late: $z = 4 - 5c - 3s$, where c is requirement clarity (the fraction of requirements signed off) and s the skill match (the fraction of the needed skills the team already has), both on a 0–1 scale.

Project A ($c = 0.9, s = 0.8$): $z = 4 - 4.5 - 2.4 = -2.9$, so $P(\text{late}) = \frac{1}{1 + e^{2.9}} = \frac{1}{1 + 18.17} = \mathbf{0.052}$.

Project B ($c = 0.5, s = 0.4$): $z = 4 - 2.5 - 1.2 = 0.3$, so $P(\text{late}) = \frac{1}{1 + e^{-0.3}} = \frac{1}{1 + 0.741} = \mathbf{0.574}$.

Reading the clarity weight. $w_c = -5$ per unit of clarity, so ten percentage points more clarity (+0.1) changes the log-odds by -0.5 and multiplies the odds of finishing late by $e^{-0.5} = \mathbf{0.61}$ — a 39% reduction in the odds, whatever the project's starting point. Note: the *odds*, not the probability. For Project A the probability barely moves; for Project B, near the middle of the curve, it moves a lot.

6.3 The Decision Boundary

The model predicts class 1 when $\sigma(z) > 0.5$, which happens exactly when $z > 0$. So the **decision boundary** is the set of points where $\mathbf{w} \cdot \mathbf{x} + b = 0$: a straight line in two dimensions, a plane in three, a hyperplane in general. Logistic regression is a *linear classifier*; it can separate classes only with a flat boundary.

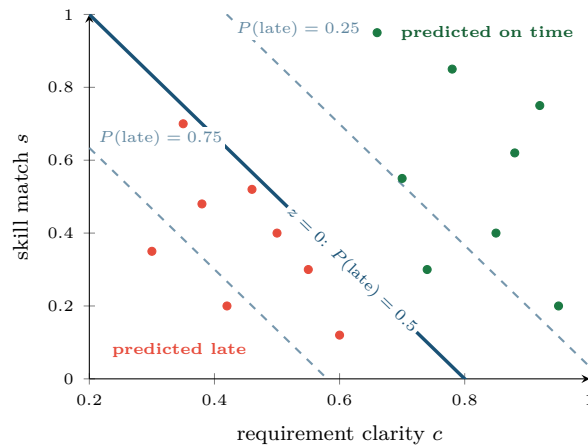


Figure 6.2: The decision boundary of the model $z = 4 - 5c - 3s$ is the line $5c + 3s = 4$. The dashed lines, where $z = \pm \ln 3$, are where the probability of finishing late is 0.75 and 0.25: probability changes gradually across the boundary.

6.4 The Loss: Cross-Entropy

Squared error is the wrong loss for probabilities: combined with the sigmoid it gives a bumpy surface with flat regions where gradient descent stalls. The right loss comes, again, from maximum likelihood. If the model gives probability p_i to class 1, the likelihood of the observed label is p_i when $y_i = 1$ and $1 - p_i$ when $y_i = 0$. Taking minus the average log of that gives:

☰ Cross-Entropy (Log Loss)

$$L(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right], \quad p_i = \sigma(\mathbf{w} \cdot \mathbf{x}_i + b)$$

For each example only one term survives: $-\ln p_i$ if the label is 1, $-\ln(1 - p_i)$ if it is 0. The loss is convex — one bowl, one minimum — so gradient descent finds the best weights from any start.

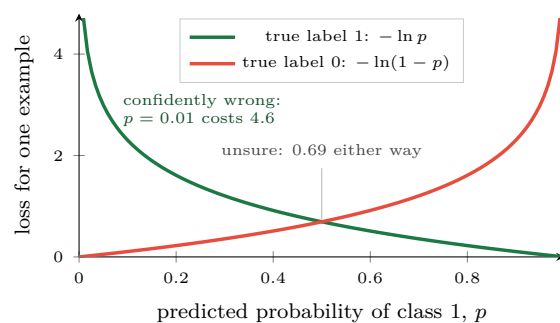


Figure 6.3: Cross-entropy punishes confident mistakes without limit. Being right at 0.9 costs 0.11; being wrong at 0.99 costs 4.6.

☰ Worked Example: Cross-Entropy on Four Predictions

- Label 1, predicted $p = 0.9$: loss $-\ln 0.9 = 0.105$.
- Label 1, predicted $p = 0.6$: loss $-\ln 0.6 = 0.511$.
- Label 0, predicted $p = 0.2$: loss $-\ln 0.8 = 0.223$.
- Label 0, predicted $p = 0.7$: loss $-\ln 0.3 = 1.204$.

Average: $(0.105 + 0.511 + 0.223 + 1.204)/4 = \mathbf{0.511}$.

The last example is the only one misclassified at a 0.5 threshold, and it contributes more than half the total. Replace it by a label-1 example predicted at $p = 0.01$ and that one example alone costs $-\ln 0.01 = 4.6$. Cross-entropy cares not just whether you are wrong, but how sure you were.

6.5 Fitting by Gradient Descent

Differentiating the cross-entropy through the sigmoid, most of the algebra cancels and leaves a result of exactly the same shape as linear regression's:

$$\frac{\partial L}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_{ij}, \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)$$

Error times feature, averaged — with the error now measured in probability.

Worked Example: One Gradient Descent Step

Four examples with one feature: $x = 1, 2, 3, 4$ and labels $y = 0, 0, 1, 1$. Start at $w = 0, b = 0$ with learning rate $\alpha = 1$.

Predictions: $z = 0$ everywhere, so $p = 0.5$ for all four, and the loss is $-\ln 0.5 = 0.693$.

Errors $p - y$: 0.5, 0.5, -0.5, -0.5.

$$\frac{\partial L}{\partial w} = \frac{1}{4} [(0.5)(1) + (0.5)(2) + (-0.5)(3) + (-0.5)(4)] = \frac{1}{4}(-2) = -0.5, \quad \frac{\partial L}{\partial b} = 0$$

Update: $w \leftarrow 0 - 1 \times (-0.5) = 0.5$; b is unchanged.

New probabilities $\sigma(0.5x)$: 0.62, 0.73, 0.82, 0.88. The loss falls to 0.654. The two class-1 examples are already right; the next steps will push b negative so that the first two fall below 0.5 as well.

When the Classes Can Be Separated Perfectly

In the worked example a single threshold, $x = 2.5$, separates the classes with no errors. The loss can then always be reduced by making the sigmoid steeper, so the weights grow without limit: after 5,000 steps $w \approx 8.9$ and still rising, while the boundary settles at $x \approx 2.5$. The fix is regularisation (Chapter 7), which scikit-learn applies by default — that is what its parameter `C` controls.

6.6 More Than Two Classes

One-vs-rest trains one binary classifier per class (“class k or not?”) and predicts the class whose classifier is most confident. **Softmax** (multinomial) regression is the direct generalisation: one linear score per class, turned into probabilities that sum to 1:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$$

Worked Example: Softmax on Three Scores

A ticket classifier gives scores $z = (2.0, 1.0, 0.1)$ for the classes *bug*, *feature request* and *question*.

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{0.1} = 1.105, \quad \text{sum} = 11.212$$

$$P = \left(\frac{7.389}{11.212}, \frac{2.718}{11.212}, \frac{1.105}{11.212} \right) = (0.659, 0.242, 0.099)$$

Adding the same constant to every score leaves the probabilities unchanged — only the differences between scores matter. The same function is the output layer of almost every neural network classifier (Chapter 13).

! Common Pitfalls

- **Reading a weight as a change in probability.** It is a change in log-odds. The effect on probability depends on where on the curve you start.
- **Assuming the 0.5 threshold is right.** It is a default. When the two errors cost different amounts, move it (Chapter 8).
- **Forgetting that the boundary is linear.** If the classes are separated by a curve, add features (squares, products) or use a non-linear model (Part III).
- **Comparing weights of unscaled features.** As in Chapter 5, standardise first.
- **Trusting probabilities from an unregularised model on separable data.** They will all be pushed to 0 or 1.

🔧 Try It Yourself: Logistic Regression by Hand and by Library

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

sigmoid = lambda z: 1 / (1 + np.exp(-z))

# --- 1. the worked example: gradient descent by hand -----
x = np.array([1, 2, 3, 4.0])
y = np.array([0, 0, 1, 1.0])
logloss = lambda p: -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
w, b = 0.0, 0.0
p = sigmoid(w * x + b)
print("before: loss", round(logloss(p), 3))
w -= 1.0 * np.mean((p - y) * x)      # gradient: mean of (p - y) * x
b -= 1.0 * np.mean(p - y)
print(f"after one step: w = {w}, b = {b}, "
      f"loss {logloss(sigmoid(w * x + b)):.3f}")
for step in range(5000):              # ... and many more steps
    p = sigmoid(w * x + b)
    w, b = w - np.mean((p - y) * x), b - np.mean(p - y)
print(f"after 5000 steps: w = {w:.2f}, b = {b:.2f}, "
      f"boundary at x = {-b / w:.2f}")

# --- 2. a real classifier, read as odds ratios -----
rng = np.random.default_rng(3)
n = 500                               # past projects
names = ["team_size", "changes", "dependencies", "clarity"]
X = np.column_stack([rng.integers(3, 16, n),          # people
                     rng.poisson(5, n),               # change requests
                     rng.integers(0, 7, n),           # external teams
                     rng.uniform(0.3, 1.0, n)])       # clarity, 0-1
z = -1.5 + 0.05 * X[:, 0] + 0.3 * X[:, 1] + 0.4 * X[:, 2] - 3.5 * X[:, 3]
y = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int) # 1 = late
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)
```

```

clf = make_pipeline(StandardScaler(), LogisticRegression())
clf.fit(X_tr, y_tr)
print("test accuracy:", round(clf.score(X_te, y_te), 3))
coef = clf[-1].coef_[0]
for i in np.argsort(-np.abs(coef)):      # strongest risk factors first
    print(f" {names[i]:13s} w = {coef[i]:5.2f} "
          f"odds x {np.exp(coef[i]):.2f} per standard deviation")
print("P(late) for 3 test projects:",
      clf.predict_proba(X_te[:3])[:, 1].round(3))

# --- 3. three classes: softmax -----
m = 300                                # support tickets
T = np.column_stack([rng.uniform(0, 4, m),      # log10 users hit
                    rng.integers(0, 2, m)])      # 1 = workaround
score = T[:, 0] - 1.2 * T[:, 1] + rng.normal(0, 0.4, m)
level = np.digitize(score, [1.0, 2.5])         # 0 low, 1 medium, 2 high
soft = make_pipeline(StandardScaler(), LogisticRegression())
soft.fit(T, level)
print("P(low, medium, high) for ticket 60:",
      soft.predict_proba(T[[60]]).round(3))

```

In part 1, change the range to 50,000 steps. Does the boundary move? Do the weights? Explain both answers.

Checkpoint — Test Yourself

1. A model gives $z = 1.2$ for a task. What is the probability that it finishes late, and what are the odds?
2. A weight is $+0.7$. By what factor does a one-unit increase in its feature multiply the odds?
3. Which costs more under cross-entropy: predicting 0.4 for a positive example, or 0.4 for a negative one? Compute both.

Chapter Summary

- Linear regression is unsuitable for classification: its outputs leave $[0, 1]$ and easy cases distort its boundary.
- Logistic regression passes a linear score through the sigmoid, $p = 1/(1 + e^{-z})$; the score is the log-odds of class 1.
- Each weight changes the log-odds by w per unit; e^w is the odds ratio.
- The decision boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$ is linear.
- Cross-entropy is the maximum likelihood loss; it is convex and punishes confident mistakes heavily. Its gradient is the mean of $(p - y) \times \text{feature}$.
- Softmax extends the model to k classes; on separable data the weights grow without limit unless regularised.

Review Questions

1. (MCQ) $\sigma(0)$ equals: (a) 0 (b) 0.5 (c) 1 (d) e .
2. (MCQ) The decision boundary of logistic regression is: (a) a sigmoid curve (b) a hyperplane (c) a circle (d) any shape.
3. (MCQ) A probability of 0.75 corresponds to odds of: (a) 0.75 (b) 1.33 (c) 3 (d) 4.
4. (Short) Explain why cross-entropy rather than squared error is used to train logistic regression.

5. **(Short)** What happens to logistic regression's weights on data that a straight line separates perfectly, and why?
6. **(Short)** Contrast one-vs-rest with softmax regression.
7. **(Applied)** With $z = -3 + 2x_1 + 0.5x_2$, compute the probability of class 1 for $(x_1, x_2) = (1, 2)$ and $(2, 0)$. Write the equation of the decision boundary and state which side each point lies on.
8. **(Applied)** Perform a second gradient descent step on the worked example, starting from $w = 0.5$, $b = 0$, with $\alpha = 1$.

Overfitting, Bias–Variance and Regularisation

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Linear Models and Generalization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** training error from true error, and **recognise** underfitting and overfitting from both.
- **Decompose** expected error into bias, variance and noise, and **compute** each from repeated predictions.
- **Diagnose** a model from its learning curves and **choose** the right remedy.
- **Apply** ridge and lasso regularisation and **explain** how the penalty strength trades bias for variance.
- **Select** a model and its hyperparameters with k -fold cross-validation, touching the test set only once.

🔗 Before You Start

Chapter 3 (training, validation and test sets; pipelines), Chapter 5 (polynomial features, MSE) and Chapter 6 (the growing weights on separable data).

📖 Key Terms in This Chapter

generalisation • training error • true error • generalisation gap • underfitting • overfitting • model complexity • bias • variance • irreducible error (noise) • learning curve • regularisation • ridge (L2) • lasso (L1) • hyperparameter • k -fold cross-validation • grid search • nested cross-validation

7.1 The Goal Is Generalisation

A model's **training error** is its error on the examples it learned from. Its **true error** is its expected error on new examples drawn from the same source — the only error anybody cares about. The difference between the two is the **generalisation gap**. Training error is easy to measure and always optimistic; true error cannot be measured at all, only estimated on data the model has never seen. Everything in this chapter follows from taking that distinction seriously.

7.2 Underfitting and Overfitting

Figure 7.1 fits polynomials of three degrees to the same ten noisy measurements of a smooth rise and fall: a team's weekly ticket volume over one release cycle, which climbs after each release and falls as the fixes land (x is the position in the cycle, y the volume on a scaled axis). Degree 1 is too rigid to follow the curve at all. Degree 9 has ten coefficients for ten points, so it passes through every one of them — training error exactly zero — and swings wildly between them. Degree 3 follows the shape and ignores the noise.

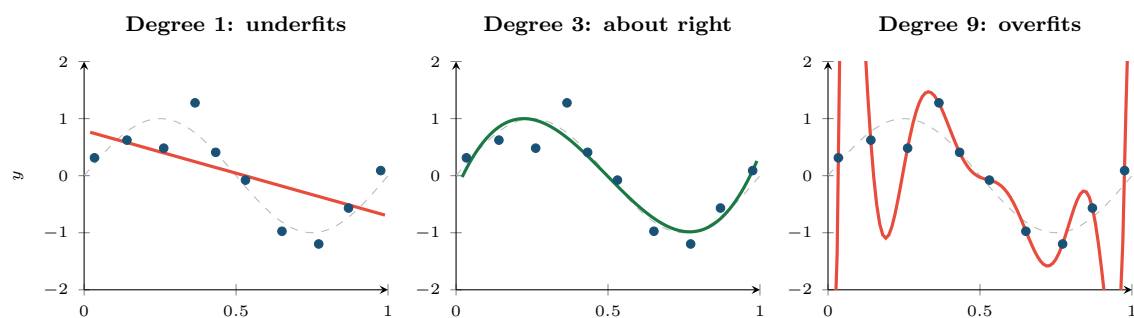


Figure 7.1: Three polynomials fitted to the same ten points (blue); the dashed curve is the true function. Training error falls from left to right and reaches zero at degree 9, which is nonetheless the worst of the three on new data.

Underfitting and Overfitting

A model **underfits** when it is too simple to capture the pattern: both training and test error are high. A model **overfits** when it captures the noise in its particular training sample as well as the pattern: training error is low, test error is high, and the gap between them is large.

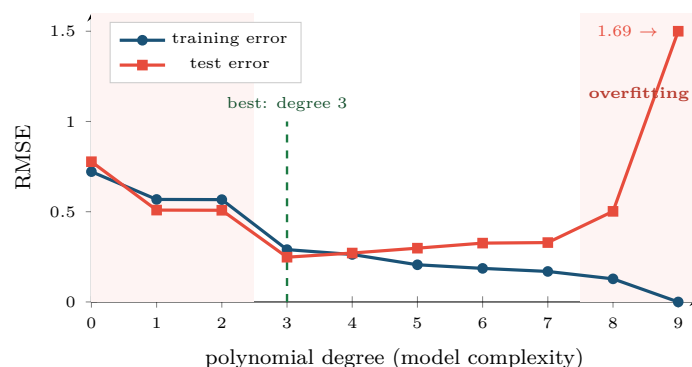


Figure 7.2: Training and test error for the polynomials of Figure 7.1, degree 0 to 9, with the test error measured on 2,000 fresh samples. Training error only ever falls; test error falls, bottoms out, and rises. The shape of that second curve is the whole of this chapter.

7.3 The Bias–Variance Decomposition

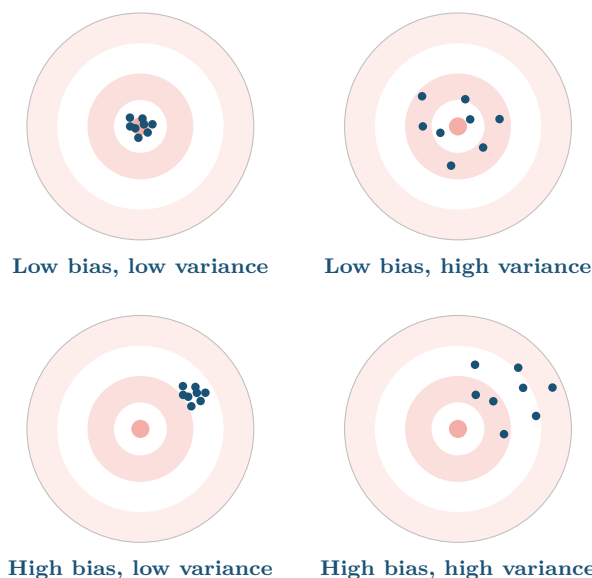
Why does test error rise again? Imagine repeating the whole experiment many times: draw a fresh training set, fit the model, predict at one fixed input x_0 . The predictions scatter. Their average may miss the truth, and they may be spread widely around their own average. Those are two different failings.

Bias, Variance and Noise

For a fixed input x_0 with true value $f(x_0)$, let $\hat{f}(x_0)$ be the prediction of a model trained on a random training set. Then the expected squared error on a new observation $y = f(x_0) + \varepsilon$ is

$$\mathbb{E}[(y - \hat{f}(x_0))^2] = \underbrace{(\mathbb{E}[\hat{f}(x_0)] - f(x_0))^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2]}_{\text{variance}} + \underbrace{\sigma_\varepsilon^2}_{\text{noise}}$$

Bias is how far the average prediction is from the truth — the error of the model’s assumptions. **Variance** is how much the prediction changes from one training set to another — its sensitivity to the particular sample. **Noise** is the **irreducible error** in the labels themselves, which no model can remove.



The centre is the truth $f(x_0)$. Each dot is the prediction of a model trained on a different random training set.

Bias is how far the *cloud’s centre* is from the bull’s-eye: simple models such as a straight line through a curve miss in the same direction every time.

Variance is how *spread out* the cloud is: flexible models such as the degree-9 polynomial land somewhere different every time.

Increasing complexity moves a model from the bottom-left target towards the top-right one.

Figure 7.3: Bias and variance as target practice. The goal is the top-left target; in practice lowering one tends to raise the other.

Worked Example: Bias and Variance From Four Training Sets

The true value at x_0 is $f(x_0) = 5$, and the label noise has variance $\sigma_\varepsilon^2 = 1$. Two models are each trained on the same four random training sets and predict at x_0 :

- **Simple model:** 3.8, 4.2, 4.0, 4.0. Mean 4.0.
 $\text{bias}^2 = (4.0 - 5)^2 = 1.00$; $\text{variance} = \frac{0.04 + 0.04 + 0 + 0}{4} = 0.02$.
- **Complex model:** 6.5, 3.4, 5.9, 4.2. Mean 5.0.
 $\text{bias}^2 = 0$; $\text{variance} = \frac{2.25 + 2.56 + 0.81 + 0.64}{4} = 1.565$.

Expected squared error = $\text{bias}^2 + \text{variance} + \text{noise}$:

- simple: $1.00 + 0.02 + 1 = \mathbf{2.02}$
- complex: $0 + 1.565 + 1 = \mathbf{2.57}$

The biased model wins. It is wrong on average, but consistently so; the unbiased model is right on average and wrong every single time. And neither can go below 1.0 — that is the noise.

The Trade-Off

Making a model more flexible lowers its bias and raises its variance; making it more rigid does the reverse. Test error is their sum plus noise, so it is U-shaped in complexity (Figure 7.2). The practical question is never “how do I eliminate bias?” or “how do I eliminate variance?” but “where is the bottom of the U, for this much data?” — because more data lowers variance and moves the bottom to the right.

7.4 Noisy Data

Noise is not only measurement error in the features. It is also *label* noise: a task recorded as late that was in fact cancelled, a bug report closed as “not a bug” by a tired reviewer. Noise sets the floor on achievable error, and it is exactly what an overfitting model memorises. Two consequences follow. First, a training error far below the noise level is a symptom, not an achievement — the model has fitted the noise. Second, the noisier the data, the simpler the model it will support: every method in Part III has a setting (tree depth and pruning, k in nearest neighbours, C in an SVM) whose job is to stop it chasing noise.

7.5 Diagnosing With Learning Curves

A **learning curve** plots training and validation error against the number of training examples. Its shape tells you which of bias and variance is the problem, and therefore what to do.

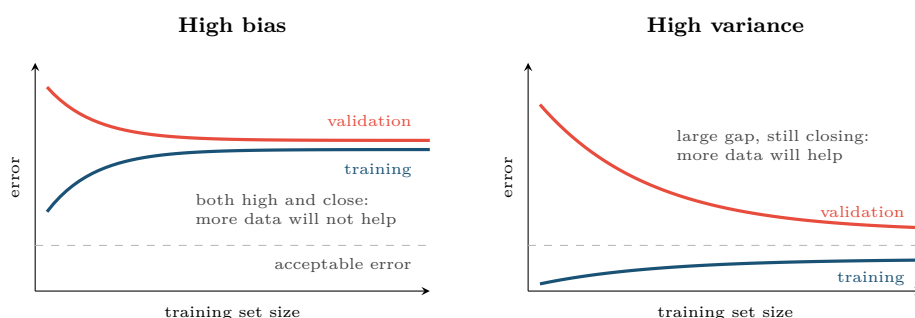


Figure 7.4: Learning curves. With high bias the curves meet early at an unacceptable level; with high variance there is a wide gap that more data would narrow.

Diagnosis	Symptom	What helps
High bias (underfitting)	Training error high; validation close to it	A more flexible model; more or better features; less regularisation. More data does <i>not</i> help
High variance (overfitting)	Training error low; validation much higher	More data; fewer features; a simpler model; more regularisation; ensembles that average (Chapter 14)
Both fine but still poor	Both curves meet at the noise level	Better data: cleaner labels, more informative features. The algorithm is not the problem

7.6 Regularisation

Choosing a lower polynomial degree is one way to control complexity, but a crude one: the degree jumps in whole steps. **Regularisation** controls it smoothly, by keeping the high-degree terms but charging the model for large weights.

Ridge and Lasso

Add a penalty on the size of the weights to the training loss:

$$\text{Ridge (L2): } \text{MSE} + \lambda \sum_j w_j^2 \qquad \text{Lasso (L1): } \text{MSE} + \lambda \sum_j |w_j|$$

The **hyperparameter** $\lambda \geq 0$ sets the strength: $\lambda = 0$ is ordinary least squares; a very large λ forces every weight to 0 (the bias b is not penalised). Ridge shrinks all weights

smoothly; lasso drives some of them to exactly zero, so it also *selects features*. Features must be standardised first, or the penalty falls unequally on them.

Worked Example: Ridge Shrinkage in One Dimension

Centred data $x = -2, -1, 0, 1, 2$ and $y = -3, -1, 0, 2, 2$, fitted by $\hat{y} = wx$. Minimising $\sum (y_i - wx_i)^2 + \lambda w^2$ gives

$$w = \frac{\sum x_i y_i}{\sum x_i^2 + \lambda} \quad \text{with} \quad \sum x_i y_i = 6 + 1 + 0 + 2 + 4 = 13, \quad \sum x_i^2 = 10$$

- $\lambda = 0$: $w = 13/10 = \mathbf{1.30}$ (least squares).
- $\lambda = 2$: $w = 13/12 = \mathbf{1.08}$.
- $\lambda = 10$: $w = 13/20 = \mathbf{0.65}$.

The penalty adds λ to the denominator, so it pulls the weight towards zero — a little when the data is plentiful ($\sum x^2$ large), a lot when it is not. That is exactly the behaviour wanted: shrink hardest where the data says least.

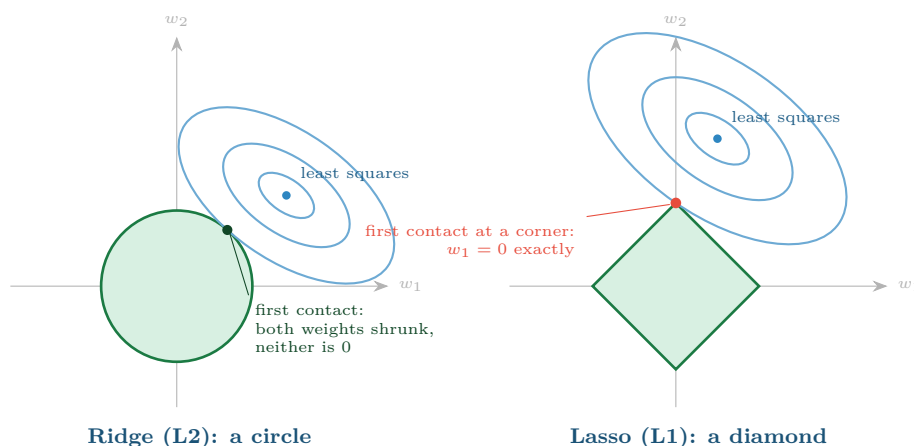


Figure 7.5: Why lasso sets weights to zero. The regularised solution is where the smallest error ellipse (blue) first touches the region allowed by the penalty (green). A circle is touched at a generic point; a diamond is usually touched at a corner, where one weight is exactly zero.

⚠ Regularisation Is Everywhere, Under Other Names

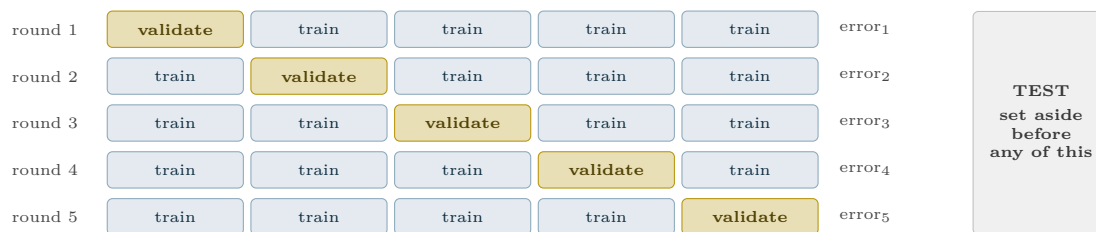
Every flexible model in this course has a knob that trades variance for bias. Maximum tree depth and pruning (Chapter 9); k in k -nearest neighbours (Chapter 11); C in SVMs (Chapter 12); weight decay, early stopping and dropout in neural networks (Chapter 13); $C = 1/\lambda$ in scikit-learn's `LogisticRegression` (Chapter 6). Once you see them as the same idea, tuning them is the same procedure — the one in the next section.

7.7 Model Selection With Cross-Validation

A **hyperparameter** is a setting chosen before training — degree, λ , k , tree depth — as opposed to a *parameter* such as a weight, which training sets. Hyperparameters are chosen by validation error. With little data a single validation split is too noisy to trust, so it is replaced by cross-validation.

k -Fold Cross-Validation

Split the training data into k equal folds (typically 5 or 10). For each fold in turn, train on the other $k - 1$ folds and measure error on the held-out fold. The cross-validation score is the mean of the k errors; report their standard deviation too. Every example is used for validation exactly once.



CV score = mean of the five errors. Repeat for every candidate setting, pick the best, refit on all the training data, then evaluate on the test set once.

Figure 7.6: Five-fold cross-validation on the training data. The test set is not part of the rotation.

Worked Example: Choosing λ by Cross-Validation

Five-fold cross-validation of a degree-12 polynomial with ridge penalties, from this chapter's lab:

λ	CV RMSE	
10^{-6}	0.388	
10^{-3}	0.326	← lowest
10^{-1}	0.330	
10	0.441	
1000	0.706	

The U-shape again: too little penalty overfits, too much underfits. Choose $\lambda = 10^{-3}$, refit on the whole training set, and only then measure the test set: RMSE 0.299. Unregularised, the same degree-12 polynomial had a CV RMSE of 2.14 — regularisation, not a lower degree, rescued it.

Notice that 10^{-3} and 10^{-1} are within 0.004 of each other, far less than the fold-to-fold variation. When candidates are this close, prefer the simpler model — here the larger penalty.

Tuning on the Test Set Is Still Cheating When a Library Does It

If you try thirty settings and report the best *test* score, the test set has become a validation set and the score is optimistic. Tune with cross-validation on the training data (`GridSearchCV` does this), then use the test set once. When even the tuning procedure must be evaluated honestly — for a published comparison — wrap the whole search in an outer cross-validation loop: **nested cross-validation**.

Common Pitfalls

- **Celebrating zero training error.** With noisy labels it proves overfitting.
- **Collecting more data to cure high bias.** If training error is already high, more of the same data will not lower it.

- **Regularising unscaled features.** The penalty then depends on the units each feature happens to be measured in.
- **Reporting the best of many test scores.** Select on cross-validation; test once.
- **Choosing between near-identical CV scores.** Differences smaller than the fold-to-fold standard deviation are noise.

Try It Yourself: Find the Bottom of the U

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import (train_test_split, KFold,
                                     cross_val_score, GridSearchCV)

# --- a release cycle: ticket volume rises and falls, plus noise -----
rng = np.random.default_rng(0)
X = rng.uniform(0, 1, (60, 1))
y = np.sin(2 * np.pi * X[:, 0]) + rng.normal(0, 0.25, 60)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25,
                                          random_state=0)

cv = KFold(5, shuffle=True, random_state=0)
rmse = "neg_root_mean_squared_error"

# --- 1. complexity: training error against cross-validated error -----
for deg in (1, 3, 6, 12):
    m = make_pipeline(PolynomialFeatures(deg), StandardScaler(),
                      LinearRegression())
    m.fit(X_tr, y_tr)
    train = np.sqrt(np.mean((m.predict(X_tr) - y_tr) ** 2))
    cvs = -cross_val_score(m, X_tr, y_tr, cv=cv, scoring=rmse)
    print(f"degree {deg:2d}: train RMSE {train:.3f}    "
          f"CV RMSE {cvs.mean():.3f} +/- {cvs.std():.3f}")

# --- 2. keep degree 12, but regularise; choose alpha by CV -----
pipe = make_pipeline(PolynomialFeatures(12), StandardScaler(), Ridge())
grid = GridSearchCV(pipe, {"ridge__alpha": [1e-6, 1e-3, 1e-1, 10, 1000]},
                    cv=cv, scoring=rmse).fit(X_tr, y_tr)
for a, s in zip(grid.cv_results_["param_ridge__alpha"],
                  grid.cv_results_["mean_test_score"]):
    print(f"alpha {a:>8g}: CV RMSE {-s:.3f}")
print("chosen:", grid.best_params_)

# --- 3. only now, once: the test set -----
pred = grid.predict(X_te)
print("test RMSE:", round(float(np.sqrt(np.mean((pred - y_te) ** 2))), 3))
```

scikit-learn calls λ alpha. Change the 60 samples to 600 and run again: which degrees stop overfitting, and why?

Checkpoint — Test Yourself

1. A model has training RMSE 0.05 and validation RMSE 0.90. Is the problem bias or variance? Name two remedies.
2. Ten models trained on different samples predict 2, 2, 2, 2, 2, 2, 2, 2, 2, 2 at a point whose true value is 3. Give the bias² and the variance.
3. Why does lasso, but not ridge, perform feature selection?

Chapter Summary

- Training error is optimistic; generalisation is judged on unseen data.
- Underfitting: both errors high. Overfitting: training error low, test error high. Test error is U-shaped in model complexity.
- Expected error = $\text{bias}^2 + \text{variance} + \text{noise}$. Flexibility trades bias for variance; noise is a floor no model can go below.
- Learning curves diagnose the problem: converged and high means bias; a wide, closing gap means variance.
- Ridge (L2) shrinks weights; lasso (L1) also sets some to zero. The penalty strength is a hyperparameter.
- Choose hyperparameters by k -fold cross-validation on the training data, refit, then test once.

Review Questions

1. **(MCQ)** Adding training data is most useful when a model has: (a) high bias (b) high variance (c) high noise (d) low variance.
2. **(MCQ)** As λ in ridge regression increases: (a) bias and variance both fall (b) bias rises and variance falls (c) bias falls and variance rises (d) neither changes.
3. **(MCQ)** Which of these is a hyperparameter? (a) a regression weight (b) the bias term b (c) the number of folds' mean error (d) the degree of a polynomial.
4. **(Short)** Explain why training error cannot be used to choose the degree of a polynomial.
5. **(Short)** What is irreducible error, and why can no model remove it?
6. **(Short)** Describe nested cross-validation and when it is needed.
7. **(Applied)** Three models predict at a point with true value 10 and noise variance 4. Over five training sets they predict: A: 8, 8, 9, 8, 7; B: 12, 7, 10, 13, 8; C: 10, 10, 11, 9, 10. Compute bias^2 , variance and expected error for each, and rank them.
8. **(Applied)** For the centred data $x = -1, 0, 1, 2$ (centre it first) and $y = 1, 2, 2, 5$ (centre it too), compute the ridge weight for $\lambda = 0, 1, 5$ and describe the pattern.

Measuring Classifier Performance

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Linear Models and Generalization

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Build** a confusion matrix and **compute** accuracy, precision, recall, specificity and F_1 from it.
- **Explain** why accuracy misleads on imbalanced classes, and **choose** a metric that matches the cost of each error.
- **Read** ROC and precision–recall curves, and **choose** a decision threshold from costs.
- **Compute** macro- and micro-averaged metrics for several classes.
- **Construct** a confidence interval for an error rate, and **test** whether two classifiers genuinely differ.

🔗 Before You Start

Chapter 2 (the base-rate calculation), Chapter 6 (probabilities and thresholds) and Chapter 7 (cross-validation; the test set used once).

📖 Key Terms in This Chapter

confusion matrix • true/false positive/negative • accuracy • precision • recall (sensitivity) • specificity • F_1 score • class imbalance • ROC curve • AUC • precision–recall curve • operating threshold • macro and micro averaging • sample error • true error • confidence interval • McNemar’s test

8.1 Measuring Classifier Accuracy Is Not One Number

The HEC outline calls this topic *measuring classifier accuracy*, and the first lesson is that accuracy is usually the wrong measure. A classifier makes two kinds of mistake, and in almost every real problem one costs far more than the other: a defective release shipped against a reviewer’s hour spent on a change that was fine, a late task nobody warned about against a needless check-in with a team that was on track. A single number that adds the two together hides exactly the information needed to decide whether the classifier is any use.

8.2 The Confusion Matrix

📄 Confusion Matrix

For two classes, cross-tabulate the predictions against the truth:

- **true positive** (TP): positive, predicted positive;
- **false positive** (FP): negative, predicted positive — a false alarm;
- **false negative** (FN): positive, predicted negative — a miss;
- **true negative** (TN): negative, predicted negative.

Every metric in this chapter is a ratio of these four counts.

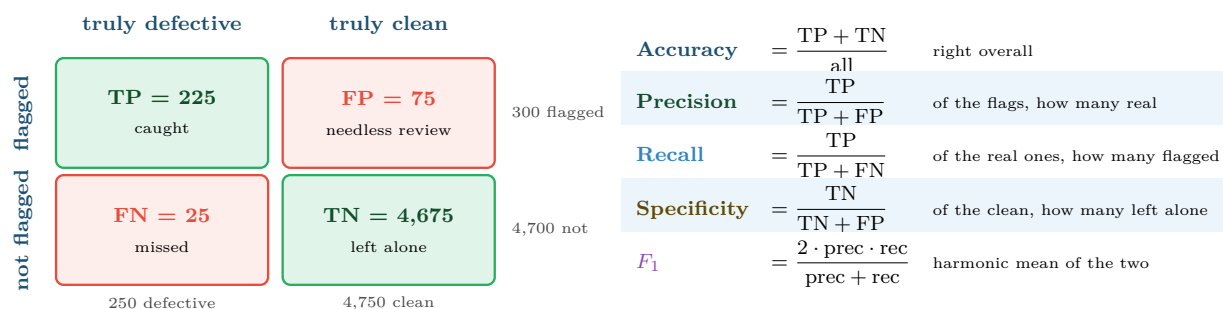


Figure 8.1: A defect predictor’s confusion matrix on 5,000 code changes, and the five metrics computed from it. Precision reads along the “flagged” row; recall reads down the “truly defective” column.

Worked Example: One Defect Predictor, Five Metrics

Before a release, the predictor of Figure 8.1 examined 5,000 code changes, of which 250 later turned out to have introduced a defect. It flagged 300 for extra review, of which 225 were defective.

Cells: $TP = 225$, $FP = 300 - 225 = 75$, $FN = 250 - 225 = 25$, $TN = 5000 - 225 - 75 - 25 = 4675$.

- **Accuracy** = $(225 + 4675)/5000 = \mathbf{98.0\%}$
- **Precision** = $225/300 = \mathbf{75.0\%}$
- **Recall** = $225/250 = \mathbf{90.0\%}$
- **Specificity** = $4675/4750 = \mathbf{98.4\%}$
- **F_1** = $\frac{2(0.75)(0.90)}{0.75 + 0.90} = \frac{1.35}{1.65} = \mathbf{0.818}$

Against the baseline. “Flag nothing” is right on the 4,750 clean changes: 95.0% accuracy. The predictor’s 98.0% is three points better than doing nothing, which sounds small; its recall of 90% — nine defective changes in ten caught, at the cost of one needless review for every three real finds — is what actually describes it.

⚠ The Accuracy Paradox

When one class is rare, accuracy is dominated by the common class. A detector for an event that occurs in 1% of cases can be 99% accurate by never detecting anything. Compute the majority-class baseline before looking at any accuracy figure, and report precision and recall instead.

8.3 Precision, Recall and the Threshold

A probabilistic classifier (Chapter 6) does not flag or clear anything by itself; it outputs a score, and a **threshold** turns scores into decisions. Lower the threshold and more cases are flagged: recall rises and precision falls. Raise it and the reverse happens. Neither can be maximised without sacrificing the other, so the threshold must be chosen from the costs of the two errors — which is a decision about the application, not about the algorithm.

Worked Example: Choosing an Operating Threshold From Costs

For the defect predictor, a missed defective change (FN) is estimated to cost 50 times as much as a needless review (FP) — a production incident against twenty minutes of a reviewer’s time. The predictor’s precision and recall at four thresholds, with 250 defective changes:

Threshold	Precision	Recall	TP	FN	FP	Cost = 50 FN + FP
0.3	0.55	0.96	240	10	196	696
0.5	0.75	0.90	225	25	75	1,325
0.7	0.86	0.78	195	55	32	2,782
0.9	0.95	0.52	130	120	7	6,007

(TP = recall $\times$ 250; FP = TP $\times$ (1 – precision)/precision.)

The cheapest threshold is 0.3, which almost halves the cost of the default 0.5 by accepting 121 more needless reviews to catch 15 more defects. With a cost ratio of 1:1 the answer would be different. The threshold is where the business decision enters the model.

8.4 Threshold-Free Summaries: ROC and PR Curves

ROC Curve and AUC

The **ROC curve** plots the true positive rate (recall) against the false positive rate (1 – specificity) at every possible threshold. The **area under the curve** (AUC) summarises it: the probability that the classifier scores a randomly chosen positive above a randomly chosen negative. A random classifier has AUC 0.5; a perfect one, 1.0.

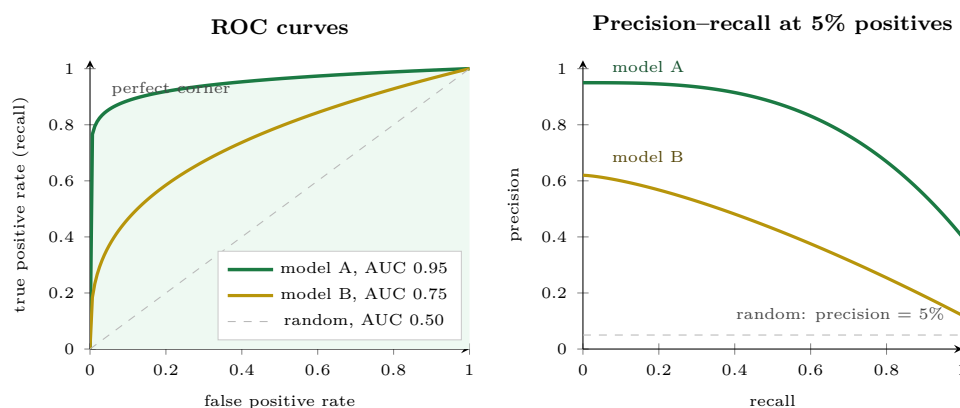


Figure 8.2: Left: ROC curves sweep every threshold; the shaded area is model A's AUC. Right: the same two models on precision and recall. With rare positives the PR plot shows how many alerts are false, which the ROC plot hides.

⚠ Prefer Precision-Recall When Positives Are Rare

The ROC false positive rate divides false alarms by the (huge) number of negatives, so hundreds of false alarms can look like a rate of 0.01. A precision-recall curve uses precision directly and does not flatter a rare-event detector. For defect, outage and overrun prediction, report the PR curve or its area (average precision), and quote precision at the recall the project needs.

8.5 More Than Two Classes

With k classes the confusion matrix is $k \times k$, and precision and recall are computed for each class in turn, treating it as positive and the rest as negative. They are then combined in one of two ways. **Macro averaging** takes the plain mean of the per-class values, so every class counts equally. **Micro averaging** pools the counts first, so every *example* counts equally — and micro-averaged recall equals accuracy.

Worked Example: Macro Against Micro

A ticket router is tested on 100 tickets: 80 bug reports, 15 feature requests and 5 security reports. It labels 76 bug reports, 9 feature requests and 1 security report correctly.

Per-class recall: bug $76/80 = 0.95$; feature $9/15 = 0.60$; security $1/5 = 0.20$.

Micro (accuracy): $(76 + 9 + 1)/100 = \mathbf{0.86}$.

Macro: $(0.95 + 0.60 + 0.20)/3 = \mathbf{0.58}$.

The micro figure is carried by the bug class; the macro figure reveals that the router almost never recognises a security report. If the rare classes matter — and security reports are exactly the tickets that must not wait — report the macro figure and the per-class table.

8.6 How Sure Is the Number? Confidence Intervals for Error

A test set is a sample, so the error measured on it is an estimate. Mitchell distinguishes the **sample error** $\text{error}_S(h)$, the fraction of the n test examples misclassified, from the **true error** $\text{error}_D(h)$ over the whole population. Because each test example is an independent yes/no trial, the number of mistakes is binomial, and for $n \geq 30$ an approximate 95% confidence interval for the true error is

$$\text{error}_S(h) \pm 1.96 \sqrt{\frac{\text{error}_S(h) (1 - \text{error}_S(h))}{n}}$$

Worked Example: How Wide Is 12% Error?

A classifier misclassifies 24 of $n = 200$ test examples: $\text{error}_S = 0.12$.

$$\sqrt{\frac{0.12 \times 0.88}{200}} = \sqrt{0.000528} = 0.0230, \quad 1.96 \times 0.0230 = 0.045$$

95% interval: 0.12 ± 0.045 , that is **7.5% to 16.5%**.

With $n = 2000$ and the same 12% error, the half-width shrinks to $1.96 \sqrt{0.12 \times 0.88 / 2000} = 0.014$: **10.6% to 13.4%**. Ten times the data gives an interval about $\sqrt{10} \approx 3.2$ times narrower. On 200 examples, a competitor at 10% error is not demonstrably better.

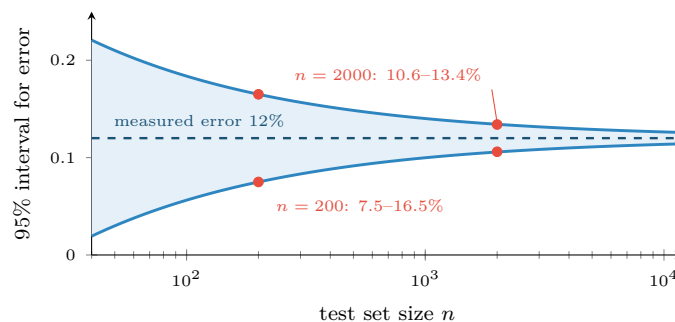


Figure 8.3: How the uncertainty in a measured 12% error rate shrinks with test-set size. The width falls as $1/\sqrt{n}$: four times the data halves it.

8.7 Is One Classifier Really Better? McNemar's Test

Two classifiers are tested on the same examples and one scores a little higher. Is it better, or luckier? Only the examples on which they *disagree* carry information. Let b be the number that classifier A gets wrong and B gets right, and c the number A gets right and B wrong. If the two are equally good, b and c should be similar. **McNemar's test** statistic

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

is compared with 3.84, the 95% point of the chi-squared distribution with one degree of freedom; a larger value means the difference is unlikely to be chance.

Worked Example: McNemar on 500 Test Tickets

The current ticket router A and a new router B are run on the same 500 tickets. They disagree on 40: B is right and A wrong on $b = 12$; A is right and B wrong on $c = 28$.

$$\chi^2 = \frac{(|12 - 28| - 1)^2}{12 + 28} = \frac{15^2}{40} = 5.63 > 3.84$$

A is significantly better at the 5% level ($p \approx 0.018$): it is right on 16 more tickets, 3.2 points of accuracy. The 460 tickets on which they agree play no part — which is why the test is sensitive even when the accuracies look close. When the comparison is across cross-validation folds instead of one test set, the equivalent tool is a paired t -test on the per-fold scores.

8.8 Handling Class Imbalance

Imbalance is a problem for training as well as evaluation: a learner minimising average loss can do very well by ignoring the rare class. Three remedies, in order of preference:

1. **Move the threshold.** Train normally, then choose the threshold from costs, as above. It changes nothing about the model and is easy to revisit.
2. **Weight the classes.** Make each error on the rare class cost more in the loss (`class_weight="balanced"` in scikit-learn).
3. **Resample the training data.** Undersample the common class, or oversample the rare one (by duplication, or by synthesising new examples with SMOTE). Resample only the training folds — never the validation or test data, whose class balance must stay real.

Common Pitfalls

- **Reporting accuracy on imbalanced data** without the majority baseline beside it.
- **Quoting precision without recall**, or either without the threshold at which it was measured.
- **Using ROC-AUC alone for rare events.** Report the PR curve.
- **Declaring a winner from a one-point difference** on a few hundred test examples. Compute the interval or run McNemar's test.
- **Resampling before splitting**, which puts copies of the same rare example into both training and test sets.

Try It Yourself: Evaluate an Imbalanced Classifier Properly

Six thousand synthetic code changes, 5% of which introduced a defect, each described by twelve numeric features (size, files touched, the author's experience and so on).

```

import numpy as np
from scipy.stats import chi2
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (confusion_matrix, precision_score, recall_score,
                             f1_score, roc_auc_score, average_precision_score)

# --- an imbalanced problem: 5% of changes are defective -----
X, y = make_classification(n_samples=6000, n_features=12, n_informative=5,
                          weights=[0.95], flip_y=0.01, random_state=1)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.4,
                                          stratify=y, random_state=1)
print("baseline accuracy (always 0):", round(1 - y_te.mean(), 3))
lr = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)
p = lr.predict_proba(X_te)[:, 1]

# --- 1. the four cells and the metrics built from them -----
pred = (p > 0.5).astype(int)
tn, fp, fn, tp = confusion_matrix(y_te, pred).ravel()
print(f"TP={tp} FP={fp} FN={fn} TN={tn}")
print(f"accuracy {np.mean(pred == y_te):.3f} "
      f"precision {precision_score(y_te, pred):.3f} "
      f"recall {recall_score(y_te, pred):.3f} F1 {f1_score(y_te, pred):.3f}")
print(f"ROC-AUC {roc_auc_score(y_te, p):.3f} "
      f"PR-AUC {average_precision_score(y_te, p):.3f}")

# --- 2. the threshold is a choice: sweep it -----
for t in (0.1, 0.3, 0.5, 0.7):
    pr = (p > t).astype(int)
    print(f"threshold {t}: precision {precision_score(y_te, pr):.2f} "
          f"recall {recall_score(y_te, pr):.2f}")

# --- 3. a 95% confidence interval for the error rate -----
n, e = len(y_te), np.mean(pred != y_te)
half = 1.96 * np.sqrt(e * (1 - e) / n)
print(f"error {e:.3f}, 95% interval [{e - half:.3f}, {e + half:.3f}]")

# --- 4. is a decision tree really different? McNemar's test -----
tree = DecisionTreeClassifier(max_depth=5, random_state=0).fit(X_tr, y_tr)
a_right = pred == y_te
b_right = tree.predict(X_te) == y_te
b = np.sum(~a_right & b_right) # logistic wrong, tree right
c = np.sum(a_right & ~b_right) # logistic right, tree wrong
stat = (abs(b - c) - 1) ** 2 / (b + c)
print(f"disagreements b={b}, c={c}; McNemar chi2 = {stat:.2f}, "
      f"p = {chi2.sf(stat, 1):.3f}")

```

The model is 96% accurate against a 95% baseline, and at the default threshold it finds only a third of the positives. Add `class_weight="balanced"` to `LogisticRegression` and compare the recall, precision and PR-AUC.

🔍 Checkpoint — Test Yourself

1. Of 400 tasks flagged as at risk, 100 did finish late; 20 late tasks were not flagged. Compute precision and recall.
2. A classifier's ROC-AUC is 0.97 on data with 0.1% positives. What should you ask for before believing it is useful?
3. A model makes 15 errors on 100 test examples. Give an approximate 95% interval for its true error.

Chapter Summary

- Every classification metric is a ratio of the four confusion-matrix cells; always compute the majority baseline first.
- Precision asks “when I flag, am I right?”; recall asks “of the real cases, how many did I flag?” The threshold trades one for the other, and should be chosen from the costs of the two errors.
- ROC-AUC summarises ranking quality across thresholds but flatters rare-event detectors; prefer precision–recall curves there.
- Macro averaging treats classes equally and exposes weak rare classes; micro averaging equals accuracy.
- A measured error is an estimate: its 95% interval is $e \pm 1.96\sqrt{e(1-e)/n}$. Compare two classifiers on the same test set with McNemar’s test.
- Handle imbalance by moving the threshold, weighting classes, or resampling the training folds only.

Review Questions

1. **(MCQ)** Recall is: (a) $TP/(TP+FP)$ (b) $TP/(TP+FN)$ (c) $TN/(TN+FP)$ (d) $(TP+TN)/all$.
2. **(MCQ)** Lowering the decision threshold usually: (a) raises precision and recall (b) raises recall, lowers precision (c) raises precision, lowers recall (d) changes neither.
3. **(MCQ)** Micro-averaged recall over all classes equals: (a) macro recall (b) accuracy (c) precision (d) F_1 .
4. **(Short)** Explain the accuracy paradox with a numeric example of your own.
5. **(Short)** Why do only the disagreements matter in McNemar’s test?
6. **(Short)** Give a situation where you would choose a threshold that makes precision 0.2, and justify it.
7. **(Applied)** A defect predictor tested on 10,000 code changes, 200 of them defective, flags 500, of which 160 are defective. Build the confusion matrix and compute accuracy, precision, recall, specificity and F_1 . Which two numbers would you put in a report to the release manager?
8. **(Applied)** Classifier A has 18% error and classifier B 15% on the same 300 test examples. Compute a 95% interval for each. On the disagreements, A is wrong and B right 25 times, and the reverse 16 times. Is B significantly better?

PART III

Supervised Learning Algorithms

Five families of supervised learner, each with a different idea of what a pattern is — a sequence of questions, a probability, a neighbourhood, a margin, a network of weighted sums — and a sixth chapter that combines them. Learn each one's inductive bias, and you will know when to reach for it.

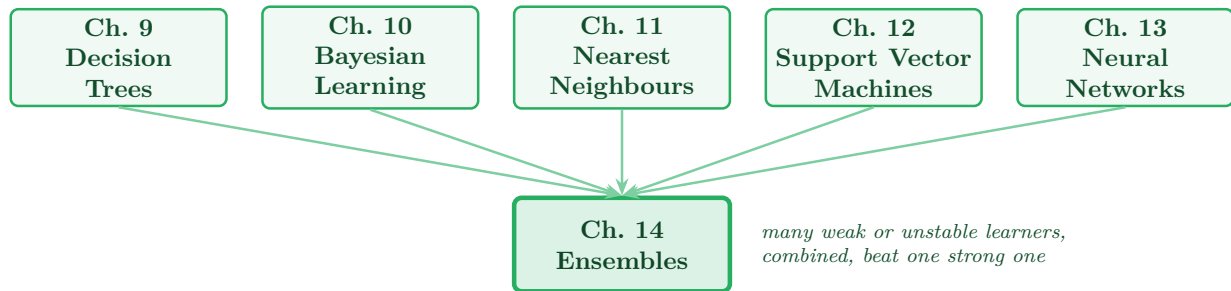


Figure P3.1: Reading path through Part III. The five learners can be taught in any order after Part II; Chapter 14 (highlighted) combines them, and is where trees in particular become state of the art.

Decision Tree Learning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Supervised Learning Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Classify** an example with a decision tree, and **read** a tree as a set of rules.
- **Compute** entropy and information gain, and **trace** ID3's choice of attribute at each node by hand.
- **Handle** continuous attributes, attributes with many values (gain ratio) and the Gini index.
- **Explain** why trees overfit noisy data, and **apply** reduced-error, rule post-pruning and cost-complexity pruning.
- **State** the inductive bias of ID3.

🔗 Before You Start

Chapter 2 (entropy), Chapter 4 (hypothesis spaces and inductive bias) and Chapter 7 (overfitting, noise, choosing a setting by cross-validation).

📖 Key Terms in This Chapter

decision tree • internal node, branch, leaf • ID3 • entropy • information gain • split information • gain ratio • Gini index • CART • threshold split • overfitting • pre-pruning • reduced-error pruning • rule post-pruning • cost-complexity pruning • Occam's razor

9.1 Trees as Hypotheses

A **decision tree** classifies an example by asking a sequence of questions about its attributes. Each **internal node** tests one attribute, each **branch** is one outcome of the test, and each **leaf** gives a class. To classify, start at the root and follow the branches that match the example until a leaf is reached.

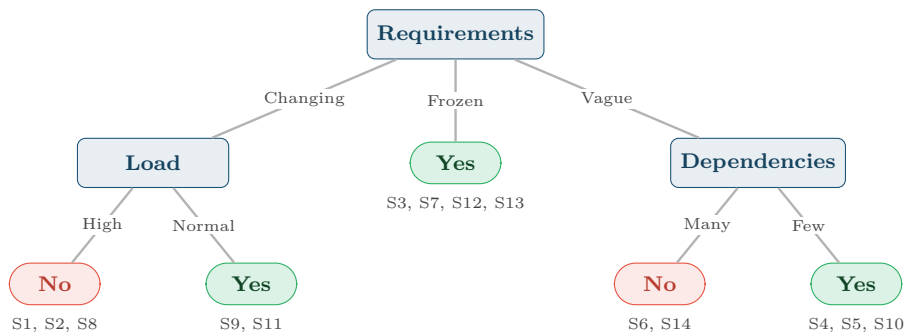


Figure 9.1: The tree ID3 learns from the fourteen past sprints. Under each leaf are the training sprints that reach it; every leaf is pure. It is equivalent to the rule: the goal is met if (requirements Changing and load Normal) or requirements Frozen or (requirements Vague and Few dependencies).

A tree is a disjunction of conjunctions: each root-to-leaf path is an AND of tests, and the tree is the OR of the paths that end in “Yes”. That makes trees far more expressive than the conjunctions of Chapter 4 — a large enough tree can represent *any* discrete function — and it makes them readable. Trees suit problems where examples are attribute–value pairs, the target is discrete, disjunctions may be needed, and the training data may contain errors or missing values.

9.2 Choosing the Question: Entropy and Information Gain

Fourteen past sprints, each described by four attributes, and whether the sprint met its goal. *Requirements* were Changing, Frozen or Vague during the sprint; *Pressure* is the deadline pressure; *Load* is whether the team was carrying more work than usual; *Dependencies* counts waits on other teams. (This is Mitchell’s *PlayTennis* table with the attributes renamed, so every number below can be checked against the set text.)

Sprint	Requirements	Pressure	Load	Dependencies	Goal met
S1	Changing	High	High	Few	No
S2	Changing	High	High	Many	No
S3	Frozen	High	High	Few	Yes
S4	Vague	Medium	High	Few	Yes
S5	Vague	Low	Normal	Few	Yes
S6	Vague	Low	Normal	Many	No
S7	Frozen	Low	Normal	Many	Yes
S8	Changing	Medium	High	Few	No
S9	Changing	Low	Normal	Few	Yes
S10	Vague	Medium	Normal	Few	Yes
S11	Changing	Medium	Normal	Many	Yes
S12	Frozen	Medium	High	Many	Yes
S13	Frozen	High	Normal	Few	Yes
S14	Vague	Medium	High	Many	No

ID3 grows the tree top-down and greedily: at each node, it asks which single attribute best separates the examples that reach it, puts that attribute there, and recurses on each branch. “Best separates” is measured with the entropy of Chapter 2.

Information Gain

The **information gain** of attribute A on example set S is the expected reduction in entropy from splitting S on A :

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

where S_v is the subset of S with $A = v$. ID3 puts the attribute with the highest gain at the node.

Worked Example: ID3’s First Choice on the Sprint Data

The root. 9 Yes, 5 No: $\text{Entropy}(S) = 0.940$ (computed in Chapter 2).

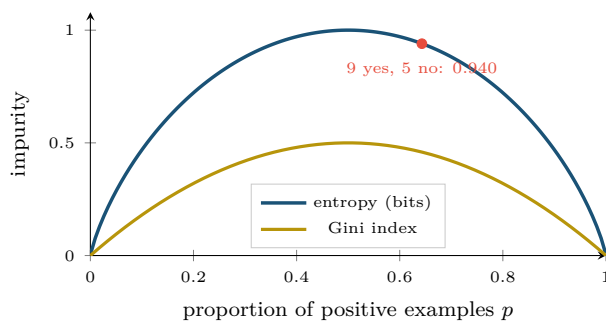


Figure 9.2: Two impurity measures for a two-class node. Both are zero for a pure node and largest for an even split; they almost always pick the same attribute.

Requirements splits S into Changing (2 Yes, 3 No: entropy 0.971), Frozen (4, 0: entropy 0) and Vague (3, 2: entropy 0.971):

$$\text{Gain}(S, \text{Requirements}) = 0.940 - \frac{5}{14}(0.971) - \frac{4}{14}(0) - \frac{5}{14}(0.971) = \mathbf{0.246}$$

Load: High (3, 4: 0.985), Normal (6, 1: 0.592):

$$\text{Gain}(S, \text{Load}) = 0.940 - \frac{7}{14}(0.985) - \frac{7}{14}(0.592) = \mathbf{0.151}$$

Dependencies: Few (6, 2: 0.811), Many (3, 3: 1.000):

$$\text{Gain}(S, \text{Dependencies}) = 0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1.000) = \mathbf{0.048}$$

Pressure: Gain = **0.029** (check it yourself).

Requirements wins and goes at the root. Its Frozen branch is already pure, so it becomes a Yes leaf.

The Changing branch (S1, S2, S8, S9, S11; entropy 0.971) is split the same way, using only those five sprints: Gain(Load) = 0.971, Gain(Pressure) = 0.571, Gain(Dependencies) = 0.020. Load separates them perfectly. On the Vague branch Dependencies does the same. Every leaf is now pure, and ID3 stops with the tree of Figure 9.1.

9.3 Beyond the Basic Algorithm

Continuous attributes. A numeric attribute is turned into a yes/no test $A \leq t$. The only thresholds worth trying are midpoints between adjacent sorted values where the class changes; each is scored by information gain like any other attribute.

Worked Example: Choosing a Threshold

Six sprints in which 40, 48, 60, 72, 80, 90 per cent of the stories had been refined (given acceptance criteria) before the sprint began, and the goal was met No, No, Yes, Yes, Yes, No. The class changes between 48 and 60 and between 80 and 90, so the candidates are $t = 54$ and $t = 85$. The set has 3 Yes and 3 No: entropy 1.

- $t = 54$: ≤ 54 holds {No, No} (entropy 0); > 54 holds 3 Yes, 1 No (entropy 0.811). Gain = $1 - \frac{4}{6}(0.811) = \mathbf{0.459}$.
 - $t = 85$: ≤ 85 holds 3 Yes, 2 No (0.971); > 85 holds {No} (0). Gain = $1 - \frac{5}{6}(0.971) = \mathbf{0.191}$.
- The test Refined ≤ 54 is chosen.

Attributes with many values. Information gain favours attributes with many values. An attribute such as *SprintID* splits the training sprints into fourteen pure singletons — maximal gain, zero use. C4.5 corrects this with the **gain ratio**, dividing the gain by the **split information**, the entropy of the split itself:

$$\text{SplitInfo}(S, A) = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}, \quad \text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}$$

For Requirements (5, 4, 5 sprints) $\text{SplitInfo} = 1.577$ and the gain ratio is $0.246/1.577 = 0.156$. For *SprintID*, $\text{SplitInfo} = \log_2 14 = 3.81$, which heavily discounts its gain.

The Gini index. CART, and scikit-learn by default, uses $\text{Gini}(S) = 1 - \sum_i p_i^2$ instead of entropy. At the root of the sprint data it is $1 - (9/14)^2 - (5/14)^2 = 0.459$, falling to 0.343 after splitting on Requirements. Figure 9.2 shows why the choice rarely matters: the two curves have the same shape.

Missing values. An example missing the tested attribute can be sent down the most common branch, or split fractionally among all branches in proportion to their training frequencies (C4.5’s approach).

9.4 Overfitting, Noisy Data and Pruning

ID3 keeps splitting until every leaf is pure. With noisy data that means growing branches whose only purpose is to fit the mislabelled examples: add one wrongly recorded sprint — (Changing, High, Normal, Many, No) — and ID3 must grow a new subtree under Changing/Normal just to isolate it. Figure 9.3 shows the effect on a dataset in which 15% of the labels were flipped.

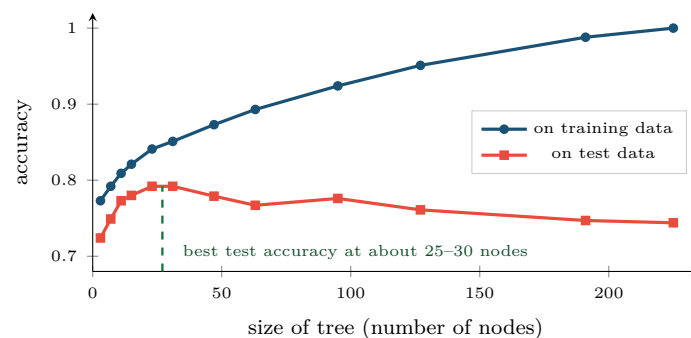


Figure 9.3: Accuracy against tree size on data with 15% label noise. Training accuracy climbs to 100% as the tree memorises the noise; test accuracy peaks early and then declines.

Pruning

Pre-pruning stops growth early: a maximum depth, a minimum number of examples per leaf, or a minimum gain. **Post-pruning** grows the full tree and then cuts it back:

- **Reduced-error pruning:** using a separate validation set, repeatedly replace the subtree whose removal most improves validation accuracy by a leaf labelled with its majority class; stop when every removal hurts.
- **Rule post-pruning** (C4.5): convert the tree into one rule per root-to-leaf path, drop any precondition whose removal does not hurt the rule’s estimated accuracy, and sort the rules by accuracy.
- **Cost-complexity pruning** (CART): minimise $\text{error} + \alpha \times (\text{number of leaves})$; larger α gives smaller trees, and α is chosen by cross-validation.

💡 Why Rules Are Easier to Prune Than Trees

Pruning a tree removes a node and everything below it — a decision about the root's test affects every path through it. Converting to rules first lets each path be simplified on its own: the test Requirements = Changing can be dropped from one rule and kept in another. It also produces a list of plain if-then statements, which is what a project manager actually wants to read — and can turn straight into a risk checklist.

9.5 The Inductive Bias of ID3

ID3 searches a *complete* hypothesis space — every discrete function is some tree — so, unlike CANDIDATE-ELIMINATION, it does not restrict what can be learned. Its bias lies in its *search*: it is greedy, never backtracks, and so prefers **shorter trees** and trees that put **high-gain attributes near the root**. That is a **preference bias** (an ordering over hypotheses), where CANDIDATE-ELIMINATION's was a **restriction bias** (a limit on which hypotheses exist). Its justification is a form of **Occam's razor**: there are far fewer short trees than long ones, so a short tree that fits the data is less likely to fit it by coincidence.

⚠ Common Pitfalls

- **Letting the tree grow until the leaves are pure** on noisy data. It will fit the noise; prune, or cap the depth.
- **Keeping an ID-like attribute**. It has maximal information gain and no predictive value. Drop it, or use the gain ratio.
- **Reading the root attribute as “the most important feature”**. Greedy splitting on correlated features picks one almost arbitrarily.
- **Expecting stability**. A small change in the data can change the root and hence the whole tree — the high variance that Chapter 14 exploits.
- **Scaling features for a tree**. Unnecessary: a threshold test is unchanged by any monotone rescaling.

🔧 Try It Yourself: ID3 by Hand, Then Pruning by Cross-Validation

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, export_text

# --- 1. ID3's first choice, computed by hand -----
rows = ["Changing High High Few No", "Changing High High Many No",
        "Frozen High High Few Yes", "Vague Medium High Few Yes",
        "Vague Low Normal Few Yes", "Vague Low Normal Many No",
        "Frozen Low Normal Many Yes", "Changing Medium High Few No",
        "Changing Low Normal Few Yes", "Vague Medium Normal Few Yes",
        "Changing Medium Normal Many Yes", "Frozen Medium High Many Yes",
        "Frozen High Normal Few Yes", "Vague Medium High Many No"]
df = pd.DataFrame([r.split() for r in rows],
                  columns=["Reqs", "Pressure", "Load", "Deps", "Met"])

def entropy(labels):
    p = labels.value_counts(normalize=True)
    return float(-(p * np.log2(p)).sum())
```

```

def gain(data, attr):
    rest = sum(len(g) / len(data) * entropy(g.Met)
               for _, g in data.groupby(attr))
    return entropy(data.Met) - rest

for a in ["Reqs", "Pressure", "Load", "Deps"]:
    print(f"Gain(S, {a:8s}) = {gain(df, a):.3f}")
changing = df[df.Reqs == "Changing"]
print("inside Changing:", {a: round(gain(changing, a), 3)
                           for a in ["Pressure", "Load", "Deps"]})

# --- 2. noisy data: a tree grown without limit, then pruned -----
X, y = make_classification(n_samples=1500, n_features=10, n_informative=4,
                          flip_y=0.15, random_state=4) # 15% labels flipped
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5,
                                          random_state=0)
full = DecisionTreeClassifier(criterion="entropy", random_state=0)
full.fit(X_tr, y_tr)
print(f"unpruned: {full.tree_.node_count:3d} nodes, "
      f"train {full.score(X_tr, y_tr):.3f}, "
      f"test {full.score(X_te, y_te):.3f}")

# cost-complexity pruning: choose alpha by 5-fold CV on the training data
alphas = full.cost_complexity_pruning_path(X_tr, y_tr).ccp_alphas[:-1]
cv = [cross_val_score(DecisionTreeClassifier(criterion="entropy",
      ccp_alpha=a, random_state=0), X_tr, y_tr, cv=5).mean() for a in alphas]
best = alphas[int(np.argmax(cv))]
pruned = DecisionTreeClassifier(criterion="entropy", ccp_alpha=best,
                              random_state=0).fit(X_tr, y_tr)
print(f"pruned: {pruned.tree_.node_count:3d} nodes, "
      f"train {pruned.score(X_tr, y_tr):.3f}, "
      f"test {pruned.score(X_te, y_te):.3f} (alpha = {best:.4f})")
print(export_text(pruned, max_depth=1))

```

Part 1 prints 0.247 for Requirements and 0.152 for Load where the worked example has 0.246 and 0.151: the code does not round the intermediate entropies. Part 2 stands in for 1,500 past sprints described by ten numeric features, 15% of them recorded with the wrong outcome. Pruning cuts the tree from 207 to 63 nodes and *raises* test accuracy from 0.72 to 0.75 while lowering training accuracy — the signature of removing overfitting.

🔍 Checkpoint — Test Yourself

1. A node holds 8 positive and 8 negative examples. A test splits it into (6+, 2−) and (2+, 6−). Compute the information gain.
2. Why does information gain favour an attribute such as *TaskID*? What fixes it?
3. What is the difference between a restriction bias and a preference bias? Which does ID3 have?

📋 Chapter Summary

- A decision tree is a disjunction of conjunctions of attribute tests; it can represent any discrete function and reads as a set of rules.
- ID3 grows the tree greedily, choosing at each node the attribute with the greatest information gain; on the sprint data, Requirements (0.246) wins at the root.
- Numeric attributes become threshold tests at class boundaries; the gain ratio corrects the preference for many-valued attributes; Gini behaves like entropy.

- Grown to purity on noisy data, trees overfit. Pre-prune with depth or leaf-size limits, or post-prune by reduced error, rules or cost complexity, choosing the strength by validation.
- ID3's inductive bias is a preference for short trees with high-gain attributes near the root — Occam's razor.

Review Questions

1. **(MCQ)** The entropy of a node with 4 positive and 4 negative examples is: (a) 0 (b) 0.5 (c) 1 (d) 2.
2. **(MCQ)** C4.5's gain ratio divides information gain by: (a) the number of examples (b) the split information (c) the Gini index (d) the tree depth.
3. **(MCQ)** Cost-complexity pruning with a larger α gives: (a) a larger tree (b) a smaller tree (c) the same tree (d) a deeper tree.
4. **(Short)** Explain why a tree that classifies every training example correctly may be a poor model.
5. **(Short)** Describe reduced-error pruning and state what data it requires.
6. **(Short)** Why do decision trees not need feature scaling?
7. **(Applied)** Compute $\text{Gain}(S, \text{Pressure})$ for the full sprint data, showing the entropy of each of the three branches.
8. **(Applied)** On the Vague branch of the sprint data (S4, S5, S6, S10, S14), compute the gain of Load, Pressure and Dependencies, and confirm that ID3 chooses Dependencies.

Bayesian Learning and Naive Bayes

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Supervised Learning Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Apply** Bayes' theorem to hypotheses, and **find** the MAP and maximum likelihood hypotheses.
- **Explain** the Bayes optimal classifier and why it can disagree with the single most probable hypothesis.
- **Classify** an example with naive Bayes by hand, and **state** its conditional independence assumption.
- **Apply** Laplace smoothing, and **use** Gaussian naive Bayes for continuous features.
- **Build** a multinomial naive Bayes text classifier working in log-probabilities.

🔗 Before You Start

Chapter 2 (conditional probability, Bayes' theorem, the Gaussian, maximum likelihood) and Chapter 9 (the sprint data, used again here).

📖 Key Terms in This Chapter

prior • likelihood • posterior • evidence • MAP hypothesis • maximum likelihood hypothesis • Bayes optimal classifier • conditional independence • naive Bayes • zero-frequency problem • Laplace smoothing • m -estimate • Gaussian naive Bayes • bag of words • multinomial naive Bayes • log-probabilities

10.1 Learning as Updating Belief

The learners so far return one hypothesis and no statement of how sure they are. Bayesian learning treats every hypothesis as uncertain: it starts from a **prior** belief about which hypotheses are plausible, observes data, and computes a **posterior** belief. Two things follow that the earlier chapters could not offer. Prior knowledge can be put in explicitly, and every prediction comes with a probability. And even where Bayesian methods are too expensive to use directly, they give the standard against which other methods are judged.

10.2 Bayes' Theorem for Hypotheses

📖 Posterior, MAP and ML Hypotheses

For a hypothesis h and training data D :

$$P(h \mid D) = \frac{P(D \mid h) P(h)}{P(D)}$$

$P(h)$ is the **prior**, $P(D | h)$ the **likelihood**, $P(D)$ the **evidence** and $P(h | D)$ the **posterior**. The **maximum a posteriori** (MAP) hypothesis maximises the posterior; since $P(D)$ is the same for every h , it can be dropped:

$$h_{MAP} = \arg \max_{h \in H} P(D | h) P(h)$$

If every hypothesis is equally likely a priori, this reduces to the **maximum likelihood** hypothesis $h_{ML} = \arg \max_h P(D | h)$.

Worked Example: A Flagged Code Change

Two hypotheses: a code change contains a critical defect, or it is clean. 0.8% of changes contain one. A static-analysis tool flags 98% of the changes that do, and 3% of those that do not. The tool flags a change.

- $P(\text{flag} | \text{defect}) P(\text{defect}) = 0.98 \times 0.008 = 0.0078$
- $P(\text{flag} | \text{clean}) P(\text{clean}) = 0.03 \times 0.992 = 0.0298$

$h_{MAP} = \text{clean}$, because $0.0298 > 0.0078$. Normalising:

$$P(\text{defect} | \text{flag}) = \frac{0.0078}{0.0078 + 0.0298} = \mathbf{0.21}$$

A flag raises the probability from 0.8% to 21% — a twenty-six-fold increase — and it is still more likely than not that the change is clean. The maximum likelihood hypothesis, which ignores the prior, would say “defect” ($0.98 > 0.03$). This is the base-rate effect of Chapter 2, now read as a choice between hypotheses.

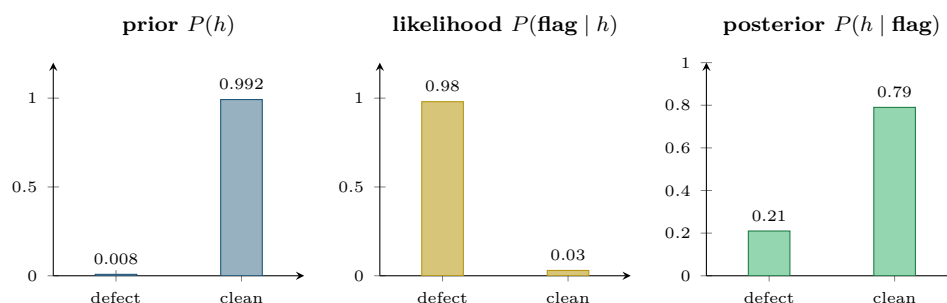


Figure 10.1: Prior times likelihood, normalised, gives the posterior. A flag that strongly favours “defect” is not enough to overturn a prior that strongly favours “clean”.

Least Squares Is a MAP Hypothesis in Disguise

If the training labels are the true function plus Gaussian noise and all hypotheses are equally likely a priori, the maximum likelihood hypothesis is the one that minimises the sum of squared errors — exactly the least-squares fit of Chapter 5. Put a Gaussian prior on the weights as well, favouring small ones, and the MAP hypothesis is the ridge regression of Chapter 7. Regularisation is a prior.

10.3 The Bayes Optimal Classifier

The MAP hypothesis answers “which hypothesis is most probable?”. That is not the same question as “which classification is most probable?”. The **Bayes optimal classifier** answers the second by letting every hypothesis vote, weighted by its posterior:

$$v_{\text{opt}} = \arg \max_v \sum_{h \in H} P(v | h) P(h | D)$$

No other method using the same hypothesis space and prior can do better on average — hence “optimal”. It is rarely computable, because H is too large to sum over, but it is the benchmark every approximation aims at.

Worked Example: When the Most Probable Hypothesis Is Outvoted

Three hypotheses remain with posteriors $P(h_1 | D) = 0.4$, $P(h_2 | D) = 0.3$ and $P(h_3 | D) = 0.3$. A new instance is classified positive by h_1 and negative by h_2 and h_3 .

- **MAP:** h_1 is the most probable hypothesis, so “positive”.
- **Bayes optimal:** $P(+) = 0.4$ and $P(-) = 0.3 + 0.3 = 0.6$, so “**negative**”.

The single best hypothesis is outvoted by the combined weight of the others. The same idea — let many hypotheses vote — is the basis of the ensembles of Chapter 14.

10.4 Naive Bayes

For classification with attributes $a_1, \dots, a_d$, the most probable class is

$$v_{MAP} = \arg \max_v P(v) P(a_1, a_2, \dots, a_d | v)$$

The prior $P(v)$ is easy to estimate: count. The joint likelihood is not: there are as many combinations of attribute values as there are possible examples, and most never appear in the training data. Naive Bayes removes the problem with one bold assumption.

The Naive Bayes Classifier

Assume the attributes are **conditionally independent** given the class: $P(a_1, \dots, a_d | v) = \prod_j P(a_j | v)$. Then

$$v_{NB} = \arg \max_v P(v) \prod_{j=1}^d P(a_j | v)$$

Each $P(a_j | v)$ is estimated from counts: the fraction of class- v training examples that have value a_j . Training is a single pass of counting.

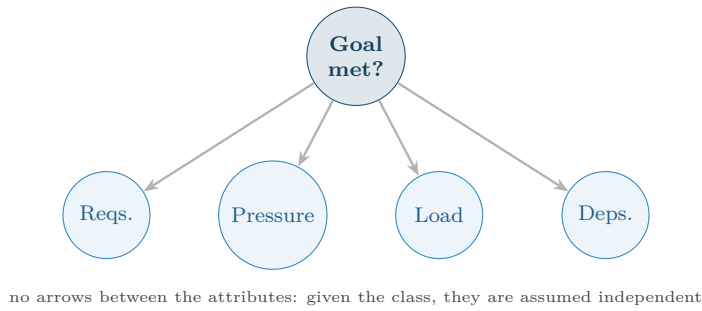
Worked Example: Naive Bayes on the Sprint Data

Will a new sprint with ⟨Changing requirements, Low pressure, High load, Many dependencies⟩ meet its goal? Use the fourteen sprints of Chapter 9.

Priors: $P(\text{Yes}) = 9/14 = 0.643$; $P(\text{No}) = 5/14 = 0.357$.

Conditionals, counted within each class:

- Yes: $P(\text{Changing} | \text{Yes}) = 2/9$, $P(\text{Low} | \text{Yes}) = 3/9$, $P(\text{High load} | \text{Yes}) = 3/9$, $P(\text{Many} | \text{Yes}) = 3/9$.
- No: $P(\text{Changing} | \text{No}) = 3/5$, $P(\text{Low} | \text{No}) = 1/5$, $P(\text{High load} | \text{No}) = 4/5$, $P(\text{Many} | \text{No}) = 3/5$.



What is estimated. One prior $P(v)$ per class, and one table $P(a_j | v)$ per attribute and class. For the sprint data that is 2 priors and $2 \times (3 + 3 + 2 + 2) = 20$ conditional probabilities — instead of a joint distribution over all 36 combinations of attribute values for each class.

Figure 10.2: The naive Bayes assumption drawn as a network: the class causes each attribute, and the attributes do not influence each other once the class is known.

Scores:

$$\text{Yes: } 0.643 \times \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{3}{9} = 0.0053 \quad \text{No: } 0.357 \times \frac{3}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5} = 0.0206$$

Prediction: No. Normalised, $P(\text{No} | x) = 0.0206 / (0.0206 + 0.0053) = \mathbf{0.795}$. (The decision tree of Chapter 9 agrees: changing requirements and a high load means the goal is missed.)

10.5 The Zero-Frequency Problem and Laplace Smoothing

Classify $\langle \text{Frozen}, \text{High}, \text{High}, \text{Many} \rangle$. No training sprint that missed its goal had Frozen requirements, so $P(\text{Frozen} | \text{No}) = 0/5 = 0$, and the whole product for No is zero — however strongly a high load and many dependencies point towards No. One unseen attribute value silences all the others.

Laplace Smoothing and the m -Estimate

Add one imaginary example of every value:

$$P(a_j = v | c) = \frac{n_{v,c} + 1}{n_c + k_j}$$

where $n_{v,c}$ counts class- c examples with that value, n_c counts class- c examples, and k_j is the number of values attribute j can take.

Mitchell's more general **m -estimate**, $(n_{v,c} + mp) / (n_c + m)$, adds m imaginary examples distributed according to a prior estimate p ; Laplace smoothing is the case $m = k_j$, $p = 1/k_j$.

Worked Example: Smoothing Rescues the Frozen Sprint

With Laplace smoothing (Requirements and Pressure have 3 values, Load and Dependencies 2):

- Yes: $0.643 \times \frac{4+1}{9+3} \times \frac{2+1}{9+3} \times \frac{3+1}{9+2} \times \frac{3+1}{9+2} = 0.643 \times 0.417 \times 0.250 \times 0.364 \times 0.364 = 0.0089$
- No: $0.357 \times \frac{0+1}{5+3} \times \frac{2+1}{5+3} \times \frac{4+1}{5+2} \times \frac{3+1}{5+2} = 0.357 \times 0.125 \times 0.375 \times 0.714 \times 0.571 = 0.0068$

$P(\text{Yes} | x) = 0.0089 / (0.0089 + 0.0068) = \mathbf{0.56}$. Without smoothing the answer was “Yes with certainty”; with it, a narrow Yes that reflects the conflict between Frozen requirements (always Yes in training) and a high load with many dependencies (usually No).

10.6 Continuous Features: Gaussian Naive Bayes

For a numeric attribute, counting values is useless — every value is new. Instead model each attribute within each class as a Gaussian, with the class's own mean $\mu_{j,c}$ and standard deviation $\sigma_{j,c}$ estimated from the training data, and use the density as the likelihood:

$$P(a_j | c) \propto \frac{1}{\sqrt{2\pi} \sigma_{j,c}} \exp\left(-\frac{(a_j - \mu_{j,c})^2}{2\sigma_{j,c}^2}\right)$$

Worked Example: One Feature, Two Classes, a Strong Prior

Modules that pass QA see 500 lines of code changed per week on average (sd 100); modules that go on to fail QA see 900 (sd 150); 5% of modules fail. This week a module had 750 lines changed.

- Pass: density $\frac{1}{100\sqrt{2\pi}} e^{-(250)^2/(2 \cdot 100^2)} = 0.00399 \times e^{-3.125} = 0.000175$
- Fail: density $\frac{1}{150\sqrt{2\pi}} e^{-(150)^2/(2 \cdot 150^2)} = 0.00266 \times e^{-0.5} = 0.00161$

The likelihood favours failure by $0.00161/0.000175 = 9.2$ to 1. Now the priors:

$$P(\text{fail} | 750) = \frac{0.05 \times 0.00161}{0.05 \times 0.00161 + 0.95 \times 0.000175} = \frac{0.0000807}{0.000247} = \mathbf{0.33}$$

Nine-to-one evidence against nineteen-to-one prior odds leaves the module more likely to pass. Figure 10.3 shows the whole picture: the boundary is at 772 lines.

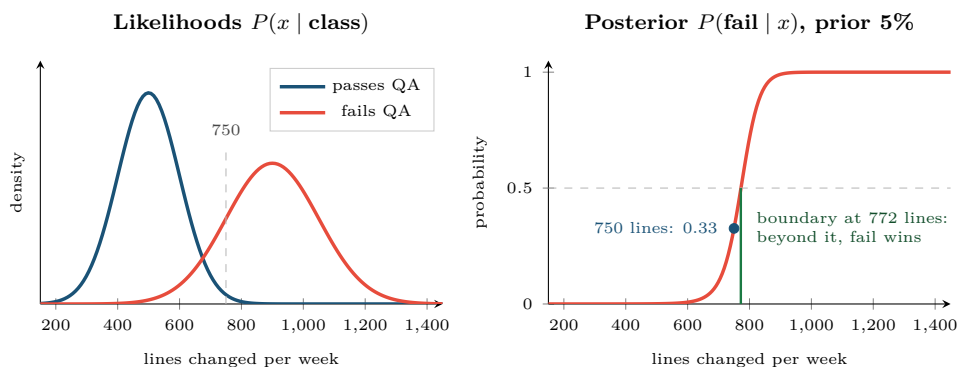


Figure 10.3: Left: the two class-conditional densities; at 750 lines the fail density is nine times higher. Right: after weighting by the 5% prior, the posterior crosses 0.5 only at 772 lines. The boundary sits beyond the midpoint of the means because the rare class needs more evidence.

10.7 Text Classification

Naive Bayes's most famous success is text: routing tickets, labelling commit messages, spotting duplicate bug reports. A document is represented as a **bag of words** — the count of each vocabulary word, order ignored — and **multinomial naive Bayes** treats each word occurrence as an independent draw from a class-specific distribution over the vocabulary:

$$P(w | c) = \frac{\text{count}(w, c) + 1}{\text{total words in } c + |V|},$$

$$v_{NB} = \arg \max_c \left[\log P(c) + \sum_{\text{words } w \text{ in doc}} \log P(w | c) \right]$$

Working in logarithms turns the product into a sum; a product of a few hundred probabilities would otherwise underflow to zero on a computer.

📊 Worked Example: A Four-Word Ticket Router

Training counts over a vocabulary of $|V| = 4$ words, from bug reports and feature requests; both classes contain 40 words and are equally likely:

	crash	error	add	option
bug	20	15	2	3
feature	2	1	18	19

With smoothing, $P(\text{crash} \mid \text{bug}) = 21/44$, $P(\text{add} \mid \text{bug}) = 3/44$, $P(\text{crash} \mid \text{feature}) = 3/44$, $P(\text{add} \mid \text{feature}) = 19/44$.

Ticket: “crash add crash”.

- bug: $\log \frac{1}{2} + 2 \log \frac{21}{44} + \log \frac{3}{44} = -0.693 - 1.479 - 2.686 = -4.858$
- feature: $\log \frac{1}{2} + 2 \log \frac{3}{44} + \log \frac{19}{44} = -0.693 - 5.371 - 0.840 = -6.904$

Bug wins by 2.05 in log-probability, a likelihood ratio of $e^{2.05} \approx 7.7$, so $P(\text{bug}) = 7.7/8.7 = 0.885$. The two “crash”es outweigh the one “add” — which also shows the router’s weakness: a feature request that happens to mention crashes is sent to the bug queue.

⚠️ Why Naive Bayes Works When Its Assumption Is False

Words are obviously not independent — “stack” and “trace” travel together. Naive Bayes therefore double-counts correlated evidence and its probabilities are badly *over-confident*. But classification needs only the right *ranking* of classes, and the double-counting usually inflates the right class as much as it helps it. Use naive Bayes’s decisions freely; do not trust its probabilities without calibration. Models that represent the dependencies explicitly — Bayesian networks — are the next step, and are covered in Mitchell’s Chapter 6.

❗ Common Pitfalls

- **Forgetting to smooth.** One zero count vetoes the class.
- **Multiplying hundreds of probabilities.** Add log-probabilities.
- **Confusing MAP with ML.** Maximum likelihood ignores the prior, and with rare classes the prior decides.
- **Reporting naive Bayes probabilities as calibrated risks.** They are too extreme; the ranking is what is reliable.
- **Feeding Gaussian NB heavily skewed features.** Transform them (log) first, or the Gaussian fits badly.

🔧 Try It Yourself: Naive Bayes by Counting, Then by Library

```
import numpy as np
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

# --- 1. naive Bayes on the sprint data, by counting -----
sprints = [
    ("Changing High High Few", "No"),
    ("Changing High High Many", "No"),
    ("Frozen High High Few", "Yes"),
    ("Vague Medium High Few", "Yes"),
```

```

        ("Vague Low Normal Few", "Yes"), ("Vague Low Normal Many", "No"),
        ("Frozen Low Normal Many", "Yes"),
        ("Changing Medium High Few", "No"),
        ("Changing Low Normal Few", "Yes"),
        ("Vague Medium Normal Few", "Yes"),
        ("Changing Medium Normal Many", "Yes"),
        ("Frozen Medium High Many", "Yes"),
        ("Frozen High Normal Few", "Yes"),
        ("Vague Medium High Many", "No")]
X = [s.split() for s, _ in sprints]
y = [c for _, c in sprints]
n_values = [3, 3, 2, 2]                                # values per attribute

def naive_bayes(x, laplace=0):
    scores = {}
    for c in ("Yes", "No"):
        rows = [r for r, lab in zip(X, y) if lab == c]
        p = len(rows) / len(X)                          # prior P(c)
        for j, v in enumerate(x):                       # P(x_j | c)
            count = sum(r[j] == v for r in rows)
            p *= (count + laplace) / (len(rows) + laplace * n_values[j])
        scores[c] = p
    total = sum(scores.values())
    return {c: round(s / total, 3) for c, s in scores.items()}, scores

post, raw = naive_bayes(["Changing", "Low", "High", "Many"])
print("unnormalised:", {c: round(s, 5) for c, s in raw.items()})
print("posterior: ", post)
frozen = ["Frozen", "High", "High", "Many"]
print("Frozen sprint, no smoothing:", naive_bayes(frozen)[1])
print("Frozen sprint, Laplace      :", naive_bayes(frozen, laplace=1)[0])

# --- 2. Gaussian naive Bayes on the code-churn example -----
rng = np.random.default_rng(0)
churn = np.r_[rng.normal(500, 100, 1900),               # modules that pass QA
              rng.normal(900, 150, 100)]                # the 5% that fail
fails = np.r_[np.zeros(1900), np.ones(100)]
gnb = GaussianNB().fit(churn.reshape(-1, 1), fails)
print("learned means:", gnb.theta_.ravel().round(0))
print("P(fail | 750 lines):", gnb.predict_proba([[750]])[0, 1].round(2))
grid = np.arange(600, 1000).reshape(-1, 1)
print("boundary near", int(grid[gnb.predict(grid) == 1][0, 0]), "lines")

# --- 3. a ticket router from twelve tickets -----
tickets = ["app crashes when saving", "error on the export page",
           "crash after the update", "null pointer error in upload",
           "report page crashes on load", "error message on checkout",
           "add an option to export pdf", "please add a dark mode",
           "option to filter by date", "add support for csv import",
           "add a bulk edit option", "please add weekly summary"]
labels = [1] * 6 + [0] * 6                               # 1 = bug
vec = CountVectorizer()
nb = MultinomialNB(alpha=1.0).fit(vec.fit_transform(tickets), labels)
for t in ["crash after export", "add export option", "crash option crash"]:
    p = nb.predict_proba(vec.transform([t]))[0, 1]
    print(f"{t!r:22s} P(bug) = {p:.3f}")

```

Part 1 reproduces both worked examples. Part 2 samples modules from the two Gaussians of the worked example and lets `GaussianNB` estimate them; its answer is close to the 0.33

and 772 lines computed by hand. In part 3, `alpha` is the smoothing constant: set it to `1e-9` and classify “crash option crash” again.

🔍 Checkpoint — Test Yourself

- 1% of releases would break production. A pre-release check flags 90% of those and 5% of good releases. Give h_{MAP} and h_{ML} for a release the check has flagged.
- What exactly does naive Bayes assume, and what goes wrong in the probabilities when the assumption fails?
- Why must naive Bayes for text work with log-probabilities?

📋 Chapter Summary

- Bayesian learning updates a prior into a posterior with the likelihood; h_{MAP} maximises $P(D | h)P(h)$ and equals h_{ML} under a uniform prior.
- Least squares is maximum likelihood under Gaussian noise; ridge is MAP with a Gaussian prior on the weights.
- The Bayes optimal classifier weights every hypothesis’s vote by its posterior and can overrule the MAP hypothesis.
- Naive Bayes assumes attributes are independent given the class, which reduces training to counting; on the sprint data it predicts No with 0.795.
- Smooth the counts (Laplace, m -estimate) to avoid zero vetoes; use Gaussian likelihoods for continuous features.
- For text, use a bag of words and log-probabilities. Trust the ranking, not the probabilities.

🔧 Review Questions

1. (MCQ) With a uniform prior over hypotheses, h_{MAP} equals: (a) h_{ML} (b) the Bayes optimal classifier (c) the most specific hypothesis (d) none of these.
2. (MCQ) Laplace smoothing is used to: (a) speed up training (b) prevent zero probabilities for unseen values (c) normalise the posterior (d) remove correlated features.
3. (MCQ) In Gaussian naive Bayes, each class–feature pair is described by: (a) a count table (b) a mean and a variance (c) a threshold (d) a weight.
4. (Short) Explain the difference between the MAP hypothesis and the Bayes optimal classification, with an example.
5. (Short) Why can naive Bayes classify well even when its probabilities are poor?
6. (Short) Describe how a user could word a feature request so that a naive Bayes router sends it to the faster bug queue.
7. (Applied) Using the sprint data and Laplace smoothing, classify ⟨Vague, High, Normal, Few⟩. Show every probability.
8. (Applied) For the Gaussian example, find the weekly churn at which the two prior-weighted densities are equal, to the nearest 10 lines, by trying values. Explain why it is not 700, the midpoint of the means.

Instance-Based Learning: k -Nearest Neighbours

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Supervised Learning Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** lazy (instance-based) from eager learning.
- **Classify** an example with k -nearest neighbours by hand, with and without distance weighting.
- **Choose** a distance measure, and **explain** why features must be scaled first.
- **Select** k by cross-validation, relating small and large k to variance and bias.
- **Explain** the curse of dimensionality and the computational cost of nearest-neighbour search.

🔗 Before You Start

Chapter 2 (Euclidean distance), Chapter 3 (standardisation) and Chapter 7 (bias, variance, cross-validation). The HEC outline lists k -NN under unsupervised learning; it is supervised — it needs labels to vote with — and is taught here with the other classifiers.

📖 Key Terms in This Chapter

instance-based learning • lazy vs. eager learning • k -nearest neighbours • majority vote • distance-weighted vote • Euclidean, Manhattan, Minkowski and Hamming distance • Voronoi cell • k -NN regression • locally weighted regression • curse of dimensionality • KD-tree • case-based reasoning

11.1 Learning by Remembering

Every learner so far has been **eager**: it studies the training data once, builds an explicit model — a line, a tree, a table of probabilities — and then discards the data. **Instance-based** learning is **lazy**. It stores the training examples and does nothing more until asked to classify a new one; only then does it look for the stored examples most similar to the query and let them decide.

The idea is old and intuitive: to predict whether a new project will finish on time, find the past projects most like it and see how they did. Its strengths follow directly. There is no training time, the model can be as complex as the data (the decision boundary follows whatever shape the examples trace), and new examples are added by simply storing them. Its weaknesses follow too: every prediction requires searching the stored data, and “most similar” is only as good as the distance measure.

11.2 The k -Nearest Neighbour Algorithm

k-Nearest Neighbours

To classify a query $\mathbf{x}_q$:

1. compute its distance to every stored training example;
2. take the k examples with the smallest distances;
3. predict the most common class among them (for regression, the mean of their target values).

Its inductive bias is *smoothness*: the classification of an instance is most similar to the classifications of instances nearby in the feature space.

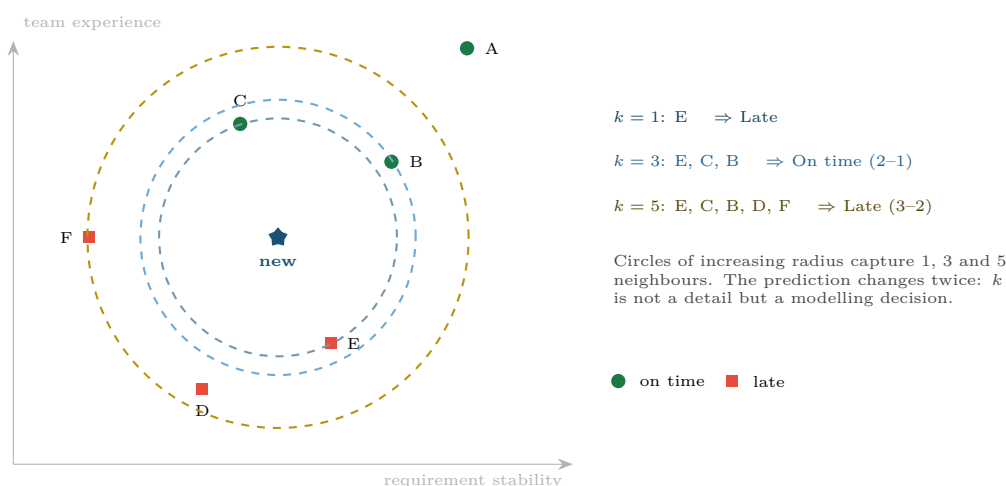


Figure 11.1: Six past projects and one new project. Which class the new project receives depends on how many neighbours are asked. (Axes scaled so that 10 points = one unit.)

Worked Example: Three Values of k , Three Answers

Six past projects, each scored from 0 to 100 for requirement stability and for the team's experience with the technology: A (90, 85) On time; B (80, 70) On time; C (60, 75) On time; D (55, 40) Late; E (72, 46) Late; F (40, 60) Late. Classify the new project Q (65, 60). Both features are on the same 0–100 scale, so no rescaling is needed.

Distances from Q, sorted:

- E: $\sqrt{7^2 + 14^2} = \sqrt{245} = 15.65$ (Late)
- C: $\sqrt{5^2 + 15^2} = \sqrt{250} = 15.81$ (On time)
- B: $\sqrt{15^2 + 10^2} = \sqrt{325} = 18.03$ (On time)
- D: $\sqrt{10^2 + 20^2} = \sqrt{500} = 22.36$ (Late)
- F: $\sqrt{25^2 + 0^2} = 25.00$ (Late)
- A: $\sqrt{25^2 + 25^2} = \sqrt{1250} = 35.36$ (On time)

$k = 1: E \Rightarrow \text{Late}$. $k = 3: E, C, B \Rightarrow \text{On time, 2 to 1}$. $k = 5: \text{add D, F} \Rightarrow \text{Late, 3 to 2}$.

Distance-weighted, $k = 5$. Weight each vote by $1/d^2$: Late = $\frac{1}{245} + \frac{1}{500} + \frac{1}{625} = 0.00768$; On time = $\frac{1}{250} + \frac{1}{325} = 0.00708$. **Late**, narrowly — E, the nearest neighbour, counts for more than F, the farthest.

11.3 Distance and Scale

Distance Measures

For vectors $\mathbf{x}$ and $\mathbf{z}$ with d features:

- **Euclidean:** $\sqrt{\sum_j (x_j - z_j)^2}$ — the straight-line default.
- **Manhattan:** $\sum_j |x_j - z_j|$ — less dominated by one large difference.
- **Minkowski:** $(\sum_j |x_j - z_j|^p)^{1/p}$ — Manhattan at $p = 1$, Euclidean at $p = 2$.
- **Hamming:** the number of attributes that differ — for categorical data.
- **Cosine distance:** $1 - \cos \theta$ — for text and other data where direction matters more than length (Chapter 2).

Always Scale Before Measuring Distance

Add the budget, in thousands of dollars (50–2,000), to the worked example, and a difference of a few hundred outweighs every difference in the two scores. The distance is decided by whichever feature has the largest units. Standardise first (Chapter 3). On the projects of this chapter's lab, 5-NN makes 24.5% cross-validated errors on raw features and 13.0% after standardising — the same algorithm, about half the error.

A second, subtler problem is **irrelevant features**. k -NN uses every feature equally, so twenty features of which two matter give a distance that is mostly noise. Decision trees choose which attributes to test; k -NN cannot, unless the features are selected or weighted beforehand — Mitchell's suggestion is to stretch each axis by a weight chosen by cross-validation.

11.4 Choosing k

The decision boundary of 1-NN runs between every pair of differently labelled neighbours: each training example owns a **Voronoi cell**, the region of space closer to it than to any other example, and the boundary is made of cell edges. It follows every stray example — low bias, high variance. Increasing k averages over more neighbours and smooths the boundary; with $k = n$ it predicts the majority class everywhere — high bias, zero variance. k is the regularisation knob of Chapter 7, and it is chosen the same way.

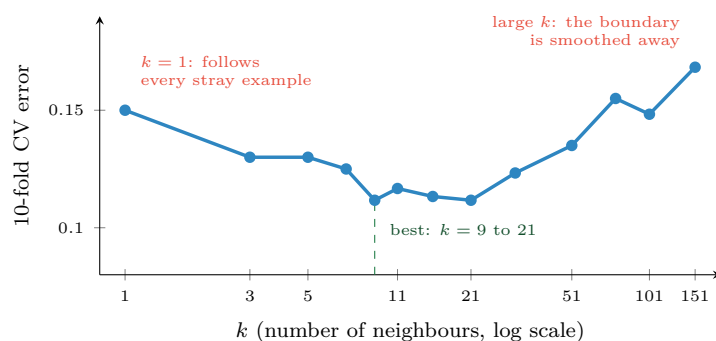


Figure 11.2: Cross-validated error of k -NN on the (standardised) project data of this chapter's lab. Small k has high variance, large k high bias; the familiar U-shape.

Two practical rules: for two classes use an odd k , so votes cannot tie; and treat $\sqrt{n}$ as a rough upper starting point, then let cross-validation decide.

11.5 Beyond Classification

k -NN regression predicts the mean (or distance-weighted mean) of the neighbours' target values. **Locally weighted regression**, in Mitchell's treatment, goes further: for each query it fits a small linear model to the nearby examples only, weighted by their distance, and uses that model's prediction. The global function is approximated by a patchwork of local ones — a model that could never be written down in advance.

Case-based reasoning applies the same lazy idea to rich symbolic descriptions: a help-desk system that retrieves the most similar past incident and adapts its solution. Similarity there is measured by matching structure, not by a formula.

11.6 The Curse of Dimensionality

Nearest-neighbour methods rest on the idea that the nearest examples are meaningfully closer than the rest. In high dimensions that stops being true.

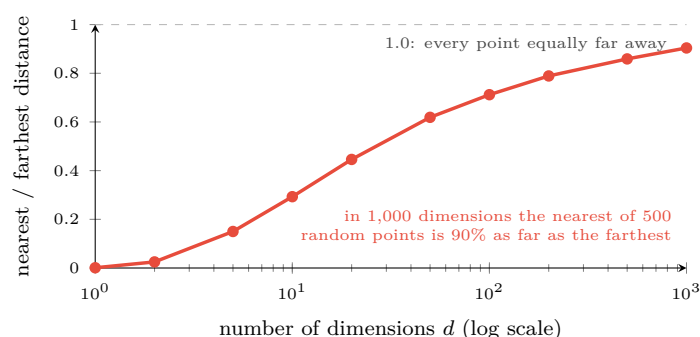


Figure 11.3: The curse of dimensionality. For 500 uniformly random points, the ratio of the nearest to the farthest distance from a query rises towards 1 as dimensions are added: “nearest” stops meaning anything.

💡 Why Distances Concentrate

Each extra dimension adds another independent term to the sum under the square root. With many terms, the sums for different pairs of points all come out close to their average — the same reason an average of many coin tosses is close to one half. So every point ends up at nearly the same distance from every other, and the nearest neighbour is barely nearer than a random one. The remedies are fewer dimensions (feature selection, or PCA in Chapter 16) or a learned notion of similarity.

11.7 The Cost of Being Lazy

Training is free; prediction is not. A naive search compares the query with all n stored examples in d dimensions, $O(nd)$ work per query — a million examples means a million distance computations for every prediction. **KD-trees** and ball trees partition space so that most examples can be ruled out without computing their distance, which works well up to perhaps twenty dimensions; beyond that, approximate nearest-neighbour indexes trade a little accuracy for large speed-ups. The whole training set must also be kept, which on a small device may be impossible.

⚠ Common Pitfalls

- **Forgetting to scale.** The feature with the biggest units decides everything.
- **Using $k = 1$ on noisy data.** Every mislabelled example claims a region of its own.
- **Using an even k for two classes.** Ties must then be broken arbitrarily.

- **Keeping irrelevant features.** They add noise to every distance; select or weight features first.
- **Ignoring prediction cost.** Fine for thousands of examples, slow for millions without an index.

Try It Yourself: Neighbours, Scaling and the Choice of k

```
import numpy as np
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# --- 1. the worked example -----
X = np.array([[90, 85], [80, 70], [60, 75], [55, 40], [72, 46], [40, 60]])
y = np.array(["OnTime", "OnTime", "OnTime", "Late", "Late", "Late"])
q = np.array([[65, 60]])
for k in (1, 3, 5):
    plain = KNeighborsClassifier(k).fit(X, y).predict(q)[0]
    weighted = KNeighborsClassifier(k, weights="distance").fit(X, y)
    print(f"k={k}: vote {plain}, distance-weighted {weighted.predict(q)[0]}")

# --- 2. scaling matters -----
rng = np.random.default_rng(11)
n = 600
Xb = np.column_stack([rng.uniform(20, 100, n),          # past projects
                      rng.uniform(20, 100, n),          # requirement stability
                      rng.uniform(50, 2000, n),          # team experience
                      rng.uniform(4, 52, n)])            # budget, thousand USD
yb = (Xb[:, 0] * Xb[:, 1] < 2500).astype(int)           # planned weeks
flip = rng.random(n) < 0.05                             # 1 = late
yb[flip] = 1 - yb[flip]                                 # 5% recorded wrongly
raw = cross_val_score(KNeighborsClassifier(5), Xb, yb, cv=10).mean()
scaled = cross_val_score(make_pipeline(StandardScaler(),
                                       KNeighborsClassifier(5)),
                        Xb, yb, cv=10).mean()
print(f"5-NN accuracy: raw {raw:.3f}, standardised {scaled:.3f}")

# --- 3. choose k by cross-validation -----
pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())
grid = GridSearchCV(pipe, {"kneighborsclassifier__n_neighbors":
                          [1, 3, 5, 7, 11, 21, 51, 101]}, cv=10)
grid.fit(Xb, yb)
for k, s in zip([1, 3, 5, 7, 11, 21, 51, 101],
                grid.cv_results_["mean_test_score"]):
    print(f" k={k:3d}: CV accuracy {s:.3f}")

# --- 4. the curse of dimensionality -----
rng = np.random.default_rng(0)
for dim in (2, 10, 100, 1000):
    P, Q = rng.uniform(size=(500, dim)), rng.uniform(size=(50, dim))
    D = np.sqrt(((Q[:, None, :] - P[None, :, :]) ** 2).sum(-1))
    ratio = (D.min(1) / D.max(1)).mean()
    print(f"d={dim:4d}: nearest/farthest = {ratio:.2f}")
```

In part 2 a project is late when stability times experience falls below 2,500; budget and planned weeks are irrelevant, but budget has by far the largest units. scikit-learn's `weights="distance"` uses $1/d$ rather than the $1/d^2$ of the worked example; here both give the same answers. Add ten columns of random noise to the project features and rerun part 3: what happens to the best accuracy?

? Checkpoint — Test Yourself

1. In the worked example, what does 3-NN predict for a new project at (50, 55)?
2. Why is k -NN called a lazy learner, and what does laziness cost at prediction time?
3. As k grows from 1 to n , what happens to bias and to variance?

✓ Chapter Summary

- k -NN stores the training data and classifies a query by a vote of its k nearest neighbours (the mean, for regression). Its bias is smoothness: nearby examples share labels.
- The prediction can change with k ; distance weighting lets closer neighbours count more.
- Scale features before computing distances, and remove irrelevant ones; choose the distance to suit the data.
- Small k means high variance, large k high bias; choose k by cross-validation, odd for two classes.
- In high dimensions all distances become similar, and nearest neighbours stop being meaningful.
- Training is free but each prediction searches the data; indexes such as KD-trees reduce the cost in low dimensions.

✍ Review Questions

1. **(MCQ)** k -NN is: (a) an eager learner (b) a lazy learner (c) an unsupervised method (d) a linear model.
2. **(MCQ)** Increasing k in k -NN generally: (a) raises variance (b) lowers bias (c) smooths the decision boundary (d) makes training slower.
3. **(MCQ)** The Manhattan distance between (1, 2) and (4, 6) is: (a) 5 (b) 7 (c) 25 (d) 12.
4. **(Short)** Explain the curse of dimensionality and its effect on k -NN.
5. **(Short)** Why does k -NN need feature scaling while a decision tree does not?
6. **(Short)** Describe locally weighted regression in your own words.
7. **(Applied)** Add a seventh project G (68, 58, On time) to the worked example. Recompute the 1-, 3- and 5-NN predictions for Q.
8. **(Applied)** A project dataset has features “team size” (2–60 people) and “budget” (100,000–2,000,000 rupees). Compute the Euclidean distance between (25, 400,000) and (55, 410,000) before and after min–max scaling, and comment.

Support Vector Machines

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Supervised Learning Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why the maximum-margin hyperplane is preferred among all separating hyperplanes, and **identify** its support vectors.
- **Compute** the margin of a linear SVM and **classify** a point with its decision function.
- **Describe** the soft-margin SVM and the role of C , and **compute** the hinge loss.
- **Explain** the kernel trick, and **verify** a polynomial kernel against its explicit feature map.
- **Tune** C and the RBF width γ by cross-validation.

🔗 Before You Start

Chapter 2 (dot products, norms), Chapter 6 (linear decision boundaries, the loss view of classification) and Chapter 7 (regularisation and tuning by cross-validation).

🔑 Key Terms in This Chapter

separating hyperplane • margin • maximum-margin classifier • support vectors • hard margin • soft margin • slack variable • C • hinge loss • dual form • feature map • kernel trick • polynomial kernel • RBF kernel • γ • support vector regression

12.1 Which Line?

When two classes can be separated by a straight line, they can usually be separated by infinitely many. Logistic regression picks the one that maximises likelihood; the perceptron of Chapter 13 picks whichever it reaches first. A **support vector machine** picks the line that stays as far as possible from the nearest examples of both classes — the one with the widest **margin**. The intuition is that a boundary with room to spare on both sides is the least likely to be crossed by small perturbations of new data, and the theory agrees: a wider margin gives a tighter bound on the error on unseen data, regardless of the number of features.

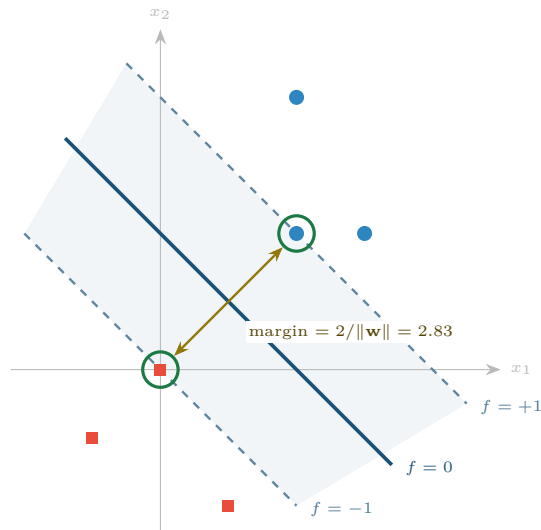
12.2 The Maximum-Margin Hyperplane

📖 Hyperplane, Margin and Support Vectors

A linear classifier predicts $\text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$ with labels $y \in \{-1, +1\}$. The distance of a point $\mathbf{x}$ from the hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$ is $|\mathbf{w} \cdot \mathbf{x} + b|/\|\mathbf{w}\|$. Scale $\mathbf{w}$ and b so that the closest points satisfy $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$; then the **margin** — the width of the empty band between the classes — is $2/\|\mathbf{w}\|$. The hard-margin SVM solves

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \text{for every } i$$

The training points that lie exactly on the edges of the band are the **support vectors**. They alone determine the solution: every other point could be moved or deleted, as long as it stayed outside the band, and the hyperplane would not change.



The worked example. Positives (blue) at (2, 2), (3, 2), (2, 4); negatives (red) at (0, 0), (-1, -1), (1, -2).

The widest band touches only (2, 2) and (0, 0) — the two **support vectors**, circled. The boundary is the line $x_1 + x_2 = 2$, halfway between them.

Delete any other point and the answer is unchanged.

Figure 12.1: The maximum-margin hyperplane. The solid line separates the classes; the dashed lines bound the widest empty band; the points on the dashed lines are the support vectors.

Worked Example: A Maximum-Margin Line by Hand

The six points of Figure 12.1. The closest opposite-class pair is (2, 2) and (0, 0), so the boundary is perpendicular to the segment between them: $\mathbf{w} = a(1, 1)$ for some $a > 0$.

Canonical scaling. Put the support vectors exactly on the margins:

$$\mathbf{w} \cdot (2, 2) + b = 4a + b = +1, \quad \mathbf{w} \cdot (0, 0) + b = b = -1$$

So $b = -1$, $a = 0.5$: $\mathbf{w} = (0.5, 0.5)$. The boundary is $0.5x_1 + 0.5x_2 - 1 = 0$, i.e. $x_1 + x_2 = 2$.
Margin: $\|\mathbf{w}\| = \sqrt{0.25 + 0.25} = 0.707$, so $2/\|\mathbf{w}\| = \mathbf{2.83}$ — exactly the distance from (0, 0) to (2, 2).

Check the constraints: (3, 2): $f = 1.5 \geq 1$; (2, 4): $f = 2 \geq 1$; (-1, -1): $f = -2 \leq -1$; (1, -2): $f = -1.5 \leq -1$. All satisfied.

Classify (1.5, 1): $f = 0.75 + 0.5 - 1 = 0.25 > 0$, so **positive**, at distance $0.25/0.707 = 0.35$ from the boundary — inside the margin band, so a point the SVM is not confident about. scikit-learn's SVC returns the same $\mathbf{w}$, b and support vectors (this chapter's lab).

12.3 Soft Margins and the Hinge Loss

Real data is rarely separable, and even when it is, one mislabelled point can drag the hard-margin boundary into a bad place. The **soft-margin** SVM lets points violate the margin at a price. Each point gets a **slack variable** $\xi_i \geq 0$ measuring how far it is on the wrong side of its margin:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$$

The hyperparameter C sets the price. A large C punishes every violation heavily: narrow margin, few violations, high variance. A small C tolerates violations to buy a wider margin: more bias, less variance. C is the SVM's regularisation knob, and it is tuned by cross-validation.

The Hinge Loss

At the optimum each slack equals $\max(0, 1 - y_i f(\mathbf{x}_i))$, so the soft-margin SVM is equivalent to minimising

$$\underbrace{\sum_i \max(0, 1 - y_i f(\mathbf{x}_i))}_{\text{hinge loss}} + \frac{1}{2C} \|\mathbf{w}\|^2$$

— a loss plus an L2 penalty, the same shape as regularised logistic regression with the cross-entropy replaced by the **hinge**. Points beyond the margin on the correct side cost nothing.

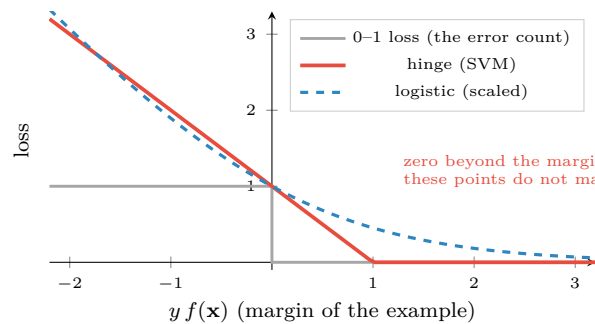


Figure 12.2: Three losses as a function of $yf(\mathbf{x})$, positive when the example is correctly classified. The hinge is exactly zero for points beyond the margin, which is why only support vectors affect the SVM; the logistic loss is never quite zero.

Worked Example: Hinge Losses

Four training points have margins $yf(\mathbf{x}) = 2, 1, 0.5, -0.5$:

- $yf = 2$: $\max(0, 1 - 2) = 0$ — well clear; irrelevant to the solution.
- $yf = 1$: $\max(0, 0) = 0$ — exactly on the margin; a support vector.
- $yf = 0.5$: **0.5** — correctly classified but inside the margin.
- $yf = -0.5$: **1.5** — misclassified.

The last three all influence the boundary; the first does not. With C large the optimiser will work hard to remove the last two losses; with C small it will accept them for a wider margin.

12.4 The Kernel Trick

Some problems no straight line can solve — one class surrounding another, say. The standard remedy (Chapter 5) is to add non-linear features, such as squares and products, and fit a linear model in the enlarged space. That works, but the enlarged space can be enormous: all products of pairs of 1,000 features is half a million features.

Kernel

A **kernel** is a function $K(\mathbf{x}, \mathbf{z})$ that equals the dot product $\phi(\mathbf{x}) \cdot \phi(\mathbf{z})$ in some **feature space** ϕ , but is computed without ever constructing ϕ . Because the SVM's optimisation and its decision function use the data only through dot products,

$$f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

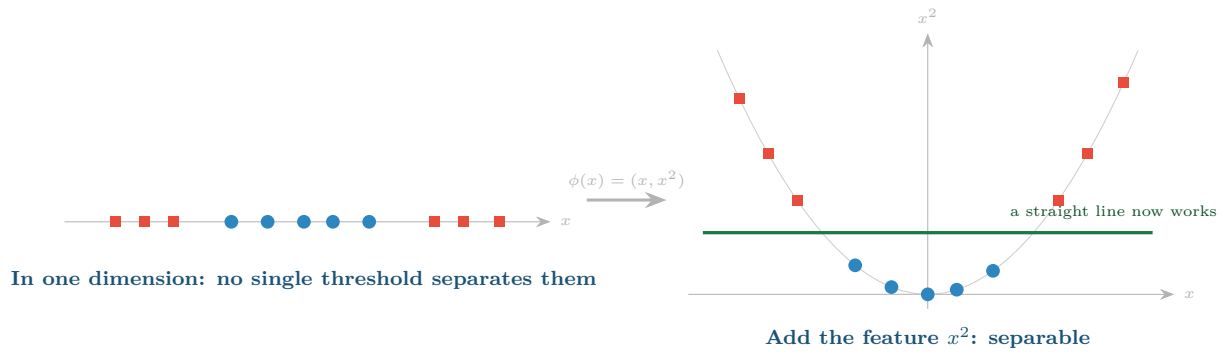


Figure 12.3: Lifting data into more dimensions. The inner class (blue) and the outer class (red) cannot be separated on the line, but after adding x^2 as a second feature a horizontal line separates them; in the original space that line is two thresholds.

replacing every dot product by a kernel fits a linear boundary in the feature space — a non-linear boundary in the original one — at the cost of computing K . This is the **kernel trick**. The sum runs over support vectors only: all other α_i are zero.

Worked Example: The Kernel Computes the Big Dot Product Cheaply

Take $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z})^2$ in two dimensions, and $\mathbf{x} = (1, 2)$, $\mathbf{z} = (3, 1)$.

The kernel: $\mathbf{x} \cdot \mathbf{z} = 3 + 2 = 5$, so $K = 25$. One dot product and one squaring.

The long way: this kernel corresponds to $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$:

$$\phi(1, 2) = (1, 2\sqrt{2}, 4), \quad \phi(3, 1) = (9, 3\sqrt{2}, 1)$$

$$\phi(\mathbf{x}) \cdot \phi(\mathbf{z}) = 9 + 2\sqrt{2} \cdot 3\sqrt{2} + 4 = 9 + 12 + 4 = \mathbf{25}$$

The same number. For degree p on d features the feature space has on the order of d^p dimensions; the kernel is still one dot product and one power.

Kernel	$K(\mathbf{x}, \mathbf{z})$	Use and settings
Linear	$\mathbf{x} \cdot \mathbf{z}$	Many features, text; fast. Tune C only
Polynomial	$(\gamma \mathbf{x} \cdot \mathbf{z} + r)^p$	Interactions of up to p features; tune p , C
RBF (Gaussian)	$\exp(-\gamma \ \mathbf{x} - \mathbf{z}\ ^2)$	The default for non-linear problems. Large γ : each point's influence is narrow, the boundary wiggles (variance). Small γ : smooth boundary (bias). Tune C and γ together

⚠ Scale the Features, Then Tune C and γ Together

The RBF kernel is a function of distance, so everything said about scaling in Chapter 11 applies. And C and γ interact — a larger γ often wants a smaller C — so search them as a grid, on a logarithmic scale (C in $\{0.1, 1, 10, 100\}$, γ in $\{0.001, 0.01, 0.1, 1\}$), by cross-validation. On the 600 projects of Chapter 11's lab this finds $C = 100$, $\gamma = 0.01$ and 93.2% accuracy — better than the best k -NN — using 183 of the projects as support vectors. C landed on the edge of the grid, so the next step would be to extend the grid upwards.

12.5 Practicalities

More than two classes. The SVM is inherently binary. `scikit-learn`'s `SVC` trains one classifier per pair of classes (one-vs-one) and lets them vote; `LinearSVC` uses one-vs-rest.

Probabilities. The decision function $f(\mathbf{x})$ is a distance, not a probability. `SVC(probability=True)` fits a sigmoid to it afterwards (Platt scaling), at extra cost.

Scale of data. Training cost grows between the square and the cube of the number of examples, so kernel SVMs suit datasets of up to tens of thousands of rows. Linear SVMs scale to millions.

Regression. **Support vector regression** ignores errors smaller than a tube of width ε around the prediction and charges linearly beyond it — the regression counterpart of the hinge.

Inductive bias. Among the hypotheses consistent with the data (up to the slack allowed), prefer the one with the largest margin.

! Common Pitfalls

- **Not scaling features.** The RBF kernel and the margin are both distances.
- **Tuning C and γ separately.** They interact; use a grid.
- **Using a huge C on noisy data.** The hard margin chases every mislabelled point.
- **Reading $f(\mathbf{x})$ as a probability.** It is a signed distance; calibrate if probabilities are needed.
- **Running a kernel SVM on a million rows.** Use a linear SVM, or a different model.

🔧 Try It Yourself: Margins, Kernels and a Tuning Grid

```
import numpy as np
from sklearn.svm import SVC
from sklearn.datasets import make_circles
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# --- 1. the worked example: a maximum-margin line -----
X = np.array([[2, 2], [3, 2], [2, 4], [0, 0], [-1, -1], [1, -2]])
y = np.array([1, 1, 1, -1, -1, -1])
svm = SVC(kernel="linear", C=1e6).fit(X, y)          # huge C = hard margin
w, b = svm.coef_[0], svm.intercept_[0]
print("w =", w.round(3), " b =", round(b, 3))
print("margin width 2/||w|| =", round(2 / np.linalg.norm(w), 3))
print("support vectors:", svm.support_vectors_.tolist())
print("f(1.5, 1) =", round(float(svm.decision_function([[1.5, 1]])[0]), 3))

# --- 2. a problem no line can solve -----
Xc, yc = make_circles(n_samples=400, noise=0.08, factor=0.4, random_state=0)
for kernel in ("linear", "poly", "rbf"):
    acc = cross_val_score(SVC(kernel=kernel, degree=2), Xc, yc, cv=5).mean()
    print(f"circles, {kernel:6s} kernel: accuracy {acc:.3f}")

# --- 3. tune C and gamma together, features scaled -----
rng = np.random.default_rng(11)                # the projects of Chapter 11
n = 600
Xb = np.column_stack([rng.uniform(20, 100, n),   # requirement stability
                      rng.uniform(20, 100, n),   # team experience
                      rng.uniform(50, 2000, n),   # budget, thousand USD
                      rng.uniform(4, 52, n)])     # planned weeks
yb = (Xb[:, 0] * Xb[:, 1] < 2500).astype(int)    # 1 = late
flip = rng.random(n) < 0.05
yb[flip] = 1 - yb[flip]
```

```

pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
grid = GridSearchCV(pipe, {"svc__C": [0.1, 1, 10, 100],
                             "svc__gamma": [0.001, 0.01, 0.1, 1]}, cv=5)

grid.fit(Xb, yb)
print("best:", grid.best_params_, "CV accuracy",
      round(grid.best_score_, 3))
print("support vectors used:", grid.best_estimator_[-1].n_support_.sum(),
      "of", len(yb))

```

In part 1, add the point (1, 1.5) with label +1 and rerun. It lies inside the old margin: which support vectors change, and what happens to the margin width?

Checkpoint — Test Yourself

1. A linear SVM has $\mathbf{w} = (3, 4)$ and $b = -5$. What is its margin width, and how is (1, 1) classified?
2. What happens to the margin and to training errors as C is increased?
3. Why can the kernel trick be applied to the SVM at all?

Chapter Summary

- Of all separating hyperplanes the SVM chooses the one with the widest margin, $2/\|\mathbf{w}\|$; it is determined by the support vectors alone.
- The soft-margin SVM allows violations priced by C , and is equivalent to hinge loss plus an L2 penalty. Large C : narrow margin, high variance.
- The hinge loss is zero beyond the margin, which is why only support vectors matter.
- Kernels compute dot products in a large feature space without building it; the RBF kernel is the default for non-linear problems.
- Scale the features and tune C and γ together on a logarithmic grid by cross-validation.

Review Questions

1. (MCQ) The support vectors of an SVM are: (a) all training points (b) the points on or inside the margin (c) the misclassified points only (d) the cluster centres.
2. (MCQ) With the RBF kernel, a very large γ tends to: (a) underfit (b) overfit (c) have no effect (d) make the model linear.
3. (MCQ) The hinge loss of a point with $yf(\mathbf{x}) = 0.3$ is: (a) 0 (b) 0.3 (c) 0.7 (d) 1.3.
4. (Short) Why is a wider margin expected to generalise better?
5. (Short) Compare the hinge loss with the logistic loss.
6. (Short) Explain the kernel trick without formulas.
7. (Applied) For support vectors (1, 3) (label +1) and (1, 1) (label -1), find $\mathbf{w}$, b and the margin of the maximum-margin line, and classify (4, 2.5).
8. (Applied) Verify that $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x} \cdot \mathbf{z})^2$ equals $\phi(\mathbf{x}) \cdot \phi(\mathbf{z})$ for $\mathbf{x} = (2, -1)$ and $\mathbf{z} = (1, 3)$.

Artificial Neural Networks

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Supervised Learning Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Compute** the output of a perceptron and **train** one with the perceptron rule.
- **Explain** linear separability and why a single perceptron cannot learn XOR.
- **Derive** the delta rule as gradient descent on squared error.
- **Perform** one forward and backward pass of backpropagation through a small multilayer network by hand.
- **Choose** activation functions and **apply** early stopping, weight decay and dropout against overfitting.

🔗 Before You Start

Chapter 2 (the chain rule, gradient descent), Chapter 6 (the sigmoid, the softmax, cross-entropy) and Chapter 7 (overfitting, regularisation).

📖 Key Terms in This Chapter

artificial neuron • perceptron • threshold unit • perceptron training rule • linear separability • XOR • delta rule • sigmoid unit • multilayer perceptron • hidden layer • forward pass • backpropagation • epoch • activation function • ReLU • early stopping • weight decay • dropout

13.1 From Neurons to Units

A biological neuron receives signals from thousands of others, and fires if their combined stimulation exceeds a threshold. Learning, roughly, changes the strength of the connections. An **artificial neural network** keeps just that much of the biology: many simple *units*, each computing a weighted sum of its inputs and passing it through a simple function, connected so that the outputs of some are the inputs of others. Everything the network knows is in the connection weights, and learning means adjusting them.

Networks are the right tool when the input is high-dimensional and raw — in project work, the full text of tickets and commit messages, or the complete event history of every task in a tracker — when the target function is complex and unknown, when some noise is acceptable, when long training times are acceptable, and when nobody needs to read the learned function. Networks of this kind, made deep, are the whole of deep learning.

13.2 The Perceptron

Perceptron

A **perceptron** computes a weighted sum of its inputs and thresholds it:

$$o(\mathbf{x}) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + \cdots + w_dx_d > 0 \\ 0 & \text{otherwise} \end{cases}$$

With a constant input $x_0 = 1$ the bias w_0 is just another weight, and the unit computes $\text{step}(\mathbf{w} \cdot \mathbf{x})$. Its decision boundary is the hyperplane $\mathbf{w} \cdot \mathbf{x} = 0$ — the same family as logistic regression and the linear SVM.

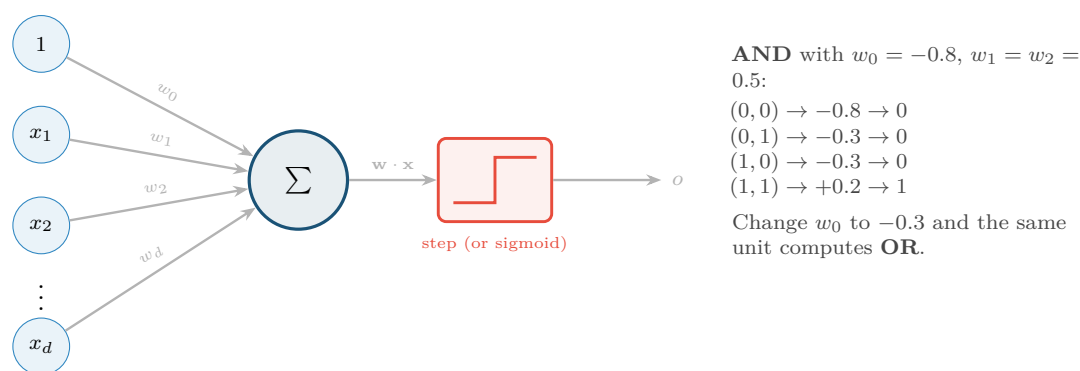


Figure 13.1: A perceptron: a weighted sum of the inputs, then a threshold. The bias is a weight on a constant input of 1. With the weights shown on the right it computes logical AND.

The Perceptron Training Rule

Start from small (or zero) weights. For each training example with target t and current output o , update every weight:

$$w_i \leftarrow w_i + \eta(t - o)x_i$$

Correct outputs change nothing; a missed positive pushes the weights towards the input, a false positive pushes them away. If the training examples are **linearly separable** — some hyperplane separates the classes perfectly — this procedure is guaranteed to find one in a finite number of steps, for any sufficiently small learning rate η .

Worked Example: Training a Perceptron to Compute AND

Inputs with bias $x_0 = 1$; weights (w_0, w_1, w_2) start at $(0, 0, 0)$; $\eta = 0.5$. The output is 1 only if $\mathbf{w} \cdot \mathbf{x} > 0$.

Epoch 1. $(0, 0)$, $(0, 1)$, $(1, 0)$: net = 0, output 0, target 0 — no change. $(1, 1)$: net 0, output 0, target 1 — update by $0.5 \times (1 - 0) \times (1, 1, 1)$: $\mathbf{w} = (0.5, 0.5, 0.5)$.

Epoch 2. $(0, 0)$: net 0.5, output 1, target 0 — subtract $0.5 \times (1, 0, 0)$: $\mathbf{w} = (0, 0.5, 0.5)$. $(0, 1)$: net 0.5, output 1 — subtract $0.5 \times (1, 0, 1)$: $\mathbf{w} = (-0.5, 0.5, 0)$. $(1, 0)$: net 0, correct. $(1, 1)$: net 0, output 0 — add $0.5 \times (1, 1, 1)$: $\mathbf{w} = (0, 1, 0.5)$.

Epochs 3–5 continue the same way, and at the end of epoch 5 the weights are $\mathbf{w} = (-1, 1, 0.5)$. **Check:** $(0, 0) \rightarrow -1$, $(0, 1) \rightarrow -0.5$, $(1, 0) \rightarrow 0$, all output 0; $(1, 1) \rightarrow +0.5$, output 1. Epoch 6 makes no changes: converged. (Trace epochs 3 to 5 yourself: it is standard examination practice.)

13.3 Linear Separability and XOR

A single perceptron draws one straight line. AND and OR can be separated by a line; XOR — true when exactly one input is true — cannot: its two positive points (0, 1) and (1, 0) sit on one diagonal and its two negatives on the other. No perceptron, however trained, can represent XOR. Minsky and Papert’s 1969 book made this point so forcefully that neural network research stalled for over a decade.

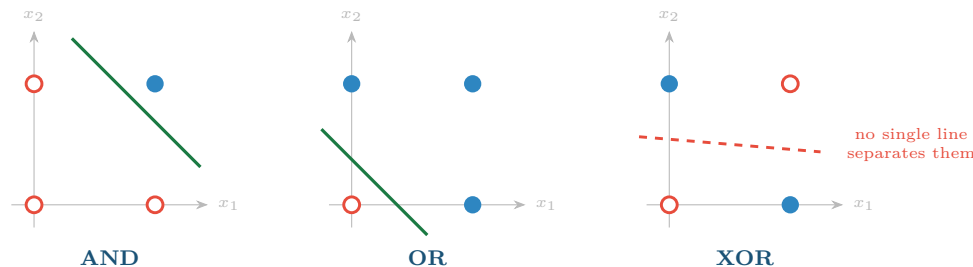


Figure 13.2: AND and OR are linearly separable; XOR is not. A second layer of units, each drawing its own line, is needed to represent it.

The cure is a second layer. XOR is “OR but not AND”: one hidden unit computes OR, another computes NAND (not-AND), and an output unit computes AND of the two. Each unit draws one line; together they carve out the diagonal. This is why networks have hidden layers — and the only problem is how to train them, since the perceptron rule needs a target for every unit and the hidden units have none.

13.4 Gradient Descent and the Delta Rule

The perceptron rule cannot be extended to hidden units because the step function has no useful derivative. The fix is to replace hard thresholds with smooth units and train by gradient descent on an error measure.

The Delta Rule

For a *linear* unit $o = \mathbf{w} \cdot \mathbf{x}$ and squared error $E = \frac{1}{2} \sum_d (t_d - o_d)^2$ over the training examples, the gradient is $\partial E / \partial w_i = - \sum_d (t_d - o_d) x_{id}$, so gradient descent updates

$$w_i \leftarrow w_i + \eta \sum_d (t_d - o_d) x_{id}$$

the **delta rule** (or LMS rule). It looks like the perceptron rule, but it is gradient descent: it converges towards the minimum-error weights even when the data is *not* separable, which the perceptron rule does not. It is exactly the gradient descent for linear regression of Chapter 5.

For a **sigmoid unit**, $o = \sigma(\mathbf{w} \cdot \mathbf{x})$, the chain rule adds one factor, the derivative of the sigmoid, which has the convenient form $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = o(1 - o)$.

13.5 Multilayer Networks and Backpropagation

Multilayer Perceptron

A **multilayer perceptron** (MLP) arranges sigmoid (or other smooth) units in layers: an input layer, one or more **hidden layers**, and an output layer. Each unit takes the outputs of the previous layer, forms a weighted sum and applies its activation. A network with one

hidden layer of enough units can approximate any continuous function to any accuracy; two hidden layers, any function at all.

Backpropagation (for sigmoid units and squared error)

For each training example:

1. **Forward pass:** compute every unit's output o , layer by layer.
2. **Output units:** $\delta_k = o_k(1 - o_k)(t_k - o_k)$.
3. **Hidden units** (backwards, layer by layer): $\delta_h = o_h(1 - o_h) \sum_k w_{kh} \delta_k$ — each hidden unit's share of the blame is the output deltas, weighted by its connections to them.
4. **Update every weight** w_{ji} from unit i to unit j : $w_{ji} \leftarrow w_{ji} + \eta \delta_j x_{ji}$, where x_{ji} is the input arriving along that connection.

One sweep through the whole training set is an **epoch**.

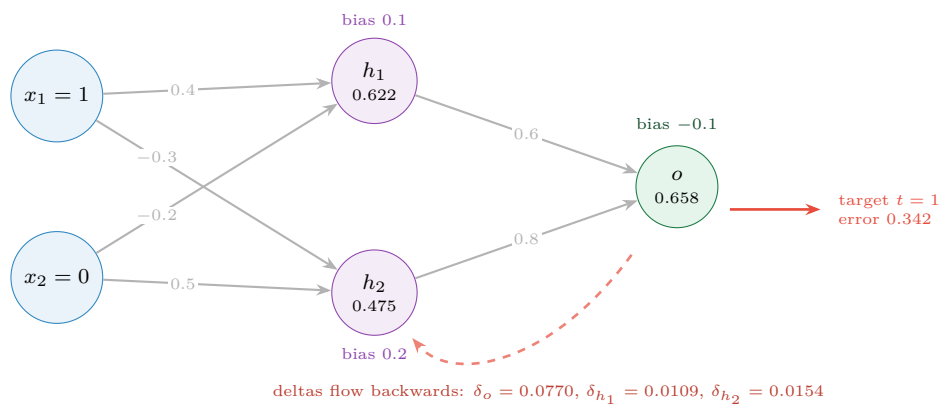


Figure 13.3: The network of the worked example, with the unit outputs of the forward pass written in each unit. Errors computed at the output are passed backwards along the same connections to assign blame to the hidden units.

Worked Example: One Step of Backpropagation

The 2-2-1 network of Figure 13.3, all sigmoid units, input $(1, 0)$, target $t = 1$, learning rate $\eta = 0.5$.

Forward.

- h_1 : net = $0.1 + 0.4(1) - 0.2(0) = 0.5$, output $\sigma(0.5) = 0.6225$.
- h_2 : net = $0.2 - 0.3(1) + 0.5(0) = -0.1$, output $\sigma(-0.1) = 0.4750$.
- o : net = $-0.1 + 0.6(0.6225) + 0.8(0.4750) = 0.6535$, output **0.6578**. Error $t - o = 0.3422$.

Backward.

- $\delta_o = 0.6578 \times (1 - 0.6578) \times 0.3422 = \mathbf{0.0770}$.
- $\delta_{h_1} = 0.6225 \times 0.3775 \times (0.6 \times 0.0770) = \mathbf{0.0109}$.
- $\delta_{h_2} = 0.4750 \times 0.5250 \times (0.8 \times 0.0770) = \mathbf{0.0154}$.

Update ($\Delta w = \eta \delta x$).

- Output weights: from h_1 : $0.6 + 0.5(0.0770)(0.6225) = 0.6240$; from h_2 : $0.8 + 0.5(0.0770)(0.4750) = 0.8183$; bias: $-0.1 + 0.5(0.0770) = -0.0615$.
- h_1 : weight from x_1 $0.4 \rightarrow 0.4054$, bias $0.1 \rightarrow 0.1054$; from x_2 unchanged, because $x_2 = 0$.
- h_2 : weight from x_1 $-0.3 \rightarrow -0.2923$, bias $0.2 \rightarrow 0.2077$.

Check. A new forward pass gives $o = 0.6727$: closer to 1. The squared error $\frac{1}{2}(t - o)^2$ falls from 0.0586 to 0.0536. Thousands of such small steps are what training a network means.

13.6 Activation Functions and Outputs

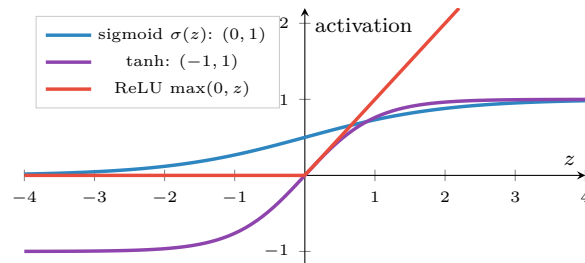


Figure 13.4: Three activation functions. The sigmoid and tanh flatten out for large $|z|$, where their gradients vanish; ReLU does not saturate for positive inputs, which is why it trains deep networks faster.

Mitchell’s networks use sigmoids throughout. Modern practice differs in two ways. Hidden layers use **ReLU**, $\max(0, z)$: its gradient is 1 for any positive input, so it does not shrink the error signal the way a saturated sigmoid does (the *vanishing gradient* that made deep sigmoid networks hard to train). And the output layer matches the task: a single sigmoid with cross-entropy for two classes (exactly logistic regression, Chapter 6), a softmax with cross-entropy for several, a linear unit with squared error for regression.

13.7 Training Well: Overfitting and Its Cures

A network with enough hidden units can memorise any training set, and it will if allowed to. Figure 13.5 shows the typical course: training error falls steadily, validation error falls for a while and then rises as the network begins fitting the noise.

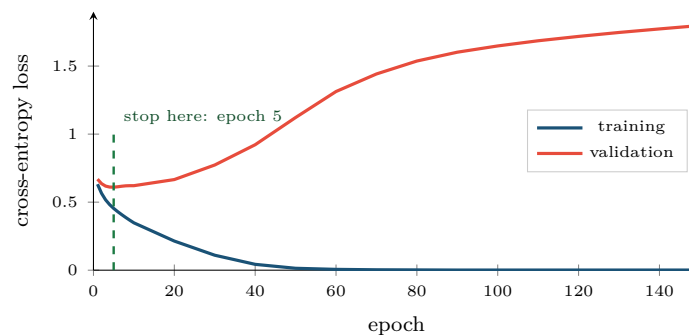


Figure 13.5: Training and validation loss of a two-hidden-layer network on data with 20% label noise. Training loss goes to zero; validation loss is lowest after five epochs and rises thereafter. Early stopping keeps the epoch-5 weights.

Technique	What it does	Setting
Early stopping	Keep the weights from the epoch with the lowest validation error	Needs a validation set; patience of a few epochs
Weight decay (L2)	Add $\lambda \sum w^2$ to the loss — ridge regression for networks (Chapter 7)	alpha in scikit-learn; tune by CV

Technique	What it does	Setting
Dropout	During training, switch off each hidden unit at random with probability p , so no unit can rely on another	$p \approx 0.2$ – 0.5 ; standard in deep learning frameworks
Fewer units	A smaller network has less capacity to memorise	Tune the width by CV
More data, augmented	Real or synthetic variations of the training examples	Paraphrased ticket text; small random jitter on numeric features

💡 What the Hidden Units Learn, and What Comes Next

Backpropagation lets the hidden layer invent its own features. Mitchell’s famous demonstration trains an 8–3–8 network to copy an 8-bit input to its output through only 3 hidden units; the hidden units discover a 3-bit binary code for the eight inputs, a representation nobody programmed. Deep learning is this idea scaled up: many layers, each building features from the one below, with architectures designed for the data — convolutional networks for images, recurrent networks and transformers for sequences. Those architectures are the subject of the Deep Learning course; this chapter is their foundation.

⚠ Common Pitfalls

- **Not scaling inputs.** Unscaled inputs saturate sigmoid units and make gradient descent crawl.
- **Initialising all weights to zero.** Every hidden unit then computes the same thing and receives the same update forever; initialise randomly.
- **Training until the training error is zero.** Watch the validation error and stop when it turns.
- **Assuming a network beats a linear model.** On small tabular data it often does not; in this chapter’s lab it gains less than one point.
- **Judging from one run.** Random initialisation makes results vary; repeat with several seeds.

🔧 Try It Yourself: Backpropagation From Scratch, Then a Real Network

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression

# --- 1. XOR: no perceptron can learn it; a 2-3-1 network can -----
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
t = np.array([[0], [1], [1], [0]], dtype=float)
sig = lambda z: 1 / (1 + np.exp(-z))
rng = np.random.default_rng(1)
W1, b1 = rng.normal(0, 1, (2, 3)), np.zeros(3)      # input -> hidden
W2, b2 = rng.normal(0, 1, (3, 1)), np.zeros(1)      # hidden -> output
eta = 1.0
for epoch in range(5001):
    h = sig(X @ W1 + b1)                            # forward pass
    o = sig(h @ W2 + b2)
    d_out = o * (1 - o) * (t - o)                   # output deltas
    d_hid = h * (1 - h) * (d_out @ W2.T)            # hidden deltas
```

```

W2 += eta * h.T @ d_out                                # weight updates
b2 += eta * d_out.sum(0)
W1 += eta * X.T @ d_hid
b1 += eta * d_hid.sum(0)
if epoch in (0, 500, 5000):
    err = 0.5 * np.sum((t - o) ** 2)
    print(f"epoch {epoch:4d}: error {err:.4f} "
          f"outputs {o.ravel().round(2)}")

# --- 2. staffing: too few people is late, and so is too many -----
rng = np.random.default_rng(0)
n = 800
scope = rng.uniform(100, 1000, n)                        # story points
team = rng.uniform(2, 20, n)                             # people
Xs = np.column_stack([scope, team])
ys = (np.abs(team - scope / 60) > 5).astype(int)          # 1 = late
flip = rng.random(n) < 0.05                             # 5% recorded wrongly
ys[flip] = 1 - ys[flip]
X_tr, X_te, y_tr, y_te = train_test_split(Xs, ys, test_size=0.3,
                                          random_state=0, stratify=ys)

sc = StandardScaler().fit(X_tr)
X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)
lin = LogisticRegression().fit(X_tr, y_tr)
print("logistic regression test accuracy:", round(lin.score(X_te, y_te), 3))
for alpha in (0.0001, 1.0):
    net = MLPClassifier(hidden_layer_sizes=(16,), alpha=alpha,
                        max_iter=3000, random_state=0).fit(X_tr, y_tr)
    print(f"network, weight decay {alpha:g}: {net.n_iter_} epochs, "
          f"test accuracy {net.score(X_te, y_te):.3f}")

```

In part 2 a project runs late when its team is far from the size its scope needs — understaffed, or so overstaffed that coordination swamps the work (Brooks's law). The late projects lie on *both* sides of a diagonal band, so no single line separates them: logistic regression manages 0.56, barely better than a coin, while the network finds the band and scores 0.90. Now add `early_stopping=True` to the network and explain why accuracy *falls* so sharply.

🔍 Checkpoint — Test Yourself

1. A perceptron has $w_0 = -1$, $w_1 = 2$, $w_2 = 1$. What does it output for $(0, 1)$ and $(1, 0)$?
2. Why can a single perceptron not learn XOR, and how does one hidden layer fix it?
3. In backpropagation, why does the hidden unit's delta contain the output unit's delta?

📋 Chapter Summary

- A perceptron thresholds a weighted sum; the perceptron rule finds a separating hyperplane whenever one exists.
- A single unit cannot represent XOR or any function that is not linearly separable; hidden layers can.
- The delta rule is gradient descent on squared error and converges even on non-separable data; sigmoid units make it differentiable.
- Backpropagation computes output deltas, passes them backwards through the weights to the hidden units, and updates every weight by $\eta \delta x$ — the chain rule applied layer by layer.
- Modern networks use ReLU hidden units and task-matched outputs (sigmoid, softmax, linear).
- Control overfitting with early stopping, weight decay, dropout, smaller networks and more data.

 Review Questions

1. **(MCQ)** Which function can a single perceptron *not* represent? (a) AND (b) OR (c) NAND (d) XOR.
2. **(MCQ)** The derivative of the sigmoid output o with respect to its input is: (a) o (b) $1 - o$ (c) $o(1 - o)$ (d) e^{-o} .
3. **(MCQ)** Early stopping chooses the weights from the epoch with the lowest: (a) training error (b) validation error (c) test error (d) learning rate.
4. **(Short)** Contrast the perceptron training rule with the delta rule.
5. **(Short)** Why must a network's weights be initialised randomly?
6. **(Short)** Explain why ReLU helps train deep networks.
7. **(Applied)** Give weights for a two-layer network of threshold units that computes XOR, and verify it on all four inputs.
8. **(Applied)** Repeat the backpropagation worked example for the input $(0, 1)$ with target $t = 0$, starting from the original weights. Report the output before and after the update.

Ensemble Learning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Supervised Learning Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why a committee of hypotheses can be more accurate than its members, and **compute** the accuracy of a majority vote.
- **Describe** bagging and random forests, and **explain** which part of the error they reduce.
- **Perform** one round of AdaBoost by hand: weighted error, learner weight and example reweighting.
- **Describe** gradient boosting and stacking.
- **Choose** between bagging and boosting for a given problem.

🔗 Before You Start

Chapter 9 (decision trees and their instability), Chapter 7 (bias and variance) and Chapter 10 (the Bayes optimal classifier as a weighted vote of hypotheses).

📖 Key Terms in This Chapter

ensemble • committee • majority (hard) vote • soft vote • independent errors • bootstrap sample • bagging • out-of-bag estimate • random forest • feature importance • boosting • weak learner • AdaBoost • decision stump • gradient boosting • stacking

14.1 Committees of Hypotheses

The Bayes optimal classifier of Chapter 10 lets every hypothesis vote, and no single hypothesis can beat it. Ensemble methods are the practical version of that idea: train several models, and combine their predictions. They are, in practice, the most accurate off-the-shelf learners for tabular data — the random forests and gradient-boosted trees of this chapter win most such competitions.

💡 Why a Committee Can Beat Its Members

If each member of a committee is right more often than not, and their mistakes are *independent*, the majority is right more often than any member: for the majority to be wrong, most members must be wrong on the same example at once. Everything in this chapter is a way of producing members that are individually decent and whose errors are as uncorrelated as possible.

📊 Worked Example: The Arithmetic of a Majority Vote

Five late-task classifiers, each 70% accurate, with independent errors. The majority is right when at least three are right:

$$P(\geq 3 \text{ right}) = \binom{5}{3} 0.7^3 0.3^2 + \binom{5}{4} 0.7^4 0.3 + \binom{5}{5} 0.7^5 = 0.309 + 0.360 + 0.168 = \mathbf{0.837}$$

Twenty-one such classifiers give 0.974; a hundred and one classifiers that are each only 60% accurate give 0.979.

The catch is “independent”. Five copies of the same classifier make the same mistakes and vote as one: 70%. Real ensemble members are correlated, so the gains are smaller than this — and the whole craft is in reducing the correlation.

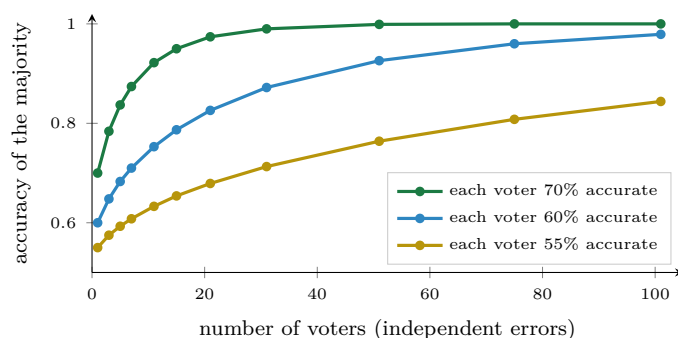


Figure 14.1: How majority voting amplifies weak but independent voters. Even 55%-accurate members reach 84% with a hundred voters — if, and only if, their mistakes are independent.

Voting comes in two forms. A **hard vote** counts class predictions. A **soft vote** averages predicted probabilities and picks the largest, which lets a confident member outweigh several unsure ones and is usually a little better.

14.2 Bagging

Bagging (Bootstrap Aggregating)

From a training set of n examples, draw B **bootstrap samples**, each of n examples drawn *with replacement*. Train one model on each sample. Predict by majority vote (classification) or by averaging (regression).

Each bootstrap sample leaves some examples out and repeats others: the chance that a given example is never drawn in n draws is $(1 - 1/n)^n \approx e^{-1} = 0.368$, so each sample contains about 63.2% of the distinct examples. The models therefore differ, and their errors are partly decorrelated. Bagging helps exactly when the base learner is *unstable*: when small changes in the training data produce very different models. That describes a fully grown decision tree (Chapter 9) precisely — high variance, low bias — and bagging averages the variance away while keeping the low bias. It does little for a stable learner such as linear regression or k -NN with large k .

A Free Validation Set: the Out-of-Bag Estimate

Each example is left out of about 37% of the bootstrap samples. Predict it using only the models that did not see it, and the resulting **out-of-bag** error is an honest estimate of test error — computed during training, with no data held back.

14.3 Random Forests

Bagged trees are still correlated: if one feature is very strong, every tree puts it at the root and the trees look alike. A **random forest** (Breiman, 2001) decorrelates them further.

Random Forest

Bag fully grown decision trees, with one change: at each split, choose the best test from a random subset of the features only — typically $\sqrt{d}$ of the d features for classification. Different trees are forced to use different features, so their errors are less correlated and the average is better. A forest also reports each feature's **importance**: the total reduction in impurity from splits on that feature, averaged over the trees.

Random forests are the safest default for tabular data: few hyperparameters (the number of trees, which only needs to be large enough, and the feature-subset size), no scaling, robust to irrelevant features, and hard to overfit by adding trees.

14.4 Boosting

Bagging trains its members independently and in parallel. Boosting trains them in sequence, each one concentrating on the examples its predecessors got wrong. Its members are **weak learners** — barely better than chance, typically **decision stumps** (one-split trees) or very shallow trees — and it is bias, not variance, that boosting mainly reduces.

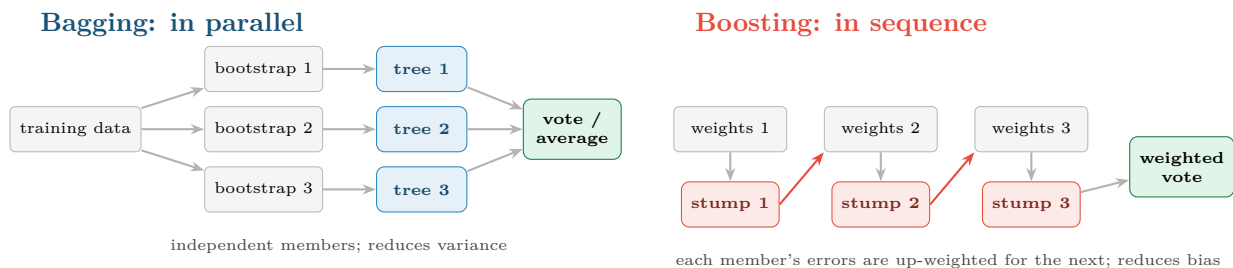


Figure 14.2: Bagging trains independent models on resampled data and averages them. Boosting trains models one after another, re-weighting the data towards the examples still being misclassified, and combines them with a weighted vote.

AdaBoost

Give every training example the weight $1/n$. For rounds $t = 1, \dots, T$:

1. Train a weak learner h_t on the weighted data.
2. Compute its weighted error ε_t (the total weight of the examples it misclassifies) and its say in the final vote, $\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$.
3. Multiply the weight of each misclassified example by e^{α_t} and of each correct one by $e^{-\alpha_t}$; renormalise to sum to 1.

Predict with the weighted vote $\text{sign}(\sum_t \alpha_t h_t(\mathbf{x}))$.

Worked Example: One Round of AdaBoost

Ten examples, each of weight 0.1. The first stump misclassifies three of them.

Weighted error: $\varepsilon_1 = 3 \times 0.1 = 0.3$.

Say of the stump: $\alpha_1 = \frac{1}{2} \ln \frac{0.7}{0.3} = \frac{1}{2} \ln 2.333 = 0.424$.

Reweight: $e^{0.424} = 1.528$ and $e^{-0.424} = 0.655$.

- three mistakes: $0.1 \times 1.528 = 0.1528$ each (total 0.458);
- seven correct: $0.1 \times 0.655 = 0.0655$ each (total 0.458).

Normalise by the total 0.917: mistakes $\rightarrow$ **0.167** each, correct $\rightarrow$ **0.071** each.

After the update the three mistakes carry exactly half the total weight — always true in AdaBoost. The next stump, trained on these weights, cannot score better than chance by repeating the first stump’s errors, so it is forced to attend to them. A stump with $\varepsilon = 0.5$ would get $\alpha = 0$: no say at all.

Gradient boosting generalises the idea to any differentiable loss. Start with a constant prediction; at each round, fit a small tree to the current *residuals* (more generally, to the negative gradient of the loss), and add a fraction η — the learning rate, typically 0.05–0.1 — of its predictions to the model. Each tree corrects what the ensemble so far gets wrong. Libraries such as XGBoost and LightGBM are highly engineered versions, and are the first thing most practitioners try on a tabular problem.

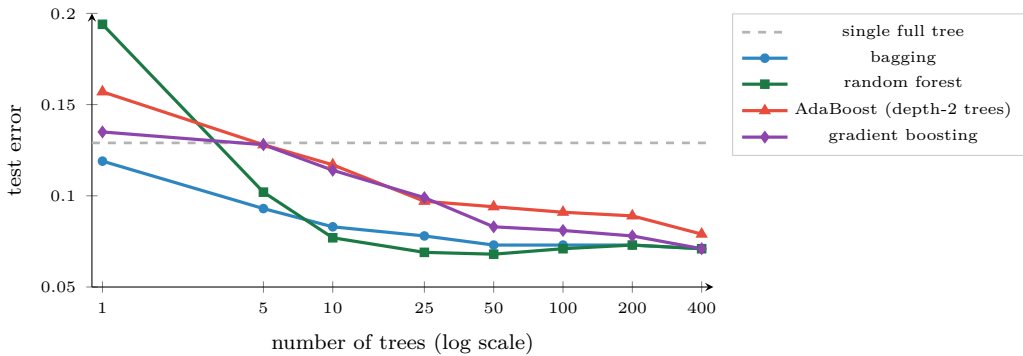


Figure 14.3: Test error against ensemble size on a 3,000-example problem with 5% label noise. Every ensemble roughly halves the error of a single tree. Averaging methods level off quickly; boosting keeps improving for longer.

14.5 Stacking

Stacking combines *different kinds* of model. Train several base models — say logistic regression, an SVM (Chapter 12) and a random forest — then train a second-level model (often logistic regression) whose inputs are the base models’ predictions and whose target is the true label. To avoid leakage, the second-level training data must be the base models’ predictions on examples they did not train on, produced by cross-validation. Stacking learns how much to trust each base model, and when.

Method	Members	Mainly reduces	Watch out for
Bagging	Deep trees on bootstrap samples, in parallel	Variance	Little gain with stable learners
Random forest	Bagged trees with random feature subsets	Variance	Importances biased towards many-valued features
AdaBoost	Stumps or shallow trees, in sequence	Bias (and variance)	Label noise: mislabelled points get ever larger weights

Method	Members	Mainly reduces	Watch out for
Gradient boosting	Shallow trees fitted to residuals	Bias (and variance)	Overfits with too many trees at a high learning rate; tune both
Stacking	Different model types plus a combiner	Whatever the base models miss	Leakage if the combiner is trained on in-sample predictions

! Common Pitfalls

- **Ensembling copies of the same model.** Correlated members vote as one; diversity is the whole point.
- **Boosting noisy labels without limit.** AdaBoost's weights chase mislabelled points; use fewer rounds, a lower learning rate, or bagging.
- **Reading impurity importances as causal.** They say what the forest used, not what drives the outcome, and they favour many-valued features.
- **Training a stacking combiner on in-sample predictions.** Use cross-validated predictions.
- **Forgetting the cost.** Hundreds of trees are slower to run and harder to explain than one; check that the gain is worth it.

🔧 Try It Yourself: One Tree Against Five Ensembles

Part 2 uses 3,000 synthetic past tasks, each described by twenty numeric features, of which 5% were recorded with the wrong outcome.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                              AdaBoostClassifier, GradientBoostingClassifier,
                              VotingClassifier)

# --- 1. one round of AdaBoost, by hand -----
w = np.full(10, 0.1) # ten examples, equal weight
wrong = np.array([1, 1, 1, 0, 0, 0, 0, 0, 0, 0], dtype=bool)
eps = w[wrong].sum() # weighted error = 0.3
alpha = 0.5 * np.log((1 - eps) / eps) # say of this learner
w = np.where(wrong, w * np.exp(alpha), w * np.exp(-alpha))
w = w / w.sum()
print(f"alpha = {alpha:.3f}; new weights: wrong {w[0]:.4f}, "
      f"right {w[-1]:.4f}; weight on the mistakes = {w[wrong].sum():.2f}")

# --- 2. one tree against five ensembles -----
X, y = make_classification(n_samples=3000, n_features=20, n_informative=8,
                          n_redundant=4, flip_y=0.05, random_state=7)
tree = DecisionTreeClassifier
models = {
    "single tree": tree(random_state=0),
    "bagging (100 trees)": BaggingClassifier(tree(), n_estimators=100,
                                             random_state=0),
    "random forest (100)": RandomForestClassifier(n_estimators=100,
                                                  random_state=0),
    "AdaBoost (depth 2)": AdaBoostClassifier(tree(max_depth=2),
                                              n_estimators=200,
```

```

        random_state=0),
    "gradient boosting": GradientBoostingClassifier(n_estimators=200,
                                                    random_state=0),
    "vote: LR + tree + RF": VotingClassifier([
        ("lr", LogisticRegression(max_iter=1000)),
        ("dt", tree(max_depth=6, random_state=0)),
        ("rf", RandomForestClassifier(n_estimators=100, random_state=0))],
        voting="soft"),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=5)
    print(f"{name:24s} accuracy {s.mean():.3f} +/- {s.std():.3f}")

# --- 3. a free validation estimate, and what the forest looked at -----
rf = RandomForestClassifier(n_estimators=300, oob_score=True,
                           random_state=0).fit(X, y)
print("random forest out-of-bag accuracy:", round(rf.oob_score_, 3))
top = np.argsort(-rf.feature_importances_)[1:5]
print("most used features:", top.tolist(),
      rf.feature_importances_[top].round(3).tolist())

```

Every ensemble beats the single tree by four to six points. Now set `flip_y=0.3` (30% label noise) and compare bagging with AdaBoost again: which suffers more, and why?

🔍 Checkpoint — Test Yourself

1. Three independent classifiers are each 80% accurate. What is the accuracy of their majority vote?
2. A stump has weighted error 0.2. Compute its α and the factors by which misclassified and correct examples are re-weighted.
3. Why does bagging help a decision tree much more than it helps logistic regression?

📋 Chapter Summary

- A majority of individually decent, independent voters beats every voter; the work of ensemble learning is making members diverse.
- Bagging trains unstable learners on bootstrap samples and averages them, reducing variance; out-of-bag examples give a free error estimate.
- Random forests add random feature subsets at each split to decorrelate the trees; they are the safest default for tabular data.
- Boosting trains weak learners in sequence on re-weighted data; AdaBoost gives each learner a say $\alpha = \frac{1}{2} \ln \frac{1-\epsilon}{\epsilon}$ and moves half the weight onto its mistakes. It reduces bias but is sensitive to label noise.
- Gradient boosting fits each new tree to the current residuals; stacking learns how to combine different model types.

🔧 Review Questions

1. (MCQ) A bootstrap sample of n examples contains about what fraction of the distinct examples? (a) 50% (b) 63% (c) 90% (d) 100%.
2. (MCQ) Random forests differ from bagged trees by: (a) using stumps (b) re-weighting examples (c) considering a random subset of features at each split (d) pruning every tree.
3. (MCQ) In AdaBoost, a weak learner with weighted error 0.5 receives a weight α of: (a) 1 (b) 0.5 (c) 0 (d) -1 .

4. **(Short)** Contrast bagging and boosting in terms of how members are trained and which error component each reduces.
5. **(Short)** Explain the out-of-bag estimate and why it is honest.
6. **(Short)** Why is AdaBoost sensitive to mislabelled training examples?
7. **(Applied)** Compute the accuracy of a majority vote of seven independent classifiers that are each 65% accurate.
8. **(Applied)** Continue the AdaBoost worked example: the second stump misclassifies two of the seven previously correct examples and none of the three previous mistakes. Compute ε_2 , α_2 and the new weights.

PART IV

Learning Without Full Labels

Labels are expensive. Part IV asks what can be learned without them — groups, directions, maps — and then what a handful of labels can do when they are combined with a great many unlabelled examples. The EM algorithm at its end returns in Part V.

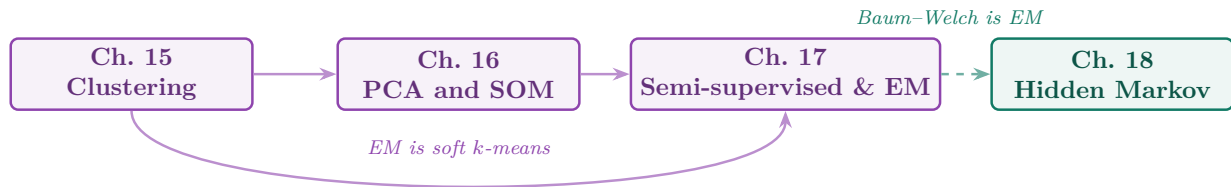


Figure P4.1: Reading path through Part IV. k -means (Chapter 15) is the hard-assignment special case of the EM algorithm in Chapter 17, which reappears in Chapter 18 to train hidden Markov models.

Clustering

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Learning Without Full Labels

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what clustering is for, and why its answer depends on choices the analyst makes.
- **Trace** k -means by hand through assignment and update steps to convergence.
- **Build** a dendrogram by hierarchical agglomerative clustering under single, complete and average linkage.
- **Choose** the number of clusters with the elbow method and the silhouette coefficient.
- **Evaluate** a clustering internally and, when labels exist, externally.

🔗 Before You Start

Chapter 2 (distance), Chapter 3 (scaling) and Chapter 11 (distance measures and the curse of dimensionality, which clustering inherits in full).

📖 Key Terms in This Chapter

clustering • partitional vs. hierarchical • centroid • k -means (Lloyd's algorithm) • within-cluster sum of squares (WCSS) • local minimum • k -means++ • agglomerative clustering • dendrogram • single, complete, average and Ward linkage • chaining • elbow method • silhouette coefficient • adjusted Rand index

15.1 Finding Groups Without Labels

Clustering divides examples into groups — **clusters** — so that examples in the same group are similar and examples in different groups are not. There are no labels to learn from and no right answer to check against. What counts as “similar” is a choice (of features, their scaling and a distance), and so is the number of groups; change either and the clusters change.

That makes clustering exploratory: a way of proposing structure for a person to examine. Kinds of project in a portfolio, tickets that describe the same underlying fault, developers with similar skill profiles, the phases a project passes through — each is a clustering that someone then named and decided was useful. Two families of algorithm cover the HEC outline: **partitional** methods such as k -means divide the data into a fixed number of groups; **hierarchical** methods build a whole tree of nested groupings.

15.2 k -Means

📄 k -Means (Lloyd's Algorithm)

Choose k and place k initial **centroids**. Repeat until nothing changes:

1. **Assign** each example to its nearest centroid.
2. **Update** each centroid to the mean of the examples assigned to it.

Each step can only lower the **within-cluster sum of squares**

$$\text{WCSS} = \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

so the algorithm always converges — but to a **local** minimum that depends on where it started.

Worked Example: k -Means by Hand

Seven past projects, each described by its budget (in hundreds of thousands of dollars) and its duration (in months): A1 (1, 1), A2 (1.5, 2), A3 (3, 4), A4 (5, 7), A5 (3.5, 5), A6 (4.5, 5), A7 (3.5, 4.5). $k = 2$, with initial centroids A2 and A4.

Iteration 1 — assign. Distances to $\mu_1 = (1.5, 2)$ and $\mu_2 = (5, 7)$: A1 1.12 / 7.21; A2 0 / 6.10; A3 2.50 / 3.61; A4 6.10 / 0; A5 3.61 / 2.50; A6 4.24 / 2.06; A7 3.20 / 2.92. So $C_1 = \{A1, A2, A3\}$, $C_2 = \{A4, A5, A6, A7\}$.

Update. $\mu_1 = (\frac{1+1.5+3}{3}, \frac{1+2+4}{3}) = (1.83, 2.33)$; $\mu_2 = (\frac{5+3.5+4.5+3.5}{4}, \frac{7+5+5+4.5}{4}) = (4.13, 5.38)$. WCSS = 12.21.

Iteration 2 — assign. A3 is now 2.03 from μ_1 and **1.78** from μ_2 : it moves to C_2 . Every other point stays.

Update. $\mu_1 = (1.25, 1.5)$, $\mu_2 = (3.9, 5.1)$. WCSS = 8.53.

Iteration 3. No point changes cluster, so the centroids stop moving: **converged**, with $C_1 = \{A1, A2\}$ and $C_2 = \{A3, \dots, A7\}$ — two small, short projects and five larger ones.

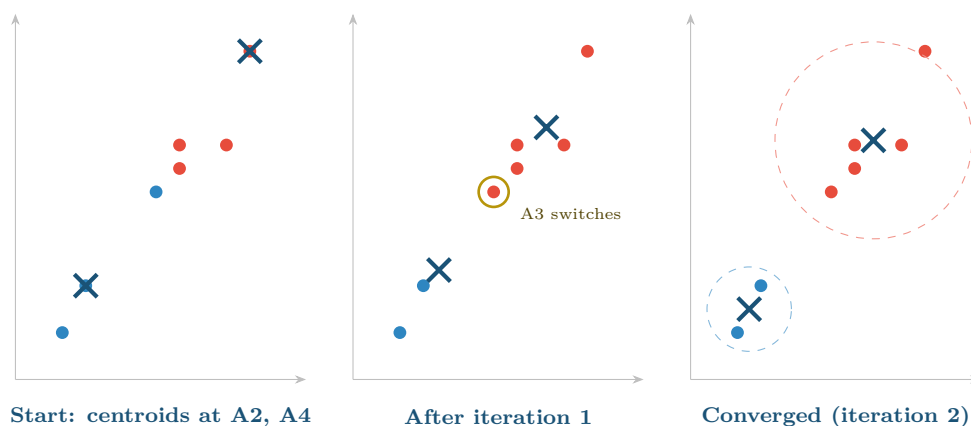


Figure 15.1: Three snapshots of k -means on the worked example. Assignment and update alternate until no point changes cluster.

⚠ Where k -Means Goes Wrong

- **Bad starts.** Different initial centroids can converge to different, worse clusterings. Run it many times and keep the lowest WCSS (`n_init`), and start with `k-means++`, which spreads the initial centroids out.
- **Its shape bias.** Every cluster is the set of points nearest a centroid, so k -means assumes round, similar-sized clusters. Long, curved or very unequal clusters are cut badly.
- **Scale.** It is built on Euclidean distance; unscaled features decide everything (Chapter 11).
- **Outliers.** A mean is pulled by extreme values; one wild point can capture a centroid of its own.

15.3 Hierarchical Agglomerative Clustering

Agglomerative Clustering and Linkage

Start with every example in a cluster of its own. Repeatedly merge the two *closest* clusters until one remains. The record of merges, drawn as a tree whose heights are the merge distances, is a **dendrogram**. The distance between two clusters is defined by the **linkage**:

- **single**: the distance between their *closest* members;
- **complete**: the distance between their *farthest* members;
- **average**: the average distance over all pairs of members;
- **Ward**: the increase in WCSS that merging them would cause — the k -means criterion, applied hierarchically.

Worked Example: Single and Complete Linkage on Five Numbers

Five tickets, described only by how many days each took to resolve: 1, 2, 4, 9, 11.

Single linkage. Closest pair: $\{1, 2\}$ at distance 1. Next, $\{1, 2\}$ to 4 is $|2 - 4| = 2$ and 9 to 11 is also 2: merge $\{1, 2, 4\}$ and $\{9, 11\}$, both at height 2. Last, the two groups are $|4 - 9| = 5$ apart. **Heights: 1, 2, 2, 5.**

Complete linkage. $\{1, 2\}$ at 1. Now $\{1, 2\}$ to 4 is $|1 - 4| = 3$ (the farthest pair), while 9 to 11 is 2, so $\{9, 11\}$ merges first, at 2; then $\{1, 2, 4\}$ at 3. Last, the farthest pair across the groups is $|1 - 11| = 10$. **Heights: 1, 2, 3, 10.**

Both find the same two groups, but complete linkage separates them by a much taller final merge. Average linkage gives a final height of $(8 + 10 + 7 + 9 + 5 + 7)/6 = 7.67$. **Cutting** a dendrogram at any height yields a flat clustering: cut complete linkage anywhere between 3 and 10 and you get $\{1, 2, 4\}$ and $\{9, 11\}$.

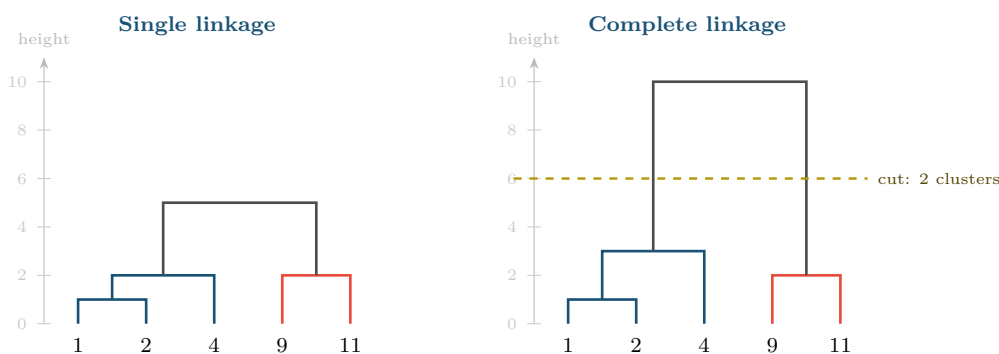


Figure 15.2: Dendrograms for the points 1, 2, 4, 9, 11. The merges happen in almost the same order, but at different heights: complete linkage keeps groups compact and makes the gap between them obvious.

Chaining, and Which Linkage to Use

Single linkage merges two clusters as soon as *any* pair of their members is close, so a thin line of points can join two genuinely separate groups into one long “chain”. That lets it find elongated or curved clusters, and also makes it fragile with noise. Complete linkage does the opposite: it favours compact, roughly equal clusters and is pulled by outliers. Average linkage sits between them. Ward’s method behaves like k -means and is a good default. Unlike k -means, hierarchical clustering needs no k in advance and gives the same answer every run — but it must compute all n^2 pairwise distances, which limits it to tens of thousands of examples.

15.4 How Many Clusters?

The elbow method. Plot WCSS against k . WCSS always falls as k rises — with $k = n$ it is zero — so look for the *elbow*, the k after which extra clusters buy little.

The silhouette. For each example, let a be its mean distance to the other members of its own cluster and b its mean distance to the members of the nearest other cluster. Its **silhouette** is

$$s = \frac{b - a}{\max(a, b)} \quad (-1 \leq s \leq 1)$$

Near 1: well inside its cluster. Near 0: on a boundary. Negative: probably in the wrong cluster. The mean silhouette over all examples scores a whole clustering; pick the k that maximises it. A point with $a = 2$ and $b = 6$ has $s = (6 - 2)/6 = 0.67$.

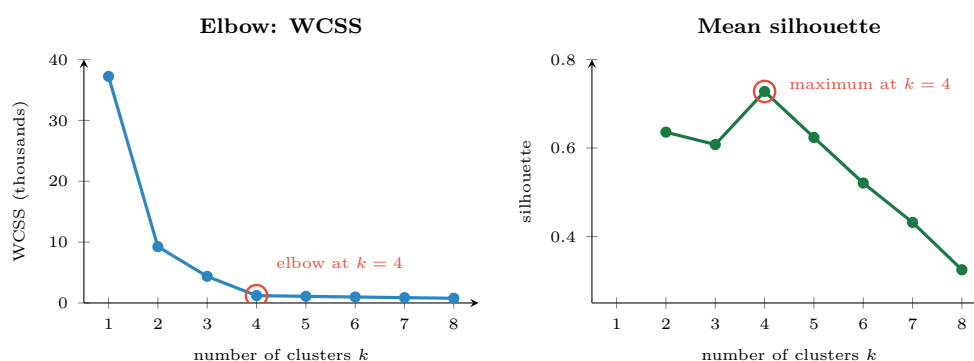


Figure 15.3: Choosing k for 600 points generated from four clusters. The WCSS curve bends sharply at $k = 4$ and the silhouette peaks there: both agree with the truth.

⚠ Judging a Clustering

Internal measures (WCSS, silhouette) use only the data, and reward whatever shape the measure assumes. **External** measures such as the **adjusted Rand index** compare the clusters with known labels, when some exist: 1 for perfect agreement, about 0 for chance. The most important test is neither: can a domain expert name the clusters, and do the names lead to a different decision for each? A clustering nobody can use is not a result.

15.5 Beyond the HEC Outline

Two methods complete the picture. **DBSCAN** grows clusters from dense regions and labels isolated points as noise, so it finds clusters of any shape without being told k — it belongs to the Data Mining course, with outlier detection. **Gaussian mixture models** replace k -means's hard assignments by probabilities; they are fitted with the EM algorithm of Chapter 17, of which k -means turns out to be a special case.

⚠ Common Pitfalls

- **Clustering unscaled features.** The feature with the largest units decides the clusters.
- **Running k -means once.** Use many starts and k -means++.
- **Choosing k by WCSS alone.** It always falls; look for the elbow and check the silhouette.
- **Treating clusters as discovered facts.** They reflect the features, scaling, distance and k chosen; a different choice gives different clusters.
- **Forcing k -means onto non-round clusters.** Use single linkage, DBSCAN or a mixture model instead.

Try It Yourself: k-Means, a Dendrogram and the Choice of k

```
import warnings
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", message="KMeans is known to have a memory")

# --- 1. the worked example: k-means from A2 and A4 -----
P = np.array([[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5],
              [3.5, 4.5]])
km = KMeans(n_clusters=2, init=P[[1, 3]], n_init=1).fit(P)
print("centroids:", km.cluster_centers_.round(3).tolist(),
      " WCSS:", round(km.inertia_, 3), " iterations:", km.n_iter_)

# --- 2. choosing k: the elbow and the silhouette -----
X, truth = make_blobs(n_samples=600, centers=4, cluster_std=1.0,
                     random_state=3) # 600 projects of four kinds
for k in range(2, 8):
    m = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(f"k={k}: WCSS {m.inertia_:8.1f} "
          f"silhouette {silhouette_score(X, m.labels_):.3f}")

# --- 3. hierarchical clustering: linkage heights on five numbers -----
x = np.array([[1.0], [2.0], [4.0], [9.0], [11.0]])
for method in ("single", "complete", "average"):
    Z = linkage(x, method)
    print(f"{method:8s} merge heights: {Z[:, 2].round(2).tolist()}")
print("complete linkage cut into 2 groups:",
      fcluster(linkage(x, "complete"), 2, criterion="maxclust").tolist())

# --- 4. against the truth, and why scale matters -----
km4 = KMeans(4, n_init=10, random_state=0).fit_predict(X)
hac = AgglomerativeClustering(n_clusters=4, linkage="ward").fit(X)
print("adjusted Rand, k-means:", round(adjusted_rand_score(truth, km4), 3),
      " ward HAC:", round(adjusted_rand_score(truth, hac.labels_), 3))
Xs = X.copy()
Xs[:, 1] *= 100 # one feature in other units
raw = KMeans(4, n_init=10, random_state=0).fit_predict(Xs)
fix = KMeans(4, n_init=10, random_state=0).fit_predict(
    StandardScaler().fit_transform(Xs))
print("rescaled feature, adjusted Rand: raw",
      round(adjusted_rand_score(truth, raw), 3), " standardised",
      round(adjusted_rand_score(truth, fix), 3))
```

Part 2 stands in for 600 projects of four kinds, described by two numeric features; the kinds are known (truth) only so that part 4 can check the clusters against them. The warning filter hides a harmless message scikit-learn prints on some Windows installations. Change `cluster_std` to 2.5 in part 2: do the elbow and the silhouette still agree?

Checkpoint — Test Yourself

- Two centroids are at 2 and 8 on a line. Assign the points 1, 4, 6, 7, 9 and compute the new centroids.
- Why does k -means always converge, and why might it converge to a poor answer?
- A point has $a = 3$ and $b = 2$. Compute its silhouette and say what it means.

 Chapter Summary

- Clustering proposes groups without labels; the result depends on features, scaling, distance and the number of clusters.
- k -means alternates assignment to the nearest centroid with moving each centroid to its cluster's mean, lowering WCSS until it converges to a local minimum; use many starts and k -means++.
- Agglomerative clustering merges the closest clusters step by step into a dendrogram; the linkage (single, complete, average, Ward) defines “closest”. Single linkage chains; complete linkage keeps groups compact.
- Choose k with the elbow of the WCSS curve and the maximum mean silhouette; judge the result by whether it can be named and used.

 Review Questions

1. (MCQ) The k -means update step moves each centroid to: (a) the median of its cluster (b) the mean of its cluster (c) the farthest point (d) a random point.
2. (MCQ) Which linkage is most prone to chaining? (a) single (b) complete (c) average (d) Ward.
3. (MCQ) A silhouette close to -1 suggests the point is: (a) well clustered (b) on a boundary (c) probably in the wrong cluster (d) an outlier with no cluster.
4. (Short) Why can the WCSS alone not be used to choose k ?
5. (Short) Give two situations where hierarchical clustering is preferable to k -means.
6. (Short) Explain the role of k -means++.
7. (Applied) Run k -means by hand on the worked example with initial centroids A1 and A7. Does it reach the same final clusters?
8. (Applied) Build single- and complete-linkage dendrograms for the points 0, 3, 4, 10, 12, 20 and state the two-cluster cut of each.

Dimensionality Reduction and Self-Organizing Maps

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Learning Without Full Labels

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why reducing the number of features helps, with reference to the curse of dimensionality.
- **Compute** principal components by hand for a small two-dimensional dataset, and the variance each explains.
- **Choose** how many components to keep, and **use** PCA inside a pipeline.
- **Describe** the self-organizing map, and **perform** one training step by hand: best-matching unit, neighbourhood and update.
- **Contrast** PCA and SOM as ways of seeing high-dimensional data.

🔗 Before You Start

Chapter 2 (vectors, variance, matrices), Chapter 11 (the curse of dimensionality), Chapter 15 (centroids and competitive assignment) and Chapter 13 (units, weights and a learning rate: a self-organizing map is a neural network, trained without backpropagation).

📖 Key Terms in This Chapter

dimensionality reduction • feature selection vs. extraction • covariance matrix • principal component • eigenvector and eigenvalue • explained variance • scree plot • projection • reconstruction • self-organizing map (Kohonen map) • competitive learning • best-matching unit • neighbourhood function • topology preservation • U-matrix

16.1 Why Fewer Dimensions

Chapter 11 showed what happens when there are too many features: distances concentrate, neighbours stop being meaningful, and every method that relies on distance degrades. High-dimensional data is also slow to process, impossible to look at, and often redundant — the forty metrics a project dashboard records each week (hours, story points, open bugs, test counts, change requests, ...) rise and fall together, because a handful of underlying facts about the project drive them all. Two responses exist. **Feature selection** keeps a subset of the original features (a tree's splits, lasso's non-zero weights). **Feature extraction** builds a smaller set of new features from all the old ones. This chapter covers the two extraction methods in the HEC outline: principal component analysis, which is linear and has an exact solution, and the self-organizing map, a neural network that produces a two-dimensional map.

16.2 Principal Component Analysis

Principal Components

The first **principal component** of a dataset is the direction along which the data varies most. The second is the direction of greatest remaining variance at right angles to the first, and so on. Computationally: centre the data, form its **covariance matrix** S ($d \times d$), and find its **eigenvectors** — directions $\mathbf{v}$ with $S\mathbf{v} = \lambda\mathbf{v}$. Sorted by **eigenvalue** λ , the eigenvectors are the principal components, and each λ is the variance of the data along its component. The fraction $\lambda_j / \sum_i \lambda_i$ is the **explained variance** of component j .

Keeping the first m components and **projecting** each example onto them — $z_j = \mathbf{v}_j \cdot (\mathbf{x} - \bar{\mathbf{x}})$ — replaces d features by m while losing as little variance as any linear projection can. Equivalently, it is the projection with the smallest **reconstruction** error: mapping the m scores back into the original space loses the least.

Worked Example: PCA by Hand in Two Dimensions

Five examples: $(1, 2)$, $(2, 1)$, $(3, 3)$, $(4, 5)$, $(5, 4)$.

Centre. The mean is $(3, 3)$; centred data: $(-2, -1)$, $(-1, -2)$, $(0, 0)$, $(1, 2)$, $(2, 1)$.

Covariance (dividing by $n - 1 = 4$):

$$\text{var}(x_1) = \frac{4+1+0+1+4}{4} = 2.5, \quad \text{var}(x_2) = 2.5, \quad \text{cov}(x_1, x_2) = \frac{2+2+0+2+2}{4} = 2$$

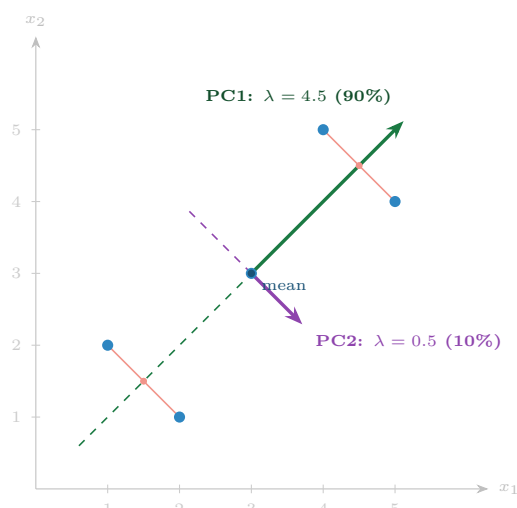
$$S = \begin{pmatrix} 2.5 & 2 \\ 2 & 2.5 \end{pmatrix}$$

Eigenvectors. For this symmetric pattern they are $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$. Check: $S(1, 1)^\top = (4.5, 4.5)^\top = 4.5(1, 1)^\top$, and $S(1, -1)^\top = (0.5, -0.5)^\top = 0.5(1, -1)^\top$. So $\lambda_1 = 4.5$, $\lambda_2 = 0.5$.

Explained variance: PC1 carries $4.5/5 = 90\%$, PC2 the remaining 10%.

Scores on PC1: $z = (x_1 + x_2)/\sqrt{2}$ for each centred point: $-2.12, -2.12, 0, 2.12, 2.12$.

One number per example now keeps 90% of the variation. The discarded direction, $x_1 - x_2$, only records which of the two coordinates is slightly larger — if that is noise, PCA has removed it.



Blue: the five examples.

Green arrow: the first principal component, the direction of greatest spread; its eigenvalue is the variance along it.

Red segments: each example's projection onto PC1. Keeping only the projected position (the score) loses the short red distance — 10% of the variance.

Purple: the second component, at right angles.

Figure 16.1: The worked example. PCA rotates the axes to line up with the directions of greatest variance; dropping PC2 projects every point onto the green line.

16.3 How Many Components?

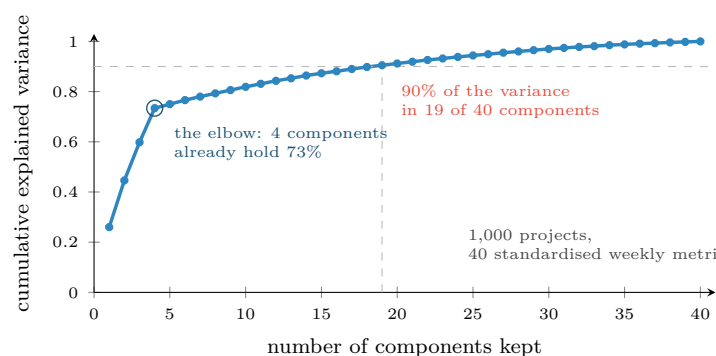


Figure 16.2: Cumulative explained variance for forty weekly project metrics. The curve rises steeply for four components and then flattens: four underlying factors drive almost everything the dashboard shows, and the rest is noise.

Three rules are in use: keep enough components to explain a target fraction of the variance (90% or 95% are common); keep the components before the *elbow* of the eigenvalue plot (the **scree plot**); or — when PCA is a step before a supervised model — treat the number of components as a hyperparameter and choose it by cross-validation. On the project metrics of this chapter's lab, logistic regression predicting which projects finish late scores 54% on one component, 63% on two, 89% on four, and slightly *less* on ten (88%) or all forty (88%): the four components before the elbow carry everything that matters, and the rest only adds noise. The 90% rule would have kept nineteen.

⚠ Standardise First, and Remember What PCA Ignores

PCA finds directions of large *variance*, so a feature measured in large units dominates the first component purely because of its units. Standardise first, unless the features share a scale that means something. And PCA is unsupervised: the direction of greatest variance need not be the direction that separates the classes. A small, discarded component can be the one that matters.

16.4 Self-Organizing Maps

A **self-organizing map** (SOM, or Kohonen map, after Teuvo Kohonen) is a neural network that learns to lay high-dimensional data out on a two-dimensional grid, so that similar examples land on nearby grid cells. It is unsupervised, and its output is a picture: a map of the data that people can read.

Self-Organizing Map

A grid of units (often 10×10), each holding a weight vector $\mathbf{w}_u$ of the same dimension as the data. Training repeats, for randomly chosen examples $\mathbf{x}$:

1. **Compete:** find the **best-matching unit** (BMU), the unit whose weight vector is closest to $\mathbf{x}$.
2. **Cooperate:** every unit u gets a **neighbourhood** weight $h(u)$, 1 at the BMU and falling with distance *on the grid* — usually $h(u) = \exp(-\text{grid distance}^2 / 2\sigma^2)$.
3. **Adapt:** move every unit towards the example: $\mathbf{w}_u \leftarrow \mathbf{w}_u + \eta h(u) (\mathbf{x} - \mathbf{w}_u)$.

Both the learning rate η and the neighbourhood width σ shrink during training: first the map organises roughly, then each unit fine-tunes.

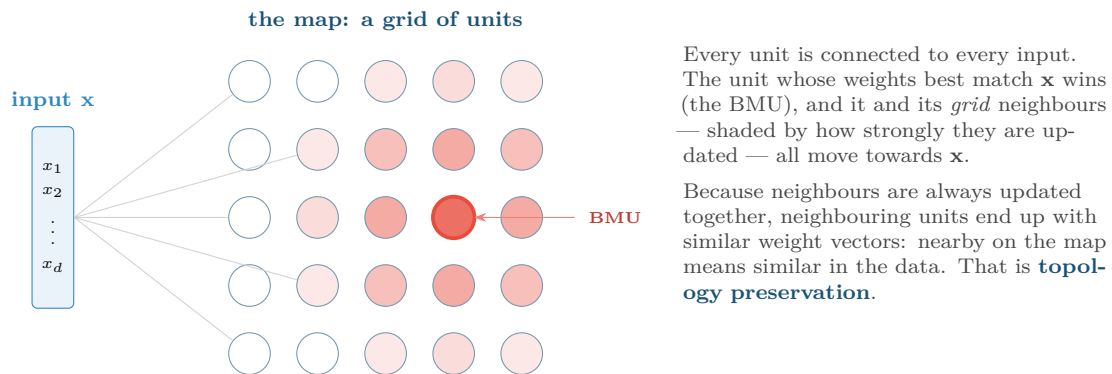


Figure 16.3: A self-organizing map. Competition picks one winner; cooperation spreads the update over its neighbourhood on the grid, which is what makes the map smooth.

Worked Example: One SOM Update

A one-dimensional map of three units with two-dimensional weights: $\mathbf{w}_1 = (0.2, 0.8)$, $\mathbf{w}_2 = (0.5, 0.5)$, $\mathbf{w}_3 = (0.9, 0.1)$. Input $\mathbf{x} = (0.8, 0.3)$; learning rate $\eta = 0.5$; neighbourhood $h = 1$ for the BMU, 0.5 for its immediate neighbours, 0 beyond.

Compete. Distances: $\|\mathbf{x} - \mathbf{w}_1\| = \sqrt{0.6^2 + 0.5^2} = 0.781$; $\|\mathbf{x} - \mathbf{w}_2\| = \sqrt{0.3^2 + 0.2^2} = 0.361$; $\|\mathbf{x} - \mathbf{w}_3\| = \sqrt{0.1^2 + 0.2^2} = 0.224$. **BMU: unit 3.**

Adapt.

- Unit 3 ($h = 1$): $\mathbf{w}_3 + 0.5 \times 1 \times ((0.8, 0.3) - (0.9, 0.1)) = (0.9, 0.1) + (-0.05, 0.10) = (0.85, 0.20)$.
- Unit 2 ($h = 0.5$, the BMU's neighbour): $(0.5, 0.5) + 0.25 \times (0.3, -0.2) = (0.575, 0.45)$.
- Unit 1 ($h = 0$, two steps away on the map): unchanged.

Unit 2 moved towards $\mathbf{x}$ although it did not win — that is the cooperation. After many such steps, unit 3 represents inputs like $\mathbf{x}$, and unit 2 inputs somewhere between unit 1's and unit 3's.

💡 Reading a Trained Map

Each unit becomes a prototype for the examples that map to it, so a SOM is also a clustering (and with $\sigma \rightarrow 0$ it is exactly on-line k -means). Colour each unit by the class of the examples it attracts and the classes appear as regions — the lab's six-by-six map of past projects puts maintenance work in one corner and the two similar kinds, new builds and integrations, next to each other elsewhere. The **U-matrix** colours each unit by the average distance to its neighbours' weights: high values mark the borders between groups, so cluster boundaries become visible walls on the map.

	PCA	Self-organizing map
Mapping	Linear projection	Non-linear; follows curved structure
Solution	Exact, unique (eigenvectors)	Iterative; depends on initialisation and schedule
Output	m new numeric features	A position on a 2-D grid, plus prototypes
Best for	Compression, pre-processing, removing correlation	Visual exploration, clustering, prototype-based summaries
New examples	Project with the same components	Map to the BMU

! Common Pitfalls

- **Running PCA on unscaled features.** The component is then decided by units.
- **Fitting PCA on all the data before splitting.** It is a fitted transformer: put it in the pipeline (Chapter 3).
- **Assuming high variance means high relevance.** PCA does not look at the labels.
- **Interpreting components as causes.** A component is a weighted mix of every feature; naming it takes care.
- **Keeping the SOM neighbourhood wide to the end.** The map never fine-tunes; shrink σ and η over training.

🔧 Try It Yourself: PCA by Hand, PCA at Work, and a Map of Past Projects

Part 2 makes 1,000 projects whose forty weekly metrics are driven by four hidden factors (size, quality, churn, staffing) plus noise, and asks which finish late. Part 3 maps 150 projects of three kinds: maintenance, new builds and integrations, each described by four standardised features.

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

# --- 1. the worked example: PCA by eigen-decomposition -----
X = np.array([[1, 2], [2, 1], [3, 3], [4, 5], [5, 4]], dtype=float)
Xc = X - X.mean(axis=0)                                # centre
S = np.cov(Xc, rowvar=False)                            # covariance, divides by n - 1
vals, vecs = np.linalg.eigh(S)                          # eigenvalues, ascending
order = np.argsort(vals)[::-1]
```

```

vals, vecs = vals[order], vecs[:, order]
print("covariance:\n", S)
print("eigenvalues:", vals, " explained:", (vals / vals.sum()).round(3))
print("PC1 direction:", vecs[:, 0].round(3))
print("scores on PC1:", (Xc @ vecs[:, 0]).round(3))

# --- 2. 40 dashboard metrics: how many components? -----
rng = np.random.default_rng(16)
n, d = 1000, 40                                # projects, weekly metrics
Z = rng.normal(size=(n, 4))                     # hidden: size, quality, churn, staff
A = rng.normal(size=(4, d))                     # how each metric reflects them
Xm = Z @ A + rng.normal(0, 1.0, (n, d))         # what the dashboard records
ym = (Z[:, 0] - Z[:, 1] + 0.5 * Z[:, 2] + rng.normal(0, 0.5, n) > 0) # late
pca = PCA().fit(StandardScaler().fit_transform(Xm))
cum = np.cumsum(pca.explained_variance_ratio_)
for target in (0.5, 0.8, 0.9, 0.95):
    print(f"{target:.0%} of variance needs {np.argmax(cum >= target) + 1} "
          f"of {d} components")
for k in (1, 2, 4, 10, 40):
    pipe = make_pipeline(StandardScaler(), PCA(k), LogisticRegression())
    acc = cross_val_score(pipe, Xm, ym, cv=5).mean()
    print(f"logistic regression on {k:2d} components: accuracy {acc:.3f}")

# --- 3. a self-organizing map in twenty lines -----
rng = np.random.default_rng(16)
kinds = np.repeat([0, 1, 2], 50)               # maintenance, build, integration
centre = np.array([[ -1.5, -1.2, -1.5, -0.5],    # small, short, little new
                   [ 0.8, 0.9, 1.2, 0.2],       # new build
                   [ 0.8, 0.6, 0.3, 1.0]])       # many integrations
Xi = centre[kinds] + rng.normal(0, 0.45, (150, 4))
Xi = StandardScaler().fit_transform(Xi)
rows, cols = 6, 6
grid = np.array([(r, c) for r in range(rows) for c in range(cols)])
rng = np.random.default_rng(0)
W = rng.normal(0, 0.5, (rows * cols, Xi.shape[1]))
steps = 3000
for t in range(steps):
    x = Xi[rng.integers(len(Xi))]
    bmu = np.argmin(((W - x) ** 2).sum(axis=1)) # best-matching unit
    eta = 0.5 * (1 - t / steps)                 # learning rate decays
    sigma = 3.0 * (1 - t / steps) + 0.5         # neighbourhood shrinks
    d2 = ((grid - grid[bmu]) ** 2).sum(axis=1)  # distance on the map
    h = np.exp(-d2 / (2 * sigma ** 2))          # neighbourhood weight
    W += eta * h[:, None] * (x - W)

# label each unit with the kind of project that most often maps to it
hits = {}
for x, y in zip(Xi, kinds):
    u = np.argmin(((W - x) ** 2).sum(axis=1))
    hits.setdefault(u, []).append(y)
names = "mbi"                                   # maintenance, build, integration
for r in range(rows):
    print(" ".join(names[np.argmax(np.bincount(hits[r * cols + c])
        if r * cols + c in hits else "." for c in range(cols)))

```

The last block prints the map: m, b and i mark units that maintenance, new-build and integration projects map to, and . an unused unit. Rerun it with $\sigma = 0.5$ throughout (no shrinking neighbourhood): is the map still organised?

? Checkpoint — Test Yourself

1. A covariance matrix has eigenvalues 6, 3 and 1. What fraction of the variance do the first two components explain?
2. Why must features usually be standardised before PCA?
3. In a SOM, what is the difference between the best-matching unit and its neighbourhood, and why are both updated?

✓ Chapter Summary

- Reducing dimensions fights the curse of dimensionality, removes redundancy and makes data visible; selection keeps features, extraction builds new ones.
- PCA's components are the eigenvectors of the covariance matrix; each eigenvalue is the variance along its component. Keep enough to explain a target fraction, or tune the number by cross-validation.
- Standardise before PCA, fit it inside the pipeline, and remember it ignores the labels.
- A SOM finds the best-matching unit for each input and moves it and its grid neighbours towards the input; with a shrinking neighbourhood it produces a smooth, topology-preserving 2-D map.
- PCA is linear, exact and for compression; the SOM is non-linear, iterative and for visual exploration.

✍ Review Questions

1. **(MCQ)** The first principal component is the direction of: (a) greatest class separation (b) greatest variance (c) smallest variance (d) the first feature.
2. **(MCQ)** In a SOM, the neighbourhood function is measured by distance: (a) in the input space (b) on the grid of units (c) between classes (d) between eigenvalues.
3. **(MCQ)** Principal components are: (a) correlated (b) at right angles to each other (c) always three in number (d) the original features.
4. **(Short)** Explain the difference between feature selection and feature extraction, with one method of each.
5. **(Short)** Why might a discarded low-variance component matter for classification?
6. **(Short)** What does a U-matrix show?
7. **(Applied)** For the points (1,3), (3,1), (4,4), (5,7) and (7,5), compute the covariance matrix, verify that (1,1) is an eigenvector, and find the fraction of the variance the first principal component explains.
8. **(Applied)** Continue the SOM worked example with the next input (0.3,0.7), using the updated weights. Which unit wins, and what are the three weight vectors afterwards?

Semi-Supervised Learning and the EM Algorithm

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — Learning Without Full Labels

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** when unlabelled examples can improve a classifier, and the assumptions that make it possible.
- **Describe** self-training, and **state** its main risk.
- **Formulate** a Gaussian mixture model with hidden cluster labels.
- **Perform** an iteration of the EM algorithm by hand: responsibilities in the E-step, weighted means in the M-step.
- **Apply** EM to a mixture of labelled and unlabelled data, and **relate** k -means to EM.

🔗 Before You Start

Chapter 2 (the Gaussian, maximum likelihood), Chapter 10 (posteriors and naive Bayes) and Chapter 15 (k -means). The algorithm of this chapter returns in Chapter 18.

📖 Key Terms in This Chapter

semi-supervised learning • smoothness, cluster and manifold assumptions • self-training (pseudo-labelling) • confirmation bias • latent (hidden) variable • mixture model • Gaussian mixture • mixing weight • responsibility • EM algorithm • E-step • M-step • local maximum • hard vs. soft assignment • label propagation

17.1 Few Labels, Many Examples

Labels are expensive. A senior engineer must read a ticket to name its root cause; a reviewer must study a commit to say whether it introduced a defect; a project manager must reconstruct why a project slipped. Unlabelled examples are nearly free: every ticket in the tracker, every commit in the history, every project in the archive. **Semi-supervised learning** uses both — a small labelled set and a large unlabelled one — to learn a better classifier than the labelled set alone would give.

How can examples with no labels possibly help? Only if the way the examples are spread out says something about where the classes lie. Three assumptions, all versions of the same idea, make that true:

- **Smoothness:** examples close together are likely to share a label.
- **Cluster:** examples form clusters, and a cluster tends to be one class — so the decision boundary should pass through low-density gaps, not through the middle of a cluster.
- **Manifold:** high-dimensional data lies near a lower-dimensional surface, and distances along that surface are what matter.

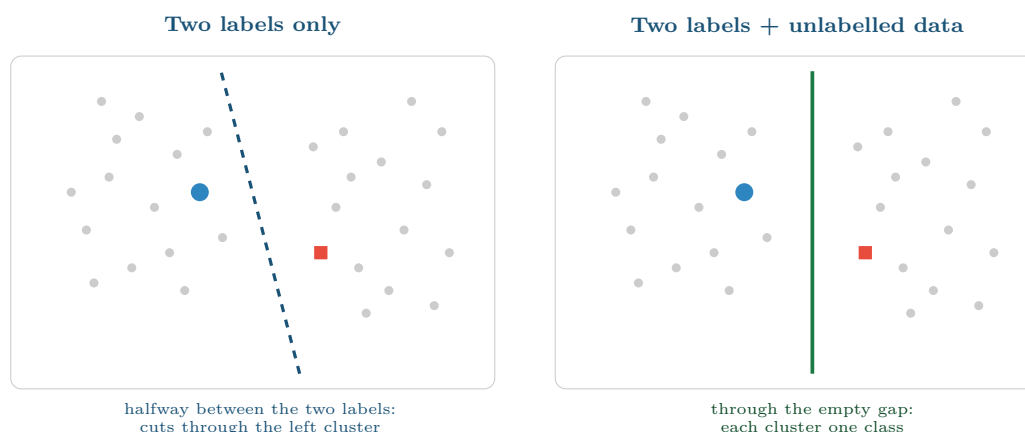


Figure 17.1: The cluster assumption. With two labelled points (blue circle, red square) a classifier puts the boundary between them, through the middle of a cluster. The unlabelled points (grey) reveal two clusters and a gap, and the boundary belongs in the gap.

⚠ Unlabelled Data Can Also Hurt

When the assumptions are false — the classes overlap in one blob, or a cluster contains both classes — unlabelled data pushes the boundary to the wrong place, and the semi-supervised model can be *worse* than one trained on the labels alone. Always compare against the labelled-only baseline on a held-out labelled test set.

17.2 Self-Training

The simplest method needs no new theory. **Self-training** (pseudo-labelling):

1. train a classifier on the labelled examples;
2. predict the unlabelled examples, and add those predicted with high confidence to the training set, with their predicted labels;
3. retrain, and repeat until no confident predictions remain.

It works with any classifier that gives probabilities, and on the tickets of this chapter's lab it lifts accuracy from 92% with 50 labels to 97% using the 1,210 unlabelled tickets as well (all 1,260 labels give 98%). Its risk is **confirmation bias**: an early confident mistake becomes a training label, is reinforced at every round, and is never corrected. A high confidence threshold limits the damage.

17.3 Mixture Models and Hidden Variables

A more principled approach models how the data was generated. Suppose each example was produced by first picking a class (or cluster) z , then drawing $\mathbf{x}$ from that class's distribution. For labelled examples z is observed; for unlabelled ones it is a **hidden** (latent) variable.

📖 Gaussian Mixture Model

A **mixture** of k Gaussians has, for each component j , a **mixing weight** π_j (the probability of choosing it, with $\sum_j \pi_j = 1$), a mean $\boldsymbol{\mu}_j$ and a covariance Σ_j . The probability density of an example is

$$p(\mathbf{x}) = \sum_{j=1}^k \pi_j \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \Sigma_j)$$

If we knew which component produced each example, fitting would be easy: average each component's own examples. If we knew the parameters, we could compute for each example

the probability that each component produced it. We know neither — and the EM algorithm alternates between the two.

17.4 The EM Algorithm

📖 Expectation–Maximisation (EM)

Start from initial parameter guesses. Repeat until the likelihood stops improving:

- **E-step:** with the current parameters, compute for every example i and component j the **responsibility** r_{ij} below — the posterior probability that component j produced example i .
- **M-step:** re-estimate each component from *all* examples, weighted by responsibility: $\mu_j = \sum_i r_{ij} \mathbf{x}_i / \sum_i r_{ij}$, $\pi_j = \frac{1}{n} \sum_i r_{ij}$, and Σ_j likewise.

$$r_{ij} = \frac{\pi_j \mathcal{N}(\mathbf{x}_i | \mu_j, \Sigma_j)}{\sum_l \pi_l \mathcal{N}(\mathbf{x}_i | \mu_l, \Sigma_l)}$$

Each iteration never decreases the likelihood of the data, so EM converges — to a **local maximum**, which depends on the starting point.

🧮 Worked Example: EM for Two Means (Mitchell’s Example, in Numbers)

Six tickets took $x = 1, 2, 3, 7, 8, 9$ hours to resolve. There seem to be two kinds — quick fixes and investigations — but nobody recorded which was which. Model: two Gaussians with known $\sigma = 1$ and equal weights; unknown means. Start from $\mu_1 = 2$, $\mu_2 = 5$.

E-step. For each x , the densities are proportional to $e^{-(x-\mu)^2/2}$:

- $x = 1$: $e^{-0.5} = 0.607$ against $e^{-8} = 0.0003 \Rightarrow r_1 = 0.999$
- $x = 2$: $e^0 = 1$ against $e^{-4.5} = 0.011 \Rightarrow r_1 = 0.989$
- $x = 3$: $e^{-0.5} = 0.607$ against $e^{-2} = 0.135 \Rightarrow r_1 = 0.818$
- $x = 7, 8, 9$: component 2 is overwhelmingly likelier $\Rightarrow r_1 \approx 0$

M-step. Responsibility-weighted means:

$$\mu_1 = \frac{0.999(1) + 0.989(2) + 0.818(3)}{0.999 + 0.989 + 0.818} = \frac{5.430}{2.806} = \mathbf{1.935}$$

$$\mu_2 = \frac{0.001(1) + 0.011(2) + 0.182(3) + 7 + 8 + 9}{0.001 + 0.011 + 0.182 + 3} = \frac{24.570}{3.194} = \mathbf{7.693}$$

Note that $x = 3$ contributes 18% of itself to component 2: a *soft* assignment.

Iteration 2 moves the means to 2.000 and 8.000, after which nothing changes. The log-likelihood of the data rises from -24.96 at the starting means to -11.82 after one iteration and -11.67 at convergence. EM has found the two groups $\{1, 2, 3\}$ and $\{7, 8, 9\}$.

💡 k -Means Is EM With Hard Assignments

Replace the E-step’s probabilities by a hard choice — responsibility 1 for the nearest mean, 0 for the others — and the M-step becomes “each mean is the average of its assigned points”. That is exactly k -means (Chapter 15). EM for Gaussian mixtures is the soft, probabilistic generalisation: clusters may overlap, may have different sizes and elongated shapes (through Σ_j), and each example gets a probability of membership instead of a verdict.

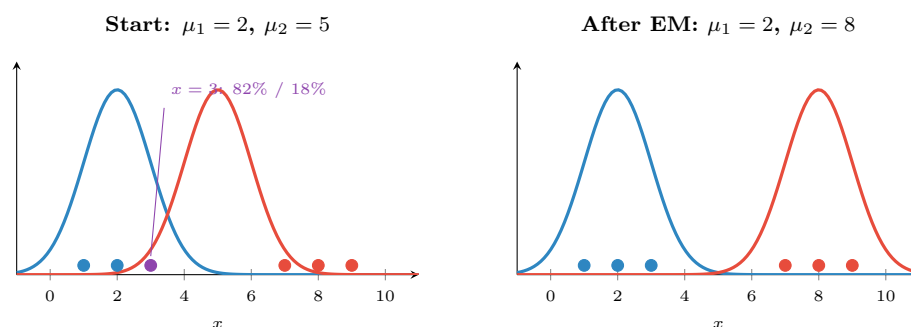


Figure 17.2: EM on the worked example. Each point is coloured by its responsibilities; $x = 3$ starts out shared between the components. Two iterations move the second Gaussian onto the group it explains.

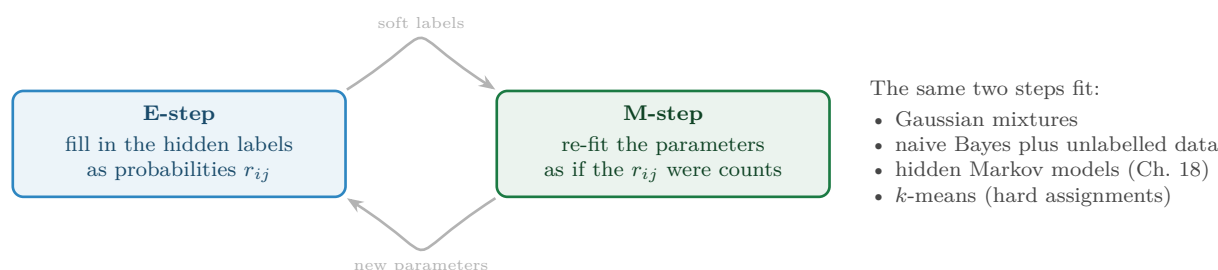


Figure 17.3: The EM loop. Whenever a model would be easy to fit if some variable were observed, EM fills that variable in with probabilities and fits as if it were observed.

17.5 EM With Labelled and Unlabelled Data

This is the method the HEC outline names. Nigam, McCallum, Thrun and Mitchell (2000) used it to classify text documents from a handful of labelled ones and thousands of unlabelled ones — exactly the position of a ticket tracker with two hundred triaged tickets and ten thousand untriaged. The recipe:

1. Fit the model — naive Bayes for text, or a Gaussian per class — on the labelled examples alone.
2. **E-step:** give each *unlabelled* example a probability for each class from the current model. Labelled examples keep their known labels (responsibility 1 for their class, 0 for the others).
3. **M-step:** refit the model on all examples, labelled ones counting in full and unlabelled ones fractionally, by their responsibilities.
4. Repeat until the parameters settle.

The labelled examples anchor each component to a class — they say *which* cluster is which — and the unlabelled examples say *where* the clusters really are. The authors report that adding unlabelled documents to small labelled sets reduced text-classification error by up to 30%.

Worked Example: Two Labels and Four Unlabelled Points

The six tickets of the previous example, but now an engineer has checked two: the 3-hour ticket was a quick fix (class A) and the 7-hour ticket an investigation (class B); the rest are unlabelled.

Labelled data alone gives $\mu_A = 3$, $\mu_B = 7$, and a boundary at 5.

E-step. $x = 1$ and $x = 2$ are far likelier under A, and $x = 8$ and $x = 9$ under B; $x = 3$ and $x = 7$ are clamped to their labels.

M-step. $\mu_A = (1 + 2 + 3)/3 = 2$ and $\mu_B = (7 + 8 + 9)/3 = 8$, to within a thousandth. The next iteration changes nothing.

The class means moved from 3 and 7 to **2 and 8** using points nobody labelled. Here the boundary stays at 5, but it would not if the two groups had different spreads or sizes — and in higher dimensions the direction of the boundary moves too.

Graph-based methods such as **label propagation** take the smoothness assumption literally: connect each example to its nearest neighbours in a graph, fix the labelled nodes, and let labels diffuse along the edges until they settle. They follow curved clusters that a Gaussian mixture cannot represent.

! Common Pitfalls

- **Skipping the labelled-only baseline.** Unlabelled data can make a model worse; prove that it helped.
- **Evaluating on pseudo-labels.** Test only on examples with genuine labels, set aside before training.
- **A low self-training threshold.** Early mistakes become permanent labels.
- **Running EM once.** It finds a local maximum; start it several times and keep the highest likelihood.
- **A mixture with the wrong shape.** If the classes are not roughly Gaussian clusters, the mixture's components will not line up with them.

🔧 Try It Yourself: EM by Hand, Then Fifty Labels and a Thousand Guesses

Part 3 uses 1,800 synthetic tickets, each described by ten numeric features of its text and history, with four possible root causes; only 50 of the training tickets are labelled.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.semi_supervised import SelfTrainingClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# --- 1. EM for two Gaussian means (sigma = 1 known, equal weights) -----
x = np.array([1.0, 2.0, 3.0, 7.0, 8.0, 9.0])
mu = np.array([2.0, 5.0]) # initial guesses
for it in range(1, 5):
    dens = np.exp(-0.5 * (x[:, None] - mu[None, :]) ** 2) # E-step
    r = dens / dens.sum(axis=1, keepdims=True) # responsibilities
    ll = np.log(dens.mean(axis=1) / np.sqrt(2 * np.pi)).sum()
    mu = (r * x[:, None]).sum(axis=0) / r.sum(axis=0) # M-step
    print(f"iteration {it}: log-likelihood {ll:8.3f} -> mu = {mu.round(3)}")

# --- 2. two labels, four unlabelled points -----
labelled = {2: 0, 3: 1} # x=3 is class A (0), x=7 class B (1)
mu = np.array([x[2], x[3]]) # labelled points alone: 3 and 7
print("from the two labels alone: mu =", mu)
for it in range(10):
    dens = np.exp(-0.5 * (x[:, None] - mu[None, :]) ** 2)
    r = dens / dens.sum(axis=1, keepdims=True)
    for i, c in labelled.items(): # a labelled point keeps its label
        r[i] = np.eye(2)[c]
    mu = (r * x[:, None]).sum(axis=0) / r.sum(axis=0)
    print("EM with the unlabelled points too: mu =", mu.round(3))
```

```
# --- 3. self-training with 50 labels out of 1,260 -----
X, y = make_classification(n_samples=1800, n_features=10, n_informative=6,
                          n_redundant=2, n_classes=4, n_clusters_per_class=1,
                          class_sep=2.0, random_state=6) # 4 root causes
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          random_state=0, stratify=y)

rng = np.random.default_rng(0)
keep = rng.choice(len(y_tr), size=50, replace=False)
y_part = np.full_like(y_tr, -1) # -1 means "unlabelled"
y_part[keep] = y_tr[keep]
base = make_pipeline(StandardScaler(), LogisticRegression())
only = base.fit(X_tr[keep], y_tr[keep]).score(X_te, y_te)
self_t = SelfTrainingClassifier(
    make_pipeline(StandardScaler(), LogisticRegression()),
    threshold=0.9).fit(X_tr, y_part)
print(f"50 labels only: {only:.3f}    self-training with the rest "
      f"unlabelled: {self_t.score(X_te, y_te):.3f}    "
      f"all {len(y_tr)} labels: "
      f"{base.fit(X_tr, y_tr).score(X_te, y_te):.3f}")
```

Change the self-training threshold to 0.5 and then 0.99. Which is better, and what does the answer say about confirmation bias?

🔍 Checkpoint — Test Yourself

1. State the cluster assumption, and give an example where it fails.
2. In one EM iteration, what does the E-step compute and what does the M-step compute?
3. How does k -means differ from EM for a Gaussian mixture?

📋 Chapter Summary

- Semi-supervised learning uses many unlabelled examples to improve a classifier trained on few labels, relying on the smoothness, cluster or manifold assumption; when those fail it can hurt.
- Self-training adds confident predictions as labels and retrains; its risk is confirmation bias.
- A Gaussian mixture explains data as coming from hidden components, each with a weight, mean and covariance.
- EM alternates an E-step (responsibilities: soft assignments) and an M-step (responsibility-weighted re-estimates), never lowering the likelihood and converging to a local maximum; k -means is its hard-assignment special case.
- With labelled and unlabelled data, labelled examples keep their labels in the E-step and anchor each component to a class.

🔧 Review Questions

1. (MCQ) In the EM algorithm, the responsibilities are computed in: (a) the M-step (b) the E-step (c) the initialisation (d) neither step.
2. (MCQ) k -means corresponds to EM with: (a) soft assignments (b) hard assignments (c) no M-step (d) labelled data only.
3. (MCQ) Self-training's main risk is: (a) underfitting (b) slow prediction (c) confirmation bias (d) the curse of dimensionality.
4. (Short) Why can unlabelled data improve a classifier at all?
5. (Short) Explain why EM can converge to different answers from different starting points, and what to do about it.

6. **(Short)** In semi-supervised EM, what role do the labelled examples play that the unlabelled ones cannot?
7. **(Applied)** Repeat the first EM iteration of the worked example with initial means $\mu_1 = 4$, $\mu_2 = 6$. Where do the means go?
8. **(Applied)** For the values 0, 1, 5, 6 with $\sigma = 1$ and equal weights, start EM from $\mu_1 = 0$, $\mu_2 = 1$ and perform two iterations. Comment on the result.

PART V

Sequences, Decisions and Reinforcement

Until now every example has been independent of the last. Part V drops that assumption. First a model of sequences that are only partly observed; then a model of choices whose consequences unfold over time; then an agent that learns those consequences by trying things.

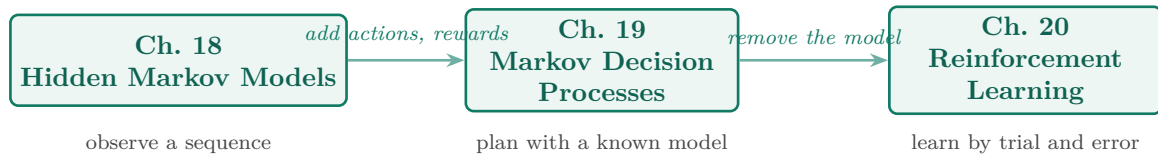


Figure P5.1: Reading path through Part V. Each chapter adds one thing to the last: an HMM observes, an MDP chooses with a known model, and reinforcement learning chooses without one.

Hidden Markov Models

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Sequences, Decisions and Reinforcement

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the Markov property, and **compute** with a Markov chain's transition matrix.
- **Specify** a hidden Markov model by its states, observations, initial, transition and emission probabilities.
- **Compute** the probability of an observation sequence with the forward algorithm, by hand.
- **Find** the most likely hidden-state sequence with the Viterbi algorithm, by hand.
- **Explain** how Baum–Welch learns an HMM's parameters, and **estimate** a probability by Monte Carlo sampling.

🔗 Before You Start

Chapter 2 (conditional probability), Chapter 10 (Bayes' theorem, posteriors) and Chapter 17 (the EM algorithm, which trains HMMs). The HEC outline lists HMMs under reinforcement learning; they belong here, at the start of Part V, because the Markov machinery they introduce is what Markov decision processes (Chapter 19) build on.

🔑 Key Terms in This Chapter

sequence • Markov property • Markov chain • transition matrix • stationary distribution • hidden state • observation (emission) • initial distribution π • emission matrix • evaluation, decoding and learning problems • forward algorithm • Viterbi algorithm • backpointer • Baum–Welch • Monte Carlo estimation

18.1 When Order Matters

Every model so far has treated examples as independent: shuffling the training set changes nothing. Much project data is a *sequence* in which the order carries the information — a project's weekly status reports, the stages a ticket passes through, the runs of a build pipeline, the commits on a feature branch. Hidden Markov models were the standard tool for sequences for thirty years, and they remain the clearest introduction to reasoning about them.

18.2 Markov Chains

📖 Markov Property and Markov Chain

A sequence of states $s_1, s_2, \dots$ has the **Markov property** if the next state depends only on the current one, not on the history: $P(s_{t+1} | s_t, s_{t-1}, \dots, s_1) = P(s_{t+1} | s_t)$. A **Markov chain** is such a sequence, described by an initial distribution π and a **transition matrix** A with $A_{ij} = P(s_{t+1} = j | s_t = i)$; each row of A sums to 1.

A project is either Healthy (H) or Distressed (D). From one week to the next a healthy project stays healthy with probability 0.8 and slips into distress with probability 0.2; a distressed one recovers with probability 0.3 and stays distressed with 0.7:

$$A = \begin{pmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{pmatrix}$$

If it is healthy now, the probability it is distressed in two weeks sums over the two routes: $H \rightarrow H \rightarrow D = 0.8 \times 0.2 = 0.16$ and $H \rightarrow D \rightarrow D = 0.2 \times 0.7 = 0.14$, total 0.30 — which is the entry of A^2 in row H, column D. Run long enough, the chain forgets where it started and settles into a **stationary distribution**: here 60% healthy, 40% distressed (check: $0.6 \times 0.2 = 0.4 \times 0.3$, so the flows between the states balance).

18.3 Hidden Markov Models

In practice a project's true state is not visible. What the project office sees is the weekly *status report*, which depends on the state but only probabilistically: a healthy project usually reports *on plan*, a distressed one usually *behind* or *blocked*, but either can produce any report. The states are *hidden*; the observations are what is recorded.

Hidden Markov Model

An HMM has hidden states S and observable symbols O , and three sets of probabilities:

- the **initial distribution** $\pi_i = P(s_1 = i)$;
- the **transition matrix** $A_{ij} = P(s_{t+1} = j \mid s_t = i)$;
- the **emission matrix** $B_{ik} = P(o_t = k \mid s_t = i)$.

The hidden states form a Markov chain, and each observation depends only on the hidden state at the same time.

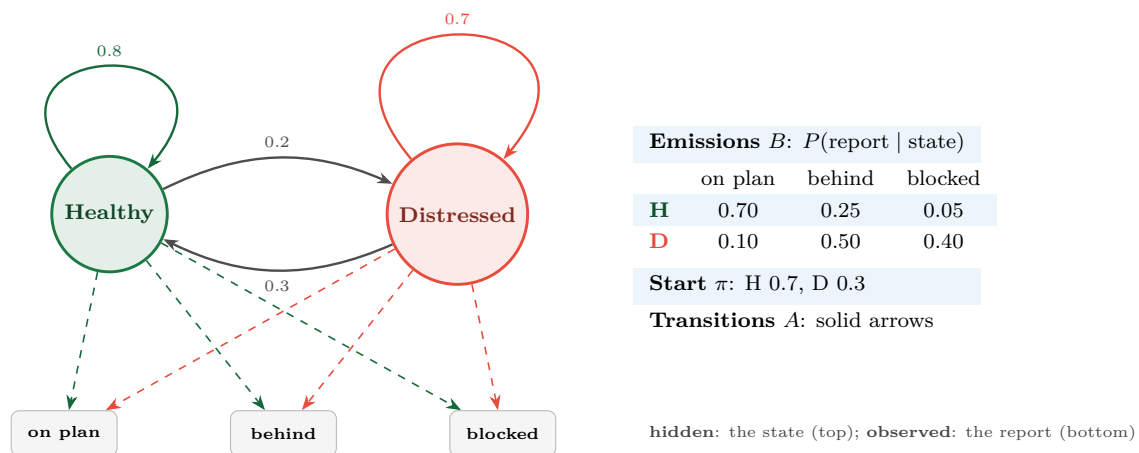


Figure 18.1: The hidden Markov model used throughout this chapter. The project office never sees the state, only a weekly status report whose probabilities depend on it.

Three Questions an HMM Answers

- **Evaluation:** how probable is an observed sequence under this model? (Is this run of status reports normal?) — the forward algorithm.
- **Decoding:** which sequence of hidden states most probably produced it? (When did the project actually slip?) — the Viterbi algorithm.

- **Learning:** which parameters make the observed sequences most probable? — the Baum–Welch algorithm, an instance of EM.

18.4 Evaluation: The Forward Algorithm

The direct way to find $P(o_1, \dots, o_T)$ sums over every possible hidden-state sequence — N^T of them, which for 2 states and 100 time steps is 2^{100} . The forward algorithm avoids the explosion by reusing partial sums.

Forward Algorithm

Let $\alpha_t(i) = P(o_1, \dots, o_t, s_t = i)$: the probability of the observations so far *and* being in state i now.

- Start: $\alpha_1(i) = \pi_i B_i(o_1)$.
- Step: $\alpha_{t+1}(j) = \left[\sum_i \alpha_t(i) A_{ij} \right] B_j(o_{t+1})$ — every way of arriving in j , times the chance of emitting what was seen.
- End: $P(o_1, \dots, o_T) = \sum_i \alpha_T(i)$.

The cost is N^2T instead of N^T .

Worked Example: How Likely Is “On Plan, Behind, Blocked”?

Three weekly reports $o = (\text{on plan, behind, blocked})$, model of Figure 18.1.

$t = 1$ (**on plan**): $\alpha_1(H) = 0.7 \times 0.70 = 0.49$; $\alpha_1(D) = 0.3 \times 0.10 = 0.03$.

$t = 2$ (**behind**):

$$\alpha_2(H) = (0.49 \times 0.8 + 0.03 \times 0.3) \times 0.25 = 0.401 \times 0.25 = 0.10025$$

$$\alpha_2(D) = (0.49 \times 0.2 + 0.03 \times 0.7) \times 0.50 = 0.119 \times 0.5 = 0.0595$$

$t = 3$ (**blocked**):

$$\alpha_3(H) = (0.10025 \times 0.8 + 0.0595 \times 0.3) \times 0.05 = 0.09805 \times 0.05 = 0.00490$$

$$\alpha_3(D) = (0.10025 \times 0.2 + 0.0595 \times 0.7) \times 0.40 = 0.0617 \times 0.4 = 0.02468$$

Answer: $P(o) = 0.00490 + 0.02468 = \mathbf{0.0296}$.

The same numbers answer a second question for free: given everything seen so far, the project is now distressed with probability $0.02468/0.0296 = \mathbf{0.83}$ — the *filtered* state estimate, which is what a portfolio dashboard wants.

18.5 Decoding: The Viterbi Algorithm

Viterbi has the same structure as the forward algorithm with one change: replace the sum over incoming paths by a *maximum*, and remember which predecessor gave it.

Viterbi Algorithm

Let $\delta_t(j)$ be the probability of the single most likely state sequence ending in state j at time t (with the observations so far).

- Start: $\delta_1(i) = \pi_i B_i(o_1)$.
- Step: $\delta_{t+1}(j) = \max_i [\delta_t(i) A_{ij}] B_j(o_{t+1})$, and store the maximising i as the **backpointer** for $(t+1, j)$.
- End: take the state with the largest δ_T , then follow the backpointers back to $t = 1$.

Worked Example: When Did the Project Slip?

The same reports: on plan, behind, blocked.

$t = 1$: $\delta_1(H) = 0.49$, $\delta_1(D) = 0.03$.

$t = 2$ (**behind**):

- into H: from H $0.49 \times 0.8 = 0.392$, from D $0.03 \times 0.3 = 0.009$; best from H. $\delta_2(H) = 0.392 \times 0.25 = 0.098$.
- into D: from H $0.49 \times 0.2 = 0.098$, from D $0.03 \times 0.7 = 0.021$; best from H. $\delta_2(D) = 0.098 \times 0.5 = 0.049$.

$t = 3$ (**blocked**):

- into H: from H $0.098 \times 0.8 = 0.0784$, from D $0.049 \times 0.3 = 0.0147$; best from H. $\delta_3(H) = 0.0784 \times 0.05 = 0.00392$.
- into D: from H $0.098 \times 0.2 = 0.0196$, from D $0.049 \times 0.7 = 0.0343$; best from D. $\delta_3(D) = 0.0343 \times 0.4 = \mathbf{0.01372}$.

Backtrack. The best final state is D (0.01372). Its backpointer is D at $t = 2$, whose backpointer is H at $t = 1$. **Most likely states: H, D, D** — the project slipped between the first and second week, when the first “behind” report appeared, not when it became blocked.

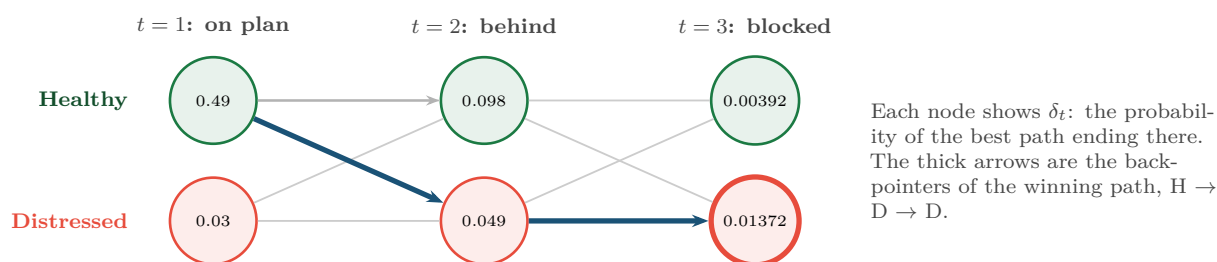


Figure 18.2: The Viterbi trellis for the worked example. Every hidden-state sequence is a path through the trellis; Viterbi keeps only the best path into each node.

18.6 Learning: Baum–Welch

Where do π , A and B come from? If the hidden states had been recorded, they would be counts: how often H followed H, how often D reported “blocked”. They were not — so the **Baum–Welch** algorithm applies EM (Chapter 17). The **E-step** runs the forward algorithm and its mirror image, the backward algorithm, to compute for every time step the probability of each hidden state and of each transition, given the whole observed sequence. The **M-step** re-estimates A and B as expected counts: the expected number of $H \rightarrow D$ transitions divided by the expected number of visits to H, and so on. As with every EM, the likelihood never decreases and the answer is a local maximum. This chapter’s lab generates 3,000 weekly reports from the model, forgets the parameters, and from a vague start recovers

$$A \approx \begin{pmatrix} 0.78 & 0.22 \\ 0.31 & 0.69 \end{pmatrix},$$

close to the true values.

18.7 Monte Carlo Estimation

There is a completely different way to answer the evaluation question: *simulate*. Generate many sequences from the model and count how often the observed one appears.

☰ Monte Carlo Estimation

To estimate a probability or an expectation, draw many independent samples from the model and average: the fraction of samples in which an event occurs estimates its probability, and the average of a quantity over the samples estimates its expected value. The error shrinks like $1/\sqrt{\text{number of samples}}$ — the same square-root law as the confidence intervals of Chapter 8.

For three observations the forward algorithm is exact and instant, and simulation is pointless: with 1,000 samples the lab's estimate is 0.025, with 10,000 it is 0.032, and with 100,000 it is 0.0296, finally matching. Monte Carlo earns its place when no exact algorithm exists — models too complex to sum over, or, as in Chapter 20, an environment whose probabilities are unknown and can only be experienced. There, averaging the outcomes of many trials is the only estimate available, and it is the foundation of Monte Carlo reinforcement learning.

⚠ Why HMMs Were Largely Replaced — and Where They Remain

An HMM's hidden state is one symbol from a small set, and each observation depends on the current state alone. Long texts, such as a ticket's whole discussion thread, have longer dependencies than that, and recurrent networks and transformers (the Deep Learning course) now dominate text. HMMs remain the right tool when the data is small, the states are meaningful and must be interpretable — a project's health, a ticket's stage, the phase of a release — and when an exact probability is needed rather than a score.

❗ Common Pitfalls

- **Multiplying probabilities along long sequences.** They underflow to zero; scale at each step or work in logarithms (Viterbi is usually run in log space, turning products into sums).
- **Confusing the forward and Viterbi results.** Forward sums over all paths (probability of the observations); Viterbi keeps the single best path.
- **Taking the most likely state at each step as the best sequence.** The individually likeliest states may form a sequence that is itself unlikely, or even impossible; decode with Viterbi.
- **Trusting Baum–Welch from one start.** It finds a local maximum; restart from several initial guesses.
- **Assuming the Markov property holds.** If the next state depends on a long history, a first-order HMM is the wrong model.

🧑 Try It Yourself: Forward, Viterbi, Monte Carlo and Baum-Welch

```
import numpy as np

# --- the worked example: a project, Healthy (0) or Distressed (1) -----
pi = np.array([0.7, 0.3])          # initial state
A = np.array([[0.8, 0.2],          # transitions, row = from
               [0.3, 0.7]])
B = np.array([[0.70, 0.25, 0.05],   # reports: on plan, behind,
               [0.10, 0.50, 0.40]]) #           blocked
obs = [0, 1, 2]                    # on plan, behind, blocked
```

```

def forward(obs, pi, A, B):
    alpha = pi * B[:, obs[0]]
    for o in obs[1:]:
        alpha = (alpha @ A) * B[:, o]
    return alpha # sums to P(obs)

def viterbi(obs, pi, A, B):
    delta, back = pi * B[:, obs[0]], []
    for o in obs[1:]:
        cand = delta[:, None] * A # cand[i, j]: from i to j
        back.append(cand.argmax(axis=0))
        delta = cand.max(axis=0) * B[:, o]
    path = [int(delta.argmax())]
    for b in reversed(back):
        path.insert(0, int(b[path[0]]))
    return path, delta.max()

alpha = forward(obs, pi, A, B)
print("P(on plan, behind, blocked) =", round(alpha.sum(), 6))
print("P(state at t=3 | obs) =", (alpha / alpha.sum()).round(3))
path, p = viterbi(obs, pi, A, B)
print("most likely states:", ["HD"[s] for s in path], "prob", round(p, 5))

# --- Monte Carlo: estimate the same probability by sampling -----
rng = np.random.default_rng(0)

def sample(T):
    s = rng.choice(2, p=pi)
    out = []
    for t in range(T):
        out.append(rng.choice(3, p=B[s]))
        s = rng.choice(2, p=A[s])
    return out

for n in (1_000, 10_000, 100_000):
    hits = sum(sample(3) == obs for _ in range(n))
    print(f"Monte Carlo, {n:6d} samples: {hits / n:.4f}")

# --- Baum-Welch (EM): relearn A and B from one long sampled sequence -----
seq = np.array(sample(3000))
A_hat = np.array([[0.5, 0.5], [0.5, 0.5]]) # deliberately vague start
B_hat = np.array([[0.5, 0.3, 0.2], [0.2, 0.3, 0.5]])
p_hat = np.array([0.5, 0.5])
T = len(seq)
for it in range(60):
    a, c = np.zeros((T, 2)), np.zeros(T) # scaled forward pass
    a[0] = p_hat * B_hat[:, seq[0]]
    c[0] = a[0].sum()
    a[0] /= c[0]
    for t in range(1, T):
        a[t] = (a[t - 1] @ A_hat) * B_hat[:, seq[t]]
        c[t] = a[t].sum()
        a[t] /= c[t]

    b = np.ones((T, 2)) # scaled backward pass
    for t in range(T - 2, -1, -1):
        b[t] = A_hat @ (B_hat[:, seq[t + 1]] * b[t + 1]) / c[t + 1]

```

```

g = a * b                                     # E-step:  $P(\text{state}_t \mid \text{obs})$ 
xi = (a[:-1, :, None] * A_hat[None] *
      (B_hat[:, seq[1:]].T * b[1:][:, None, :]) / c[1:, None, None])
A_hat = xi.sum(0) / g[:-1].sum(0)[:, None]    # M-step: expected counts
B_hat = np.array([g[seq == k].sum(0) for k in range(3)]).T
B_hat /= g.sum(0)[:, None]
p_hat = g[0]
print("relearned A:\n", A_hat.round(2))
print("relearned B:\n", B_hat.round(2))

```

The forward and Viterbi results reproduce both worked examples. Change the start of Baum–Welch to `B_hat = [[0.2, 0.3, 0.5], [0.5, 0.3, 0.2]]`: the states come out the other way round. Why is that not an error?

🔍 Checkpoint — Test Yourself

1. In the Markov chain of this chapter, a project is distressed now. What is the probability it is healthy two weeks later?
2. What does $\alpha_t(i)$ represent, and how does $\delta_t(i)$ differ from it?
3. Why does the direct sum over all hidden sequences become impractical, and how does the forward algorithm avoid it?

📋 Chapter Summary

- A Markov chain's next state depends only on the current one; its transition matrix gives multi-step probabilities and a stationary distribution.
- An HMM adds observations emitted from hidden states: π , A and B define it completely.
- The forward algorithm computes the probability of an observation sequence in N^2T steps, and yields the filtered state probabilities.
- Viterbi replaces the forward sum by a maximum with backpointers, giving the most likely hidden-state sequence.
- Baum–Welch learns the parameters by EM; Monte Carlo sampling estimates any probability by simulation, with error falling as $1/\sqrt{n}$.

🔧 Review Questions

1. (MCQ) In an HMM, the emission matrix gives: (a) $P(s_{t+1} \mid s_t)$ (b) $P(o_t \mid s_t)$ (c) $P(s_1)$ (d) $P(s_t \mid o_t)$.
2. (MCQ) The Viterbi algorithm differs from the forward algorithm by replacing a sum with: (a) a product (b) a maximum (c) an average (d) a logarithm.
3. (MCQ) Baum–Welch is an instance of: (a) gradient descent (b) the EM algorithm (c) k -means (d) Q-learning.
4. (Short) State the three problems an HMM can solve and the algorithm for each.
5. (Short) Why can choosing the most likely state at each time step separately give a worse answer than Viterbi?
6. (Short) When is Monte Carlo estimation preferable to an exact algorithm?
7. (Applied) Compute $P(\text{behind, behind})$ for the model of this chapter with the forward algorithm, and the filtered probability that the project is distressed after the second report.
8. (Applied) Run Viterbi on the reports (blocked, on plan, on plan) and interpret the decoded state sequence.

Markov Decision Processes

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Sequences, Decisions and Reinforcement

Learning Outcomes

By the end of this chapter you will be able to:

- **Formulate** a sequential decision problem as a Markov decision process: states, actions, transitions, rewards and discount.
- **Compute** the discounted return of a sequence of rewards, and **explain** the role of γ .
- **Define** state values and action values, and **write** the Bellman equations for them.
- **Perform** value iteration by hand for a small MDP and **read off** the optimal policy.
- **Describe** policy iteration and **compare** it with value iteration.

Before You Start

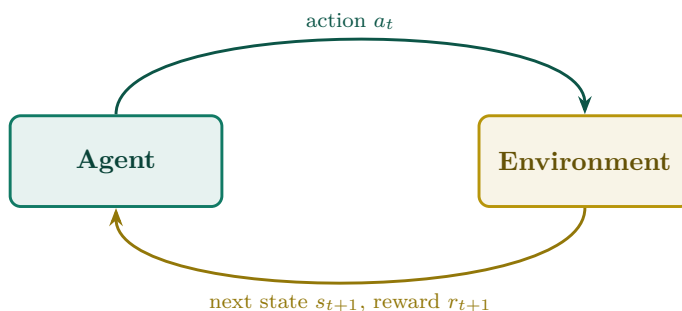
Chapter 18 (Markov chains and transition matrices) and, from *Artificial Intelligence*, agents and search. An MDP is a Markov chain in which an agent chooses the transition probabilities by its actions, and is paid for doing so.

Key Terms in This Chapter

agent • environment • state • action • reward • Markov decision process • transition model • policy • return • discount factor γ • state-value function V • action-value function Q • Bellman expectation equation • Bellman optimality equation • value iteration • policy evaluation • policy improvement • policy iteration

19.1 Decisions Whose Consequences Unfold

Classification makes one decision per example, and the next example does not depend on it. Many project decisions are not like that. Refactoring ships nothing this sprint but keeps the team fast in later ones; adding a developer to a late project slows it before it speeds it up; releasing on a Friday saves a day now and risks a weekend of firefighting. The value of an action depends on the situations it leads to, and on the actions that will be taken there. The framework for such problems is the **Markov decision process**, and it is the foundation of reinforcement learning (Chapter 20).



At each step the agent observes the state, chooses an action, and the environment responds with a new state and a numeric reward. The agent's goal is to choose actions that maximise the *total* reward over time, not the next reward alone.

Figure 19.1: The agent–environment loop. Every problem in Part V's last two chapters has this shape.

19.2 The Markov Decision Process

Markov Decision Process

An MDP consists of:

- a set of **states** S and a set of **actions** A ;
- a **transition model** $P(s' | s, a)$ — the probability that action a in state s leads to state s' ;
- a **reward** $R(s, a)$ received for taking a in s (or $R(s, a, s')$);
- a **discount factor** $0 \leq \gamma < 1$.

It has the Markov property: what happens next depends only on the current state and action. A **policy** π says what to do in each state, $a = \pi(s)$; solving the MDP means finding the policy that maximises the expected discounted return.

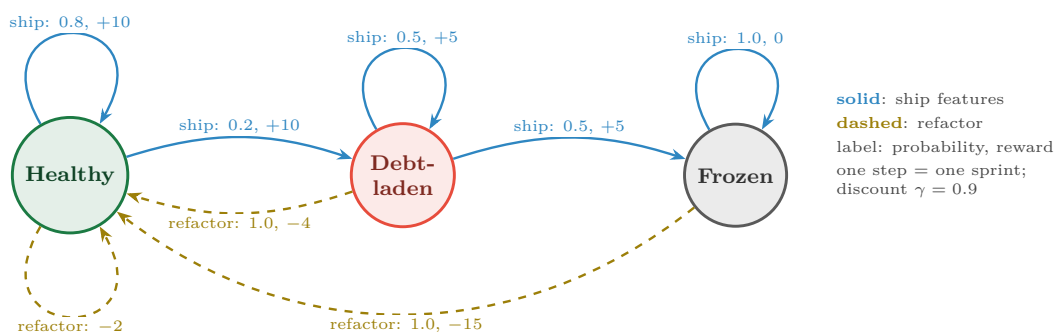


Figure 19.2: The technical-debt MDP of the worked examples. Shipping features earns value but lets debt build up; refactoring costs a sprint's value and restores the codebase.

The codebase is *Healthy* (H), *Debt-laden* (D) — every change takes longer than it should — or *Frozen* (F), so fragile that each change breaks something else and the team does nothing but fight fires. Each sprint the team either ships features, earning value (10 when healthy, 5 when debt-laden, nothing when frozen) but risking more debt, or spends the sprint refactoring, which costs value (2, 4, or 15 for a rewrite of a frozen codebase) and restores health.

Return and Discounting

The **return** from time t is the discounted sum of future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

With $\gamma < 1$ the sum is finite even for tasks that never end, and a reward k steps away is worth γ^k of the same reward now. A small γ makes the agent short-sighted; γ close to 1 makes it patient.

Worked Example: Two Returns

With $\gamma = 0.9$:

- Rewards 10, 10, 10, 5, 5, 0, 0, ... (keep shipping until the codebase freezes): $G = 10 + 9 + 8.1 + 5(0.729) + 5(0.656) = 27.1 + 3.645 + 3.281 = \mathbf{34.0}$.
- Rewards 10, 10, -4, 10, 10, ... (refactor once, then keep shipping): the first two give 19; the refactoring costs $4 \times 0.81 = 3.24$; and a healthy codebase thereafter earns, at the least,

$10(0.729 + 0.656 + \dots) = 10 \times 0.729 / (1 - 0.9) = 72.9$ if it never gathers debt again. The refactoring pays for itself many times over.

The second calculation needs to know what happens after the refactoring — which is exactly what the value functions below capture.

19.3 Value Functions and the Bellman Equations

☰ State and Action Values

Under a policy π , the **state value** $V^\pi(s)$ is the expected return starting from s and following π ; the **action value** $Q^\pi(s, a)$ is the expected return from taking a in s and following π afterwards. They satisfy the **Bellman expectation equation**:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s' | s, \pi(s)) V^\pi(s')$$

— the value of a state is the immediate reward plus the discounted value of where you go next. The best achievable values satisfy the **Bellman optimality equation**:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right] = \max_a Q^*(s, a)$$

and the optimal policy is to act greedily with respect to them: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

💡 Why a Recursive Definition Is Useful

The Bellman equation turns a sum over infinitely many futures into a relation between neighbouring states. For a known policy it is a set of linear equations — one per state — that can be solved directly. For the optimal values it is not linear (because of the max), but it can be solved by repeatedly applying it, which is value iteration.

19.4 Value Iteration

☰ Value Iteration

Start with $V(s) = 0$ for every state. Repeat until the values stop changing: for every state, set

$$V(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V(s') \right]$$

Then read off the policy: in each state, the action achieving the maximum. After k sweeps, V is the best return achievable in k steps; as k grows it converges to V^* .

📊 Worked Example: Value Iteration on the Technical-Debt MDP

$\gamma = 0.9$; start from $V = (0, 0, 0)$ for (H, D, F).

Sweep 1. With all values 0, only immediate rewards count: ship gives 10, 5, 0 and refactor −2, −4, −15. $V = (10, 5, 0)$; best action everywhere: ship.

Sweep 2.

- H: ship $10 + 0.9(0.8 \times 10 + 0.2 \times 5) = 10 + 8.1 = 18.1$; refactor $-2 + 0.9 \times 10 = 7$. $V(H) = 18.1$.
- D: ship $5 + 0.9(0.5 \times 5 + 0.5 \times 0) = 7.25$; refactor $-4 + 0.9 \times 10 = 5$. $V(D) = 7.25$.
- F: ship $0 + 0.9 \times 0 = 0$; refactor $-15 + 9 = -6$. $V(F) = 0$.

Sweep 3.

- H: ship $10 + 0.9(0.8 \times 18.1 + 0.2 \times 7.25) = 24.34$; refactor $-2 + 16.29 = 14.29$. $V(H) = 24.34$.
- D: ship $5 + 0.9(0.5 \times 7.25 + 0) = 8.26$; refactor $-4 + 0.9 \times 18.1 = \mathbf{12.29}$. **Refactoring now wins.**
- F: ship 0; refactor $-15 + 16.29 = \mathbf{1.29}$. **Refactoring wins.**

Converged (after about 200 sweeps): $V^* = (78.64, 66.78, 55.78)$, and the optimal policy is **ship when healthy, refactor when debt-laden or frozen**.

Check with the Bellman equation for that policy: $V(H) = 10 + 0.9(0.8V(H) + 0.2V(D))$ and $V(D) = -4 + 0.9V(H)$ give $V(H) = 9.28/0.118 = 78.64$. For comparison, “always ship” is worth only $V(H) = 41.56$: the codebase eventually freezes and delivers nothing for ever.

19.5 Policy Iteration

Value iteration improves values and only implicitly improves the policy. **Policy iteration** alternates two explicit steps:

1. **Policy evaluation:** compute V^π for the current policy exactly, by solving the Bellman expectation equations — one linear equation per state.
2. **Policy improvement:** make the policy greedy with respect to V^π : $\pi(s) \leftarrow \arg \max_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^\pi(s')]$.

Stop when the policy no longer changes. Each improvement is guaranteed not to make any state worse, and there are finitely many policies, so it terminates — usually in a handful of iterations. On the technical-debt MDP it starts from “always ship” ($V(H) = 41.56$), improves once to “refactor when debt-laden or frozen”, evaluates that ($V(H) = 78.64$), finds nothing to change, and stops: two evaluations against about two hundred sweeps of value iteration, each evaluation costlier.

	Value iteration	Policy iteration
Each iteration	One cheap sweep: a max over actions per state	A full policy evaluation (solve $ S $ linear equations) plus an improvement
Iterations needed	Many; converges gradually	Few; stops exactly
Best when	Many states, cheap sweeps	Fewer states, or a good starting policy
Needs the model?	Yes: P and R	Yes: P and R

⚠ Both Methods Need the Model

Value and policy iteration are *planning* algorithms: they compute the optimal policy from a known $P(s' | s, a)$ and $R(s, a)$. In most real problems nobody writes those down — the probability that a refactoring really clears the debt, that a new hire speeds the team up, or that a client accepts a release is unknown. Chapter 20 removes the model: the agent learns the same value functions from experience, by trying actions and observing what happens.

❗ Common Pitfalls

- **Maximising the next reward.** The point of an MDP is that the best action now may earn less now.
- **Choosing γ carelessly.** A small γ ignores long-term consequences; $\gamma = 1$ can make values infinite in tasks that never end.

- **A state that is not Markov.** If the right action depends on history the state does not record, add that history to the state.
- **Stopping value iteration early and trusting the values.** The policy often settles long before the values do; check both.
- **Rewarding the wrong thing.** An agent maximises exactly the reward it is given. Reward the outcome you want, not a proxy it can game.

Try It Yourself: Value Iteration and Policy Iteration

```
import numpy as np

# --- the technical-debt MDP: states H, D, F; actions ship (0), refactor (1)
S, A = ["H", "D", "F"], ["ship", "refactor"]
P = np.zeros((2, 3, 3))          # P[a, s, s']
P[0, 0] = [0.8, 0.2, 0.0]        # ship when healthy
P[0, 1] = [0.0, 0.5, 0.5]        # ship when debt-laden
P[0, 2] = [0.0, 0.0, 1.0]        # ship when frozen: stays frozen
P[1, :, 0] = 1.0                 # refactor: back to healthy
R = np.array([[10.0, 5.0, 0.0],   # R[a, s]: ship
               [-2.0, -4.0, -15.0]]) # refactor
gamma = 0.9

# --- value iteration -----
V = np.zeros(3)
for it in range(1, 1000):
    Q = R + gamma * P @ V          # Q[a, s]
    V_new = Q.max(axis=0)
    if it <= 3:
        print(f"iteration {it}: V = {V_new.round(3)} "
              f"policy = {[A[a] for a in Q.argmax(axis=0)]}")
    if np.abs(V_new - V).max() < 1e-8:
        break
    V = V_new
print(f"converged after {it} iterations: V = {V.round(2)}")
print("optimal policy:", dict(zip(S, [A[a] for a in Q.argmax(axis=0)])))

# --- policy iteration: evaluate exactly, then improve -----
policy = np.zeros(3, dtype=int)    # start: always ship
while True:
    Pp = P[policy, np.arange(3)]   # 3 x 3 under the policy
    Rp = R[policy, np.arange(3)]
    V = np.linalg.solve(np.eye(3) - gamma * Pp, Rp) # V = R + g P V
    print("evaluate", [A[a] for a in policy], "->", V.round(2))
    new = (R + gamma * P @ V).argmax(axis=0) # improve greedily
    if (new == policy).all():
        break
    policy = new
```

Both methods reproduce the worked example. Now change `gamma` to 0.5 — a team that cares about this sprint far more than the ones after it — and rerun. What happens to the policy, and what does that say about teams that are always under deadline pressure?

Checkpoint — Test Yourself

1. With $\gamma = 0.5$, what is the return of the rewards 4, 4, 4, 0, 0, ...?
2. What is the difference between $V^\pi(s)$ and $Q^\pi(s, a)$?
3. In the technical-debt MDP, why does “always ship” score so much less than the optimal policy, when shipping always earns more immediately than refactoring?

 Chapter Summary

- An MDP has states, actions, a transition model, rewards and a discount; a policy maps states to actions.
- The return is the discounted sum of future rewards; γ sets how far ahead the agent looks.
- The Bellman equations relate each state's value to its neighbours'; the optimal policy acts greedily with respect to the optimal action values.
- Value iteration repeatedly applies the Bellman optimality update; policy iteration alternates exact evaluation and greedy improvement and usually needs very few rounds.
- Both are planning methods that require the model; reinforcement learning learns without it.

 Review Questions

1. **(MCQ)** The discount factor γ controls: (a) the learning rate (b) how much future rewards count relative to immediate ones (c) the probability of a transition (d) the number of actions.
2. **(MCQ)** Value iteration requires: (a) no model (b) the transition and reward model (c) labelled examples (d) a neural network.
3. **(MCQ)** The optimal policy chooses in each state the action that maximises: (a) $R(s, a)$ (b) $Q^*(s, a)$ (c) $V^*(s)$ (d) $P(s' | s, a)$.
4. **(Short)** Explain the Bellman expectation equation in words.
5. **(Short)** Why does policy iteration usually need fewer iterations than value iteration?
6. **(Short)** Give an example of a reward that an agent could satisfy in an unintended way.
7. **(Applied)** Perform sweep 4 of value iteration on the technical-debt MDP, starting from the sweep-3 values (24.34, 12.29, 1.29).
8. **(Applied)** In the technical-debt MDP, raise the cost of refactoring a debt-laden codebase to -20 . Write the Bellman equations for the policy “ship when healthy or debt-laden, refactor when frozen” and solve them for $V(H)$.

Reinforcement Learning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Sequences, Decisions and Reinforcement

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** how reinforcement learning differs from planning with a known model and from supervised learning.
- **Estimate** values by Monte Carlo averaging of observed returns.
- **Explain** the exploration–exploitation trade-off, and **apply** greedy, ε -greedy and UCB action selection to a bandit problem.
- **Perform** temporal-difference and Q-learning updates by hand.
- **Contrast** Q-learning with SARSA, and **describe** how function approximation leads to deep reinforcement learning.

🔗 Before You Start

Chapter 19 (MDPs, returns, value functions, the Bellman equations) and Chapter 18 (Monte Carlo estimation). Chapter 7 for the trade-off between trying new things and using what is known, which returns here in a new form.

📖 Key Terms in This Chapter

reinforcement learning • model-free • episode • Monte Carlo prediction • first-visit Monte Carlo • exploration vs. exploitation • multi-armed bandit • greedy • ε -greedy • upper confidence bound (UCB) • incremental mean • temporal difference (TD) • TD error • bootstrapping • Q-learning • SARSA • on-policy vs. off-policy • function approximation • deep Q-network

20.1 Learning From Consequences

Chapter 19 solved decision problems when the rules of the world were written down: every transition probability and every reward. **Reinforcement learning** (RL) solves them when they are not. The agent acts, observes what happens and what it is paid, and improves its behaviour from that experience alone. Nobody tells it the right action — unlike supervised learning, there are no labels, only a reward that may arrive many steps after the action that earned it. That is how Samuel’s checkers program improved in 1959 (Chapter 1), and how AlphaGo beat a world champion in 2016.

RL is **model-free** in the sense used here: it learns value functions or policies directly, without first estimating $P(s' | s, a)$. Two ideas carry the whole chapter. **Averaging**: the value of a state is the average return that follows it, so average the returns you observe (Monte Carlo). **Bootstrapping**: the Bellman equation says a state’s value is the reward plus the next state’s value, so nudge your estimate towards that (temporal-difference learning).

20.2 Monte Carlo Methods

Chapter 18 introduced *Monte Carlo estimation*: estimate an expectation by sampling and averaging, with error shrinking like $1/\sqrt{n}$. The simplest illustration needs no MDP at all. In the model of Chapter 18, a healthy project stays healthy for another week with probability 0.8; what is the chance it stays healthy for ten weeks? Exactly, $0.8^{10} = 0.107$. By simulation: run the ten weeks 100 times and count the runs that stay healthy — the lab gets 0.120; a million runs give 0.1071. The same procedure works when no exact formula exists, which is the point.

☰ Monte Carlo Prediction

To estimate $V^\pi(s)$ without a model: run many **episodes** (complete sequences from start to end) following π ; each time s is visited, record the return G that followed; estimate $V^\pi(s)$ as the average of the recorded returns. **First-visit** Monte Carlo counts only the first visit to s in each episode. With an ε -greedy policy (below) and averaging $Q(s, a)$ instead of $V(s)$, the same idea learns to *control*, not just predict.

☰ Worked Example: Averaging Returns

Three episodes pass through state s , and the returns that followed the first visit were $G = 7, 3$ and 5 . The Monte Carlo estimate is $V(s) = (7 + 3 + 5)/3 = 5$.

The average can be kept *incrementally* — no need to store every return:

$$V_n = V_{n-1} + \frac{1}{n}(G_n - V_{n-1})$$

After the first episode $V = 7$; after the second $7 + \frac{1}{2}(3 - 7) = 5$; after the third $5 + \frac{1}{3}(5 - 5) = 5$. The form “old estimate plus a step towards the new evidence” appears in every method in this chapter.

Monte Carlo is simple and unbiased, but it must wait until an episode ends to learn anything, and returns summed over long episodes are noisy. The temporal-difference methods later in the chapter fix both.

20.3 Exploration and Exploitation

An agent that always takes the action it currently believes best *exploits* what it knows, but may never discover that another action is better. One that tries actions at random *explores*, but wastes effort on actions it already knows are poor. Every RL agent must balance the two, and the cleanest setting in which to study the balance is the bandit.

☰ Multi-Armed Bandit

k actions (“arms”, after slot machines), each paying a random reward from an unknown distribution; there is only one state. The agent pulls one arm per step and tries to maximise its total reward. Its estimate $Q(a)$ of each arm’s value is the average reward received from it so far, updated incrementally: $Q(a) \leftarrow Q(a) + \frac{1}{N(a)}(r - Q(a))$.

Strategy	Rule	Behaviour
Greedy	Always pull $\arg \max_a Q(a)$	Locks onto whichever arm looked good first; may never find the best

Strategy	Rule	Behaviour
ε -greedy	With probability ε pull a random arm; otherwise greedy	Keeps exploring forever at rate ε ; simple and robust
UCB	Pull $\arg \max_a \left[Q(a) + c \sqrt{\frac{\ln t}{N(a)}} \right]$	Explores arms in proportion to how <i>uncertain</i> their estimate is: optimism under uncertainty

Worked Example: Which Estimation Technique Next Sprint?

A team is trying three estimation techniques — A planning poker, B t-shirt sizing, C three-point estimates — one per sprint. The reward is 1 when the sprint's estimate lands within 10% of the actual effort, 0 otherwise. So far the rewards average: A 0.5 (used in 10 sprints), B 0.6 (5 sprints), C 0.3 (2 sprints). Total pulls $t = 17$; $\ln 17 = 2.833$.

Greedy: B, the highest average.

ε -greedy ($\varepsilon = 0.1$): B with probability $0.9 + 0.1/3 = 0.933$; A and C with 0.033 each.

UCB ($c = 1$):

- A: $0.5 + \sqrt{2.833/10} = 0.5 + 0.532 = 1.032$
- B: $0.6 + \sqrt{2.833/5} = 0.6 + 0.753 = 1.353$
- C: $0.3 + \sqrt{2.833/2} = 0.3 + 1.190 = \mathbf{1.490}$

UCB chooses **C**: its average is lowest, but it has been tried only twice, so its true value is the most uncertain. If C really is poor, a few more sprints will lower its bonus and UCB will stop choosing it; if it is good, greedy would never have found out.

Update after B is used again and its estimate lands (reward 1): $N(B) = 6$ and $Q(B) = 0.6 + \frac{1}{6}(1 - 0.6) = 0.667$.

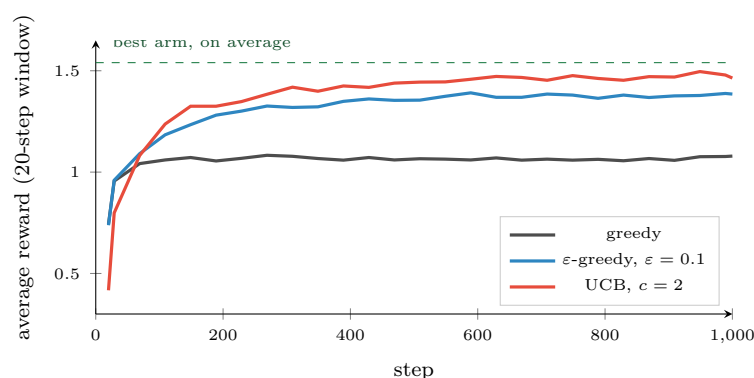


Figure 20.1: Ten-armed bandits (600 random problems, averaged). Greedy settles early on a mediocre arm; ε -greedy keeps improving; UCB explores hardest at first — its worst start — and earns the most thereafter.

20.4 Temporal-Difference Learning

Monte Carlo waits for the end of an episode to see the return. **Temporal-difference** (TD) learning updates after every single step, using the Bellman equation: the value of a state should equal the reward plus the discounted value of the next state, so move the estimate towards that.

☰ TD(0) Update

After moving from s to s' with reward r :

$$V(s) \leftarrow V(s) + \alpha \underbrace{[r + \gamma V(s') - V(s)]}_{\text{TD target}}$$

The bracket is the **TD error**: how much better or worse things turned out than expected. Using the current estimate $V(s')$ inside the target is called **bootstrapping** — learning a guess from a guess. It lets TD learn during an episode, and in tasks that never end.

📊 Worked Example: One TD Update

$V(s) = 5$ and $V(s') = 10$. The agent moves from s to s' and receives $r = 2$; $\gamma = 0.9$, $\alpha = 0.1$.

$$\text{TD target} = 2 + 0.9 \times 10 = 11, \quad \text{TD error} = 11 - 5 = 6, \quad V(s) \leftarrow 5 + 0.1 \times 6 = \mathbf{5.6}$$

The step was better than expected, so the estimate of s rises — by a tenth of the surprise, because one step is weak evidence.

20.5 Q-Learning

To choose actions without a model, the agent needs action values $Q(s, a)$, not state values: with Q , the best action is simply $\arg \max_a Q(s, a)$, with no need to know where each action leads.

☰ Q-Learning (Watkins, 1989)

Initialise $Q(s, a)$ arbitrarily (say 0). In each step: in state s choose a (e.g. ε -greedily from Q), observe reward r and next state s' , and update

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Under mild conditions — every action tried in every state infinitely often, and a suitably decreasing α — Q converges to the optimal Q^* of Chapter 19, whatever exploratory policy generated the experience.

📊 Worked Example: Three Q-Learning Updates on the Technical-Debt MDP

The agent does not know the transition probabilities of Figure 19.2; it only experiences them, one sprint at a time. Q starts at 0 everywhere; $\alpha = 0.5$, $\gamma = 0.9$.

Step 1: in H, ship; reward +10; stays in H. $Q(H, \text{ship}) \leftarrow 0 + 0.5 [10 + 0.9 \times \max(0, 0) - 0] = \mathbf{5}$.

Step 2: in H, ship; reward +10; debt builds up, now D. $Q(H, \text{ship}) \leftarrow 5 + 0.5 [10 + 0.9 \times \max(Q(D, \cdot)) - 5] = 5 + 0.5 \times 5 = \mathbf{7.5}$.

Step 3: in D, refactor; reward -4; back to H. $Q(D, \text{ref}) \leftarrow 0 + 0.5 [-4 + 0.9 \times 7.5 - 0] = 0.5 \times 2.75 = \mathbf{1.375}$.

The refactoring was penalised -4, yet its Q-value is *positive*: the update already credits it with the value of the healthy state it leads to. Run for 200,000 steps (the lab), Q-learning arrives at the same policy value iteration computed from the full model — ship when healthy, refactor when debt-laden or frozen — having never seen a single transition probability.

💡 Q-Learning and SARSA: Off-Policy and On-Policy

Q-learning's target uses $\max_{a'} Q(s', a')$ — the value of the *best* next action, whether or not the agent will take it. It learns the value of the optimal policy while behaving exploratorily: **off-policy**. **SARSA** uses instead the action a' the agent actually takes next, $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$ (the name lists the quintuple s, a, r, s', a'). It learns the value of the policy it follows, exploration included: **on-policy**. Where an exploratory step is disastrous — pushing an untested build to production, say — SARSA learns a safer policy than Q-learning, because it accounts for its own occasional random steps.

20.6 From Tables to Deep Reinforcement Learning

A table with one entry per state–action pair is fine for three codebase states and hopeless for the full state of a project portfolio — every team, task and deadline. **Function approximation** replaces the table by a model $\hat{Q}(s, a; \mathbf{w})$ — a linear model over state features, or a neural network (Chapter 13) — trained with the same TD targets as regression targets. A deep neural network trained this way is a **deep Q-network** (DQN), and the field is **deep reinforcement learning**, the technology behind AlphaGo. Approximation brings instability that the tabular case does not have, which is why deep RL systems need engineering tricks (experience replay, target networks) that belong to a deep RL course.

⚠️ What Makes Real Reinforcement Learning Hard

- **Samples are expensive.** Q-learning needed 200,000 steps on a three-state problem; a real team runs perhaps twenty-five sprints a year. Most practical RL learns in a simulator first.
- **Exploration can be unsafe.** “Try a random action” is unacceptable for a production release or a client’s budget.
- **The reward is the specification.** An agent will exploit any gap between the reward and what was meant.

❗ Common Pitfalls

- **Never exploring.** A greedy agent can lock onto a poor action for ever.
- **Exploring for ever at a high rate.** Decay ε over time once estimates are reliable.
- **Confusing Q-learning’s target with SARSA’s.** Max over next actions versus the next action actually taken.
- **A learning rate that is too large.** Values oscillate; too small and learning takes for ever.
- **Judging a policy by one run.** Rewards are random; average many episodes, as in the bandit figure.

🔧 Try It Yourself: Monte Carlo, Bandits and Q-Learning

```
import numpy as np

rng = np.random.default_rng(0)

# --- 1. Monte Carlo inference: estimate an expectation by averaging -----
# the chance that a project stays healthy for 10 weeks (it slips with
# probability 0.2 each week): exact answer 0.8 ** 10 = 0.107
for n in (100, 10_000, 1_000_000):
    runs = rng.random((n, 10)) > 0.2          # True = stayed healthy
    print(f"{n:9,d} simulated runs: P(healthy 10 weeks) ~ "
```

```

f"{runs.all(axis=1).mean():.4f}")

# --- 2. exploration vs exploitation: a 10-armed bandit -----
def bandit(strategy, steps=1000, arms=10, runs=300):
    total = np.zeros(steps)
    for _ in range(runs):
        true = rng.normal(0, 1, arms)          # unknown mean payoffs
        Q, N = np.zeros(arms), np.zeros(arms)
        for t in range(1, steps + 1):
            if strategy == "greedy":
                a = int(np.argmax(Q))
            elif strategy == "e-greedy":
                a = (int(rng.integers(arms)) if rng.random() < 0.1
                     else int(np.argmax(Q)))
            else:
                # UCB, c = 2
                ucb = Q + 2 * np.sqrt(np.log(t) / np.maximum(N, 1e-9))
                a = int(np.argmax(np.where(N == 0, np.inf, ucb)))
            r = rng.normal(true[a], 1)
            N[a] += 1
            Q[a] += (r - Q[a]) / N[a]          # incremental average
            total[t - 1] += r
    return total / runs

for s in ("greedy", "e-greedy", "ucb"):
    avg = bandit(s)
    print(f"{s:8s}: mean reward, steps 1-100 {avg[:100].mean():.2f}; "
          f"steps 901-1000 {avg[900:].mean():.2f}")

# --- 3. Q-learning on the technical-debt MDP, without being told P -----
P = {("H", "ship"): ([0.8, 0.2, 0.0], 10),
      ("D", "ship"): ([0.0, 0.5, 0.5], 5),
      ("F", "ship"): ([0.0, 0.0, 1.0], 0),
      ("H", "refactor"): ([1, 0, 0], -2), ("D", "refactor"): ([1, 0, 0], -4),
      ("F", "refactor"): ([1, 0, 0], -15)}
S, A = ["H", "D", "F"], ["ship", "refactor"]

def env_step(s, a):
    probs, reward = P[(S[s], A[a])]          # the agent only sees samples
    return int(rng.choice(3, p=probs)), reward

Q = np.zeros((3, 2))
alpha, gamma, eps = 0.1, 0.9, 0.1
s = 0
for t in range(200_000):
    a = int(rng.integers(2)) if rng.random() < eps else int(Q[s].argmax())
    s2, r = env_step(s, a)
    Q[s, a] += alpha * (r + gamma * Q[s2].max() - Q[s, a])  # Q-learning
    s = s2
print("learned Q (rows H, D, F; columns ship, refactor):")
print(Q.round(1))
print("learned policy:", dict(zip(S, [A[a] for a in Q.argmax(axis=1)])))

```

The learned Q-values approach the true Q^* (for H: ship 78.6, refactor 68.8) but do not reach them exactly, because α stays at 0.1 and the agent keeps exploring. Try $\varepsilon = 0$: does Q-learning still discover that refactoring pays?

? Checkpoint — Test Yourself

1. Returns of 10, 4 and 7 followed three visits to a state. What is the Monte Carlo estimate of its value, and what does it become after a fourth return of 3?
2. Why can a purely greedy bandit strategy perform badly for ever?
3. $Q(s, a) = 4$; the agent receives $r = 1$ and lands in s' where the best action value is 6. With $\alpha = 0.5$ and $\gamma = 0.9$, what is the new $Q(s, a)$?

✓ Chapter Summary

- Reinforcement learning finds good policies from experience, without a model of the environment and without labels.
- Monte Carlo methods average the returns observed after visiting a state; they are unbiased but wait for episodes to end.
- Exploration versus exploitation: greedy action selection can lock in a poor choice; ϵ -greedy explores at a fixed rate; UCB explores where estimates are most uncertain.
- TD learning updates after every step towards $r + \gamma V(s')$, bootstrapping from its own estimates.
- Q-learning updates action values towards $r + \gamma \max_{a'} Q(s', a')$ and converges to Q^* off-policy; SARSA is its on-policy counterpart.
- Function approximation, with neural networks, extends RL to huge state spaces: deep reinforcement learning.

✍ Review Questions

1. **(MCQ)** Q-learning is: (a) on-policy (b) off-policy (c) a supervised method (d) a planning method that needs the model.
2. **(MCQ)** In UCB, an arm that has been tried few times receives: (a) a smaller bonus (b) a larger bonus (c) no bonus (d) a negative value.
3. **(MCQ)** The TD error is: (a) $r - V(s)$ (b) $r + \gamma V(s') - V(s)$ (c) $V(s') - V(s)$ (d) $G - r$.
4. **(Short)** Compare Monte Carlo and TD learning: when does each update, and what does each target use?
5. **(Short)** Explain “optimism under uncertainty” with reference to UCB.
6. **(Short)** Why is exploration dangerous in some real applications, and what is done about it?
7. **(Applied)** For the estimation example, compute the UCB scores with $c = 2$ and state which technique is chosen. Then suppose it is used and the estimate misses (reward 0): recompute its average.
8. **(Applied)** Continue the Q-learning worked example with step 4: in H, refactor, reward -2 , stays in H. Compute the new $Q(H, \text{refactor})$ and state which action is now greedy in H.

PART VI

Responsible Practice

A model that works is not yet a model that should be used. Part VI asks who a learned model can harm and how to check, and then asks you to build one end to end — framed, evaluated honestly, documented, and defended in front of the class.

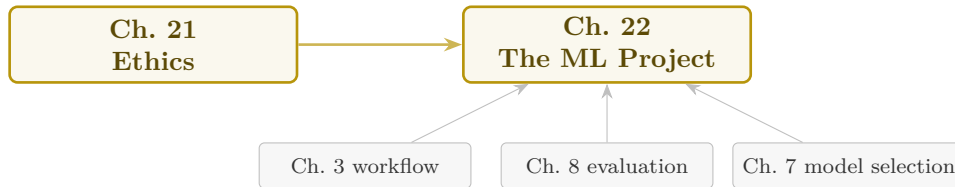


Figure P6.1: Reading path through Part VI. The project draws directly on the workflow, the model selection and the evaluation of Parts I and II, and must answer the questions of Chapter 21.

Responsible and Ethical Machine Learning

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Responsible Practice

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Identify** where bias can enter a machine learning system, from data collection to deployment.
- **Compute** and **compare** group fairness measures — selection rate, equal opportunity and predictive parity — from confusion matrices.
- **Explain** why the common fairness criteria cannot all be satisfied at once when base rates differ.
- **Discuss** privacy, transparency, accountability and security as obligations of anyone who deploys a model.
- **Apply** a responsible-ML checklist to a project, and **write** a model card.

🔗 Before You Start

Chapter 8 (confusion matrices, precision, recall, thresholds) and Chapter 3 (data and features). This chapter delivers the HEC 2025 outcome “discuss ethical considerations while applying ML to societal problems”.

📖 Key Terms in This Chapter

algorithmic bias • historical, representation and measurement bias • proxy variable • feedback loop • protected attribute • demographic parity • disparate impact (four-fifths rule) • equal opportunity • equalised odds • predictive parity • impossibility of fairness • privacy • re-identification • explainability • model card • accountability • adversarial example • data poisoning

21.1 Why a Machine Learning Course Has an Ethics Chapter

A model is a decision procedure applied at scale. A biased hiring manager affects the candidates they interview; a biased screening model affects every candidate, the same way, every year, and looks objective while doing it. The earlier chapters taught how to make a model accurate. This one asks the questions an accurate model does not answer: accurate for whom, measured how, used to decide what, and with what recourse for the people it gets wrong? None of these questions is a matter of opinion to be left to someone else. Each has technical content, and the engineer who built the model is often the only person who can answer it.

21.2 Where Bias Enters

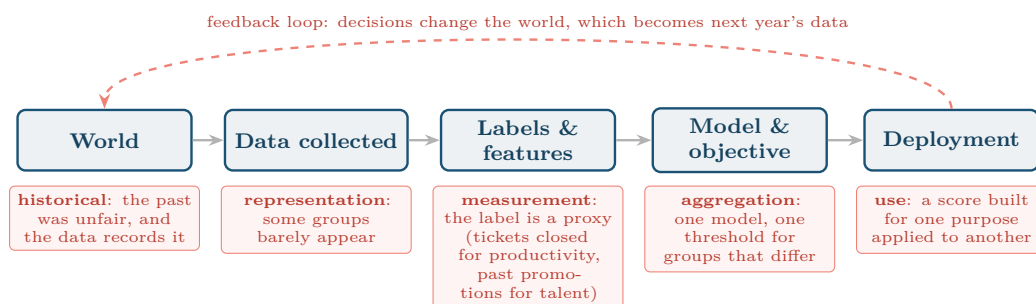


Figure 21.1: Five places bias enters a machine learning system. Removing a protected attribute addresses none of them.

Two points from Figure 21.1 catch almost every team. First, a model does not need to see a **protected attribute** (gender, religion, ethnicity, region) to discriminate on it. Other features act as **proxy variables**: university, home district, name, even a gap in a CV. In this chapter's lab, three ordinary features recover group membership with 97% accuracy. Second, **feedback loops**: a model that gives the most visible projects to the developers it rates highest hands them the achievements that make next year's model rate them highest again. The model's prediction makes itself come true.

21.3 Measuring Fairness

Fairness cannot be measured without choosing what to compare. Four group criteria are in common use, each a comparison of confusion-matrix quantities between groups:

Criterion	Equal across groups	Meaning
Demographic parity	Selection rate $P(\hat{y} = 1)$	Each group is selected in the same proportion. The four-fifths rule of US employment law flags a ratio below 0.8 as disparate impact
Equal opportunity	True positive rate (recall)	Qualified people are selected at the same rate in every group
Equalised odds	TPR and FPR	As above, and unqualified people are wrongly selected at the same rate
Predictive parity	Precision $P(y = 1 \mid \hat{y} = 1)$	A positive prediction means the same in every group

Worked Example: Auditing a Hiring Screen

A software company's model screens 1,000 applicants for developer roles from each of two groups. Group A has 400 truly qualified applicants; group B, whose members had fewer chances to build a strong record (fewer internships, less access to paid courses), has 200.

Group	TP	FN	FP	TN	Selected
A	320	80	60	540	380
B	120	80	40	760	160

- **Selection rate:** A $380/1000 = 38\%$; B $160/1000 = 16\%$. Ratio $0.16/0.38 = 0.42$ — far below 0.8: disparate impact.

- **True positive rate:** A $320/400 = 0.80$; B $120/200 = 0.60$. A qualified B applicant is rejected 40% of the time, twice as often as a qualified A applicant (20%). Equal opportunity fails.
- **False positive rate:** A $60/600 = 0.10$; B $40/800 = 0.05$.
- **Precision:** A $320/380 = 0.84$; B $120/160 = 0.75$.

Reading it. Part of the gap in selection reflects a real difference in base rates (40% against 20% qualified); but the gap in *true positive rate* does not — it is the model treating equally qualified people differently. That is the finding to act on first.

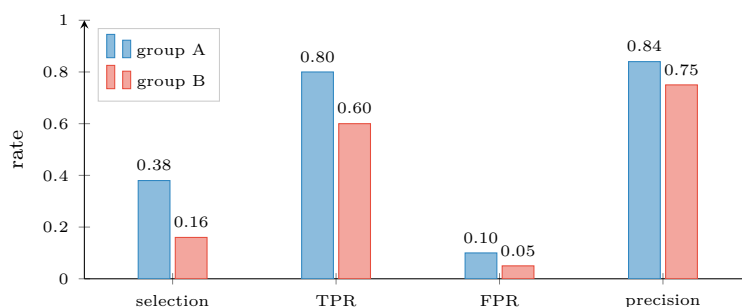


Figure 21.2: The worked audit. Every criterion shows a gap, but they mean different things: the selection gap partly reflects different base rates; the TPR gap is unequal treatment of equally qualified applicants.

⚠ The Impossibility of Fairness

When two groups have different base rates, no classifier that makes any errors can satisfy equal TPR, equal FPR and equal precision at the same time (Chouldechova, 2017; Kleinberg, Mullainathan and Raghavan, 2016). Choosing a fairness criterion is therefore a *decision*, not a calculation: equal opportunity protects qualified applicants; predictive parity protects the meaning of a positive score. In the lab, lowering group B's threshold to 0.26 equalises the TPR at 0.93 — and B's precision falls from 0.83 to 0.63. Say which criterion you chose, why, and who pays for it.

21.4 Privacy

Training data often describes people, and a model can leak it. Removing names is not anonymisation: a small number of attributes — postcode, date of birth and gender — identifies most individuals in a population, and linking a “de-identified” dataset to a public one **re-identifies** them. Models themselves can memorise and reveal rare training examples. Good practice: collect only what the task needs; keep identifiers out of features; aggregate or add noise where exact values are not needed (*differential privacy* makes that noise mathematically rigorous); and control who can query a model trained on sensitive data. Legal duties vary by jurisdiction — Pakistan's Prevention of Electronic Crimes Act 2016, the Personal Data Protection Bill drafted by the Ministry of IT and Telecommunication, the EU's GDPR for data about European residents — and a project handling personal data should identify which apply before collecting anything.

21.5 Transparency and Explainability

People affected by a decision are entitled to an explanation they can act on, and the people deploying a model need to know when it is relying on something it should not. Three levels help:

- **Interpretable models** where the stakes justify the accuracy cost: logistic regression’s odds ratios (Chapter 6), a shallow tree’s rules (Chapter 9).
- **Global explanations** of complex models: permutation importance — how much does performance drop when one feature is shuffled? — and partial dependence.
- **Local explanations** of a single prediction: which features pushed *this* applicant’s score down (SHAP values and LIME are the common tools).

An explanation of what the model *uses* is not an explanation of what *causes* the outcome (Chapter 5); say which one you are giving.

Model Card

A **model card** is a short document published with a model: what it is for, and what it must not be used for; the data it was trained on; its performance overall *and for each relevant group*; the fairness criterion chosen and the result; known limitations; and who is responsible for it. The project of Chapter 22 must include one.

21.6 Accountability and Security

Accountability means that a person, not “the algorithm”, is responsible for a decision, and that the people affected can contest it. Keep a human in the loop for consequential decisions; log what the model predicted and why; and provide a route for appeal that can actually overturn the prediction.

Security is an ethical issue too, because a model that can be manipulated will be. **Adversarial examples** — inputs changed slightly and deliberately — flip a model’s output (Chapter 13).

Data poisoning inserts crafted examples into the training data, a particular risk for models retrained on user-supplied data and for self-training (Chapter 17). A model deployed where someone benefits from fooling it must be tested against someone trying.

Stage	Questions to answer before deployment
Purpose	What decision will this support? Who is affected? Would a simpler rule, or no model, be better?
Data	Who is missing or under-represented? What is the label really measuring? Was it collected with consent, and may it be used this way?
Features	Could any feature be a proxy for a protected attribute? Does any exist only because of the outcome (Chapter 3)?
Evaluation	Performance for each group, not only overall. Which fairness criterion, and why?
Deployment	Who reviews the model’s decisions? How can they be contested? How will drift and feedback loops be detected?
Security	Who benefits from fooling it? Has it been tested against them?

Common Pitfalls

- “We removed the protected attribute, so the model is fair.” Proxies carry it back in.

- **Reporting only overall accuracy.** A model can be accurate on average and poor for a minority group; report per group.
- **Picking the fairness metric the model happens to pass.** Choose it from the harm you want to prevent, before looking.
- **Treating anonymised data as safe.** Linking attributes re-identifies people.
- **Deploying without a way to be wrong.** Every model errs; the question is what happens to the person it errs about.

Try It Yourself: Auditing a Model for Fairness

A synthetic hiring dataset in which group B had fewer chances to build a strong record. The group is never given to the model as a feature.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score

# --- a synthetic hiring dataset with a protected group -----
rng = np.random.default_rng(0)
n = 6000
group = rng.integers(0, 2, n)          # 0 = group A, 1 = group B
# group B had fewer internships and projects to show (historical bias)
record = rng.normal(0.6 - 0.25 * group, 0.15, n) # CV: internships, projects
aptitude = rng.normal(0.5, 0.15, n)             # coding test: equal
district = group + rng.normal(0, 0.3, n)         # a proxy for the group
qualified = (0.5 * record + 0.5 * aptitude
             + rng.normal(0, 0.05, n) > 0.5).astype(int)
X = np.column_stack([record, aptitude, district])
X_tr, X_te, y_tr, y_te, g_tr, g_te = train_test_split(
    X, qualified, group, test_size=0.5, random_state=0)

def report(name, pred):
    print(name)
    for g, label in ((0, "A"), (1, "B")):
        m = g_te == g
        sel = pred[m].mean()                # selection rate
        tpr = pred[m][y_te[m] == 1].mean()  # recall
        fpr = pred[m][y_te[m] == 0].mean()
        prec = y_te[m][pred[m] == 1].mean()
        print(f"  group {label}: base rate {y_te[m].mean():.2f} "
              f"selected {sel:.2f}  TPR {tpr:.2f}  FPR {fpr:.2f} "
              f"precision {prec:.2f}")

# --- 1. the model, group column excluded (it was never a feature) -----
model = LogisticRegression().fit(X_tr, y_tr)
p = model.predict_proba(X_te)[ :, 1]
report("threshold 0.5 for everyone:", (p > 0.5).astype(int))

# --- 2. removing the group column does not remove the group -----
acc = cross_val_score(LogisticRegression(), X, group, cv=5).mean()
print(f"the features recover group membership with accuracy {acc:.2f}")

# --- 3. equalise the true positive rate with a group-specific threshold --
t_b = 0.5
tpr_a = (p[(g_te == 0) & (y_te == 1)] > 0.5).mean()
while (p[(g_te == 1) & (y_te == 1)] > t_b).mean() < tpr_a:
    t_b -= 0.01
pred = np.where(g_te == 1, p > t_b, p > 0.5).astype(int)
```

```
report(f"threshold 0.5 for A, {t_b:.2f} for B (equal opportunity):", pred)
```

Equalising the TPR costs group B precision and raises its false positive rate — the impossibility result in action. Write two sentences, as if for the hiring committee, explaining the trade-off you would recommend.

🔍 Checkpoint — Test Yourself

1. A hiring model selects 45% of one group and 30% of another. Does it pass the four-fifths rule?
2. Why does removing a protected attribute from the features not guarantee a fair model?
3. Which fairness criterion protects qualified applicants from being rejected, and which confusion-matrix quantity does it compare?

📋 Chapter Summary

- Bias enters through history, representation, measurement, aggregation and use, and feedback loops can amplify it; proxies reintroduce removed attributes.
- Group fairness compares confusion-matrix rates across groups: selection (demographic parity), TPR (equal opportunity), TPR and FPR (equalised odds), precision (predictive parity).
- With different base rates these criteria conflict; choosing one is a decision to be justified, not computed.
- Protect privacy by minimising and aggregating data; explain models at the right level; keep a human accountable and a route to contest.
- Document every deployed model with a model card, including per-group performance, and test it against people with a reason to fool it.

🔧 Review Questions

1. **(MCQ)** Equal opportunity requires equal: (a) selection rates (b) true positive rates (c) precision (d) accuracy.
2. **(MCQ)** Using “tickets closed per month” as the label for “good developer” is an example of: (a) representation bias (b) measurement bias (c) a feedback loop only (d) data poisoning.
3. **(MCQ)** A selection-rate ratio of 0.42 between two groups: (a) passes the four-fifths rule (b) fails it (c) proves intent to discriminate (d) says nothing.
4. **(Short)** Explain a feedback loop with an example of your own from IT project management.
5. **(Short)** Why can equal opportunity and predictive parity not both hold when base rates differ?
6. **(Short)** List the sections of a model card and justify one of them.
7. **(Applied)** For the worked audit, find how many more group-B qualified applicants must be selected to equalise the TPR at 0.80, and compute B’s new selection rate if no additional false positives are admitted.
8. **(Applied)** Choose one model from Chapters 5–20 and write its model card for a realistic deployment, using the checklist of this chapter.

The Machine Learning Project

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Responsible Practice

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Scope** a machine learning project: turn a vague aim into a task, performance measure, data source, baseline and success criterion.
- **Plan** the project in milestones and **write** a one-page proposal.
- **Run** experiments reproducibly: one fixed split, cross-validated comparisons, an experiment log, seeds and version control.
- **Report** results honestly, with a baseline, uncertainty, per-group performance and a model card.
- **Present** and **defend** the project's choices.

🔗 Before You Start

Everything. In particular Chapter 1 (T, P and E), Chapter 3 (the workflow and leakage), Chapter 7 (model selection), Chapter 8 (metrics and uncertainty) and Chapter 21 (the checklist and the model card). The project delivers HEC 2023 CLO-5 — “develop a reasonable size project using suitable machine learning technique” — and the HEC 2025 outcome “design and develop end-to-end ML solutions for real-world datasets”.

📖 Key Terms in This Chapter

problem framing • success criterion • proposal • milestone • fixed test set • experiment log • reproducibility • random seed • version control • ablation • error analysis • model card • technical report • presentation

22.1 What the Project Is For

Each chapter has taught one piece: one algorithm, one metric, one pitfall. Real work needs all of them at once, in the right order, on data nobody has cleaned in advance, for a question that matters to someone. The project is where that happens. It is worth 15% of the course (see the Course Specification), is done in pairs, runs from Week 11 to Week 16, and is judged less on the final accuracy than on whether every number in it can be trusted.

22.2 Choosing and Scoping a Problem

A good project problem has four properties: a **decision** someone would make with the prediction; **data** you can legally obtain, with labels, preferably at least a thousand examples; a **baseline** that is easy to compute; and a **scope** that fits five weeks. Sources of data:

- the issue trackers of open-source projects (the Apache Software Foundation's public Jira, GitHub issues), which record tickets, their text, their history and how long they took;
- public software-engineering datasets — effort-estimation and defect data such as those of the PROMISE repository — and the general collections of the UCI Machine Learning Repository, OpenML and Kaggle;

- data from your own employer’s project tracker — only with written permission, and never with personal identifiers (Chapter 21).

💡 Project Ideas

- **Resolution time.** Predict how long a ticket will take to resolve from its text and metadata (Chapter 5, with the word features of Chapter 10), and compare with the team’s own estimates.
- **Ticket routing.** Send each ticket to the right component or team from its text; report macro-averaged metrics, because the rare queues matter (Chapter 8).
- **Effort estimation.** Predict story hours from story points and task features (Chapter 5, Chapter 11), and test whether the model beats “hours per point” from last quarter.
- **Projects at risk.** Classify projects likely to overrun from their early metrics — the worked example below.
- **Defect prediction.** Flag code changes likely to introduce a defect; the classes are imbalanced, so report the precision–recall curve.
- **Duplicate reports.** Find earlier tickets that describe the same fault (cosine similarity and nearest neighbours, Chapter 11).

📊 Worked Example: From a Vague Idea to a Project

Idea: “Use machine learning to deliver more projects on time.”

Decision. Who acts on the prediction? The project office, which can review about 15 projects in depth each quarter.

Task *T*. One month into each project, rank the projects that started this quarter by their risk of overrunning their deadline by more than 20%.

Experience *E*. Four years of completed projects (about 900) with their month-one metrics — change requests, staffing, velocity, open defects — and whether they overran, exported from the company’s project tracker with written permission and without personal identifiers.

Performance *P*. Recall among the top 15 ranked projects (how many eventual overruns the office would review), with precision reported beside it.

Baseline. Rank by the number of change requests alone — the rule the office already uses.

Success criterion. Beat the change-request rule’s recall at 15 by at least three projects on the held-out year, with no project type (web, mobile, data) whose recall is below 60% of the others’.

Every later choice — features, model, threshold, fairness check — now has something to be judged against. A project without that paragraph cannot fail, and so cannot succeed either.

22.3 The Plan

💡 The One-Page Proposal (due end of Week 11)

1. **Problem and decision:** who will use the prediction, for what.
2. ***T*, *P*, *E*,** as in the worked example.
3. **Data:** source, size, label, permission, known problems.
4. **Baseline and success criterion.**
5. **Planned models** (at least one from each of two different chapters).
6. **Risks and ethics:** who could be harmed; which fairness check.

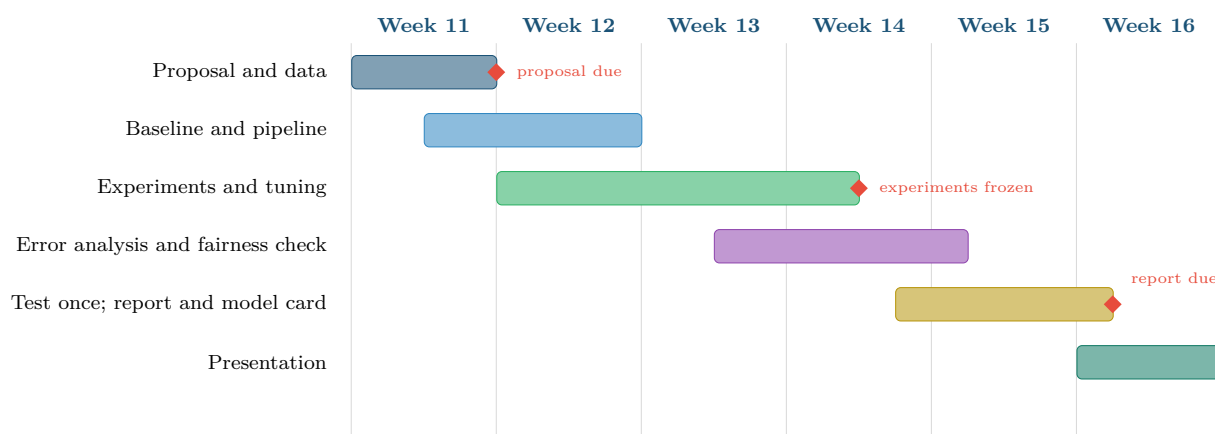


Figure 22.1: The project across the last six weeks of the semester. The test set is opened only once, after experiments are frozen in Week 14.

22.4 Running Experiments Honestly

📋 Rules for Reproducible Experiments

- **Split once, first**, and store the test indices; the test set is not opened until the experiments are frozen.
- **Every model is a pipeline** (Chapter 3) evaluated by the same cross-validation folds.
- **Log every run**: date, model, settings, CV mean and standard deviation, and one line on why it was run. The log, not memory, is the record.
- **Fix the random seeds** and record the library versions.
- **Use version control** (Git): commit the code with each logged run, so any number in the report can be regenerated.

📅 Worked Example: Reading an Experiment Log

The lab’s template writes this log for the projects-at-risk example, whose metric is recall on the overrun class:

Model	CV recall (mean)	(sd)
baseline (majority class)	0.000	0.000
logistic regression	0.799	0.047
random forest	0.804	0.061

The baseline scores zero: it predicts “on time” for every project, so it never catches an overrun — the accuracy paradox of Chapter 8 expressed in the metric that matters. **The two models** — logistic regression (Chapter 6) and a random forest (Chapter 14) — **are indistinguishable**: 0.799 and 0.804 differ by far less than one fold-to-fold standard deviation. Choose the simpler, more explainable one — logistic regression, whose odds ratios the project office can read — tune it (the template finds $C = 0.1$ with balanced class weights, which lifts cross-validated recall to 0.881), and only then open the test set, once: recall 0.857, which is 48 of the 56 overruns caught.

Error analysis is the step most projects skip and the one that teaches most: look at the examples the model gets wrong, and ask what they have in common. An **ablation** — removing one feature

or one step and measuring the change — shows what each part of the pipeline contributes. Both belong in the report.

22.5 The Report and the Presentation

Section	Contents
1. Problem	The decision, T , P , E , success criterion (from the proposal, updated)
2. Data	Source, permission, size, label, cleaning, the split; a table of features
3. Method	Baseline; models compared; the pipeline; how hyperparameters were chosen
4. Results	The experiment log; the chosen model's test performance <i>with a confidence interval</i> (Chapter 8); comparison with the baseline and success criterion
5. Analysis	Error analysis; ablation; per-group performance and the fairness check
6. Limitations	What the model must not be used for; what would change the conclusion
Appendix	The model card (Chapter 21); how to rerun everything

The presentation is ten minutes and five slides: the decision and why it matters; the data; what was compared and how; the result against the baseline, with its uncertainty; and one limitation. Every member of the pair must be able to answer any question on any part.

22.6 Assessment

Criterion	Marks	Full marks require	CLO
Framing and proposal	10	A decision, $T/P/E$, baseline and success criterion stated before any modelling	1, 6
Data and pipeline	15	Legal data; leak-free pipeline; one fixed split	6
Experiments	20	At least two model families; CV comparison; logged; reproducible from the repository	2–4, 6
Evaluation	20	Right metric; baseline; confidence interval; test set used once	5
Analysis and ethics	15	Error analysis; per-group results; model card	5, 7
Report	10	Clear, complete, honest about limitations	6
Presentation and defence	10	Both members answer confidently	6
Total	100	Weighted to 15% of the course	

! Common Pitfalls

- **Choosing the dataset before the question.** Start from a decision; find data for it.
- **Opening the test set to “check progress”.** It is then a validation set, and the final number is optimistic.
- **No baseline.** A 92% model means nothing until the baseline is known.
- **Reporting only the best run.** Report the log, including what did not work.
- **A notebook that runs only in the order it was written.** Restart and run everything before submitting; better, turn it into a script.

🔧 Try It Yourself: A Project Template, End to End

Copy this script as the skeleton of your project and replace the data and the candidate models. It writes an experiment log, the saved model and a model card into a folder called `project_output`.

```

"""Project skeleton: from raw data to a saved, documented model."""
import csv
import datetime
import json
import os

import joblib
import sklearn
from sklearn.datasets import make_classification
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, recall_score
from sklearn.model_selection import (GridSearchCV, StratifiedKFold,
                                     cross_val_score, train_test_split)
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

SEED = 42                                # one seed, used everywhere
OUT = "project_output"
os.makedirs(OUT, exist_ok=True)

# --- 1. data, and a split made once and never changed -----
# a stand-in for your tracker export: 900 projects, 8 month-one metrics
X, y = make_classification(n_samples=900, n_features=8, n_informative=5,
                          n_redundant=1, weights=[0.75], flip_y=0.02,
                          random_state=5) # y: 1 = overran, must not miss
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=SEED)
cv = StratifiedKFold(5, shuffle=True, random_state=SEED)
metric = "recall"                        # chosen in the proposal

# --- 2. baseline, then candidates, all by cross-validation -----
candidates = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression()),
    "forest": RandomForestClassifier(n_estimators=300, random_state=SEED),
}
with open(os.path.join(OUT, "experiments.csv"), "w", newline="") as fh:
    log = csv.writer(fh)
    log.writerow(["model", f"cv_{metric}_mean", f"cv_{metric}_std"])
    for name, model in candidates.items():
        s = cross_val_score(model, X_tr, y_tr, cv=cv, scoring=metric)
        log.writerow([name, round(s.mean(), 4), round(s.std(), 4)])
        print(f"{name:9s} CV {metric} {s.mean():.3f} +/- {s.std():.3f}")

```

```

# --- 3. tune the chosen family -----
params = {"logisticregression__C": [0.01, 0.1, 1, 10],
          "logisticregression__class_weight": [None, "balanced"]}
grid = GridSearchCV(candidates["logistic"], params, cv=cv,
                    scoring=metric).fit(X_tr, y_tr)
print("chosen settings:", grid.best_params_)

# --- 4. the test set, once -----
pred = grid.predict(X_te)
test_recall = recall_score(y_te, pred)
print(classification_report(y_te, pred,
                           target_names=["on time", "overran"], digits=3))

# --- 5. save the model and write its model card -----
joblib.dump(grid.best_estimator_, os.path.join(OUT, "model.joblib"))
card = {
    "model": "logistic regression, standardised features",
    "intended use": "rank projects for a project-office review; never to "
                  "judge a team or a manager",
    "data": f"{len(y)} past projects, 8 month-one metrics",
    "split": f"75/25 stratified, seed {SEED}; 5-fold CV for selection",
    "settings": grid.best_params_,
    f"test {metric}": round(float(test_recall), 3),
    "limitations": "one company's projects; re-validate after any change "
                  "to how projects are run",
    "date": datetime.date.today().isoformat(),
    "library": f"scikit-learn {sklearn.__version__}",
}
with open(os.path.join(OUT, "model_card.json"), "w") as fh:
    json.dump(card, fh, indent=2, default=str)
print("saved:", sorted(os.listdir(OUT)))

```

The template reproduces the experiment log of the worked example. Before adapting it, change `metric` to `"precision"` and rerun: does the chosen model change?

🔍 Checkpoint — Test Yourself

1. Write a success criterion for the late-task warning of Chapter 1 that a project manager could check.
2. Two models score 0.912 ± 0.030 and 0.905 ± 0.028 in cross-validation. Which do you choose, and on what grounds?
3. Name three things that must be recorded for an experiment to be reproducible.

📋 Chapter Summary

- Start from a decision, and write T , P , E , a baseline and a success criterion before modelling.
- Split once; compare pipelines by cross-validation on fixed folds; log every run; fix seeds; commit code.
- Choose between near-equal models on simplicity; open the test set once; report the result with its uncertainty and beside the baseline.
- Error analysis, ablations and per-group results turn a score into understanding.
- The report, the model card and the defence are part of the work, not decoration.

 Review Questions

1. **(MCQ)** The test set should be used: (a) after every experiment (b) once, after experiments are frozen (c) to choose hyperparameters (d) never.
2. **(MCQ)** An ablation study: (a) removes components to measure their contribution (b) adds more data (c) tunes the learning rate (d) cleans labels.
3. **(MCQ)** The most important part of a project proposal is: (a) the choice of algorithm (b) the decision, metric and success criterion (c) the number of features (d) the programming language.
4. **(Short)** Why should an experiment log include runs that did not improve the model?
5. **(Short)** What is error analysis, and what can it reveal that a single metric cannot?
6. **(Short)** Explain why the worked example chose logistic regression over the random forest.
7. **(Applied)** Write the one-page proposal for an IT project-management problem of your choice, using the template of this chapter.
8. **(Applied)** For the worked example's test result (48 of 56 overruns caught), compute a 95% confidence interval for the recall and discuss what it means for the claim "the model catches 86% of overruns".

Appendices

Python and scikit-learn Reference

Everything the labs use, on a few pages. Each lab is also a runnable file in the `code/` folder, named after its chapter.

A.1 Setting Up

Install the Anaconda distribution (Python 3.10 or later); it includes NumPy, pandas, SciPy, Matplotlib, scikit-learn and Jupyter. Check the installation with:

```
import numpy, pandas, scipy, sklearn
print(numpy.__version__, pandas.__version__, sklearn.__version__)
```

The labs were tested with NumPy 2.3 and scikit-learn 1.7. Run a lab file from a terminal with `python ch05_least_squares_three_ways.py`, or paste it into a notebook cell.

A.2 NumPy in Ten Lines

Code	What it does
<code>x = np.array([1.0, 2.0, 3.0])</code>	A vector
<code>X = np.array([[1, 2], [3, 4]])</code>	A matrix: rows are examples
<code>X.shape, X[:, 0], X[1]</code>	Dimensions; first column; second row
<code>w @ x, X @ w</code>	Dot product; matrix–vector product (Chapter 2)
<code>x.mean(), x.std(), x.sum(axis=0)</code>	Summaries; <code>axis=0</code> runs down columns
<code>np.linalg.norm(x - z)</code>	Euclidean distance
<code>np.linalg.solve(A, b)</code>	Solve $Ax = b$ (Chapter 5, Chapter 19)
<code>np.linalg.eigh(S)</code>	Eigenvalues and eigenvectors of a symmetric matrix (Chapter 16)
<code>rng = np.random.default_rng(0)</code>	A seeded random generator: <code>rng.random</code> , <code>rng.normal</code> , <code>rng.choice</code>
<code>np.argmax(q), np.argsort(-v)</code>	Index of the largest; indices from largest to smallest

A.3 pandas in Ten Lines

Code	What it does
<code>df = pd.read_csv("file.csv")</code>	Load a table
<code>df.head(), df.info(), df.describe()</code>	First rows; types and missing counts; summary statistics
<code>df["col"], df[["a", "b"]]</code>	One column (a Series); several (a DataFrame)
<code>df[df.col > 5]</code>	Rows meeting a condition

Code	What it does
<code>df.col.value_counts(normalize=True)</code>	Class proportions — check for imbalance
<code>df.isna().sum()</code>	Missing values per column
<code>df.groupby("g").col.mean()</code>	Mean per group (per-group evaluation, Chapter 21)
<code>X = df.drop(columns="y"), y = df["y"]</code>	Features and label
<code>pd.get_dummies(df, columns=["c"])</code>	One-hot encoding outside a pipeline (inside, use <code>OneHotEncoder</code>)

A.4 The scikit-learn Pattern

Every lab follows the same shape. Learn it once:

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_tr, X_te, y_tr, y_te = train_test_split(      # 1. split first
    X, y, test_size=0.25, stratify=y, random_state=0)
model = make_pipeline(StandardScaler(),        # 2. prepare + model
    LogisticRegression(max_iter=1000))
scores = cross_val_score(model, X_tr, y_tr, cv=5) # 3. compare by CV
model.fit(X_tr, y_tr)                          # 4. fit on training
print(scores.mean(), model.score(X_te, y_te))  # 5. test once
```

A.5 Models, Chapter by Chapter

Ch	Class	Key settings (hyperparameters)
5	<code>LinearRegression</code>	none; <code>Ridge(alpha)</code> , <code>Lasso(alpha)</code> add a penalty
6	<code>LogisticRegression</code>	$C = 1/\lambda$; <code>class_weight="balanced"</code> ; <code>max_iter</code>
9	<code>DecisionTreeClassifier</code>	<code>criterion ("gini", "entropy")</code> , <code>max_depth</code> , <code>min_samples_leaf</code> , <code>ccp_alpha</code>
10	<code>GaussianNB</code> , <code>MultinomialNB</code>	<code>alpha</code> (smoothing) for multinomial
11	<code>KNeighborsClassifier</code>	<code>n_neighbors</code> , <code>weights ("uniform", "distance")</code> , <code>metric</code>
12	<code>SVC</code>	<code>kernel</code> , <code>C</code> , <code>gamma</code> , <code>degree</code> ; <code>probability=True</code>
13	<code>MLPClassifier</code>	<code>hidden_layer_sizes</code> , <code>alpha</code> (weight decay), <code>early_stopping</code> , <code>learning_rate_init</code>
14	<code>RandomForestClassifier</code>	<code>n_estimators</code> , <code>max_features</code> , <code>oob_score=True</code>
14	<code>AdaBoostClassifier</code> , <code>GradientBoostingClassifier</code>	<code>n_estimators</code> , <code>learning_rate</code> , base tree depth
15	<code>KMeans</code> , <code>AgglomerativeClustering</code>	<code>n_clusters</code> , <code>n_init</code> ; <code>linkage</code>
16	<code>PCA</code>	<code>n_components</code> (an integer, or a fraction of variance such as 0.9)

Ch	Class	Key settings (hyperparameters)
17	GaussianMixture, SelfTrainingClassifier	n_components, covariance_type; threshold

A.6 Evaluation Calls

Code	Returns
confusion_matrix(y, pred).ravel()	TN, FP, FN, TP for two classes
classification_report(y, pred)	Precision, recall, F_1 per class; macro and weighted averages
roc_auc_score(y, proba)	ROC-AUC from probabilities
average_precision_score(y, proba)	Area under the precision-recall curve
mean_absolute_error, mean_squared_error, r2_score	Regression metrics
silhouette_score(X, labels)	Mean silhouette of a clustering
adjusted_rand_score(truth, labels)	Agreement of a clustering with known labels
GridSearchCV(model, grid, cv=5)	Cross-validated search; .best_params_, .best_score_

A.7 Errors Students Meet Every Semester

Message	Usual cause and fix
ConvergenceWarning	The optimiser stopped early: scale the features, or raise <code>max_iter</code>
could not convert string to float	A categorical column reached the model: one-hot encode it
Input contains NaN	Missing values: add <code>SimpleImputer</code> to the pipeline
Expected 2D array, got 1D	One example or one feature: use <code>x.reshape(1, -1)</code> or <code>x[:, [j]]</code>
Accuracy suspiciously high	Leakage (Chapter 3) or an imbalanced baseline (Chapter 8)
Different results every run	No <code>random_state</code> : fix the seeds

Formula Sheet

Every formula used in the handout, grouped by chapter. The chapter number in brackets says where it is explained. This sheet may be printed and taken into examinations if the examiner permits.

B.1 Mathematics (Chapter 2)

Quantity	Formula
Dot product	$\mathbf{w} \cdot \mathbf{x} = \sum_j w_j x_j$
Euclidean norm, distance	$\ \mathbf{x}\ = \sqrt{\sum_j x_j^2}, \quad d(\mathbf{x}, \mathbf{z}) = \ \mathbf{x} - \mathbf{z}\ $
Cosine similarity	$\cos \theta = \mathbf{x} \cdot \mathbf{z} / (\ \mathbf{x}\ \ \mathbf{z}\)$
Derivative rules	$(w^k)' = kw^{k-1}; \quad (f + g)' = f' + g'; \quad (f(g(w)))' = f'(g(w)) g'(w)$
Gradient descent	$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla L(\mathbf{w})$
Bayes' theorem	$P(A B) = P(B A)P(A)/P(B)$
Total probability	$P(B) = P(B A)P(A) + P(B \neg A)P(\neg A)$
Mean, variance	$\bar{x} = \frac{1}{n} \sum x_i, \quad \text{Var} = \mathbb{E}[(X - \mu)^2]$
Gaussian density	$p(x) = \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$
Entropy	$H = -\sum_i p_i \log_2 p_i$

B.2 Workflow and Linear Models (Chapters 3, 5, 6)

Quantity	Formula
Standardisation	$z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$
Min-max scaling	$x' = (x - \min) / (\max - \min)$
Linear model	$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$
Mean squared error	$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$
Least squares, one feature	$w = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b = \bar{y} - w\bar{x}$
Normal equation	$\boldsymbol{\theta} = (X^\top X)^{-1} X^\top \mathbf{y}$
MSE gradient	$\partial \text{MSE} / \partial w_j = \frac{2}{n} \sum (\hat{y}_i - y_i) x_{ij}$
MAE, RMSE, R^2	$\frac{1}{n} \sum y_i - \hat{y}_i ; \quad \sqrt{\text{MSE}}; \quad 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z}), \quad \sigma' = \sigma(1 - \sigma)$
Log-odds, odds ratio	$z = \ln \frac{p}{1-p}; \quad \text{odds multiplied by } e^{w_j} \text{ per unit of } x_j$
Cross-entropy	$L = -\frac{1}{n} \sum [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$
Cross-entropy gradient	$\partial L / \partial w_j = \frac{1}{n} \sum (p_i - y_i) x_{ij}$
Softmax	$P(y = k \mathbf{x}) = e^{z_k} / \sum_j e^{z_j}$

B.3 Generalisation and Evaluation (Chapters 7, 8)

Quantity	Formula
Bias–variance	$\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \sigma_\varepsilon^2$
Ridge, lasso	$\text{MSE} + \lambda \sum w_j^2$; $\text{MSE} + \lambda \sum w_j $
Ridge, one dimension	$w = \sum x_i y_i / (\sum x_i^2 + \lambda)$ (centred data)
Accuracy	$(\text{TP} + \text{TN}) / \text{all}$
Precision, recall	$\text{TP} / (\text{TP} + \text{FP})$; $\text{TP} / (\text{TP} + \text{FN})$
Specificity, FPR	$\text{TN} / (\text{TN} + \text{FP})$; $\text{FPR} = 1 - \text{specificity}$
F_1	$2 \text{prec} \cdot \text{rec} / (\text{prec} + \text{rec})$
95% interval for error	$e \pm 1.96 \sqrt{e(1-e)/n}$
McNemar’s test	$\chi^2 = (b - c - 1)^2 / (b + c)$; significant at 5% if > 3.84

B.4 Supervised Algorithms (Chapters 9–14)

Quantity	Formula
Information gain	$\text{Gain}(S, A) = H(S) - \sum_v \frac{ S_v }{ S } H(S_v)$
Split information, gain ratio	$-\sum_v \frac{ S_v }{ S } \log_2 \frac{ S_v }{ S }$; $\text{Gain} / \text{SplitInfo}$
Gini index	$1 - \sum_i p_i^2$
MAP hypothesis	$h_{MAP} = \arg \max_h P(D h)P(h)$
Naive Bayes	$v_{NB} = \arg \max_v P(v) \prod_j P(a_j v)$
Laplace smoothing	$(n_{v,c} + 1) / (n_c + k)$
k -NN weighting	$\text{weight} = 1/d^2$ (or $1/d$)
Minkowski distance	$(\sum_j x_j - z_j ^p)^{1/p}$
SVM margin	$2/\ \mathbf{w}\ $ with $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1$
Hinge loss	$\max(0, 1 - yf(\mathbf{x}))$
Kernels	$(\mathbf{x} \cdot \mathbf{z} + r)^p$; $\exp(-\gamma\ \mathbf{x} - \mathbf{z}\ ^2)$
Perceptron rule	$w_i \leftarrow w_i + \eta(t - o)x_i$
Delta rule	$w_i \leftarrow w_i + \eta \sum_d (t_d - o_d)x_{id}$
Backpropagation	$\delta_k = o_k(1 - o_k)(t_k - o_k)$; $\delta_h = o_h(1 - o_h) \sum_k w_{kh} \delta_k$; $\Delta w_{ji} = \eta \delta_j x_{ji}$
Majority vote	$P(\text{correct}) = \sum_{k > n/2} \binom{n}{k} p^k (1-p)^{n-k}$
AdaBoost	$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$; mistakes $\times e^{\alpha_t}$, correct $\times e^{-\alpha_t}$, renormalise

B.5 Unsupervised, Sequences and Decisions (Chapters 15–21)

Quantity	Formula
WCSS	$\sum_j \sum_{\mathbf{x} \in C_j} \ \mathbf{x} - \boldsymbol{\mu}_j\ ^2$
Silhouette	$s = (b - a) / \max(a, b)$
PCA	eigenvectors of the covariance matrix; explained variance $\lambda_j / \sum_i \lambda_i$

Quantity	Formula
SOM update	$\mathbf{w}_u \leftarrow \mathbf{w}_u + \eta h(u)(\mathbf{x} - \mathbf{w}_u), \quad h(u) = \exp(-d_{\text{grid}}^2/2\sigma^2)$
EM responsibility	$r_{ij} = \pi_j \mathcal{N}(\mathbf{x}_i \boldsymbol{\mu}_j, \Sigma_j) / \sum_l \pi_l \mathcal{N}(\mathbf{x}_i \boldsymbol{\mu}_l, \Sigma_l)$
EM M-step (means)	$\boldsymbol{\mu}_j = \sum_i r_{ij} \mathbf{x}_i / \sum_i r_{ij}$
HMM forward	$\alpha_1(i) = \pi_i B_i(o_1); \quad \alpha_{t+1}(j) = [\sum_i \alpha_t(i) A_{ij}] B_j(o_{t+1})$
Viterbi	$\delta_{t+1}(j) = \max_i [\delta_t(i) A_{ij}] B_j(o_{t+1}), \text{ with backpointers}$
Return	$G_t = \sum_k \gamma^k r_{t+k+1}$
Bellman optimality	$V^*(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s' s, a) V^*(s')]$
Incremental mean	$Q \leftarrow Q + \frac{1}{n}(r - Q)$
UCB	$Q(a) + c\sqrt{\ln t / N(a)}$
TD(0)	$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$
Q-learning	$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
SARSA	$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
Selection-rate ratio	$P(\hat{y} = 1 B) / P(\hat{y} = 1 A); \text{ four-fifths rule: flag if below } 0.8$

Answers to Checkpoints

Answers to the three-question Checkpoint box in each chapter. Attempt them before reading. Where an answer requires judgement, the reasoning matters more than the conclusion.

Chapter 1 — What Machine Learning Is

1. No. The threshold was chosen by a person and the tool's performance does not improve with experience: there is no E . It is automation, not learning.
2. (a) regression; (b) clustering (unsupervised); (c) classification; (d) reinforcement learning.
3. Because for any unseen instance, every hypothesis consistent with the data that calls it positive has a twin, equally consistent, that calls it negative. The data says nothing about unseen instances; only an assumption — a bias — can.

Chapter 2 — Mathematics for Machine Learning

1. $\sqrt{(4-1)^2 + (6-2)^2} = \sqrt{9+16} = 5$.
2. $L'(w) = 2(w+1) = 6$ at $w = 2$, so $w \leftarrow 2 - 0.25 \times 6 = 0.5$. The minimum is at -1 : the gap fell from 3 to 1.5, so yes, closer.
3. $H = -(0.5 \log_2 0.5 + 2 \times 0.25 \log_2 0.25) = 0.5 + 0.5 + 0.5 = 1.5$ bits.

Chapter 3 — The Machine Learning Workflow and Toolkit

1. At prediction time only the training data is known. Statistics computed with test rows leak information about the test set into training and make the evaluation optimistic; new data must be transformed with the stored training statistics.
2. One-hot encode it into three 0/1 columns. Codes 1, 2, 3 would tell a model that Waterfall is “three times” Scrum and that Kanban lies between them — an order and distances that do not exist.
3. The majority-class baseline: if 97% of projects are never cancelled, predicting “not cancelled” is 97% accurate. Then ask for recall on the cancelled projects.

Chapter 4 — Concept Learning and the Version Space

1. Yes: any release satisfying $\langle \text{Pass, Done, ?, Ready, ?, ?} \rangle$ also satisfies $\langle \text{Pass, ?, ?, Ready, ?, ?} \rangle$. The reverse is false: a release whose tests pass and whose rollback is ready, but whose review is still pending, satisfies the first but not the second.
2. $\langle \text{Pass, Done, ?, Ready, Quiet, Same} \rangle$ — the hypothesis after Release 2, since Release 3 is negative and ignored.
3. For each concept in its version space that labels the unseen instance positive, there is another, identical except for that instance, labelling it negative; both are consistent with the training data. The votes are always equal.

Chapter 5 — Linear Regression

1. $\bar{x} = 1$, $\bar{y} = 3$; $w = \frac{(-1)(-2)+0+(1)(2)}{1+0+1} = 2$; $b = 3 - 2 = 1$. So $\hat{y} = 2x + 1$, which passes through all three points.
2. The model's squared error on the test set is 20% larger than that of simply predicting the training mean: it is worse than the baseline.
3. Because $\hat{y}$ is a weighted sum of its features — here x and x^2 — so it is linear in the weights, and least squares applies unchanged.

Chapter 6 — Logistic Regression

1. $p = 1/(1 + e^{-1.2}) = 1/1.301 = 0.769$; odds $= e^{1.2} = 3.32$ (about 10 to 3).
2. $e^{0.7} = 2.01$: the odds roughly double.
3. A positive predicted at 0.4 costs $-\ln 0.4 = 0.916$; a negative predicted at 0.4 costs $-\ln 0.6 = 0.511$. The positive costs more, because 0.4 is further from its true label.

Chapter 7 — Overfitting, Bias–Variance and Regularisation

1. Variance (overfitting). Remedies: more data, stronger regularisation or a simpler model, fewer features, early stopping, an averaging ensemble.
2. $\text{Bias}^2 = (2 - 3)^2 = 1$. Variance $= 0$: every prediction is identical.
3. The L1 constraint region is a diamond with corners on the axes; the error contours usually first touch it at a corner, where some weights are exactly zero. The L2 region is a circle with no corners, so weights shrink but rarely reach zero.

Chapter 8 — Measuring Classifier Performance

1. TP = 100, FP = 300, FN = 20. Precision $= 100/400 = 0.25$; recall $= 100/120 = 0.83$.
2. The precision–recall curve, or precision at the recall the application needs, and the number of alerts per day. With 0.1% positives a high ROC-AUC is compatible with most alerts being false.
3. $e = 0.15$: $1.96\sqrt{0.15 \times 0.85/100} = 0.070$, so roughly 8% to 22%.

Chapter 9 — Decision Tree Learning

1. The parent has entropy 1. Each child has entropy $-0.75 \log_2 0.75 - 0.25 \log_2 0.25 = 0.811$ and holds half the examples, so Gain $= 1 - 0.811 = 0.189$.
2. Every ID is unique, so splitting on it gives one pure leaf per task: maximum gain, no generalisation. Use the gain ratio, whose split-information term penalises many-valued attributes — or remove identifiers.
3. A restriction bias limits which hypotheses can be represented (CANDIDATE-ELIMINATION's conjunctions); a preference bias searches a complete space but prefers some hypotheses. ID3 has a preference bias: shorter trees, high-gain attributes near the root.

Chapter 10 — Bayesian Learning and Naive Bayes

1. Would break: $0.9 \times 0.01 = 0.009$; good: $0.05 \times 0.99 = 0.0495$. So h_{MAP} is *good*; h_{ML} is *would break* ($0.9 > 0.05$). The posterior probability that the release would break is $0.009/0.0585 = 0.15$.
2. That the attributes are independent of each other given the class. When they are correlated the same evidence is counted several times, and the probabilities are pushed towards 0 and 1 — over-confident — even though the ranking of classes is often still right.
3. The product of hundreds of small probabilities underflows to zero in floating-point arithmetic. Logarithms turn the product into a sum of manageable numbers.

Chapter 11 — Instance-Based Learning: k-Nearest Neighbours

1. Distances from (50, 55): F 11.18 (Late), D 15.81 (Late), C 22.36 (On time), E 23.77, B 33.54, A 50.0. The three nearest vote Late, Late, On time: **Late**.
2. It does no work until asked to classify, and then searches the stored examples. The cost is paid at prediction time — a distance to every stored example for each query — and all the training data must be kept.
3. Bias rises and variance falls. At $k = 1$ the boundary follows every example; at $k = n$ every query gets the majority class.

Chapter 12 — Support Vector Machines

1. $\|\mathbf{w}\| = 5$, so the margin is $2/5 = 0.4$. $f(1, 1) = 3 + 4 - 5 = 2 > 0$: positive.
2. The margin narrows and the model tolerates fewer violations, so training errors fall — and the risk of overfitting rises.
3. Because both the SVM's optimisation and its decision function use the training examples only through dot products $\mathbf{x}_i \cdot \mathbf{x}_j$, and each dot product can be replaced by a kernel.

Chapter 13 — Artificial Neural Networks

1. (0, 1): $-1 + 0 + 1 = 0$, not greater than 0, so output 0. (1, 0): $-1 + 2 + 0 = 1 > 0$, so output 1.
2. XOR's positive and negative points lie on opposite diagonals; no single line separates them. A hidden layer with one unit for OR and one for NAND, combined by an AND output unit, represents it.
3. A hidden unit affects the error only through the output units it feeds, so by the chain rule its share of the blame is the output deltas weighted by its connections to them: $\delta_h = o_h(1 - o_h) \sum_k w_{kh} \delta_k$.

Chapter 14 — Ensemble Learning

1. $3(0.8^2)(0.2) + 0.8^3 = 0.384 + 0.512 = 0.896$.
2. $\alpha = \frac{1}{2} \ln(0.8/0.2) = \frac{1}{2} \ln 4 = 0.693$. Misclassified examples are multiplied by $e^{0.693} = 2$, correct ones by $e^{-0.693} = 0.5$, before renormalising.
3. A tree is unstable: different bootstrap samples give very different trees, whose errors partly cancel when averaged. Logistic regression is stable: models on different bootstrap samples are nearly identical, so averaging them changes little.

Chapter 15 — Clustering

1. 1 and 4 go to 2; 6, 7 and 9 go to 8. New centroids: $(1 + 4)/2 = 2.5$ and $(6 + 7 + 9)/3 = 7.33$.
2. Neither step can increase the WCSS and there are finitely many possible assignments, so it must stop. But it stops at a local minimum that depends on the starting centroids.
3. $s = (2 - 3)/3 = -0.33$. The point is, on average, closer to another cluster than to its own: it is probably in the wrong cluster.

Chapter 16 — Dimensionality Reduction and Self-Organizing Maps

1. $(6 + 3)/(6 + 3 + 1) = 0.9$.
2. PCA seeks directions of large variance, and a feature measured in large units has large variance whether or not it is informative; it would dominate the components purely because of its units.
3. The BMU is the single unit whose weights are closest to the input (competition). Its neighbourhood is the units near it *on the grid*. Updating the neighbours as well makes adjacent units similar, which produces a smooth, topology-preserving map.

Chapter 17 — Semi-Supervised Learning and the EM Algorithm

1. Examples in the same cluster tend to share a label, so the decision boundary should pass through low-density regions. It fails when classes overlap inside one cluster — for example tickets with different root causes whose text and history look the same (a timeout caused by the network and one caused by a slow database query).
2. The E-step computes, with the current parameters, each example's responsibilities (the probability that each component produced it). The M-step re-estimates the parameters from the responsibility-weighted examples.
3. k -means makes hard assignments and estimates only means (implicitly round, equal clusters); EM for a Gaussian mixture makes soft assignments and also estimates covariances and mixing weights, maximising the likelihood.

Chapter 18 — Hidden Markov Models

1. $D \rightarrow H \rightarrow H = 0.3 \times 0.8 = 0.24$; $D \rightarrow D \rightarrow H = 0.7 \times 0.3 = 0.21$; total 0.45.
2. $\alpha_t(i)$ is the probability of the observations up to t and being in state i , summed over every path; $\delta_t(i)$ is the probability of the single best path ending in i — a maximum instead of a sum.
3. There are N^T state sequences, exponentially many. The forward algorithm stores α_t for each state and reuses it at the next step, which costs N^2T .

Chapter 19 — Markov Decision Processes

1. $4 + 0.5 \times 4 + 0.25 \times 4 = 7$.
2. $V^\pi(s)$ is the expected return from s following π ; $Q^\pi(s, a)$ is the expected return from taking a first and then following π . So $V^\pi(s) = Q^\pi(s, \pi(s))$.
3. Because a team that keeps shipping on a debt-laden codebase eventually freezes it, and a frozen codebase delivers nothing for ever however hard the team “ships”. The small immediate gain of shipping is outweighed by the lost future value that refactoring preserves.

Chapter 20 — Reinforcement Learning

1. $(10 + 4 + 7)/3 = 7$; after a return of 3, $7 + (3 - 7)/4 = 6$.
2. An arm that happened to pay badly the first time is never tried again, so its low estimate is never corrected; greedy selection can lock onto a worse arm for ever.
3. $Q \leftarrow 4 + 0.5(1 + 0.9 \times 6 - 4) = 4 + 0.5 \times 2.4 = 5.2$.

Chapter 21 — Responsible and Ethical Machine Learning

1. $30/45 = 0.67 < 0.8$: it fails the four-fifths rule.
2. Other features act as proxies and let the model reconstruct the attribute, and the labels themselves may encode past unfairness.
3. Equal opportunity; it compares true positive rates (recall) across groups.

Chapter 22 — The Machine Learning Project

1. For example: “Over the next two sprints, the warning flags at least 70% of the tasks that go on to finish late, and no more than one flag in three is a task that finishes on time.”
2. They differ by 0.007, far less than one standard deviation of either: they are indistinguishable. Choose on other grounds — simplicity, speed, explainability, fairness.
3. Any three of: the random seeds; the data version and the split indices; the code version (commit); the library versions; the hyperparameters.

Glossary

Every key term in the handout, with the chapter that introduces it.

Term	Meaning	Ch
Accuracy	Fraction of predictions that are correct; misleading when classes are imbalanced	8
Action value $Q(s, a)$	Expected return from taking action a in state s and acting well afterwards	19
AdaBoost	Boosting that re-weights examples towards the previous learners' mistakes	14
Adjusted Rand index	Agreement between a clustering and known labels, corrected for chance	15
Adversarial example	An input altered slightly and deliberately to change a model's output	21
Agent	The learner and decision maker in reinforcement learning	19
AUC	Area under the ROC curve: the chance a random positive is ranked above a random negative	8
Backpropagation	Computing a network's error gradients layer by layer, from the output back	13
Bagging	Training models on bootstrap samples and averaging them	14
Bandit (multi-armed)	Repeated choice among actions with unknown payoffs; one state	20
Baseline	The score of the simplest possible predictor; every model must beat it	3
Baum–Welch	EM for learning an HMM's parameters	18
Bayes optimal classifier	Posterior-weighted vote of all hypotheses; no method does better on average	10
Bellman equation	A state's value as immediate reward plus discounted value of the next state	19
Best-matching unit	The SOM unit whose weights are closest to the input	16
Bias (statistical)	Error from a model's assumptions: its average prediction minus the truth	7
Bias (of a model)	The intercept b of a linear model	5
Bootstrap sample	n examples drawn with replacement from n	14
Candidate-Elimination	Maintains the S and G boundaries of the version space	4
Centroid	The mean of a cluster's examples	15
Classification	Supervised learning with a categorical target	1
Concept learning	Inferring a boolean function from examples	4
Confusion matrix	Table of TP, FP, FN, TN	8
Cross-entropy	$-\ln$ of the probability given to the true label; logistic regression's loss	6

Term	Meaning	Ch
Cross-validation (k -fold)	Rotating each of k folds as validation data and averaging	7
Curse of dimensionality	In many dimensions distances concentrate and neighbours lose meaning	11
Data leakage	Information unavailable at prediction time reaching training or evaluation	3
Decision boundary	Where a classifier's prediction changes class	6
Decision tree	A sequence of attribute tests ending in class leaves	9
Delta rule	Gradient descent on squared error for a linear unit	13
Demographic parity	Equal selection rates across groups	21
Dendrogram	Tree of merges produced by hierarchical clustering	15
Discount factor γ	Weight given to a reward one step later	19
EM algorithm	Alternates soft assignment of hidden variables (E) and re-estimation (M)	17
Emission matrix	In an HMM, $P(\text{observation} \mid \text{state})$	18
Ensemble	A combination of several models	14
Entropy	Uncertainty of a distribution in bits, $-\sum p \log_2 p$	2
Epoch	One pass through the whole training set	13
Equal opportunity	Equal true positive rates across groups	21
ε -greedy	Act greedily, except explore at random with probability ε	20
Exploration vs. exploitation	Trying new actions against using the best known one	20
Feature	An input variable (column)	1
Find-S	Returns the most specific hypothesis consistent with the positives	4
Forward algorithm	Computes the probability of an HMM observation sequence	18
Gain ratio	Information gain divided by split information	9
Gaussian mixture	A density that is a weighted sum of Gaussians	17
Generalisation	Performance on data the model has not seen	7
Gini index	Impurity $1 - \sum p_i^2$	9
Gradient	Vector of partial derivatives; points uphill	2
Gradient boosting	Adding trees fitted to the current residuals	14
Gradient descent	Repeated small steps against the gradient	2
Hidden Markov model	Hidden Markov chain emitting observations	18
Hierarchical clustering	Merging clusters step by step into a dendrogram	15
Hinge loss	$\max(0, 1 - yf(\mathbf{x}))$; the SVM's loss	12
Hyperparameter	A setting chosen before training, by validation	7
Hypothesis, hypothesis space	A candidate function; the set of all candidates	1
ID3	Greedy top-down tree growing by information gain	9
Imputation	Filling in missing values	3
Inductive bias	The assumptions a learner needs to generalise beyond its data	1, 4

Term	Meaning	Ch
Information gain	Expected reduction in entropy from a split	9
Kernel (trick)	Computing a dot product in a feature space without building it	12
k -means	Alternating nearest-centroid assignment and centroid update	15
k -nearest neighbours	Predicting from the k closest stored examples	11
Label	The target value of an example	1
Laplace smoothing	Adding one imaginary count of every value	10
Lasso (L1)	Regression with a $\lambda \sum w $ penalty; sets some weights to zero	7
Lazy learning	Deferring all work until a query arrives	11
Learning curve	Training and validation error against training set size	7
Learning rate	Step size in gradient descent	2
Likelihood	Probability of the data as a function of the parameters	2
Linear separability	Classes separable by a hyperplane	13
Linkage	The between-cluster distance in hierarchical clustering	15
Log-odds (logit)	$\ln \frac{p}{1-p}$; what logistic regression models linearly	6
MAP hypothesis	The hypothesis maximising $P(D h)P(h)$	10
Margin	Width of the empty band around an SVM's boundary	12
Markov decision process	States, actions, transitions, rewards and a discount	19
Markov property	The next state depends only on the current one	18
Maximum likelihood	Choosing parameters that make the data most probable	2
McNemar's test	Tests whether two classifiers differ, from their disagreements	8
Model card	A document describing a model's use, data, performance and limits	21
Monte Carlo estimation	Estimating an expectation by sampling and averaging	18, 20
Naive Bayes	Bayes classification assuming conditionally independent attributes	10
Normal equation	$X^\top X \boldsymbol{\theta} = X^\top \mathbf{y}$	5
Odds ratio	Factor by which odds multiply per unit of a feature, e^w	6
One-hot encoding	One 0/1 column per category	3
Out-of-bag estimate	Error on examples a bagged model did not train on	14
Overfitting	Fitting noise: low training error, high test error	7
Perceptron	A thresholded weighted sum	13
Pipeline	Chained preparation steps and model, fitted together	3
Policy	A rule choosing an action in each state	19
Policy iteration	Alternating exact policy evaluation and greedy improvement	19
Precision	Of the positive predictions, the fraction correct	8
Principal component	Direction of greatest remaining variance	16
Proxy variable	A feature that stands in for a protected attribute	21

Term	Meaning	Ch
Pruning	Removing parts of a tree to reduce overfitting	9
Q-learning	Off-policy TD learning of action values	20
Random forest	Bagged trees choosing among random feature subsets	14
Recall	Of the actual positives, the fraction found	8
Regression	Supervised learning with a numeric target	1
Regularisation	Penalising complexity to trade variance for bias	7
Reinforcement learning	Learning actions from rewards, without a model	1, 20
Residual	$y - \hat{y}$	5
Responsibility	In EM, the posterior probability that a component produced an example	17
Return	Discounted sum of future rewards	19
Ridge (L2)	Regression with a $\lambda \sum w^2$ penalty	7
ROC curve	True against false positive rate across thresholds	8
SARSA	On-policy TD learning of action values	20
Self-organizing map	A grid of units trained to map data topology-preservingly	16
Self-training	Adding confident predictions as labels and retraining	17
Semi-supervised learning	Learning from a few labelled and many unlabelled examples	17
Sigmoid	$1/(1 + e^{-z})$	6
Silhouette	$(b - a)/\max(a, b)$: how well an example sits in its cluster	15
Softmax	Turns k scores into probabilities summing to 1	6
Standardisation	Rescaling to mean 0, standard deviation 1	3
Stratified split	A split preserving class proportions	3
Supervised learning	Learning from labelled examples	1
Support vector	A training point on the SVM's margin	12
TD learning	Updating values towards $r + \gamma V(s')$ after each step	20
Test set	Data held back and used once for the final estimate	3
Threshold	The score above which a classifier predicts positive	6, 8
UCB	Action choice adding an uncertainty bonus to each estimate	20
Underfitting	Too simple a model: high training and test error	7
Unsupervised learning	Finding structure in unlabelled data	1
Validation set	Data for comparing models and choosing settings	3
Value iteration	Repeated Bellman optimality updates	19
Variance (of a model)	Sensitivity of predictions to the particular training sample	7
Version space	All hypotheses consistent with the training data	4
Viterbi algorithm	Finds the most likely hidden-state sequence of an HMM	18
Weak learner	A model only slightly better than chance, used in boosting	14

Resources

E.1 Textbooks

Book	Use it for
Tom M. Mitchell, <i>Machine Learning</i> , McGraw-Hill, 1997 (HEC set text)	Chapters 1, 4, 9, 10, 11, 13 of this handout follow its treatment; its EnjoySport and PlayTennis tables are the release and sprint data of Chapters 4 and 9, renamed
Kevin P. Murphy, <i>Machine Learning: A Probabilistic Perspective</i> , MIT Press, 2012 (HEC set text)	The probabilistic view: Bayesian learning, mixtures and EM, HMMs
Kevin P. Murphy, <i>Probabilistic Machine Learning: An Introduction</i> , MIT Press, 2022	The updated successor to the set text; freely available from the author
Gareth James et al., <i>An Introduction to Statistical Learning with Applications in Python</i> , Springer, 2023	The gentlest rigorous treatment of regression, classification, resampling and trees; free online
Aurélien Géron, <i>Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow</i> , 3rd ed., O'Reilly, 2022	Practical scikit-learn; the labs' style
Richard S. Sutton and Andrew G. Barto, <i>Reinforcement Learning: An Introduction</i> , 2nd ed., MIT Press, 2018	Chapters 19–20; bandits, Monte Carlo, TD, Q-learning; free online
Christopher M. Bishop, <i>Pattern Recognition and Machine Learning</i> , Springer, 2006	A deeper mathematical treatment of mixtures, kernels and neural networks

E.2 Online Courses

HEC 2025's AI specialization recommends online courses as companions to its core courses, and for *Machine Learning* names the Machine Learning Specialization (DeepLearning.AI and Stanford, on Coursera) and Columbia University's Machine Learning course on edX. Both align well with Parts II and III of this handout. Stanford's CS229 lecture notes are a free, more mathematical supplement.

E.3 Datasets

Source	Notes
Synthetic project data	Every lab builds its own tasks, projects, tickets or code changes with NumPy or scikit-learn's <code>make_*</code> generators, so it runs offline and its answers are known

Source	Notes
Open-source issue trackers	The Apache Software Foundation’s public Jira and GitHub issues: real tickets with text, history, status changes and resolution times
PROMISE repository	Public software-engineering datasets, among them effort-estimation and defect-prediction data
UCI, OpenML, Kaggle	General collections; search them for software-project, effort and defect data, and read other people’s notebooks critically
Your employer’s tracker	Often the most meaningful data; requires written permission and the removal of personal identifiers (Chapter 21)

E.4 Tools

Anaconda (Python distribution), Jupyter notebooks for exploration, a plain editor or VS Code for scripts, Git for version control, and the scikit-learn user guide — the single most useful reference for the practical side of this course.

E.5 The Summer 2026 Handout

The department’s *Machine Learning — Core Concepts* handout (Summer 2026) is a 24-page overview written for an eight-week summer session. It covers the supervised, unsupervised and neural-network material of Parts II–IV in outline, but not concept learning, version spaces, self-organizing maps, semi-supervised EM, hidden Markov models or Markov decision processes, which the HEC outline requires. Use it as a revision summary alongside this volume, not in place of it.

Sixteen-Week Teaching Schedule

A mapping of the 22 chapters onto a sixteen-week semester of two lecture hours and three laboratory hours a week — the HEC 3 (2–3) pattern — with assessment points. Adjust to your own calendar; the dependencies are what matter.

F.1 The Schedule

Wk	Lecture topic	Chapters	Lab	Assessment
1	What machine learning is; its mathematics	1, 2	A rule against a learned model; the chapter in NumPy	—
2	The workflow and the toolkit	3	A leak-proof pipeline	—
3	Concept learning and the version space	4	Candidate elimination in forty lines	Quiz 1 (Ch 1–3)
4	Linear regression	5	Least squares three ways	—
5	Logistic regression	6	Logistic regression by hand and by library	Assignment 1 set
6	Overfitting, bias–variance, regularisation	7	Find the bottom of the U	—
7	Measuring classifier performance	8	Evaluate an imbalanced classifier	Assignment 1 due ; Quiz 2 (Ch 4–7)
8	Decision trees	9	ID3 by hand, pruning by cross-validation	Midterm (Ch 1–8)
9	Bayesian learning and naive Bayes	10	Naive Bayes by counting and by library	—
10	Nearest neighbours; support vector machines	11, 12	Neighbours, scaling and k ; margins and kernels	Assignment 2 set
11	Neural networks; the project brief	13, 22	Backpropagation from scratch	Project set ; proposal due
12	Ensemble learning	14	One tree against five ensembles	Assignment 2 due
13	Clustering; PCA and self-organizing maps	15, 16	k -means and a dendrogram; PCA and a SOM	Quiz 3 (Ch 9–14)
14	Semi-supervised learning and EM; hidden Markov models	17, 18	EM by hand; forward, Viterbi, Baum–Welch	Project experiments frozen

Wk	Lecture topic	Chapters	Lab	Assessment
15	Markov decision processes; reinforcement learning	19, 20	Value and policy iteration; bandits and Q-learning	—
16	Responsible machine learning; project presentations	21	Auditing a model for fairness	Project report and presentation

⚠ If You Must Cut Something

Cut *depth*, never Chapters 7 and 8. Generalisation and evaluation are the hinge of the course: a student who cannot tell a good model from a lucky one will misreport every later result. If time is short, compress Chapter 4 to the version space and candidate elimination worked example, teach Chapter 16's self-organizing map in one lecture, and combine Chapters 19 and 20 — but keep the Q-learning update, which the HEC outline's reinforcement-learning outcome requires.

F.2 Dependency Map

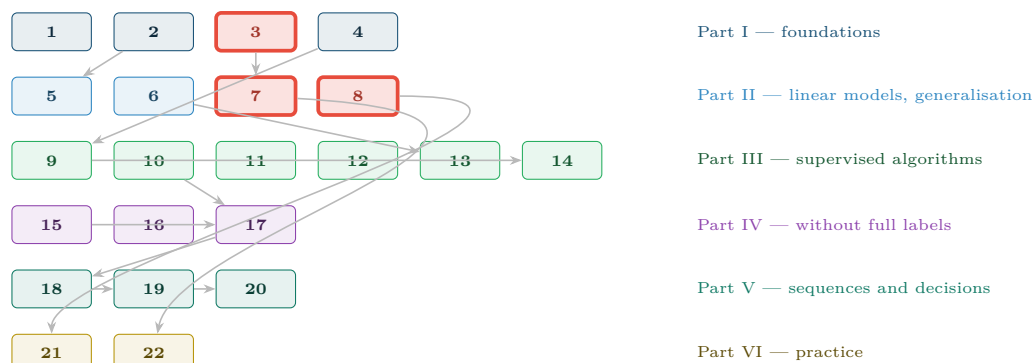


Figure F.1: Chapter dependencies. The outlined chapters (3, 7 and 8) are the ones every later chapter relies on; the arrows mark the strongest direct dependencies.

F.3 Laboratory Sessions

Each week's three laboratory hours follow the same pattern: the chapter's *Try It Yourself* lab first (about one hour, from the runnable file in `code/`), then the chapter's applied review questions worked in pairs, then — from Week 11 — project time with the instructor circulating. Every lab runs on a standard Anaconda installation (Python 3.10 or later, NumPy, pandas, SciPy, scikit-learn); none needs a GPU or an internet connection.